In [ ]:

```
# ============================================================================
# UNIFIED EVALUATION — .m equation × .py simulator  (single Colab cell)
# ----------------------------------------------------------------------------
# Paste this entire cell into Google Colab.  No uploads needed.
# Dependencies: numpy, scipy, matplotlib  (all pre-installed in Colab)
# ============================================================================

import os, math, csv, json, random, statistics, warnings
from collections import defaultdict, deque
from dataclasses import dataclass, field
from typing import Optional
import numpy as np
from scipy import stats
from scipy.integrate import quad
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

warnings.filterwarnings('ignore')
np.random.seed(42)

# ═══════════════════════════════════════════════════════════════════════════
# SECTION 1 — EQUATIONS FROM THE .m FILE  (the analytic backbone)
# ═══════════════════════════════════════════════════════════════════════════
# Logistic degradation y = L / (1 + exp(-k*(x - x0)))  — from untitled2.m
LOGISTIC_L, LOGISTIC_K, LOGISTIC_X0 = 302.2328, 0.0143, 101.5374
# Quadratic fit y = a*x^2 + b*x + c                    — from untitled2.m
QUAD_A = -0.0005518667521387619
QUAD_B =  0.7129391105869533
QUAD_C =  93.5322316774016

def logistic_curve(x):
    return LOGISTIC_L / (1.0 + math.exp(-LOGISTIC_K * (x - LOGISTIC_X0)))

def logistic_curve_vec(x):
    return LOGISTIC_L / (1.0 + np.exp(-LOGISTIC_K * (np.asarray(x) - LOGI

def quadratic_curve(x):
    return QUAD_A * x**2 + QUAD_B * x + QUAD_C

def severity_index(latency_ms, loss_rate, w_tau=0.5, w_p=0.5,
                   tau_min=0.0, tau_max=600.0):
    return w_tau * (latency_ms - tau_min) / (tau_max - tau_min) + w_p * l

# ═══════════════════════════════════════════════════════════════════════════
# SECTION 2 — SCENARIO CONSTANTS (from traffic_control_algorithm.py)
# ═══════════════════════════════════════════════════════════════════════════
COMMUNICATION_SCENARIOS = {
    'S1': {'latency_ms':   0.0, 'loss_rate': 0.00, 'label': 'Ideal'},
    'S2': {'latency_ms': 100.0, 'loss_rate': 0.05, 'label': 'Moderate'},
    'S3': {'latency_ms': 300.0, 'loss_rate': 0.15, 'label': 'Severe'},
    'S4': {'latency_ms': 600.0, 'loss_rate': 0.30, 'label': 'Extreme'},
}
```

```
NATURAL_SCENARIOS = {
    'N-clear':       {'label':'Clear',   'capacity':1.00,'demand_mult':1.
    'N-rain':        {'label':'Rain',    'capacity':0.85,'demand_mult':1.
    'N-snow':        {'label':'Snow',    'capacity':0.55,'demand_mult':0.
    'N-accident':    {'label':'Accid',   'capacity':0.65,'demand_mult':1.
    'N-rush-am':     {'label':'AMpk',    'capacity':1.00,'demand_mult':1.
    'N-sensor-fail': {'label':'NoDet',   'capacity':1.00,'demand_mult':1.
}
DECISION_INTERVAL_S = 10.0
GREEN_MIN_S, GREEN_MAX_S, YELLOW_S = 10.0, 60.0, 3.0
GRID_ROWS, GRID_COLS = 3, 3
SATURATION_FLOW_VPS = 0.5

# ═══════════════════════════════════════════════════════════════════
# SECTION 3 — CONDENSED TRAFFIC CONTROL ALGORITHM  (from .py)
# ═══════════════════════════════════════════════════════════════════
@dataclass
class Message:
    sender: str; receiver: str; t_send_ms: float; seq: int
    phase: int; queue_ns: float; queue_ew: float; elapsed_s: float

class ImpairmentChannel:
    """Truncated log-normal latency + Gilbert-Elliott correlated loss."""
    def __init__(self, latency_mean_ms, loss_rate, rng=None):
        self.latency_mean = latency_mean_ms
        self.loss_rate = loss_rate
        self.rng = rng or random.Random()
        self.last_seq = defaultdict(lambda: -1)
        self.ge_state = defaultdict(lambda: 'G')
        self.p_gb, self.p_bg, self.p_b = 0.30, 0.70, 0.90
        pi_b = self.p_gb / (self.p_gb + self.p_bg); pi_g = 1.0 - pi_b
        self.p_g = max(0.0, (loss_rate - pi_b*self.p_b) / max(pi_g, 1e-6))
        self.sent_count = 0; self.delivered_count = 0

    def _latency(self):
        if self.latency_mean <= 0: return 0.0
        mu, sg = math.log(self.latency_mean), 0.5
        for _ in range(16):
            tau = self.rng.lognormvariate(mu, sg)
            if 0.1*self.latency_mean <= tau <= 5*self.latency_mean:
                return tau
        return self.latency_mean

    def _loss(self, link):
        state = self.ge_state[link]
        if state == 'G':
            p_loss = self.p_g
            if self.rng.random() < self.p_gb: self.ge_state[link] = 'B'
        else:
            p_loss = self.p_b
            if self.rng.random() < self.p_bg: self.ge_state[link] = 'G'
        return self.rng.random() < p_loss

    def transmit(self, msg):
```

```
        self.sent_count += 1
        if self._loss(f"{msg.sender}->{msg.receiver}"): return None
        self._latency()
        if msg.seq <= self.last_seq[msg.sender]: return None
        self.last_seq[msg.sender] = msg.seq
        self.delivered_count += 1
        return msg

class Intersection:
    def __init__(self, node_id, rng):
        self.id, self.rng = node_id, rng
        self.q = {'N':0.0,'S':0.0,'E':0.0,'W':0.0}
        self.W = {'N':0.0,'S':0.0,'E':0.0,'W':0.0}
        self.phase, self.phase_elapsed = 0, 0.0
        self.arrivals = self.departures = self.completed = 0.0
        self.seq = 0; self.row = self.col = 0

    def step(self, dt, arrive_ns, arrive_ew, green_active_phase, green_ti
        for a in ('N','S'):
            self.q[a] += 0.5*arrive_ns*dt; self.arrivals += 0.5*arrive_ns
        for a in ('E','W'):
            self.q[a] += 0.5*arrive_ew*dt; self.arrivals += 0.5*arrive_ew
        serviceable = (('N','S') if green_active_phase == 0 else
                       ('E','W') if green_active_phase == 1 else ())
        for a in serviceable:
            d = min(self.q[a], SATURATION_FLOW_VPS * green_time)
            self.q[a] -= d; self.departures += d; self.completed += d
        for a in self.q: self.W[a] += self.q[a] * dt
        self.phase_elapsed += dt
        if green_active_phase is not None and green_active_phase != self.
            self.phase, self.phase_elapsed = green_active_phase, 0.0

    def total_queue(self): return sum(self.q.values())
    def queue_ns(self):    return self.q['N'] + self.q['S']
    def queue_ew(self):    return self.q['E'] + self.q['W']
    def snapshot(self):
        return {'id':self.id,'seq':self.seq,'phase':self.phase,
                'queue_ns':self.queue_ns(),'queue_ew':self.queue_ew(),
                'elapsed_s':self.phase_elapsed}

class TrafficNetwork:
    def __init__(self, rng):
        self.rng = rng; self.nodes = {}
        for r in range(GRID_ROWS):
            for c in range(GRID_COLS):
                nid = f"J{r+1}{c+1}"
                self.nodes[nid] = Intersection(nid, rng)
                self.nodes[nid].row, self.nodes[nid].col = r, c

    def neighbour_snapshots(self, node_id):
        r, c = self.nodes[node_id].row, self.nodes[node_id].col
        out = []
        for dr, dc in ((-1,0),(1,0),(0,-1),(0,1)):
            nr, nc = r+dr, c+dc
```

```
            if 0 <= nr < GRID_ROWS and 0 <= nc < GRID_COLS:
                out.append(self.nodes[f"J{nr+1}{nc+1}"].snapshot())
        return out

class TrafficControlAlgorithm:
    CONTROLLERS: dict = {}
    CATEGORIES = ('classical','coordinated','learning','mixed','field')

    def __init__(self, algorithm_type='adaptive', scenario='S1',
                 natural='N-clear', seed=0, car_following='SAFE',
                 beta=0.5, lambda_decay_per_ms=0.01,
                 stale_threshold_ms=500.0, p_drop=0.10, training=False):
        if algorithm_type not in self.CONTROLLERS:
            raise ValueError(f"Unknown algorithm '{algorithm_type}'. "
                             f"Available: {sorted(self.CONTROLLERS)}")
        override = NATURAL_SCENARIOS[natural].get('comm_override')
        if override: scenario = override
        self.algorithm_type = algorithm_type
        self.scenario = scenario; self.natural = natural
        self.comm_cfg = COMMUNICATION_SCENARIOS[scenario]
        self.nat_cfg  = NATURAL_SCENARIOS[natural]
        self.rng = random.Random(seed)
        self.beta = beta; self.lambda_decay = lambda_decay_per_ms
        self.stale_thresh = stale_threshold_ms
        self.p_drop = p_drop; self.training = training
        self.channel = ImpairmentChannel(self.comm_cfg['latency_ms'],
                                         self.comm_cfg['loss_rate'],
                                         rng=self.rng)
        self.aoi_ms = defaultdict(float)
        self.fwd_history = defaultdict(lambda: deque(maxlen=8))
        self.optimization_events = 0
        self.phase_decisions = []
        self.seq = 0; self.current_time_ms = 0.0
        self.cycle_length_s = 60.0
        self.m = {'travel_time_sum_s':0.0,'queue_sum':0.0,
                  'waiting_sum_s':0.0,'vehicles_completed':0.0,'samples':

    @classmethod
    def register(cls, name, category, inputs, fn, doc=''):
        cls.CONTROLLERS[name] = (category, tuple(inputs), fn, doc)

    @classmethod
    def compose(cls, name_a, name_b, weight_a=0.5, new_name=None):
        _, in_a, fn_a, _ = cls.CONTROLLERS[name_a]
        _, in_b, fn_b, _ = cls.CONTROLLERS[name_b]
        inputs = tuple(sorted(set(in_a) | set(in_b)))
        w = max(0.0, min(1.0, weight_a))
        cname = new_name or f"{name_a}|{name_b}:{w:.2f}"
        def blended(self, **kw):
            pa, da = fn_a(self, **{k:kw[k] for k in in_a})
            pb, db = fn_b(self, **{k:kw[k] for k in in_b})
            ph = pa if w >= 0.5 else pb
            return ph, w*da + (1-w)*db
        cls.CONTROLLERS[cname] = ('composed', inputs, blended, '')
```

```
        return cname

    @classmethod
    def list_controllers(cls, category=None):
        if category is None: return sorted(cls.CONTROLLERS)
        return sorted(n for n,(c,_,_,_) in cls.CONTROLLERS.items() if c =

    def build_observation(self, local_state, neighbours):
        self.seq += 1; received = []
        for nb in (neighbours or []):
            msg = Message(nb.get('id','nb'),'self',self.current_time_ms,
                          nb.get('seq',0), nb.get('phase',0),
                          nb.get('queue_ns',0.0), nb.get('queue_ew',0.0),
                          nb.get('elapsed_s',0.0))
            delivered = self.channel.transmit(msg)
            if delivered is None:
                self.aoi_ms[msg.sender] += DECISION_INTERVAL_S * 1000.0
                continue
            aoi = self.aoi_ms[msg.sender] + self.comm_cfg['latency_ms']
            self.aoi_ms[msg.sender] = aoi
            received.append((delivered, aoi))
            self.fwd_history[delivered.sender].append(
                (self.current_time_ms, delivered.queue_ns, delivered.queu
        return {'local': local_state, 'received': received}

    def control_signal(self, intersection_id, vehicle_count_ns,
                       vehicle_count_ew, waiting_ns, waiting_ew,
                       current_time, current_congestion,
                       gap_ns=0.0, gap_ew=0.0, neighbours=None,
                       row=0, col=0, current_phase=0):
        self.current_time_ms = current_time * 1000.0
        cfg = self.nat_cfg
        nat_total = (vehicle_count_ns + vehicle_count_ew) * cfg['demand_m
        bias = cfg['direction_bias']
        veh_ns = nat_total * (0.5 + 0.5*bias)
        veh_ew = nat_total - veh_ns
        congestion_eff = min(0.99, current_congestion / max(cfg['capacity
        if cfg['sensor_reliability'] < 1.0:
            if self.rng.random() > cfg['sensor_reliability']:
                veh_ns, veh_ew, waiting_ns, waiting_ew = 0.0, 0.0, 0.0, 0
        obs = self.build_observation(
            {'phase':current_phase,'elapsed_s':current_time % 30.0,
             'queue_ns':veh_ns,'queue_ew':veh_ew}, neighbours)
        cat, inputs, fn, _ = self.CONTROLLERS[self.algorithm_type]
        context = {'intersection_id':intersection_id,'veh_ns':veh_ns,'veh
                   'waiting_ns':waiting_ns,'waiting_ew':waiting_ew,
                   'current_time':current_time,'current_congestion':curre
                   'congestion_eff':congestion_eff,'gap_ns':gap_ns,'gap_e
                   'obs':obs,'row':row,'col':col,'current_phase':current_
        kwargs = {k: context[k] for k in inputs}
        phase, duration = fn(self, **kwargs)
        duration = max(GREEN_MIN_S, min(GREEN_MAX_S, duration))
        self.phase_decisions.append({
            'intersection':intersection_id,'phase':phase,'duration':durat
```

```
            'algorithm':self.algorithm_type,'scenario':self.scenario,
            'natural':self.natural,'time':current_time})
        self.m['samples'] += 1
        self.m['waiting_sum_s'] += waiting_ns + waiting_ew
        self.m['queue_sum'] += veh_ns + veh_ew
        cleared = min(2.5, 0.025 * (veh_ns + veh_ew) * cfg['capacity'])
        self.m['vehicles_completed'] += cleared
        cf_in_sys = (112.39 if congestion_eff < 0.5 else 171.28) \
                    + 0.35*(veh_ns+veh_ew)*(0.5+congestion_eff)
        self.m['travel_time_sum_s'] += (
            0.6*cf_in_sys + 0.4*(0.5*(waiting_ns+waiting_ew) + 100.0))
        return phase, duration

    def get_metrics(self):
        s = max(self.m['samples'], 1); sim_s = s * DECISION_INTERVAL_S
        sent = max(self.channel.sent_count, 1)
        mean_aoi = (sum(self.aoi_ms.values())/len(self.aoi_ms)
                    if self.aoi_ms else 0.0)
        return {
            'algorithm_type': self.algorithm_type,
            'scenario':       self.scenario,
            'natural':        self.natural,
            'travel_time_s':       round(self.m['travel_time_sum_s']/s, 2
            'queue_veh':           round(self.m['queue_sum']/s, 2),
            'waiting_time_s':      round(self.m['waiting_sum_s']/s, 2),
            'throughput_vph':      round(self.m['vehicles_completed']/max
            'mean_aoi_ms':         round(mean_aoi, 1),
            'delivery_ratio':      round(self.channel.delivered_count/sen
            'severity_index':      round(severity_index(
                self.comm_cfg['latency_ms'], self.comm_cfg['loss_rate']),
            'optimization_events': self.optimization_events,
        }

# ---- register controller expressions ----------------------------------
def _register_controllers():
    def reg(name, cat, inputs, doc=''):
        def deco(fn):
            TrafficControlAlgorithm.register(name, cat, inputs, fn, doc)
            return fn
        return deco

    @reg('fixed-time','classical',('current_time',))
    def fixed_time(self, current_time):
        return int((current_time % 60.0) / 30.0), 30.0

    @reg('actuated','classical',('veh_ns','veh_ew','gap_ns','gap_ew'))
    def actuated(self, veh_ns, veh_ew, gap_ns, gap_ew):
        if veh_ns >= veh_ew: phase, demand, gap = 0, veh_ns, gap_ns
        else:                phase, demand, gap = 1, veh_ew, gap_ew
        total = max(veh_ns + veh_ew, 1.0)
        green = max(GREEN_MIN_S, min(GREEN_MAX_S, GREEN_MAX_S*demand/tota
        if gap >= 3.0: green = GREEN_MIN_S
        self.optimization_events += 1
        return phase, green
```

```
    @reg('adaptive','classical',('veh_ns','veh_ew','congestion_eff'))
    def adaptive(self, veh_ns, veh_ew, congestion_eff):
        ns_r = veh_ns / max(veh_ns + veh_ew, 1.0)
        if congestion_eff > 0.7:
            ns_t, ew_t = (35.0, 20.0) if ns_r > 0.6 else (20.0, 35.0)
        else:
            ns_t = ew_t = 30.0
        phase = 0 if ns_r > 0.5 else 1
        self.optimization_events += 1
        return phase, (ns_t if phase == 0 else ew_t)

    @reg('coordinated','coordinated',
         ('veh_ns','veh_ew','congestion_eff','row','col'))
    def coordinated(self, veh_ns, veh_ew, congestion_eff, row, col):
        Y = min(0.85, 0.35 + 0.5*congestion_eff)
        L = 2.0 * YELLOW_S
        cycle = max(40.0, min(120.0, (1.5*L + 5.0)/max(1.0 - Y, 0.05)))
        self.cycle_length_s = cycle
        ns_r = veh_ns / max(veh_ns + veh_ew, 1.0)
        green = max(GREEN_MIN_S, cycle - YELLOW_S)
        phase = 0 if ns_r > 0.5 else 1
        self.optimization_events += 1
        return phase, green * (ns_r if phase == 0 else 1 - ns_r)

    @reg('cooperative-marl','learning',
         ('veh_ns','veh_ew','congestion_eff','obs'))
    def cooperative_marl(self, veh_ns, veh_ew, congestion_eff, obs):
        agg_ns, agg_ew = veh_ns, veh_ew
        for msg, _ in obs['received']:
            agg_ns += msg.queue_ns; agg_ew += msg.queue_ew
        phase = 0 if agg_ns >= agg_ew else 1
        self.optimization_events += 1
        return phase, 30.0 + 15.0*congestion_eff

    @reg('resilient-marl','learning',
         ('veh_ns','veh_ew','congestion_eff','obs'))
    def resilient_marl(self, veh_ns, veh_ew, congestion_eff, obs):
        agg_ns, agg_ew = veh_ns, veh_ew
        for msg, aoi in obs['received']:
            if self.training and self.rng.random() < self.p_drop: continu
            w = math.exp(-self.lambda_decay * aoi)
            agg_ns += w * msg.queue_ns; agg_ew += w * msg.queue_ew
        for nb_id, hist in self.fwd_history.items():
            if not hist or self.aoi_ms[nb_id] <= self.stale_thresh: conti
            if len(hist) >= 2:
                t0, ns0, ew0 = hist[-2]; t1, ns1, ew1 = hist[-1]
                dt = max(t1 - t0, 1.0)
                pred_ns = ns1 + (ns1 - ns0)/dt * self.aoi_ms[nb_id]
                pred_ew = ew1 + (ew1 - ew0)/dt * self.aoi_ms[nb_id]
            else:
                pred_ns, pred_ew = hist[-1][1], hist[-1][2]
            w = math.exp(-self.lambda_decay * self.aoi_ms[nb_id])
            agg_ns += w * pred_ns; agg_ew += w * pred_ew
```

```
        phase = 0 if agg_ns >= agg_ew else 1
        self.optimization_events += 1
        return phase, 30.0 + 12.0*congestion_eff

    @reg('max-pressure','field',('veh_ns','veh_ew','current_phase'))
    def max_pressure(self, veh_ns, veh_ew, current_phase):
        diff = (veh_ns - 0.6*veh_ew) - (veh_ew - 0.6*veh_ns)
        switch = (diff < -3.0) if current_phase == 0 else (diff > 3.0)
        new_phase = (1 - current_phase) if switch else current_phase
        dur = 20.0 + 0.6*abs(diff) + 0.4*(veh_ns + veh_ew)
        self.optimization_events += 1
        return new_phase, max(GREEN_MIN_S, min(GREEN_MAX_S, dur))

    @reg('mpc','field',('veh_ns','veh_ew','congestion_eff'))
    def mpc(self, veh_ns, veh_ew, congestion_eff):
        H, best_phase, best_cost = 4, 0, float('inf')
        arrival = (veh_ns + veh_ew) / max(DECISION_INTERVAL_S, 1.0) * 0.3
        for cand in (0, 1):
            qn, qe, cost = veh_ns, veh_ew, 0.0
            for _ in range(H):
                if cand == 0:
                    qn = max(0.0, qn - 3.0 + 0.5*arrival)
                    qe = max(0.0, qe + 0.5*arrival)
                else:
                    qn = max(0.0, qn + 0.5*arrival)
                    qe = max(0.0, qe - 3.0 + 0.5*arrival)
                cost += qn + qe + 0.05*(qn*qn + qe*qe)/20.0
            if cost < best_cost: best_cost, best_phase = cost, cand
        dur = 25.0 + 15.0*congestion_eff + 0.2*(veh_ns + veh_ew)
        self.optimization_events += 1
        return best_phase, min(GREEN_MAX_S, max(GREEN_MIN_S, dur))

    @reg('weather-adaptive','field',('veh_ns','veh_ew','congestion_eff'))
    def weather_adaptive(self, veh_ns, veh_ew, congestion_eff):
        cap = self.nat_cfg['capacity']
        qn = veh_ns / max(cap, 0.1); qe = veh_ew / max(cap, 0.1)
        ns_r = qn / max(qn + qe, 1.0)
        cycle = 60.0 + 30.0*(1.0 - cap)
        green = max(GREEN_MIN_S, cycle - 2*YELLOW_S - 2.0*(1.0 - cap))
        phase = 0 if ns_r > 0.5 else 1
        self.optimization_events += 1
        return phase, green * (ns_r if phase == 0 else 1 - ns_r)

_register_controllers()
TrafficControlAlgorithm.compose('cooperative-marl','coordinated',0.5,
                                new_name='cooperative-coordinated')
TrafficControlAlgorithm.compose('resilient-marl','coordinated',0.5,
                                new_name='resilient-coordinated')

# ═══════════════════════════════════════════════════════════════════
# SECTION 4 — EPISODE DRIVER + MULTI-SEED  (from .py)
# ═══════════════════════════════════════════════════════════════════
def simulate_episode(controller, n_steps=200, base_demand_ns=0.15,
                     base_demand_ew=0.10, congestion_start=0.40,
```

```
                    congestion_end=0.90, seed=None):
    rng = random.Random(seed) if seed is not None else controller.rng
    net = TrafficNetwork(rng); dt = DECISION_INTERVAL_S
    net_wait_sum = net_queue_samples = 0.0; net_samples = 0
    for t in range(n_steps):
        c = (congestion_start
             + (congestion_end - congestion_start) * t / max(n_steps - 1,
        cur_time = t * dt
        for nid, node in net.nodes.items():
            phase, dur = controller.control_signal(
                intersection_id=nid,
                vehicle_count_ns=node.queue_ns(),
                vehicle_count_ew=node.queue_ew(),
                waiting_ns=node.W['N'] + node.W['S'],
                waiting_ew=node.W['E'] + node.W['W'],
                current_time=cur_time, current_congestion=c,
                gap_ns=1.0 if node.queue_ns() > 0 else 5.0,
                gap_ew=1.0 if node.queue_ew() > 0 else 5.0,
                neighbours=net.neighbour_snapshots(nid),
                row=node.row, col=node.col, current_phase=node.phase)
            node.step(dt, arrive_ns=base_demand_ns*(0.8 + 0.6*c),
                      arrive_ew=base_demand_ew*(0.8 + 0.6*c),
                      green_active_phase=phase, green_time=min(dur, dt))
        for node in net.nodes.values():
            net_wait_sum += sum(node.W.values())
            net_queue_samples += node.total_queue()
        net_samples += len(net.nodes)
    controller.m['waiting_sum_s']      = net_wait_sum / max(net_samples,
    controller.m['queue_sum']          = net_queue_samples / max(net_samp
    controller.m['vehicles_completed'] = sum(n.completed for n in net.nod
    controller.m['samples']            = n_steps
    return controller

def run_multi_seed(algorithm_type, scenario='S1', natural='N-clear',
                   seeds=(1,2,3,4,5), n_steps=200,
                   base_demand_ns=0.30, base_demand_ew=0.30,
                   car_following='SAFE'):
    keys = ('travel_time_s','queue_veh','waiting_time_s',
            'throughput_vph','mean_aoi_ms','delivery_ratio')
    per_seed = defaultdict(list)
    for s in seeds:
        c = TrafficControlAlgorithm(algorithm_type, scenario, natural,
                                    seed=s, car_following=car_following)
        simulate_episode(c, n_steps=n_steps,
                         base_demand_ns=base_demand_ns,
                         base_demand_ew=base_demand_ew, seed=s)
        m = c.get_metrics()
        for k in keys: per_seed[k].append(float(m[k]))
    out = {'algorithm_type':algorithm_type,'scenario':scenario,
           'natural':natural,'n_seeds':len(seeds)}
    for k, xs in per_seed.items():
        mean = statistics.fmean(xs)
        if len(xs) > 1:
            sd = statistics.stdev(xs); ci95 = 1.96*sd/math.sqrt(len(xs))
```

```
        else:
            sd = ci95 = 0.0
        out[f'{k}_mean'] = round(mean, 3)
        out[f'{k}_sd']   = round(sd, 3)
        out[f'{k}_ci95'] = round(ci95, 3)
    return out

# ═══════════════════════════════════════════════════════════════════
# SECTION 5 — EQUATION AUDIT (8 representative eqs from amplitude_equ.txt
# ═══════════════════════════════════════════════════════════════════
@dataclass
class Cfg:
    L: float = LOGISTIC_L;  k: float = LOGISTIC_K;  x0: float = LOGISTIC_
    tau_min: float = 0.0;  tau_max: float = 600.0
    mu_tau: float = math.log(100.0);  sigma_tau: float = 0.5
    pG: float = 0.05;  pB: float = 0.60
    p_trans: float = 0.10;  q_trans: float = 0.30
    dt: float = 0.35;  T_sim: float = 60.0
    lambda_decay: float = 0.01;  beta: float = 0.5;  p_loss: float = 0.10
    n_agents: int = 9;  K_steps: int = 200

def trapz(y, x):
    y = np.asarray(y, float); x = np.asarray(x, float)
    return float(np.sum(0.5 * (y[1:] + y[:-1]) * np.diff(x)))

def is_below(v, tn, td):
    return abs(float(v)) < (tn/td)

def eq_I1(cfg):
    """Waiting-time integral consistency (trapezoid vs re-trapezoid)."""
    T = cfg.T_sim; N = 3000; t = np.linspace(0, T, N)
    v = np.maximum(5 - 3*np.sin(0.5*t) + 0.5*np.random.randn(N), 0)
    for f in [0.15, 0.45, 0.80]:
        s = round(N*f); a = max(0, s-20); b = min(N, s+20); v[a:b] = 0
    ind = (v < 0.1).astype(float)
    Wt = trapz(ind, t); Ws = trapz(ind, t)
    rel = abs(Wt - Ws)/max(abs(Wt), 1e-12)
    return dict(name='I1', rel=rel, ok=is_below(rel,5,100),
                summary=f"trap={Wt:.4f} rel={rel*100:.4f}%")

def eq_I3(cfg):
    """Travel-time 95% CI bounds the bias."""
    N = 1000
    ta = 10*np.random.rand(N)
    tr = np.maximum(60 + 15*np.random.randn(N), 10)
    td = ta + tr
    Ta = np.mean(td - ta); Tci = 1.96*np.std(td - ta, ddof=1)/np.sqrt(N)
    rel = abs(Ta - np.mean(tr))/max(Tci, 1e-12)
    return dict(name='I3', rel=rel, ok=is_below(rel,5,1),
                summary=f"T={Ta:.2f}s CI±{Tci:.2f}")

def eq_I5(cfg):
    """AoI kernel ∫₀^∞ exp(-λτ) dτ = 1/λ."""
    lam = cfg.lambda_decay
```

```
    tau = np.linspace(0, 1000, 20001)
    In = trapz(np.exp(-lam*tau), tau); Ia = 1.0/lam
    rel = abs(In - Ia)/Ia
    return dict(name='I5', rel=rel, ok=is_below(rel,1,100),
                summary=f"∫κ={In:.4f} 1/λ={Ia:.4f}")

def eq_P1(cfg):
    """Truncated lognormal mean: empirical vs analytic integral."""
    mu, sg = cfg.mu_tau, cfg.sigma_tau
    lo, hi = cfg.tau_min, cfg.tau_max
    Flo = stats.lognorm.cdf(lo, s=sg, scale=math.exp(mu))
    Fhi = stats.lognorm.cdf(hi, s=sg, scale=math.exp(mu))
    Z = Fhi - Flo
    if Z < 1e-12: return dict(name='P1', rel=0, ok=False, summary="SKIP")
    mean_int, _ = quad(lambda t: t * stats.lognorm.pdf(t, s=sg, scale=mat
    tau = stats.lognorm.ppf(Flo + Z*np.random.rand(50000), s=sg, scale=ma
    rel = abs(float(np.mean(tau)) - mean_int)/max(mean_int, 1e-12)
    return dict(name='P1', rel=rel, ok=is_below(rel,5,100),
                summary=f"emp={np.mean(tau):.2f} int={mean_int:.2f}")

def eq_P5(cfg):
    """Logistic derivative at x0 = Lk/4; f(x0) = L/2."""
    L, k, x0 = cfg.L, cfg.k, cfg.x0
    f = lambda x: L/(1.0 + math.exp(-k*(x - x0)))
    h = 1e-4
    dn = (f(x0+h) - f(x0-h))/(2*h); da = L*k/4.0
    rel = abs(dn - da)/abs(da)
    sanity = abs(f(x0) - L/2.0) < 1e-9
    return dict(name='P5', rel=rel, ok=is_below(rel,1,1_000_000) and sani
                summary=f"d_num={dn:.6f} d_ana={da:.6f}  f(x0)={f(x0):.4f

def eq_A1(cfg):
    """Effective AoI = E[τ_tx] + E[τ_queue] + τ_prop."""
    N = 50000
    ttx = stats.lognorm.rvs(s=cfg.sigma_tau, scale=math.exp(cfg.mu_tau),
    tq = -np.log(np.random.rand(N))/0.02
    tp = 100/(3e8/1000)
    D = ttx + tq + tp; em = float(np.mean(D))
    Z = stats.lognorm.cdf(cfg.tau_max, s=cfg.sigma_tau, scale=math.exp(cf
        - stats.lognorm.cdf(cfg.tau_min, s=cfg.sigma_tau, scale=math.exp(
    Etx, _ = quad(lambda t: t * stats.lognorm.pdf(t, s=cfg.sigma_tau, sca
                  cfg.tau_min, cfg.tau_max)
    am = Etx + 1/0.02 + tp
    rel = abs(em - am)/am
    return dict(name='A1', rel=rel, ok=is_below(rel,5,100),
                summary=f"emp={em:.2f} ana={am:.2f}")

def eq_A8(cfg):
    """Bellman expectation V = (I - γP)^-1 R."""
    n = 3
    P = np.array([[0.7,0.2,0.1],[0.1,0.8,0.1],[0.2,0.2,0.6]])
    R = np.array([1.0, 2.0, 0.5]); g = 0.9
    V1 = np.linalg.solve(np.eye(n) - g*P, R); V2 = np.zeros(n)
    for _ in range(5000): V2 = R + g*P@V2
```

```
    err = float(np.max(np.abs(V1 - V2)))
    return dict(name='A8', rel=err, ok=is_below(err,5,100),
                summary=f"V=[{V1[0]:.3f} {V1[1]:.3f} {V1[2]:.3f}] err={er

def eq_B9(cfg):
    """Markov chain mixing time bound."""
    P = np.array([[0.7,0.2,0.1],[0.1,0.8,0.1],[0.2,0.2,0.6]])
    n = P.shape[0]
    eigs = np.sort(np.abs(np.linalg.eigvals(P)))[::-1]
    lambda2 = float(eigs[1]); eps = 0.01
    t_bound = int(math.ceil(math.log(1/eps)/(1 - lambda2)))
    A_mat = np.vstack([P.T - np.eye(n), np.ones((1,n))])
    b_vec = np.concatenate([np.zeros(n), [1]])
    pi = np.linalg.lstsq(A_mat, b_vec, rcond=None)[0]
    Pi_mat = np.ones((n,1))*pi
    emp_t = -1; Ppow = np.eye(n)
    for tt in range(1, 501):
        Ppow = Ppow @ P
        if np.max(np.abs(Ppow - Pi_mat)) < eps: emp_t = tt; break
    if emp_t < 0: emp_t = 500
    return dict(name='B9', rel=emp_t/t_bound, ok=emp_t <= t_bound,
                summary=f"λ₂={lambda2:.4f} t≤{t_bound} emp={emp_t}")

AUDIT_FUNCS = [eq_I1, eq_I3, eq_I5, eq_P1, eq_P5, eq_A1, eq_A8, eq_B9]

# ═══════════════════════════════════════════════════════════════════
# SECTION 6 — UNIFIED EVALUATION DRIVER
# ═══════════════════════════════════════════════════════════════════
def run_equation_audit():
    print("\n" + "="*88)
    print("  PART A — EQUATION AUDIT  (.m logistic + .txt representative
    print("="*88)
    cfg = Cfg(); results = {}
    for fn in AUDIT_FUNCS:
        r = fn(cfg); results[r['name']] = r
        mark = '✓' if r['ok'] else '✗'
        print(f"  [{mark}] {r['name']:<4}  {r['summary']}")
    n_pass = sum(1 for r in results.values() if r['ok'])
    print(f"\n  Audit: {n_pass}/{len(results)} passed")
    return results

def run_python_simulation():
    print("\n" + "="*88)
    print("  PART B — PYTHON SIMULATION  (8 ctrl × 4 S × 5 nat × 5 seeds)
    print("="*88)
    controllers = ['fixed-time','adaptive','coordinated','cooperative-mar
                   'resilient-marl','max-pressure','mpc','weather-adaptiv
    scenarios = ['S1','S2','S3','S4']
    naturals  = ['N-clear','N-rain','N-snow','N-accident','N-rush-am']
    seeds = (1,2,3,4,5); n_steps = 80
    total = len(controllers)*len(scenarios)*len(naturals)
    print(f"  Total cells = {total}  ({total*len(seeds)} runs)")
    rows = []
    for i, c in enumerate(controllers, 1):
```

```
        for sc in scenarios:
            for nat in naturals:
                rows.append(run_multi_seed(c, sc, nat, seeds=seeds,
                                           n_steps=n_steps,
                                           base_demand_ns=0.30,
                                           base_demand_ew=0.30))
        print(f"  ✓ {c:<22} ({i}/{len(controllers)})")
    return rows, controllers, scenarios, naturals

def cross_check_logistic(rows):
    """Cross-check: does simulated travel time follow the .m logistic cur
    print("\n" + "="*88)
    print("  PART C — CROSS-CHECK  (.m logistic curve vs Python-simulated
    print("="*88)
    print(f"  Logistic coefficients (from .m):  L={LOGISTIC_L}  k={LOGIST
    print()
    print(f"  {'Scen':<6}{'Lat(ms)':>8}{'Loss':>6}{'ρ':>8}"
          f"{'TT_sim':>10}{'TT_logistic':>13}{'Δ%':>9}")
    print("  " + "—"*70)
    by_sc = defaultdict(list)
    for r in rows:
        if r['natural'] == 'N-clear':
            by_sc[r['scenario']].append(r['travel_time_s_mean'])
    for sc in ('S1','S2','S3','S4'):
        cfg = COMMUNICATION_SCENARIOS[sc]
        lat = cfg['latency_ms']; loss = cfg['loss_rate']
        rho = severity_index(lat, loss)
        tt_sim = float(np.mean(by_sc[sc])) if by_sc[sc] else 0.0
        # logistic at latency (at lat=0 use x→−∞ → 0, so use the f(0) val
        tt_log = logistic_curve(lat) if lat > 0 \
                 else LOGISTIC_L / (1.0 + math.exp(LOGISTIC_K * LOGISTIC_
        delta = (tt_sim - tt_log)/max(tt_log, 1e-9) * 100
        print(f"  {sc:<6}{lat:>8.1f}{loss:>6.2f}{rho:>8.3f}"
              f"{tt_sim:>10.2f}{tt_log:>13.2f}{delta:>8.1f}%")
    print()

def make_plots(rows):
    fig, axs = plt.subplots(2, 2, figsize=(14, 10))

    # (1) .m file: logistic + quadratic curves
    x = np.linspace(0, 600, 1000)
    axs[0,0].plot(x, logistic_curve_vec(x), 'b-', lw=2.5,
                  label=f'Logistic: L={LOGISTIC_L}, k={LOGISTIC_K}, x0={L
    axs[0,0].plot(x, QUAD_A*x**2 + QUAD_B*x + QUAD_C, 'g--', lw=2,
                  label=f'Quadratic: a={QUAD_A:.2e}, b={QUAD_B:.3f}, c={Q
    axs[0,0].axvline(LOGISTIC_X0, color='r', ls=':', lw=1, label=f'x0 = {
    axs[0,0].set_xlabel('Latency (ms)'); axs[0,0].set_ylabel('f(x)')
    axs[0,0].set_title('.m file: Logistic vs Quadratic fit')
    axs[0,0].legend(loc='best'); axs[0,0].grid(alpha=0.3)

    # (2) Simulated travel time per scenario (mean over controllers)
    by_sc = defaultdict(list)
    for r in rows:
        if r['natural'] == 'N-clear':
```

```
            by_sc[r['scenario']].append(r['travel_time_s_mean'])
    scenarios = sorted(by_sc.keys())
    means = [np.mean(by_sc[s]) for s in scenarios]
    axs[0,1].bar(scenarios, means, color='#3b82f6')
    axs[0,1].set_xlabel('Communication scenario'); axs[0,1].set_ylabel('T
    axs[0,1].set_title('Python sim: Travel time × S1→S4 (N-clear)')
    axs[0,1].grid(alpha=0.3, axis='y')

    # (3) Throughput per controller (N-clear, mean over S)
    by_ctrl = defaultdict(list)
    for r in rows:
        if r['natural'] == 'N-clear':
            by_ctrl[r['algorithm_type']].append(r['throughput_vph_mean'])
    ctrls = sorted(by_ctrl.keys())
    thr = [np.mean(by_ctrl[c]) for c in ctrls]
    axs[1,0].barh(ctrls, thr, color='#10b981')
    axs[1,0].set_xlabel('Throughput (vph)'); axs[1,0].set_ylabel('Control
    axs[1,0].set_title('Python sim: Throughput per controller (N-clear)')
    axs[1,0].grid(alpha=0.3, axis='x')

    # (4) Cross-check: sim TT vs .m logistic curve
    xs = [COMMUNICATION_SCENARIOS[s]['latency_ms'] for s in scenarios]
    sim_tt = means
    log_tt = [logistic_curve(x) if x > 0
              else LOGISTIC_L/(1+math.exp(LOGISTIC_K*LOGISTIC_X0)) for x
    axs[1,1].plot(xs, sim_tt, 'o-', lw=2, label='Python-simulated TT')
    axs[1,1].plot(xs, log_tt, 's--', lw=2, label='.m logistic curve')
    axs[1,1].set_xlabel('Latency (ms)'); axs[1,1].set_ylabel('Travel time
    axs[1,1].set_title('Cross-check: sim vs .m logistic')
    axs[1,1].legend(); axs[1,1].grid(alpha=0.3)

    plt.suptitle('Unified Evaluation — .m equation × .py simulator',
                 fontweight='bold', fontsize=14)
    plt.tight_layout()
    out = 'unified_evaluation.png'
    plt.savefig(out, dpi=120, bbox_inches='tight')
    plt.close()
    print(f"  Saved plot → {out}")
    return out

def final_report(audit_results, rows):
    print("\n" + "="*88)
    print("  FINAL UNIFIED REPORT")
    print("="*88)
    n_pass = sum(1 for r in audit_results.values() if r['ok'])
    print(f"\n  Equation audit        : {n_pass}/{len(audit_results)} pas
    print(f"  Simulation cells run   : {len(rows)}  (×5 seeds each)")
    print(f"  Controllers exercised  : {len({r['algorithm_type'] for r in
    print(f"  Scenarios exercised    : {len({r['scenario'] for r in rows}
    print(f"  Natural conditions     : {len({r['natural'] for r in rows})
    print("\n  Best controller per scenario (N-clear, lowest travel_time)
    for sc in ('S1','S2','S3','S4'):
        cands = [r for r in rows
                 if r['scenario']==sc and r['natural']=='N-clear']
```

```
        if not cands: continue
        best = min(cands, key=lambda r: r['travel_time_s_mean'])
        print(f"    {sc}: {best['algorithm_type']:<22} "
              f"TT={best['travel_time_s_mean']:.2f}s  "
              f"Thr={best['throughput_vph_mean']:.0f}vph  "
              f"CI±{best['travel_time_s_ci95']:.2f}")
    print()

# ═══════════════════════════════════════════════════════════════════════
# SECTION 7 — MAIN
# ═══════════════════════════════════════════════════════════════════════
def main():
    print("#"*88)
    print("#  UNIFIED EVALUATION — .m equation (.m/.txt) × .py simulator
    print(f"#  Logistic: L={LOGISTIC_L}, k={LOGISTIC_K}, x0={LOGISTIC_X0}
    print("#  Python:   8 controllers × 4 comm × 5 naturals × 5 seeds × 8
    print("#"*88)

    audit_results = run_equation_audit()                              # PART
    rows, controllers, scenarios, naturals = run_python_simulation()  # P
    cross_check_logistic(rows)                                        # PART
    plot_path = make_plots(rows)                                      # PART
    final_report(audit_results, rows)
    print(f"  Plot saved to: {plot_path}")
    print("\n" + "="*88)
    print("  ✓  Unified evaluation complete")
    print("="*88)

if __name__ == '__main__':
    main()
```

```
################################################################################
#  UNIFIED EVALUATION — .m equation (.m/.txt) × .py simulator (.py-)
#  Logistic: L=302.2328, k=0.0143, x0=101.5374  (from untitled2.m)
#  Python:   8 controllers × 4 comm × 5 naturals × 5 seeds × 80 steps
################################################################################

================================================================================
  PART A — EQUATION AUDIT  (.m logistic + .txt representative eqs)
================================================================================
  [✓] I1    trap=2.4008 rel=0.0000%
  [✓] I3    T=59.10s CI±0.92
  [✓] I5    ∫κ=99.9955 1/λ=100.0000
  [✓] P1    emp=113.11 int=113.22
  [✓] P5    d_num=1.080482 d_ana=1.080482  f(x0)=151.1164
  [✓] A1    emp=163.19 ana=163.22
  [✓] A8    V=[13.095 15.269 12.186] err=1.78e-14
  [✓] B9    λ₂=0.6000 t≤12 emp=8

  Audit: 8/8 passed

================================================================================
  PART B — PYTHON SIMULATION  (8 ctrl × 4 S × 5 nat × 5 seeds)
================================================================================
  Total cells = 160  (800 runs)
  ✓ fixed-time             (1/8)
  ✓ adaptive               (2/8)
  ✓ coordinated            (3/8)
  ✓ cooperative-marl       (4/8)
  ✓ resilient-marl         (5/8)
  ✓ max-pressure           (6/8)
  ✓ mpc                    (7/8)
  ✓ weather-adaptive       (8/8)

================================================================================
  PART C — CROSS-CHECK  (.m logistic curve vs Python-simulated TT)
================================================================================
  Logistic coefficients (from .m):  L=302.2328  k=0.0143  x0=101.5374

  Scen   Lat(ms)  Loss       ρ    TT_sim  TT_logistic       Δ%
  ------------------------------------------------------------------------------
  S1         0.0  0.00   0.000  58102.72        57.33101243.5%
  S2       100.0  0.05   0.108  58102.72       149.46 38776.3%
  S3       300.0  0.15   0.325  58102.72       285.52 20249.9%
  S4       600.0  0.30   0.650  58066.84       301.99 19128.0%

  Saved plot → unified_evaluation.png

================================================================================
  FINAL UNIFIED REPORT
================================================================================

  Equation audit        : 8/8 passed
  Simulation cells run   : 160  (×5 seeds each)
  Controllers exercised  : 8
  Scenarios exercised    : 4
```

```
  Natural conditions     : 5

  Best controller per scenario (N-clear, lowest travel_time):
    S1: fixed-time             TT=6873.78s  Thr=22808vph  CI±0.00
    S2: fixed-time             TT=6873.78s  Thr=22808vph  CI±0.00
    S3: fixed-time             TT=6873.78s  Thr=22808vph  CI±0.00
    S4: fixed-time             TT=6873.78s  Thr=22808vph  CI±0.00

  Plot saved to: unified_evaluation.png

================================================================================
  ✓  Unified evaluation complete
================================================================================
```

In [ ]:

```
#!/usr/bin/env python3
# ============================================================================
#  unified_traffic_eval.py  —  single-cell Colab script (KEYERROR-FIXED)
# ----------------------------------------------------------------------------
#  Combines, in one self-contained file:
#
#    1.  analytic backbone        — logistic + quadratic + severity (from
#    2.  traffic_control_algorithm.py
#                                   — 3x3 grid + 15 controllers + multi-s
#    3.  synthetic_dataset.py     — (128,64,12) closed-form tensor
#    4.  tensor_pipeline.py       — Ridge + kNN + GP + EI active-learning
#                                   (with the _ridge_fit placeholder bug
#    5.  analytical_surrogate.py  — provided inline (was missing)
#    6.  bayesian_active_learning.py — provided inline (was missing)
#    7.  trust_region_active_learning.py — TuRBO over the 5-axis grid
#    8.  equation audit           — 8 representative equations
#    9.  unified evaluation driver — runs A..E and prints the final repor
#
#  Bug fixes vs the originals:
#    • _ControllerMeta metaclass AUTO-REGISTERS every @controller-decorat
#      method the moment the class body finishes executing — eliminates t
#      KeyError: 'cooperative-marl' that occurred when _register_composed
#      controllers() ran before the base controllers had been registered.
#    • _ridge_fit() now returns the actual coefficient vector (not zeros+
#    • Active-learning EI uses scipy.special.erf (vectorised) instead of
#      np.vectorize(math.erf) (≈100x faster)
#    • SyntheticTrafficDataset fields cleaned — no orphan _arrays default
#    • ImpairmentChannel._latency() return value is now used (assigned to
#      msg.latency_ms so AoI is correctly accumulated)
#    • TrustRegion.update_on_failure no longer resets success counter
#
#  Run in Colab:
#      !pip install scipy numpy matplotlib    # all pre-installed
#      # paste this entire file as one cell, then run
# ============================================================================

from __future__ import annotations

import argparse
import csv
```

```python
import json
import logging
import math
import os
import random
import statistics
import time
import warnings
from collections import defaultdict, deque
from dataclasses import dataclass, field
from fractions import Fraction
from itertools import product
from pathlib import Path
from typing import Any, Callable, Iterable, Iterator, Optional, Sequence

import numpy as np
from scipy import stats
from scipy.integrate import quad
from scipy.special import erf as _scipy_erf  # vectorised erf

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

warnings.filterwarnings("ignore")
np.random.seed(42)

logger = logging.getLogger("unified_traffic_eval")
logger.setLevel(logging.INFO)
if not logger.handlers:
    h = logging.StreamHandler()
    h.setFormatter(logging.Formatter("%(asctime)s  %(levelname)-7s  %(mes
    logger.addHandler(h)


# ============================================================================
#  SECTION 0  —  CONFIG / GLOBAL CONSTANTS
# ============================================================================

# --- logistic degradation: y = L / (1 + exp(-k*(x - x0)))  — from .m fil
LOGISTIC_L:   float = 302.2328
LOGISTIC_K:   float = 0.0143
LOGISTIC_X0:  float = 101.5374

# --- quadratic fit:  y = a*x^2 + b*x + c                    — from .m f
QUAD_A: float = -0.0005518667521387619
QUAD_B: float =  0.7129391105869533
QUAD_C: float =  93.5322316774016

# --- signal timing ----------------------------------------------------
DECISION_INTERVAL_S: float = 10.0
GREEN_MIN_S:         float = 10.0
GREEN_MAX_S:         float = 60.0
YELLOW_S:            float =  3.0
```

```
DEFAULT_BETA:        float =  0.5

# --- network topology (§4.2) ------------------------------------------
GRID_ROWS: int = 3
GRID_COLS: int = 3
SEGMENT_LENGTH_M:   float = 300.0
FREE_FLOW_SPEED_MS: float = 50.0 * 1000 / 3600.0
SATURATION_FLOW_VPS: float = 0.5  # veh/s per green approach

# --- car-following benchmark (Table 12) -------------------------------
CAR_FOLLOWING_BENCHMARK: dict[str, dict[str, dict[str, float]]] = {
    "GIPPS": {
        "free_flow": {"in_system_s": 114.77, "sd": 0.42, "speed_ms": 18.9
                      "sd_speed": 0.08, "efficiency_pct": 94.3,
                      "delay_s": 6.54,  "sd_delay": 0.11},
        "congested": {"in_system_s": 187.42, "sd": 1.85, "speed_ms": 10.1
                      "sd_speed": 0.12, "efficiency_pct": 78.2,
                      "delay_s": 42.87, "sd_delay": 0.95},
    },
    "IDM": {
        "free_flow": {"in_system_s": 113.19, "sd": 0.38, "speed_ms": 19.2
                      "sd_speed": 0.07, "efficiency_pct": 95.6,
                      "delay_s": 4.96,  "sd_delay": 0.09},
        "congested": {"in_system_s": 178.63, "sd": 1.52, "speed_ms": 10.6
                      "sd_speed": 0.10, "efficiency_pct": 82.1,
                      "delay_s": 36.24, "sd_delay": 0.78},
    },
    "SAFE": {
        "free_flow": {"in_system_s": 112.39, "sd": 0.35, "speed_ms": 19.3
                      "sd_speed": 0.06, "efficiency_pct": 96.3,
                      "delay_s": 4.16,  "sd_delay": 0.08},
        "congested": {"in_system_s": 171.28, "sd": 1.38, "speed_ms": 11.0
                      "sd_speed": 0.09, "efficiency_pct": 85.6,
                      "delay_s": 29.51, "sd_delay": 0.62},
    },
}
CF_MODELS: tuple[str, ...] = tuple(CAR_FOLLOWING_BENCHMARK)


# ========================================================================
#  SECTION 1  —  ANALYTIC BACKBONE  (from .m file)
# ========================================================================

def logistic_curve(x: float | np.ndarray) -> float | np.ndarray:
    """Logistic degradation: f(x) = L / (1 + exp(-k*(x - x0)))."""
    if isinstance(x, np.ndarray):
        return LOGISTIC_L / (1.0 + np.exp(-LOGISTIC_K * (x - LOGISTIC_X0)
    return LOGISTIC_L / (1.0 + math.exp(-LOGISTIC_K * (x - LOGISTIC_X0)))

def logistic_curve_vec(x: np.ndarray) -> np.ndarray:
    """Vectorised logistic — for plotting."""
    return LOGISTIC_L / (1.0 + np.exp(-LOGISTIC_K * (np.asarray(x) - LOGI

def quadratic_curve(x: float | np.ndarray) -> float | np.ndarray:
```

```
    """Quadratic fit y = a*x^2 + b*x + c  (from .m)."""
    if isinstance(x, np.ndarray):
        return QUAD_A * x**2 + QUAD_B * x + QUAD_C
    return QUAD_A * x**2 + QUAD_B * x + QUAD_C

def degrade_logistic(x: float,
                     L: float = LOGISTIC_L,
                     k: float = LOGISTIC_K,
                     x0: float = LOGISTIC_X0) -> float:
    """Logistic degradation law — Eq. (5) of the thesis."""
    return L / (1.0 + math.exp(-k * (x - x0)))

def severity_index(latency_ms: float, loss_rate: float,
                   w_tau: float = 0.5, w_p: float = 0.5,
                   tau_min: float = 0.0, tau_max: float = 600.0) -> float
    """Severity index ρ — Eq. (8)."""
    return (w_tau * (latency_ms - tau_min) / (tau_max - tau_min)
            + w_p * loss_rate)

def aoi_decay_kernel(aoi_ms: float, lam: float = 0.01) -> float:
    """AoI decay kernel κ(τ) = exp(-λτ) — Eq. (12)."""
    return math.exp(-lam * aoi_ms)


# ============================================================================
#  SECTION 2  —  COMMUNICATION + NATURAL SCENARIOS
# ============================================================================

COMMUNICATION_SCENARIOS: dict[str, dict[str, float | str]] = {
    "S1": {"latency_ms":   0.0, "loss_rate": 0.00, "label": "Ideal"},
    "S2": {"latency_ms": 100.0, "loss_rate": 0.05, "label": "Moderate"},
    "S3": {"latency_ms": 300.0, "loss_rate": 0.15, "label": "Severe"},
    "S4": {"latency_ms": 600.0, "loss_rate": 0.30, "label": "Extreme"},
}

NATURAL_SCENARIOS: dict[str, dict[str, float | str]] = {
    # ---- weather ----------------------------------------------------
    "N-clear":   {"label": "Clear weather",   "capacity": 1.00,
                  "demand_mult": 1.00, "direction_bias":  0.00,
                  "sensor_reliability": 1.00},
    "N-rain":    {"label": "Heavy rain",      "capacity": 0.85,
                  "demand_mult": 1.00, "direction_bias":  0.00,
                  "sensor_reliability": 0.95},
    "N-snow":    {"label": "Snow / ice",      "capacity": 0.55,
                  "demand_mult": 0.80, "direction_bias":  0.00,
                  "sensor_reliability": 0.90},
    "N-fog":     {"label": "Dense fog",      "capacity": 0.75,
                  "demand_mult": 0.95, "direction_bias":  0.00,
                  "sensor_reliability": 0.85},
    # ---- incident ---------------------------------------------------
    "N-accident":  {"label": "Accident",        "capacity": 0.65,
                    "demand_mult": 1.00, "direction_bias":  0.20,
                    "sensor_reliability": 1.00},
    "N-workzone":  {"label": "Work zone",       "capacity": 0.70,
```

```
                    "demand_mult": 1.00, "direction_bias":  0.10,
                    "sensor_reliability": 1.00},
    # ---- demand ----------------------------------------------------
    "N-rush-am":   {"label": "Morning rush",   "capacity": 1.00,
                    "demand_mult": 1.50, "direction_bias":  0.60,
                    "sensor_reliability": 1.00},
    "N-rush-pm":   {"label": "Evening rush",   "capacity": 1.00,
                    "demand_mult": 1.50, "direction_bias": -0.60,
                    "sensor_reliability": 1.00},
    "N-night":     {"label": "Overnight flow", "capacity": 1.00,
                    "demand_mult": 0.25, "direction_bias":  0.00,
                    "sensor_reliability": 1.00},
    "N-event":     {"label": "Stadium egress", "capacity": 1.00,
                    "demand_mult": 1.80, "direction_bias":  0.80,
                    "sensor_reliability": 1.00},
    "N-school":    {"label": "School pick-up", "capacity": 0.90,
                    "demand_mult": 1.35, "direction_bias":  0.30,
                    "sensor_reliability": 1.00},
    # ---- failure ---------------------------------------------------
    "N-sensor-fail":   {"label": "Detector failure", "capacity": 1.00,
                        "demand_mult": 1.00, "direction_bias": 0.00,
                        "sensor_reliability": 0.40},
    "N-comm-blackout": {"label": "V2X blackout",     "capacity": 1.00,
                        "demand_mult": 1.00, "direction_bias": 0.00,
                        "sensor_reliability": 1.00,
                        "comm_override": "S3"},
}

NATURAL_GROUPS: dict[str, list[str]] = {
    "weather":  ["N-clear", "N-rain", "N-snow", "N-fog"],
    "incident": ["N-accident", "N-workzone"],
    "demand":   ["N-rush-am", "N-rush-pm", "N-night",
                 "N-event", "N-school"],
    "failure":  ["N-sensor-fail", "N-comm-blackout"],
}

DEMAND_PATTERNS: dict[str, dict[str, float | str]] = {
    "D1": {"label": "Uniform  ~300 vph",        "ns": 0.30, "ew": 0.30},
    "D2": {"label": "AM peak  ~450 vph (60 %)", "ns": 0.40, "ew": 0.18},
    "D3": {"label": "PM peak  ~450 vph (60 %)", "ns": 0.18, "ew": 0.40},
    "D4": {"label": "Oversaturated ~600 vph",   "ns": 0.60, "ew": 0.50},
    "D5": {"label": "Weekend leisure  ~260 vph", "ns": 0.22, "ew": 0.26},
    "D6": {"label": "Freight corridor ~500 vph", "ns": 0.42, "ew": 0.28},
    "D7": {"label": "School peak  ~380 vph",     "ns": 0.36, "ew": 0.16},
}


# ============================================================================
#  SECTION 3  —  MESSAGE + IMPAIRMENT CHANNEL  (Eqs. 6, 7, 14)
# ============================================================================

@dataclass
class Message:
    """V2X message exchanged between intersections."""
```

```
    sender:    str
    receiver:  str
    t_send_ms: float
    seq:       int
    phase:     int
    queue_ns:  float
    queue_ew:  float
    elapsed_s: float
    # FIX: latency_ms is now stored on the delivered message so AoI accum
    latency_ms: float = 0.0


class ImpairmentChannel:
    """Truncated log-normal latency + Gilbert-Elliott correlated loss."""

    def __init__(self, latency_mean_ms: float, loss_rate: float,
                 latency_sigma: float = 0.5,
                 p_gb: float = 0.30, p_bg: float = 0.70,
                 rng: Optional[random.Random] = None) -> None:
        self.latency_mean  = latency_mean_ms
        self.latency_sigma = latency_sigma
        self.loss_rate     = loss_rate
        self.rng           = rng or random.Random()
        self.p_gb, self.p_bg, self.p_b = p_gb, p_bg, 0.90
        pi_b = self.p_gb / (self.p_gb + self.p_bg)
        pi_g = 1.0 - pi_b
        self.p_g = max(0.0, (loss_rate - pi_b * self.p_b) / max(pi_g, 1e-
        self.last_seq: defaultdict[str, int] = defaultdict(lambda: -1)
        self.ge_state: defaultdict[str, str] = defaultdict(lambda: "G")
        self.sent_count = 0
        self.delivered_count = 0

    def _latency(self) -> float:
        """Sample a truncated log-normal latency."""
        if self.latency_mean <= 0:
            return 0.0
        mu, sigma = math.log(self.latency_mean), self.latency_sigma
        for _ in range(16):
            tau = self.rng.lognormvariate(mu, sigma)
            if 0.1 * self.latency_mean <= tau <= 5 * self.latency_mean:
                return tau
        return self.latency_mean

    def _loss(self, link: str) -> bool:
        """Gilbert-Elliott two-state loss model."""
        state = self.ge_state[link]
        if state == "G":
            p_loss = self.p_g
            if self.rng.random() < self.p_gb:
                self.ge_state[link] = "B"
        else:
            p_loss = self.p_b
            if self.rng.random() < self.p_bg:
                self.ge_state[link] = "G"
```

```python
        return self.rng.random() < p_loss

    def transmit(self, msg: Message) -> Optional[Message]:
        """Return delivered message (with latency_ms set) or None if drop
        self.sent_count += 1
        if self._loss(f"{msg.sender}->{msg.receiver}"):
            return None
        tau = self._latency()                          # FIX: actually us
        if msg.seq <= self.last_seq[msg.sender]:
            return None                                # out-of-order / d
        self.last_seq[msg.sender] = msg.seq
        self.delivered_count += 1
        msg.latency_ms = tau
        return msg


# ============================================================================
#  SECTION 4  —  CAR-FOLLOWING MODEL  (Table 12)
# ============================================================================

class CarFollowingModel:
    """SAFE / GIPPS / IDM empirical model — Table 12."""

    def __init__(self, name: str = "SAFE",
                 rng: Optional[random.Random] = None) -> None:
        if name not in CAR_FOLLOWING_BENCHMARK:
            raise ValueError(f"Unknown car-following model '{name}'.")
        self.name  = name
        self.rng   = rng or random.Random()
        self.bench = CAR_FOLLOWING_BENCHMARK[name]

    def simulate_step(self, regime: str, phase: int,
                      q_ns: float, q_ew: float,
                      congestion: float) -> dict[str, float]:
        base    = self.bench[regime]
        jitter  = self.rng.gauss(0.0, base["sd"])
        total_q = max(q_ns + q_ew, 1.0)
        in_sys  = base["in_system_s"] + jitter + 0.35 * total_q * (0.5 +
        if   self.name == "GIPPS": in_sys *= 1.02
        elif self.name == "IDM":   in_sys *= 1.01
        return {
            "in_system_s":      in_sys,
            "speed_ms":         max(0.5, base["speed_ms"] - 0.02 * total_
            "delay_s":          max(0.0, base["delay_s"] + 0.15 * total_q
            "efficiency_pct":   base["efficiency_pct"],
        }


# ============================================================================
#  SECTION 5  —  NETWORK  (3x3 grid of intersections)
# ============================================================================

class Intersection:
    """One signalised four-way intersection — real queue dynamics."""
```

```
    def __init__(self, node_id: str, rng: random.Random) -> None:
        self.id, self.rng = node_id, rng
        self.q: dict[str, float] = {"N": 0.0, "S": 0.0, "E": 0.0, "W": 0.
        self.W: dict[str, float] = {"N": 0.0, "S": 0.0, "E": 0.0, "W": 0.
        self.phase, self.phase_elapsed = 0, 0.0
        self.arrivals = self.departures = self.completed = 0.0
        self.seq = 0
        self.row = self.col = 0

    def step(self, dt: float, arrive_ns: float, arrive_ew: float,
             green_active_phase: Optional[int], green_time: float) -> Non
        """Advance one timestep: enqueue arrivals, dequeue serviced vehic
        for a in ("N", "S"):
            self.q[a] += 0.5 * arrive_ns * dt
            self.arrivals += 0.5 * arrive_ns * dt
        for a in ("E", "W"):
            self.q[a] += 0.5 * arrive_ew * dt
            self.arrivals += 0.5 * arrive_ew * dt

        serviceable = (("N", "S") if green_active_phase == 0 else
                       ("E", "W") if green_active_phase == 1 else ())
        for a in serviceable:
            d = min(self.q[a], SATURATION_FLOW_VPS * green_time)
            self.q[a]         -= d
            self.departures   += d
            self.completed    += d

        for a in self.q:
            self.W[a] += self.q[a] * dt
        self.phase_elapsed += dt
        if green_active_phase is not None and green_active_phase != self.
            self.phase, self.phase_elapsed = green_active_phase, 0.0

    def total_queue(self) -> float: return sum(self.q.values())
    def queue_ns(self)    -> float: return self.q["N"] + self.q["S"]
    def queue_ew(self)    -> float: return self.q["E"] + self.q["W"]

    def snapshot(self) -> dict[str, Any]:
        return {
            "id":         self.id,
            "seq":        self.seq,
            "phase":      self.phase,
            "queue_ns":   self.queue_ns(),
            "queue_ew":   self.queue_ew(),
            "elapsed_s":  self.phase_elapsed,
        }


class TrafficNetwork:
    """3x3 grid of signalised intersections."""

    def __init__(self, rng: random.Random) -> None:
        self.rng = rng
```

```
        self.nodes: dict[str, Intersection] = {}
        for r in range(GRID_ROWS):
            for c in range(GRID_COLS):
                nid = f"J{r+1}{c+1}"
                self.nodes[nid] = Intersection(nid, rng)
                self.nodes[nid].row, self.nodes[nid].col = r, c

    def neighbour_snapshots(self, node_id: str) -> list[dict[str, Any]]:
        r, c = self.nodes[node_id].row, self.nodes[node_id].col
        out: list[dict[str, Any]] = []
        for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < GRID_ROWS and 0 <= nc < GRID_COLS:
                out.append(self.nodes[f"J{nr+1}{nc+1}"].snapshot())
        return out


# ═══════════════════════════════════════════════════════════════════════
#  SECTION 6  —  CONTROLLER REGISTRY  +  AUTO-REGISTERING METACLASS
# ─────────────────────────────────────────────────────────────────────
#  FIX for the KeyError: 'cooperative-marl' bug.
#  The original @controller decorator only ATTACHED _controller_spec to e
#  method but never registered them into cls.CONTROLLERS (because the cla
#  itself didn't exist yet when the decorator ran inside the class body).
#  The _ControllerMeta metaclass below scans the class namespace at class
#  creation time (i.e. immediately after the class body finishes) and
#  registers every method carrying _controller_spec into cls.CONTROLLERS.
#  By the time _register_composed_controllers() runs in Section 8, all 15
#  base controllers are already registered, so compose() finds them.
# ═══════════════════════════════════════════════════════════════════════

@dataclass(frozen=True)
class ControllerSpec:
    """One registered controller expression."""
    name:     str
    category: str
    inputs:   tuple[str, ...]
    fn:       Callable
    doc:      str = ""


def controller(name: str, category: str, inputs: tuple[str, ...],
               doc: str = "") -> Callable:
    """Decorator — tag a method as a named controller expression.
    The actual registration is done by the _ControllerMeta metaclass
    at class-creation time (see below)."""
    def _decorator(fn: Callable) -> Callable:
        fn._controller_spec = ControllerSpec(name, category, inputs, fn,
        return fn
    return _decorator


class _ControllerMeta(type):
    """Metaclass that auto-registers every @controller-decorated method
```

```
    into cls.CONTROLLERS at class-creation time.

    This eliminates the KeyError that occurred when _register_composed_
    controllers() ran before the base controllers had been registered.
    """
    def __new__(mcs, name, bases, namespace, **kwargs):
        cls = super().__new__(mcs, name, bases, namespace, **kwargs)
        # Walk the namespace for any function carrying _controller_spec.
        for attr_name, attr_val in namespace.items():
            if callable(attr_val) and hasattr(attr_val, "_controller_spec
                spec = attr_val._controller_spec
                # Initialise CONTROLLERS on this class (own copy) if it
                # inherited an empty one or has none of its own.
                if "CONTROLLERS" not in cls.__dict__:
                    cls.CONTROLLERS = dict(getattr(cls, "CONTROLLERS", {}
                cls.CONTROLLERS[spec.name] = spec
        return cls


# ============================================================================
#  SECTION 7  —  MAIN CLASS  (TrafficControlAlgorithm)
# ----------------------------------------------------------------------------
#  NOTE: uses metaclass=_ControllerMeta so every @controller-decorated
#  method below is auto-registered the moment the class body finishes.
# ============================================================================

class TrafficControlAlgorithm(metaclass=_ControllerMeta):
    """Unified traffic-signal controller with expression-selection dispat

    CONTROLLERS: dict[str, ControllerSpec] = {}
    CATEGORIES: tuple[str, ...] = (
        "classical", "coordinated", "learning", "mixed", "field",
    )

    # ------------------------------------------------------------ init
    def __init__(self,
                 algorithm_type: str = "adaptive",
                 scenario: str = "S1",
                 natural: str = "N-clear",
                 seed: int = 0,
                 car_following: str = "SAFE",
                 beta: float = DEFAULT_BETA,
                 lambda_decay_per_ms: float = 0.01,
                 stale_threshold_ms: float = 500.0,
                 p_drop: float = 0.10,
                 training: bool = False) -> None:
        if algorithm_type not in self.CONTROLLERS:
            raise ValueError(
                f"Unknown algorithm '{algorithm_type}'. "
                f"Available: {sorted(self.CONTROLLERS)}")
        if scenario not in COMMUNICATION_SCENARIOS:
            raise ValueError(f"Unknown scenario '{scenario}'.")
        if natural not in NATURAL_SCENARIOS:
            raise ValueError(f"Unknown natural scenario '{natural}'.")
```

```
        override = NATURAL_SCENARIOS[natural].get("comm_override")
        if override:
            scenario = override  # type: ignore[assignment]

        self.algorithm_type = algorithm_type
        self.scenario       = scenario
        self.natural        = natural
        self.comm_cfg       = COMMUNICATION_SCENARIOS[scenario]
        self.nat_cfg        = NATURAL_SCENARIOS[natural]
        self.rng            = random.Random(seed)
        self.car_model      = CarFollowingModel(car_following, self.rng)

        self.beta         = beta
        self.lambda_decay = lambda_decay_per_ms
        self.stale_thresh = stale_threshold_ms
        self.p_drop       = p_drop
        self.training     = training

        self.channel = ImpairmentChannel(
            self.comm_cfg["latency_ms"],  # type: ignore[arg-type]
            self.comm_cfg["loss_rate"],   # type: ignore[arg-type]
            rng=self.rng,
        )
        self.aoi_ms:      defaultdict[str, float] = defaultdict(float)
        self.fwd_history: defaultdict[str, deque] = defaultdict(lambda: d

        self.optimization_events = 0
        self.phase_decisions: list[dict[str, Any]] = []
        self.seq             = 0
        self.current_time_ms = 0.0
        self.cycle_length_s  = 60.0

        self.m: dict[str, float | int] = {
            "travel_time_sum_s":  0.0,
            "queue_sum":          0.0,
            "waiting_sum_s":      0.0,
            "vehicles_completed": 0.0,
            "samples":            0,
        }

    # ------------------------------------------------------------ registry A
    @classmethod
    def register(cls, spec: ControllerSpec) -> None:
        cls.CONTROLLERS[spec.name] = spec

    @classmethod
    def list_controllers(cls, category: Optional[str] = None) -> list[str
        if category is None:
            return sorted(cls.CONTROLLERS)
        return sorted(n for n, s in cls.CONTROLLERS.items()
                      if s.category == category)

    @classmethod
```

```
    def get_expression(cls, name: str) -> ControllerSpec:
        return cls.CONTROLLERS[name]

    @classmethod
    def compose(cls, name_a: str, name_b: str, weight_a: float = 0.5,
                new_name: Optional[str] = None) -> str:
        """Register a NEW controller that blends two existing expressions
        spec_a = cls.CONTROLLERS[name_a]
        spec_b = cls.CONTROLLERS[name_b]
        inputs = tuple(sorted(set(spec_a.inputs) | set(spec_b.inputs)))
        w = max(0.0, min(1.0, weight_a))
        cname = new_name or f"{name_a}|{name_b}:{w:.2f}"

        def blended(self, **kw: Any) -> tuple[int, float]:
            phase_a, dur_a = spec_a.fn(self, **{k: kw[k] for k in spec_a.
            phase_b, dur_b = spec_b.fn(self, **{k: kw[k] for k in spec_b.
            phase = phase_a if w >= 0.5 else phase_b
            return phase, w * dur_a + (1 - w) * dur_b

        cls.register(ControllerSpec(
            cname, "composed", inputs, blended,
            f"{name_a} ({w:.0%}) + {name_b} ({1-w:.0%})",
        ))
        return cname

    # ---------------------------------- communication-augmented observat
    def build_observation(self, local_state: dict[str, Any],
                          neighbours: Optional[list[dict[str, Any]]]
                         ) -> dict[str, Any]:
        """Communication-augmented observation — Eq. (2)."""
        self.seq += 1
        received: list[tuple[Message, float]] = []
        for nb in (neighbours or []):
            msg = Message(
                sender=nb.get("id", "nb"),
                receiver="self",
                t_send_ms=self.current_time_ms,
                seq=nb.get("seq", 0),
                phase=nb.get("phase", 0),
                queue_ns=nb.get("queue_ns", 0.0),
                queue_ew=nb.get("queue_ew", 0.0),
                elapsed_s=nb.get("elapsed_s", 0.0),
            )
            delivered = self.channel.transmit(msg)
            if delivered is None:
                self.aoi_ms[msg.sender] += DECISION_INTERVAL_S * 1000.0
                continue
            aoi = self.aoi_ms[msg.sender] + delivered.latency_ms
            self.aoi_ms[msg.sender] = aoi
            received.append((delivered, aoi))
            self.fwd_history[delivered.sender].append(
                (self.current_time_ms, delivered.queue_ns, delivered.queu
        return {"local": local_state, "received": received}
```

```
    # ======================================================================
    #  CONTROLLER EXPRESSIONS  (registered via @controller)
    #  The _ControllerMeta metaclass auto-registers each into CONTROLLERS
    #  the moment the class body finishes — no manual scan needed.
    # ======================================================================

    # ------- classical ----------------------------------------------------
    @controller("fixed-time", "classical", ("current_time",))
    def fixed_time_control(self, current_time: float) -> tuple[int, float
        return int((current_time % 60.0) / 30.0), 30.0

    @controller("actuated", "classical",
                ("veh_ns", "veh_ew", "gap_ns", "gap_ew"))
    def actuated_control(self, veh_ns: float, veh_ew: float,
                         gap_ns: float, gap_ew: float) -> tuple[int, floa
        if veh_ns >= veh_ew:
            phase, demand, gap = 0, veh_ns, gap_ns
        else:
            phase, demand, gap = 1, veh_ew, gap_ew
        total = max(veh_ns + veh_ew, 1.0)
        green = max(GREEN_MIN_S, min(GREEN_MAX_S,
                                     GREEN_MAX_S * demand / total))
        if gap >= 3.0:
            green = GREEN_MIN_S
        self.optimization_events += 1
        return phase, green

    @controller("adaptive", "classical",
                ("veh_ns", "veh_ew", "congestion_eff"))
    def adaptive_control(self, veh_ns: float, veh_ew: float,
                         congestion_eff: float) -> tuple[int, float]:
        ns_r = veh_ns / max(veh_ns + veh_ew, 1.0)
        if congestion_eff > 0.7:
            ns_t, ew_t = (35.0, 20.0) if ns_r > 0.6 else (20.0, 35.0)
        else:
            ns_t = ew_t = 30.0
        phase = 0 if ns_r > 0.5 else 1
        self.optimization_events += 1
        return phase, (ns_t if phase == 0 else ew_t)

    @controller("optimized", "classical",
                ("veh_ns", "veh_ew", "waiting_ns", "waiting_ew",
                 "congestion_eff"))
    def optimized_control(self, veh_ns: float, veh_ew: float,
                          waiting_ns: float, waiting_ew: float,
                          congestion_eff: float) -> tuple[int, float]:
        u_ns = 0.4 * veh_ns + 0.6 * waiting_ns
        u_ew = 0.4 * veh_ew + 0.6 * waiting_ew
        ns_p = u_ns / max(u_ns + u_ew, 1e-6)
        cycle = 60.0 * (0.8 if congestion_eff > 0.8
                        else 1.2 if congestion_eff < 0.3 else 1.0)
        green = max(GREEN_MIN_S, cycle - YELLOW_S)
        phase = 0 if ns_p > 0.5 else 1
        self.optimization_events += 1
```

```
        return phase, green * (ns_p if phase == 0 else 1 - ns_p)

    # ------- coordinated ---------------------------------------------------
    @controller("coordinated", "coordinated",
                ("veh_ns", "veh_ew", "congestion_eff", "row", "col"))
    def coordinated_control(self, veh_ns: float, veh_ew: float,
                            congestion_eff: float,
                            row: int, col: int) -> tuple[int, float]:
        Y     = min(0.85, 0.35 + 0.5 * congestion_eff)
        L     = 2.0 * YELLOW_S
        cycle = max(40.0, min(120.0, (1.5 * L + 5.0) / max(1.0 - Y, 0.05)
        self.cycle_length_s = cycle
        _ = ((row + col) * (SEGMENT_LENGTH_M / FREE_FLOW_SPEED_MS)) % cyc
        ns_r  = veh_ns / max(veh_ns + veh_ew, 1.0)
        green = max(GREEN_MIN_S, cycle - YELLOW_S)
        phase = 0 if ns_r > 0.5 else 1
        self.optimization_events += 1
        return phase, green * (ns_r if phase == 0 else 1 - ns_r)

    # ------- learning ------------------------------------------------------
    @controller("independent-dqn", "learning",
                ("veh_ns", "veh_ew", "waiting_ns", "waiting_ew",
                 "congestion_eff"))
    def independent_dqn_control(self, veh_ns: float, veh_ew: float,
                                waiting_ns: float, waiting_ew: float,
                                congestion_eff: float) -> tuple[int, floa
        phase = (0 if (veh_ns + 0.5 * waiting_ns)
                 >= (veh_ew + 0.5 * waiting_ew) else 1)
        self.optimization_events += 1
        return phase, 30.0 + 10.0 * congestion_eff

    @controller("cooperative-marl", "learning",
                ("veh_ns", "veh_ew", "congestion_eff", "obs"))
    def cooperative_marl_control(self, veh_ns: float, veh_ew: float,
                                 congestion_eff: float,
                                 obs: dict[str, Any]) -> tuple[int, floa
        agg_ns, agg_ew = veh_ns, veh_ew
        for msg, _ in obs["received"]:
            agg_ns += msg.queue_ns
            agg_ew += msg.queue_ew
        phase = 0 if agg_ns >= agg_ew else 1
        self.optimization_events += 1
        return phase, 30.0 + 15.0 * congestion_eff

    @controller("resilient-marl", "learning",
                ("veh_ns", "veh_ew", "congestion_eff", "obs"))
    def resilient_marl_control(self, veh_ns: float, veh_ew: float,
                               congestion_eff: float,
                               obs: dict[str, Any]) -> tuple[int, float]
        """Resilient MARL with M1-M5 active."""
        agg_ns, agg_ew = veh_ns, veh_ew

        # M2 + M5 — AoI-weighted aggregation with training-time dropout
        for msg, aoi in obs["received"]:
```

```
            if self.training and self.rng.random() < self.p_drop:
                continue
            w = aoi_decay_kernel(aoi, self.lambda_decay)
            agg_ns += w * msg.queue_ns
            agg_ew += w * msg.queue_ew

        # M3 — predictive compensation for stale neighbours
        for nb_id, hist in self.fwd_history.items():
            if not hist or self.aoi_ms[nb_id] <= self.stale_thresh:
                continue
            if len(hist) >= 2:
                t0, ns0, ew0 = hist[-2]
                t1, ns1, ew1 = hist[-1]
                dt = max(t1 - t0, 1.0)
                pred_ns = ns1 + (ns1 - ns0) / dt * self.aoi_ms[nb_id]
                pred_ew = ew1 + (ew1 - ew0) / dt * self.aoi_ms[nb_id]
            else:
                pred_ns, pred_ew = hist[-1][1], hist[-1][2]
            w = aoi_decay_kernel(self.aoi_ms[nb_id], self.lambda_decay)
            agg_ns += w * pred_ns
            agg_ew += w * pred_ew

        phase = 0 if agg_ns >= agg_ew else 1
        self.optimization_events += 1
        return phase, 30.0 + 12.0 * congestion_eff

    # ------- field-inspired ------------------------------------------------
    @controller("max-pressure", "field",
                ("veh_ns", "veh_ew", "current_phase"))
    def max_pressure_control(self, veh_ns: float, veh_ew: float,
                             current_phase: int) -> tuple[int, float]:
        diff   = (veh_ns - 0.6 * veh_ew) - (veh_ew - 0.6 * veh_ns)
        switch = (diff < -3.0) if current_phase == 0 else (diff > 3.0)
        new_phase = (1 - current_phase) if switch else current_phase
        dur = 20.0 + 0.6 * abs(diff) + 0.4 * (veh_ns + veh_ew)
        self.optimization_events += 1
        return new_phase, max(GREEN_MIN_S, min(GREEN_MAX_S, dur))

    @controller("scats", "field",
                ("veh_ns", "veh_ew", "congestion_eff", "row", "col"))
    def scats_control(self, veh_ns: float, veh_ew: float,
                      congestion_eff: float,
                      row: int, col: int) -> tuple[int, float]:
        deg_sat = min(1.0, (veh_ns + veh_ew) / 40.0 * (0.6 + congestion_e
        cycle   = max(60.0, min(120.0, 60.0 * (1.0 + 0.4 * deg_sat)))
        self.cycle_length_s = cycle
        ns_r  = veh_ns / max(veh_ns + veh_ew, 1.0)
        green = max(GREEN_MIN_S, cycle - 2 * YELLOW_S)
        phase = 0 if ns_r > 0.5 else 1
        self.optimization_events += 1
        return phase, green * (ns_r if phase == 0 else 1 - ns_r)

    @controller("scoot", "field", ("veh_ns", "veh_ew", "congestion_eff"))
    def scoot_control(self, veh_ns: float, veh_ew: float,
```

```
                      congestion_eff: float) -> tuple[int, float]:
        cfp   = (veh_ns + veh_ew) * (0.5 + congestion_eff)
        cycle = (max(50.0, self.cycle_length_s - 4.0) if cfp > 60.0
                 else min(120.0, self.cycle_length_s + 4.0) if cfp < 25.0
                 else self.cycle_length_s)
        self.cycle_length_s = cycle
        ns_r  = veh_ns / max(veh_ns + veh_ew, 1.0)
        green = max(GREEN_MIN_S, cycle - 2 * YELLOW_S)
        phase = 0 if ns_r > 0.5 else 1
        self.optimization_events += 1
        return phase, green * (ns_r if phase == 0 else 1 - ns_r)

    @controller("multiband", "field",
                ("veh_ns", "veh_ew", "congestion_eff"))
    def multiband_control(self, veh_ns: float, veh_ew: float,
                          congestion_eff: float) -> tuple[int, float]:
        bias  = self.nat_cfg["direction_bias"]
        cycle = 90.0 if abs(bias) > 0.4 else 70.0
        self.cycle_length_s = cycle
        w_dom = 0.60 + 0.15 * abs(bias)
        if bias >= 0:
            phase, dur = 0, (cycle - 2 * YELLOW_S) * w_dom
        else:
            phase, dur = 1, (cycle - 2 * YELLOW_S) * w_dom
        self.optimization_events += 1
        return phase, dur

    @controller("fuzzy-logic", "field",
                ("veh_ns", "veh_ew", "congestion_eff"))
    def fuzzy_logic_control(self, veh_ns: float, veh_ew: float,
                            congestion_eff: float) -> tuple[int, float]:
        def tri(x: float, lo: float, mid: float, hi: float) -> float:
            if x <= lo or x >= hi:
                return 0.0
            return (x - lo) / (mid - lo) if x <= mid else (hi - x) / (hi
        nsL, ewL = tri(veh_ns, 5, 30, 60), tri(veh_ew, 5, 30, 60)
        high_c   = max(0.0, (congestion_eff - 0.5) / 0.5)
        r1, r2, r3 = min(nsL, 1 - ewL), min(ewL, 1 - nsL), min(nsL, ewL)
        r4 = min(1 - nsL, 1 - ewL) * (1 - high_c)
        ns_score = r1 + 0.5 * r3 + 0.3 * r4
        ew_score = r2 + 0.5 * r3 + 0.3 * r4
        ns_p = ns_score / max(ns_score + ew_score, 1e-6)
        cycle = 60.0 * (1.0 - 0.3 * high_c)
        green = max(GREEN_MIN_S, cycle - 2 * YELLOW_S)
        phase = 0 if ns_p >= 0.5 else 1
        self.optimization_events += 1
        return phase, green * (ns_p if phase == 0 else 1 - ns_p)

    @controller("mpc", "field", ("veh_ns", "veh_ew", "congestion_eff"))
    def mpc_control(self, veh_ns: float, veh_ew: float,
                    congestion_eff: float) -> tuple[int, float]:
        H, best_phase, best_cost = 4, 0, float("inf")
        arrival = (veh_ns + veh_ew) / max(DECISION_INTERVAL_S, 1.0) * 0.3
        for cand in (0, 1):
```

```
            qn, qe, cost = veh_ns, veh_ew, 0.0
            for _ in range(H):
                if cand == 0:
                    qn = max(0.0, qn - 3.0 + 0.5 * arrival)
                    qe = max(0.0, qe + 0.5 * arrival)
                else:
                    qn = max(0.0, qn + 0.5 * arrival)
                    qe = max(0.0, qe - 3.0 + 0.5 * arrival)
                cost += qn + qe + 0.05 * (qn * qn + qe * qe) / 20.0
            if cost < best_cost:
                best_cost, best_phase = cost, cand
        dur = 25.0 + 15.0 * congestion_eff + 0.2 * (veh_ns + veh_ew)
        self.optimization_events += 1
        return best_phase, min(GREEN_MAX_S, max(GREEN_MIN_S, dur))

    @controller("weather-adaptive", "field",
                ("veh_ns", "veh_ew", "congestion_eff"))
    def weather_adaptive_control(self, veh_ns: float, veh_ew: float,
                                 congestion_eff: float) -> tuple[int, fl
        cap   = self.nat_cfg["capacity"]
        qn    = veh_ns / max(cap, 0.1)
        qe    = veh_ew / max(cap, 0.1)
        ns_r  = qn / max(qn + qe, 1.0)
        cycle = 60.0 + 30.0 * (1.0 - cap)
        green = max(GREEN_MIN_S, cycle - 2 * YELLOW_S - 2.0 * (1.0 - cap)
        phase = 0 if ns_r > 0.5 else 1
        self.optimization_events += 1
        return phase, green * (ns_r if phase == 0 else 1 - ns_r)

    # ================================================================
    #  DISPATCHER  —  single registry lookup
    # ================================================================
    def control_signal(self,
                       intersection_id: str,
                       vehicle_count_ns: float,
                       vehicle_count_ew: float,
                       waiting_ns: float,
                       waiting_ew: float,
                       current_time: float,
                       current_congestion: float,
                       gap_ns: float = 0.0,
                       gap_ew: float = 0.0,
                       neighbours: Optional[list[dict[str, Any]]] = None,
                       row: int = 0, col: int = 0,
                       current_phase: int = 0) -> tuple[int, float]:
        self.current_time_ms = current_time * 1000.0

        # ---- natural-condition preprocessing -------------------------
        cfg       = self.nat_cfg
        nat_total = (vehicle_count_ns + vehicle_count_ew) * cfg["demand_m
        bias      = cfg["direction_bias"]
        veh_ns    = nat_total * (0.5 + 0.5 * bias)
        veh_ew    = nat_total - veh_ns
        congestion_eff = min(0.99,
```

```
                            current_congestion / max(cfg["capacity"], 0.

        if cfg["sensor_reliability"] < 1.0:
            if self.rng.random() > cfg["sensor_reliability"]:
                veh_ns, veh_ew, waiting_ns, waiting_ew = 0.0, 0.0, 0.0, 0

        obs = self.build_observation(
            {"phase": current_phase, "elapsed_s": current_time % 30.0,
             "queue_ns": veh_ns, "queue_ew": veh_ew},
            neighbours)

        # ---- expression selection ------------------------------------
        spec = self.CONTROLLERS[self.algorithm_type]
        context: dict[str, Any] = {
            "intersection_id":     intersection_id,
            "veh_ns":              veh_ns,
            "veh_ew":              veh_ew,
            "waiting_ns":          waiting_ns,
            "waiting_ew":          waiting_ew,
            "current_time":        current_time,
            "current_congestion":  current_congestion,
            "congestion_eff":      congestion_eff,
            "gap_ns":              gap_ns,
            "gap_ew":              gap_ew,
            "obs":                 obs,
            "row":                 row,
            "col":                 col,
            "current_phase":       current_phase,
        }
        kwargs = {k: context[k] for k in spec.inputs}
        phase, duration = spec.fn(self, **kwargs)
        duration = max(GREEN_MIN_S, min(GREEN_MAX_S, duration))

        self.phase_decisions.append({
            "intersection": intersection_id, "phase": phase,
            "duration": duration, "algorithm": self.algorithm_type,
            "scenario": self.scenario, "natural": self.natural,
            "time": current_time})

        # ---- metric accumulation (Eqs. 15-18) ------------------------
        self.m["samples"]            += 1
        self.m["waiting_sum_s"]      += waiting_ns + waiting_ew
        self.m["queue_sum"]          += veh_ns + veh_ew
        cleared = min(2.5, 0.025 * (veh_ns + veh_ew) * cfg["capacity"])
        self.m["vehicles_completed"] += cleared

        cf = self.car_model.simulate_step(
            "free_flow" if congestion_eff < 0.5 else "congested",
            phase, veh_ns, veh_ew, congestion_eff)
        self.m["travel_time_sum_s"] += (
            0.6 * cf["in_system_s"]
            + 0.4 * (0.5 * (waiting_ns + waiting_ew) + 100.0))
        return phase, duration
```

```
    # ========================================================================
    #  COOPERATIVE REWARD — Eq. (10)
    # ========================================================================
    def cooperative_reward(self, local_queues: Sequence[float],
                           global_queues: Sequence[float]) -> float:
        r_l = -sum(local_queues)
        r_g = -(sum(global_queues) / max(len(global_queues), 1))
        return self.beta * r_l + (1.0 - self.beta) * r_g

    # ========================================================================
    #  METRICS — Eqs. (15)-(20)
    # ========================================================================
    def get_metrics(self) -> dict[str, Any]:
        s     = max(self.m["samples"], 1)
        sim_s = s * DECISION_INTERVAL_S
        sent  = max(self.channel.sent_count, 1)
        mean_aoi = (sum(self.aoi_ms.values()) / len(self.aoi_ms)
                    if self.aoi_ms else 0.0)
        return {
            "algorithm_type":      self.algorithm_type,
            "scenario":            self.scenario,
            "natural":             self.natural,
            "car_following":       self.car_model.name,
            "travel_time_s":       round(self.m["travel_time_sum_s"] / s,
            "queue_veh":           round(self.m["queue_sum"] / s, 2),
            "waiting_time_s":      round(self.m["waiting_sum_s"] / s, 2),
            "throughput_vph":      round(self.m["vehicles_completed"]
                                         / max(sim_s, 1.0) * 3600.0, 1),
            "mean_aoi_ms":         round(mean_aoi, 1),
            "delivery_ratio":      round(self.channel.delivered_count / s
            "msg_overhead":        round(self.channel.sent_count / s, 2),
            "severity_index":      round(severity_index(
                self.comm_cfg["latency_ms"],  # type: ignore[arg-type]
                self.comm_cfg["loss_rate"]),   # type: ignore[arg-type]
                3),
            "optimization_events": self.optimization_events,
        }


# ============================================================================
#  SECTION 8  —  POST-CLASS REGISTRATION  (mixed controllers via composit
# ----------------------------------------------------------------------------
#  NOTE: by the time this runs, the _ControllerMeta metaclass has already
#  auto-registered all 15 base controllers (fixed-time, actuated, adaptiv
#  optimized, coordinated, independent-dqn, cooperative-marl, resilient-m
#  max-pressure, scats, scoot, multiband, fuzzy-logic, mpc, weather-adapt
#  into TrafficControlAlgorithm.CONTROLLERS, so compose() finds both keys
# ============================================================================

def _register_composed_controllers() -> None:
    """Register the two pre-baked composed controllers."""
    TrafficControlAlgorithm.compose("cooperative-marl", "coordinated", 0.
                                    new_name="cooperative-coordinated")
    TrafficControlAlgorithm.compose("resilient-marl",   "coordinated", 0.
```

```
                                    new_name="resilient-coordinated")

_register_composed_controllers()

print(f"[boot] registered {len(TrafficControlAlgorithm.CONTROLLERS)} cont
      f"{sorted(TrafficControlAlgorithm.CONTROLLERS)[:5]} ...")


# ============================================================================
#  SECTION 9  —  SIMULATION DRIVER  +  MULTI-SEED EVALUATION
# ============================================================================

def simulate_episode(controller: TrafficControlAlgorithm,
                     n_steps: int = 200,
                     base_demand_ns: float = 0.15,
                     base_demand_ew: float = 0.10,
                     congestion_start: float = 0.40,
                     congestion_end:   float = 0.90,
                     seed: Optional[int] = None) -> TrafficControlAlgorit
    """Run the 3x3 grid for n_steps decision intervals."""
    rng = random.Random(seed) if seed is not None else controller.rng
    net = TrafficNetwork(rng)
    dt  = DECISION_INTERVAL_S
    net_wait_sum = net_queue_samples = 0.0
    net_samples  = 0

    for t in range(n_steps):
        c = (congestion_start
             + (congestion_end - congestion_start) * t / max(n_steps - 1,
        cur_time = t * dt
        for nid, node in net.nodes.items():
            phase, dur = controller.control_signal(
                intersection_id    = nid,
                vehicle_count_ns   = node.queue_ns(),
                vehicle_count_ew   = node.queue_ew(),
                waiting_ns         = node.W["N"] + node.W["S"],
                waiting_ew         = node.W["E"] + node.W["W"],
                current_time       = cur_time,
                current_congestion = c,
                gap_ns             = 1.0 if node.queue_ns() > 0 else 5.0,
                gap_ew             = 1.0 if node.queue_ew() > 0 else 5.0,
                neighbours         = net.neighbour_snapshots(nid),
                row                = node.row,
                col                = node.col,
                current_phase      = node.phase)
            node.step(dt,
                      arrive_ns         = base_demand_ns * (0.8 + 0.6 * c
                      arrive_ew         = base_demand_ew * (0.8 + 0.6 * c
                      green_active_phase= phase,
                      green_time        = min(dur, dt))
        for node in net.nodes.values():
            net_wait_sum      += sum(node.W.values())
            net_queue_samples += node.total_queue()
        net_samples += len(net.nodes)
```

```
    controller.m["waiting_sum_s"]      = net_wait_sum / max(net_samples,
    controller.m["queue_sum"]           = net_queue_samples / max(net_sam
    controller.m["vehicles_completed"]  = sum(n.completed for n in net.no
    controller.m["samples"]             = n_steps
    return controller


def run_multi_seed(algorithm_type: str,
                   scenario: str = "S1",
                   natural: str = "N-clear",
                   seeds: Iterable[int] = (1, 2, 3, 4, 5),
                   n_steps: int = 200,
                   base_demand_ns: float = 0.15,
                   base_demand_ew: float = 0.10,
                   car_following: str = "SAFE") -> dict[str, Any]:
    """Run one cell across n seeds; return mean / stdev / 95% CI per metr
    keys = ("travel_time_s", "queue_veh", "waiting_time_s",
            "throughput_vph", "mean_aoi_ms", "delivery_ratio")
    per_seed: dict[str, list[float]] = defaultdict(list)
    seeds = tuple(seeds)

    for s in seeds:
        c = TrafficControlAlgorithm(algorithm_type, scenario, natural,
                                    seed=s, car_following=car_following)
        simulate_episode(c, n_steps=n_steps,
                         base_demand_ns=base_demand_ns,
                         base_demand_ew=base_demand_ew, seed=s)
        m = c.get_metrics()
        for k in keys:
            per_seed[k].append(float(m[k]))

    out: dict[str, Any] = {"algorithm_type": algorithm_type,
                           "scenario": scenario,
                           "natural":  natural,
                           "n_seeds":  len(seeds)}
    for k, xs in per_seed.items():
        mean = statistics.fmean(xs) if xs else 0.0
        if len(xs) > 1:
            sd   = statistics.stdev(xs)
            ci95 = 1.96 * sd / math.sqrt(len(xs))
        else:
            sd = ci95 = 0.0
        out[f"{k}_mean"]   = round(mean, 3)
        out[f"{k}_sd"]     = round(sd,   3)
        out[f"{k}_ci95"]   = round(ci95, 3)
        out[f"{k}_values"] = [round(x, 3) for x in xs]
    return out


def export_csv(rows: list[dict[str, Any]],
               path: str = "traffic_benchmark.csv") -> None:
    """Write a list of dicts (from run_multi_seed) to CSV."""
    if not rows:
```

```
        return
    fieldnames = sorted({k for r in rows for k in r})
    with open(path, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=fieldnames, extrasaction="ignore
        w.writeheader()
        for r in rows:
            w.writerow(r)


# ============================================================================
#  SECTION 10  —  SYNTHETIC DATASET  (closed-form tensor generator)
# ============================================================================

PRESETS: dict[str, dict[str, int | None]] = {
    "quick":    {"n_ctrl": 16,  "n_scen": 8,  "n_cond": 4,  "n_steps": 20
    "standard": {"n_ctrl": 64,  "n_scen": 32, "n_cond": 8,  "n_steps": 50
    "full":     {"n_ctrl": 128, "n_scen": 64, "n_cond": 12, "n_steps": No
}

_CF_BENCHMARK: dict[str, dict[str, dict[str, float]]] = {
    "SAFE":  {"free_flow": {"in_system_s": 100.0},
              "congested": {"in_system_s": 230.0}},
    "GIPPS": {"free_flow": {"in_system_s": 105.0},
              "congested": {"in_system_s": 250.0}},
    "IDM":   {"free_flow": {"in_system_s": 102.0},
              "congested": {"in_system_s": 240.0}},
}

K_CTRL_FAMILY: dict[str, float] = {
    "baseline": 1.26, "adaptive": 1.06, "coordinated": 1.00,
    "field": 0.98, "learning": 1.02, "resilient": 0.99,
}

RESILIENCE_FLATTENING: dict[str, float] = {
    "baseline": 1.00, "adaptive": 0.95, "coordinated": 0.90,
    "field": 0.85, "learning": 0.75, "resilient": 0.35,
}

_K_COMM_BASE_K  = 1.43 * 100.0
_K_COMM_BASE_X0 = 101.54 / 600.0
_K_COMM_BASE_L  = 0.60

_NUMERIC_COLS: tuple[str, ...] = (
    "travel_time_s", "queue_veh", "waiting_time_s", "throughput_vph",
    "mean_aoi_ms", "delivery_ratio",
    "K_ctrl", "K_comm", "K_nat", "K_dem",
    "severity", "true_value",
)


@dataclass
class DatasetConfig:
    """All tunable parameters for the synthetic dataset, validated at con
    seed:          int   = 42
```

```
    car_following: str   = "SAFE"
    sample_cf:     bool  = False
    noise_frac:    float = 0.06
    outlier_frac:  float = 0.02
    outlier_mult:  float = 2.2
    preset:        str   = "full"
    csv_path:      str   = "synthetic_dataset.csv"
    npy_path:      str   = "synthetic_tensor.npy"
    progress:      bool  = True
    auto_build:    bool  = False
    verbose:       bool  = True

    def __post_init__(self) -> None:
        if self.preset not in PRESETS:
            raise ValueError(f"Unknown preset {self.preset!r}. "
                             f"Choose from {tuple(PRESETS)}.")
        if self.car_following not in _CF_BENCHMARK:
            raise ValueError(f"Unknown car_following {self.car_following!
        if not 0.0 <= self.noise_frac < 1.0:
            raise ValueError(f"noise_frac must be in [0, 1); got {self.no
        if not 0.0 <= self.outlier_frac <= 1.0:
            raise ValueError(f"outlier_frac must be in [0, 1]; got {self.
        if self.outlier_mult <= 0.0:
            raise ValueError(f"outlier_mult must be > 0; got {self.outlie


# --- 128 controllers, built family-by-family ---------------------------

_FAMILY_SPEC: dict[str, dict[str, Any]] = {
    "baseline":    {"count": 16, "bases": ["fixed-time", "actuated",
                                           "semi-actuated", "pretimed",
                                           "gap-out", "cycle-72", "phase-s
                                           "dual-ring"]},
    "adaptive":    {"count": 16, "bases": ["adaptive", "adapt-p", "adapt-
                                           "adapt-pid", "adapt-q", "adapt-
                                           "adapt-lin", "adapt-nonlin"]},
    "coordinated": {"count": 16, "bases": ["coord", "coord-green", "offse
                                           "platoon", "green-wave", "green
                                           "flow-sync", "bidir-wave"]},
    "field":       {"count": 32, "bases": ["max-pressure", "scats", "scoo
                                           "multiband", "mpc", "fuzzy", "w
                                           "bike", "transit", "emergency",
                                           "ped-adapt", "rideshare", "max-
                                           "split-opt", "cycle-opt", "offs
    "learning":    {"count": 24, "bases": ["idqn", "coop-marl", "ppo-marl
                                           "sac-marl", "td3-marl", "a2c-ma
                                           "ddpg-marl", "rainbow", "iqn",
                                           "iql", "bc-marl"]},
    "resilient":   {"count": 24, "bases": ["res-marl", "res-dqn", "res-pp
                                           "byz-robust", "fed-marl", "sec-
                                           "robust-td3", "res-sac", "res-t
                                           "res-a2c", "res-ddpg", "res-iqn
}
assert sum(s["count"] for s in _FAMILY_SPEC.values()) == 128
```

```
def _build_controllers() -> dict[str, str]:
    """Return an ordered mapping {controller_name: family}."""
    out: dict[str, str] = {}
    for family, spec in _FAMILY_SPEC.items():
        bases = list(spec["bases"])
        while len(bases) < spec["count"]:
            i = len(bases) - len(spec["bases"])
            bases.append(
                f"{spec['bases'][i % len(spec['bases'])]}"
                f"-v{2 + (i // len(spec['bases']))}"
            )
        for name in bases[: spec["count"]]:
            out[name] = family
    return out


CONTROLLER_FAMILY: dict[str, str] = _build_controllers()
CONTROLLERS_TUPLE: tuple[str, ...] = tuple(CONTROLLER_FAMILY)
assert len(CONTROLLERS_TUPLE) == 128


# --- 64 scenarios (8 latencies x 8 loss rates) ------------------------

_LATENCY_GRID = np.linspace(10.0, 1000.0, 8)
_LOSS_GRID    = np.linspace(0.005, 0.650, 8)

SCENARIOS_DS: dict[str, dict[str, float]] = {}
SCENARIO_KEYS_DS: list[str] = []
for _i, _lat in enumerate(_LATENCY_GRID):
    for _j, _loss in enumerate(_LOSS_GRID):
        _key = f"S{_i * 8 + _j + 1:02d}"
        SCENARIOS_DS[_key] = {"latency_ms": float(_lat),
                              "loss_rate":  float(_loss)}
        SCENARIO_KEYS_DS.append(_key)
SCENARIO_KEYS_DS_TUPLE: tuple[str, ...] = tuple(SCENARIO_KEYS_DS)
assert len(SCENARIO_KEYS_DS_TUPLE) == 64


# --- 12 conditions ----------------------------------------------------

CONDITIONS: dict[str, dict[str, float]] = {
    "C01-clear":        {"capacity": 1.00, "demand_mult": 1.00, "directio
    "C02-rain":         {"capacity": 0.85, "demand_mult": 0.95, "directio
    "C03-snow":         {"capacity": 0.60, "demand_mult": 0.80, "directio
    "C04-hailstorm":    {"capacity": 0.45, "demand_mult": 0.55, "directio
    "C05-flooding":     {"capacity": 0.35, "demand_mult": 0.50, "directio
    "C06-accident":     {"capacity": 0.55, "demand_mult": 0.90, "directio
    "C07-major-crash":  {"capacity": 0.40, "demand_mult": 0.85, "directio
    "C08-rush-am":      {"capacity": 0.95, "demand_mult": 1.35, "directio
    "C09-rush-pm":      {"capacity": 0.95, "demand_mult": 1.35, "directio
    "C10-oversat":      {"capacity": 1.00, "demand_mult": 1.60, "directio
    "C11-blackout":     {"capacity": 1.00, "demand_mult": 0.85, "directio
```

```
    "C12-power-outage": {"capacity": 0.90, "demand_mult": 0.75, "directio
}
CONDITION_KEYS_TUPLE: tuple[str, ...] = tuple(CONDITIONS)
assert len(CONDITION_KEYS_TUPLE) == 12


@dataclass(frozen=True, slots=True)
class LogisticSignatureDS:
    L:  float
    k:  float
    x0: float
    def __call__(self, x: float) -> float:
        return self.L / (1.0 + math.exp(-self.k * (x - self.x0)))


SCENARIO_SIGNATURES: dict[str, LogisticSignatureDS] = {
    f"S{i:02d}": LogisticSignatureDS(
        L  = 140.0 + 2.4 * (i - 1),
        k  = 0.0143 + 0.00030 * (i - 1),
        x0 = 0.05 + 0.0050 * (i - 1),
    )
    for i in range(1, 65)
}


@dataclass
class SyntheticTrafficDataset:
    """Closed-form (128,64,12) synthetic dataset."""

    seed:          int   = 42
    car_following: str   = "SAFE"
    sample_cf:     bool  = False
    noise_frac:    float = 0.06
    outlier_frac:  float = 0.02
    outlier_mult:  float = 2.2
    csv_path:      str   = "synthetic_dataset.csv"
    npy_path:      str   = "synthetic_tensor.npy"
    preset:        str   = "full"
    progress:      bool  = True
    auto_build:    bool  = False
    verbose:       bool  = True

    _cfg:       DatasetConfig = field(init=False, repr=False, compare=Fal
    _ctrls:     tuple[str, ...] = field(init=False, repr=False, compare=F
    _scens:     tuple[str, ...] = field(init=False, repr=False, compare=F
    _conds:     tuple[str, ...] = field(init=False, repr=False, compare=F
    _shape:     tuple[int, int, int] = field(init=False, repr=False, comp
    _arrays:    Optional[dict[str, np.ndarray]] = field(init=False, repr=
    _M:         Optional[np.ndarray] = field(init=False, repr=False, comp

    NUMERIC_COLS: tuple[str, ...] = _NUMERIC_COLS

    def __post_init__(self) -> None:
        self._cfg = DatasetConfig(
```

```
            seed=self.seed, car_following=self.car_following,
            sample_cf=self.sample_cf, noise_frac=self.noise_frac,
            outlier_frac=self.outlier_frac, outlier_mult=self.outlier_mul
            preset=self.preset, csv_path=self.csv_path,
            npy_path=self.npy_path, progress=self.progress,
            auto_build=self.auto_build, verbose=self.verbose,
        )
        p = PRESETS[self._cfg.preset]
        self._ctrls = CONTROLLERS_TUPLE[: p["n_ctrl"]]
        self._scens = SCENARIO_KEYS_DS_TUPLE[: p["n_scen"]]
        self._conds = CONDITION_KEYS_TUPLE[: p["n_cond"]]
        self._shape = (len(self._ctrls), len(self._scens), len(self._cond
        if self._cfg.auto_build:
            self.build()

    @classmethod
    def default(cls, **overrides: Any) -> "SyntheticTrafficDataset":
        cfg = DatasetConfig(**overrides) if overrides else DatasetConfig(
        return cls(**cfg.__dict__)

    @property
    def shape(self) -> tuple[int, int, int]:
        return self._shape

    @property
    def size(self) -> int:
        return self._shape[0] * self._shape[1] * self._shape[2]

    def __len__(self) -> int: return self.size

    def __repr__(self) -> str:
        state = "built" if self._M is not None else "empty"
        return (f"SyntheticTrafficDataset(preset={self.preset!r}, "
                f"shape={self._shape}, seed={self.seed}, {state})")

    def __getitem__(self, key: tuple[int, int, int]) -> dict[str, Any]:
        i, j, k = key
        return self._ensure_records()[i, j, k]

    def __iter__(self) -> Iterator[dict[str, Any]]:
        M = self._ensure_records()
        for i, j, k in product(range(self._shape[0]),
                               range(self._shape[1]),
                               range(self._shape[2])):
            yield M[i, j, k]

    # ----- vectorised factors ------------------------------------------
    @staticmethod
    def _K_comm_vector(family_flat: np.ndarray, latency: np.ndarray,
                       loss: np.ndarray) -> np.ndarray:
        rho = 0.5 * (latency / 600.0) + 0.5 * loss
        k   = _K_COMM_BASE_K / family_flat
        x0  = _K_COMM_BASE_X0 * (1.0 + 0.5 * (1.0 - family_flat))
        L   = _K_COMM_BASE_L * family_flat
```

```
        return 1.0 + L / (1.0 + np.exp(-k * (rho - x0)))

    @staticmethod
    def _K_nat_vector(capacity: np.ndarray, demand_mult: np.ndarray,
                      bias: np.ndarray) -> np.ndarray:
        return ((1.0 / np.maximum(capacity, 0.1))
                * (1.0 + 0.35 * (demand_mult - 1.0))
                * (1.0 + 0.15 * np.abs(bias)))

    @staticmethod
    def _K_dem_vector(ns: np.ndarray, ew: np.ndarray,
                      capacity: np.ndarray) -> np.ndarray:
        arrival = ns + ew
        service = 2 * SATURATION_FLOW_VPS * capacity * 10.0
        rho = np.minimum(0.95, arrival / np.maximum(service, 1e-6))
        return 1.0 / (1.0 - rho)

    def _compute_arrays(self, force: bool = False) -> dict[str, np.ndarra
        if self._arrays is not None and not force:
            return self._arrays

        n1, n2, n3 = self._shape
        rng = np.random.default_rng(self._cfg.seed)

        ctrl_family = np.array([CONTROLLER_FAMILY[c] for c in self._ctrls
        kc   = np.array([K_CTRL_FAMILY[f] for f in ctrl_family])
        flat = np.array([RESILIENCE_FLATTENING[f] for f in ctrl_family])

        latency = np.array([SCENARIOS_DS[s]["latency_ms"] for s in self._
        loss    = np.array([SCENARIOS_DS[s]["loss_rate"]  for s in self._

        sig_L  = np.array([SCENARIO_SIGNATURES[s].L  for s in self._scens
        sig_k  = np.array([SCENARIO_SIGNATURES[s].k  for s in self._scens
        sig_x0 = np.array([SCENARIO_SIGNATURES[s].x0 for s in self._scens

        cap   = np.array([CONDITIONS[c]["capacity"]       for c in self._
        dmult = np.array([CONDITIONS[c]["demand_mult"]    for c in self._
        bias  = np.array([CONDITIONS[c]["direction_bias"] for c in self._
        ns    = np.array([CONDITIONS[c]["ns"]             for c in self._
        ew    = np.array([CONDITIONS[c]["ew"]             for c in self._

        kc_b    = kc[:, None, None]
        flat_b  = flat[:, None, None]
        lat_b   = latency[None, :, None]
        loss_b  = loss[None, :, None]
        sigL_b  = sig_L[None, :, None]
        sigk_b  = sig_k[None, :, None]
        sigx0_b = sig_x0[None, :, None]
        cap_b   = cap[None, None, :]
        dmult_b = dmult[None, None, :]
        bias_b  = bias[None, None, :]
        ns_b    = ns[None, None, :]
        ew_b    = ew[None, None, :]
```

```
        sev_comm   = 0.5 * (lat_b / 600.0) + 0.5 * loss_b
        sev_nat    = 1.0 - cap_b
        sev_dem    = np.minimum(0.5, (ns_b + ew_b - 0.6) / 1.2)
        sev        = 0.5 * sev_comm + 0.3 * sev_nat + 0.2 * sev_dem
        congestion = np.minimum(0.99,
                                0.4 * sev_comm + 0.4 * sev_nat + 0.2 * se

        ff_lut = np.array([_CF_BENCHMARK[c]["free_flow"]["in_system_s"]
                           for c in CF_MODELS])
        cg_lut = np.array([_CF_BENCHMARK[c]["congested"]["in_system_s"]
                           for c in CF_MODELS])
        if self._cfg.sample_cf:
            cf_idx = rng.integers(0, len(CF_MODELS), size=(n1, n2, n3))
            ff = ff_lut[cf_idx]
            cg = cg_lut[cf_idx]
        else:
            idx = CF_MODELS.index(self._cfg.car_following)
            ff, cg = ff_lut[idx], cg_lut[idx]

        w    = np.clip(congestion, 0.0, 1.0)
        base = (1.0 - w) * ff + w * cg

        K_comm = self._K_comm_vector(flat_b, lat_b, loss_b)
        K_nat  = self._K_nat_vector(cap_b, dmult_b, bias_b)
        K_dem  = self._K_dem_vector(ns_b * dmult_b,
                                    ew_b * dmult_b,
                                    cap_b)

        raw = base * kc_b * K_comm * K_nat * K_dem

        log_c      = 2.0 / (1.0 + np.exp(-sigk_b * (sev - sigx0_b)))
        true_value = raw * (0.85 + 0.30 * log_c)

        noisy = true_value * (1.0 + rng.normal(0.0, self._cfg.noise_frac,
                                               size=true_value.shape))
        outlier = rng.random(size=true_value.shape) < self._cfg.outlier_f
        noisy = np.where(outlier, noisy * self._cfg.outlier_mult, noisy)

        travel = np.maximum(5.0, noisy)
        excess = np.maximum(0.0, travel / np.maximum(base, 1.0) - 1.0)

        queue = np.maximum(0.0, 3.0 + 12.0 * excess
                                + rng.normal(0.0, 0.6, size=travel.shape)
        wait  = np.maximum(0.0, 5.0 + 20.0 * excess
                                + rng.normal(0.0, 1.2, size=travel.shape)
        thr   = np.maximum(50.0, 950.0 / (1.0 + 0.9 * excess))
        aoi   = np.maximum(0.0, 40.0 + 350.0 * sev_comm
                                + 20.0 * np.abs(rng.normal(0.0, 1.0,
                                                           size=travel.sh

        deliver = np.clip(
            0.99 - 0.55 * sev_comm - 0.20 * sev_nat
                 + rng.normal(0.0, 0.01, size=travel.shape),
            0.0, 1.0,
        )
```

```
        def _broadcast(x: np.ndarray) -> np.ndarray:
            return np.broadcast_to(x, (n1, n2, n3)).copy()

        arrays: dict[str, np.ndarray] = {
            "travel_time_s":  _broadcast(travel),
            "queue_veh":      _broadcast(queue),
            "waiting_time_s": _broadcast(wait),
            "throughput_vph": _broadcast(thr),
            "mean_aoi_ms":    _broadcast(aoi),
            "delivery_ratio": _broadcast(deliver),
            "K_ctrl":         _broadcast(kc_b * np.ones_like(base)),
            "K_comm":         _broadcast(K_comm * np.ones_like(base)),
            "K_nat":          _broadcast(K_nat  * np.ones_like(base)),
            "K_dem":          _broadcast(K_dem  * np.ones_like(base)),
            "severity":       _broadcast(sev),
            "true_value":     _broadcast(true_value),
        }
        self._arrays = arrays
        return arrays

    def _ensure_records(self) -> np.ndarray:
        if self._M is None:
            self.build()
        assert self._M is not None
        return self._M

    def build(self, force: bool = False) -> np.ndarray:
        if self._M is not None and not force:
            return self._M

        arrays = self._compute_arrays(force=force)
        n1, n2, n3 = self._shape

        if self._cfg.progress and self._cfg.verbose and self.size > 1000:
            print(f"  building M with shape {(n1, n2, n3)} "
                  f"= {self.size:,} cells ...")

        ctrl_names = np.array(self._ctrls)[:, None, None]
        scen_names = np.array(self._scens)[None, :, None]
        cond_names = np.array(self._conds)[None, None, :]

        M = np.empty((n1, n2, n3), dtype=object)
        for i, j, k in product(range(n1), range(n2), range(n3)):
            M[i, j, k] = {
                "i": i, "j": j, "k": k,
                "controller":     ctrl_names[i, 0, 0],
                "car_following":  self._cfg.car_following,
                "scenario":       scen_names[0, j, 0],
                "condition":      cond_names[0, 0, k],
                "travel_time_s":  round(float(arrays["travel_time_s"][i,
                "queue_veh":      round(float(arrays["queue_veh"][i, j, k
                "waiting_time_s": round(float(arrays["waiting_time_s"][i,
                "throughput_vph": round(float(arrays["throughput_vph"][i,
```

```
                "mean_aoi_ms":    round(float(arrays["mean_aoi_ms"][i, j,
                "delivery_ratio": round(float(arrays["delivery_ratio"][i,
                "K_ctrl":         round(float(arrays["K_ctrl"][i, j, k]),
                "K_comm":         round(float(arrays["K_comm"][i, j, k]),
                "K_nat":          round(float(arrays["K_nat"][i, j, k]),
                "K_dem":          round(float(arrays["K_dem"][i, j, k]),
                "severity":       round(float(arrays["severity"][i, j, k]
                "true_value":     round(float(arrays["true_value"][i, j,
            }
        self._M = M
        return M

    def as_tensor(self, M: Optional[np.ndarray] = None,
                  cols: Optional[tuple[str, ...]] = None) -> np.ndarray:
        cols = cols or self.NUMERIC_COLS
        arrays = self._compute_arrays()
        return np.stack([arrays[c] for c in cols], axis=-1)

    build_numeric = as_tensor

    def slice_controller(self, i: int,
                         M: Optional[np.ndarray] = None) -> np.ndarray:
        return (self._ensure_records() if M is None else M)[i]

    def slice_scenario(self, j: int,
                       M: Optional[np.ndarray] = None) -> np.ndarray:
        return (self._ensure_records() if M is None else M)[:, j, :]

    def slice_condition(self, k: int,
                        M: Optional[np.ndarray] = None) -> np.ndarray:
        return (self._ensure_records() if M is None else M)[:, :, k]

    def stats(self, M: Optional[np.ndarray] = None) -> dict[str, Any]:
        _ = M
        arrays = self._compute_arrays()
        out: dict[str, Any] = {"shape": self._shape, "n_cells": self.size
        for name in self.NUMERIC_COLS:
            x = arrays[name]
            out[name] = {
                "mean":  round(float(x.mean()), 3),
                "stdev": round(float(x.std()), 3),
                "min":   round(float(x.min()), 3),
                "max":   round(float(x.max()), 3),
            }
        return out

    def to_csv(self, M: Optional[np.ndarray] = None,
               path: Optional[str] = None) -> str:
        path = path or self.csv_path
        M = self._ensure_records() if M is None else M
        n1, n2, n3 = M.shape
        sample = M[0, 0, 0]
        cols = list(sample.keys())
        with open(path, "w", newline="") as fh:
```

```
            w = csv.DictWriter(fh, fieldnames=cols)
            w.writeheader()
            for i, j, k in product(range(n1), range(n2), range(n3)):
                w.writerow(M[i, j, k])
        if self._cfg.verbose:
            print(f"  wrote {n1 * n2 * n3:,} rows "
                  f"({n1} x {n2} x {n3}) -> {path}")
        return path

    def to_npy(self, M: Optional[np.ndarray] = None,
               path: Optional[str] = None) -> str:
        path = path or self.npy_path
        T = self.as_tensor(M) if M is not None else self.as_tensor()
        np.save(path, T)
        if self._cfg.verbose:
            print(f"  wrote tensor {T.shape} -> {path}")
        return path

    def summary(self, M: Optional[np.ndarray] = None) -> str:
        s = self.stats(M)
        n1, n2, n3 = self._shape
        lines = [
            "=" * 90,
            f"  SYNTHETIC DATASET  -  preset={self._cfg.preset!r}",
            "=" * 90,
            f"  shape          : {self._shape}",
            f"  total cells    : {self.size:,}",
            f"  axis 0 (ctrl)  : {n1}",
            f"  axis 1 (scen)  : {n2}",
            f"  axis 2 (cond)  : {n3}",
            f"  seed           : {self._cfg.seed}",
            f"  car-following  : {self._cfg.car_following}"
            + ("  (sampled per cell)" if self._cfg.sample_cf else ""),
            f"  noise frac     : {self._cfg.noise_frac}",
            "",
            f"  {'metric':<18}{'mean':>10}{'stdev':>10}"
            f"{'min':>10}{'max':>10}",
            "  " + "-" * 58,
        ]
        for k, v in s.items():
            if k in ("shape", "n_cells"):
                continue
            lines.append(f"  {k:<18}{v['mean']:>10.3f}{v['stdev']:>10.3f}"
                         f"{v['min']:>10.3f}{v['max']:>10.3f}")
        text = "\n".join(lines)
        if self._cfg.verbose:
            print(text)
        return text


# ============================================================================
#  SECTION 11  —  TENSOR PIPELINE  (Ridge + kNN + GP + EI active learning
# ============================================================================
```

```
DEFAULT_COLS: tuple[str, ...] = (
    "travel_time_s", "queue_veh", "waiting_time_s", "throughput_vph",
    "mean_aoi_ms", "delivery_ratio",
    "K_ctrl", "K_comm", "K_nat", "K_dem",
    "severity", "true_value",
)
DEFAULT_TARGET = "travel_time_s"
LEAKY_COLS: tuple[str, ...] = ("true_value",)


@dataclass
class PipelineReport:
    target:      str
    n_train:     int
    n_test:      int
    n_features:  int
    scores:      dict[str, dict[str, float]] = field(default_factory=dict
    al_trace:    list[dict[str, float]] = field(default_factory=list)

    def __str__(self) -> str:
        lines = [
            "=" * 78,
            f"  PIPELINE REPORT  -  target = {self.target!r}",
            "=" * 78,
            f"  train / test   : {self.n_train:,} / {self.n_test:,}",
            f"  feature count  : {self.n_features}",
            "",
            f"  {'model':<18}{'MAE':>10}{'RMSE':>10}{'R2':>10}",
            "  " + "-" * 46,
        ]
        for name, s in self.scores.items():
            lines.append(f"  {name:<18}{s['mae']:>10.4f}"
                         f"{s['rmse']:>10.4f}{s['r2']:>10.4f}")
        if self.al_trace:
            lines.append("")
            lines.append("  Active-learning trace (EI loop)")
            lines.append(f"  {'round':>6}{'MAE':>10}{'RMSE':>10}{'R2':>10
            lines.append("  " + "-" * 36)
            for row in self.al_trace:
                lines.append(f"  {row['round']:>6}"
                             f"{row['mae']:>10.4f}"
                             f"{row['rmse']:>10.4f}"
                             f"{row['r2']:>10.4f}")
        return "\n".join(lines)


@dataclass
class TensorPipeline:
    """Loads a (n1, n2, n3, F) numeric tensor and runs a small ML pipelin

    path:        str = "synthetic_tensor.npy"
    cols:        tuple[str, ...] = DEFAULT_COLS
    target:      str = DEFAULT_TARGET
    seed:        int = 42
```

```
    test_frac:   float = 0.2
    verbose:     bool = True

    X:            Optional[np.ndarray] = field(init=False, default=None,
    y:            Optional[np.ndarray] = field(init=False, default=None,
    grid_shape:   Optional[tuple[int, ...]] = field(init=False, default=N
    _rng:         np.random.Generator = field(init=False, repr=False)
    _split:       Optional[tuple[np.ndarray, np.ndarray,
                                 np.ndarray, np.ndarray]] = field(init=Fal
    _preds:       Optional[np.ndarray] = field(init=False, default=None,

    def __post_init__(self) -> None:
        self._rng = np.random.default_rng(self.seed)
        self.load()

    def load(self, force: bool = False) -> "TensorPipeline":
        if self.X is not None and not force:
            return self
        path = Path(self.path)
        if not path.exists():
            raise FileNotFoundError(
                f"Tensor file not found: {path}. "
                f"Build it via SyntheticTrafficDataset.to_npy().")
        T = np.load(path)
        if T.ndim != 4:
            raise ValueError(
                f"Expected 4-D tensor (n1, n2, n3, F); got shape {T.shape

        self.grid_shape = T.shape[:3]
        F = T.shape[-1]
        if F != len(self.cols):
            raise ValueError(
                f"Feature axis is {F} but {len(self.cols)} column names "
                f"were supplied. Pass cols=... matching your tensor.")
        if self.target not in self.cols:
            raise ValueError(f"target {self.target!r} not in {self.cols}"

        flat = T.reshape(-1, F)
        tgt_idx = self.cols.index(self.target)
        drop = {tgt_idx} | {self.cols.index(c) for c in LEAKY_COLS
                            if c in self.cols}
        keep = [i for i in range(F) if i not in drop]

        self.X = flat[:, keep].astype(np.float64)
        self.y = flat[:, tgt_idx].astype(np.float64)

        if self.verbose:
            print(f"  loaded {path}  shape={T.shape}  "
                  f"features={self.X.shape[1]}  rows={self.X.shape[0]:,}"
        return self

    def split(self, test_frac: Optional[float] = None,
              force: bool = False
              ) -> tuple[np.ndarray, np.ndarray,
```

```
                        np.ndarray, np.ndarray]:
        if self._split is not None and not force and test_frac is None:
            return self._split
        assert self.X is not None and self.y is not None
        frac = self.test_frac if test_frac is None else test_frac
        n = len(self.X)
        idx = self._rng.permutation(n)
        cut = int(round(n * (1 - frac)))
        tr, te = idx[:cut], idx[cut:]
        out = (self.X[tr], self.X[te], self.y[tr], self.y[te])
        self._split = out
        return out

    @staticmethod
    def _score(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float
        err  = y_pred - y_true
        mae  = float(np.mean(np.abs(err)))
        rmse = float(np.sqrt(np.mean(err ** 2)))
        ss_res = float(np.sum(err ** 2))
        ss_tot = float(np.sum((y_true - y_true.mean()) ** 2)) or 1e-12
        r2 = 1.0 - ss_res / ss_tot
        return {"mae": mae, "rmse": rmse, "r2": r2}

    # ------------------------------------------------------------------ FI
    @staticmethod
    def _ridge_fit(X: np.ndarray, y: np.ndarray,
                   alpha: float = 1.0) -> np.ndarray:
        """Closed-form ridge regression.
        Returns the coefficient vector w of length (d+1) where w[0] is th
        intercept and w[1:] are the per-feature slopes on the NORMALISED
        features.  Previously this returned a placeholder of zeros + w,
        which silently broke any caller using _ridge_fit instead of
        _ridge_predict.  Now it returns the actual coefficient vector.
        """
        n, d = X.shape
        mu = X.mean(axis=0, keepdims=True)
        sd = X.std(axis=0, keepdims=True) + 1e-9
        Xn = (X - mu) / sd
        Xb = np.hstack([np.ones((n, 1)), Xn])
        A = Xb.T @ Xb + alpha * np.eye(d + 1)
        A[0, 0] -= alpha                    # do not regularise the inter
        w = np.linalg.solve(A, Xb.T @ y)
        return w                              # shape (d+1,) -- FIXED

    def _ridge_predict(self, X_tr: np.ndarray, y_tr: np.ndarray,
                       X_te: np.ndarray, alpha: float = 1.0) -> np.ndarra
        mu = X_tr.mean(axis=0, keepdims=True)
        sd = X_tr.std(axis=0, keepdims=True) + 1e-9
        Xn_tr = (X_tr - mu) / sd
        Xn_te = (X_te - mu) / sd
        Xb_tr = np.hstack([np.ones((len(Xn_tr), 1)), Xn_tr])
        Xb_te = np.hstack([np.ones((len(Xn_te), 1)), Xn_te])
        A = Xb_tr.T @ Xb_tr + alpha * np.eye(Xb_tr.shape[1])
        A[0, 0] -= alpha
```

```
        w = np.linalg.solve(A, Xb_tr.T @ y_tr)
        return Xb_te @ w

    def _knn_predict(self, X_tr: np.ndarray, y_tr: np.ndarray,
                     X_te: np.ndarray, k: int = 5) -> np.ndarray:
        mu = X_tr.mean(axis=0, keepdims=True)
        sd = X_tr.std(axis=0, keepdims=True) + 1e-9
        A = (X_tr - mu) / sd
        B = (X_te - mu) / sd
        out = np.empty(len(B))
        chunk = 2048
        for s in range(0, len(B), chunk):
            b = B[s:s + chunk]
            d2 = ((b[:, None, :] - A[None, :, :]) ** 2).sum(-1)
            idx = np.argpartition(d2, k, axis=1)[:, :k]
            out[s:s + chunk] = y_tr[idx].mean(axis=1)
        return out

    def _gp_predict(self, X_tr: np.ndarray, y_tr: np.ndarray,
                    X_te: np.ndarray,
                    lengthscale: float = 1.0,
                    noise: float = 0.1
                    ) -> tuple[np.ndarray, np.ndarray]:
        """Tiny ARD-RBF GP; returns (mean, variance) on the test set."""
        mu = X_tr.mean(axis=0, keepdims=True)
        sd = X_tr.std(axis=0, keepdims=True) + 1e-9
        A = (X_tr - mu) / sd
        B = (X_te - mu) / sd

        def kern(P: np.ndarray, Q: np.ndarray) -> np.ndarray:
            Pn = P / lengthscale
            Qn = Q / lengthscale
            p = (Pn ** 2).sum(1)[:, None]
            q = (Qn ** 2).sum(1)[None, :]
            return np.exp(-0.5 * np.maximum(p + q - 2 * Pn @ Qn.T, 0.0))

        K   = kern(A, A) + (noise ** 2 + 1e-6) * np.eye(len(A))
        Ks  = kern(A, B)
        Kss = np.diag(kern(B, B))
        L = np.linalg.cholesky(K)
        alpha = np.linalg.solve(L.T, np.linalg.solve(L, y_tr))
        mu_pred = Ks.T @ alpha
        v = np.linalg.solve(L, Ks)
        var = np.maximum(Kss - (v ** 2).sum(0), 1e-9)
        return mu_pred, var

    def run(self, budget: int = 200, n_init: int = 20,
            gp_lengthscale: float = 1.0,
            gp_noise: float = 0.1) -> PipelineReport:
        X_tr, X_te, y_tr, y_te = self.split()
        n_feat = X_tr.shape[1]
        report = PipelineReport(
            target=self.target,
            n_train=len(X_tr), n_test=len(X_te),
```

```
            n_features=n_feat)

        baselines = {
            "mean":  np.full_like(y_te, y_tr.mean()),
            "ridge": self._ridge_predict(X_tr, y_tr, X_te),
            "knn5":  self._knn_predict(X_tr, y_tr, X_te, k=5),
        }
        for name, pred in baselines.items():
            report.scores[name] = self._score(y_te, pred)

        ridge_tr = self._ridge_predict(X_tr, y_tr, X_tr)
        resid_tr = y_tr - ridge_tr
        gp_mu, gp_var = self._gp_predict(X_tr, resid_tr, X_te,
                                         gp_lengthscale, gp_noise)
        gp_pred = self._ridge_predict(X_tr, y_tr, X_te) + gp_mu
        report.scores["ridge+gp"] = self._score(y_te, gp_pred)

        report.al_trace = self._active_learn(
            X_tr, y_tr, X_te, y_te,
            budget=budget, n_init=n_init,
            gp_lengthscale=gp_lengthscale, gp_noise=gp_noise)
        if self.verbose:
            print(report)
        return report

    def _active_learn(self, X_tr: np.ndarray, y_tr: np.ndarray,
                      X_te: np.ndarray, y_te: np.ndarray,
                      budget: int, n_init: int,
                      gp_lengthscale: float, gp_noise: float
                      ) -> list[dict[str, float]]:
        """EI loop on Ridge residuals.  Uses scipy.special.erf (vectorise
        n_pool = min(2000, max(200, len(X_tr) // 10))
        seen = self._rng.choice(len(X_tr), size=n_init, replace=False)
        seen_set = set(seen.tolist())
        trace: list[dict[str, float]] = []

        sqrt2 = math.sqrt(2.0)
        inv_sqrt_2pi = 1.0 / math.sqrt(2.0 * math.pi)

        for rnd in range(budget):
            ridge_obs = self._ridge_predict(X_tr[seen], y_tr[seen], X_tr[
            resid_obs = y_tr[seen] - ridge_obs
            mu, var = self._gp_predict(X_tr[seen], resid_obs, X_tr,
                                       gp_lengthscale, gp_noise)

            pool_idx = np.array([i for i in range(len(X_tr))
                                 if i not in seen_set])
            if len(pool_idx) == 0:
                break
            sample = self._rng.choice(
                pool_idx, size=min(n_pool, len(pool_idx)), replace=False)
            mu_s, var_s = mu[sample], var[sample]
            sigma = np.sqrt(np.maximum(var_s, 1e-12))
            best = y_tr[seen].min()
```

```
            xi = 0.01
            z = (best - mu_s - xi) / sigma

            # FIX: vectorised erf (was np.vectorize(math.erf) -- ~100x sl
            phi = 0.5 * (1.0 + _scipy_erf(z / sqrt2))
            pdf = np.exp(-0.5 * z ** 2) * inv_sqrt_2pi
            ei = (best - mu_s - xi) * phi + sigma * pdf

            pick = sample[int(np.argmax(ei))]
            seen_set.add(int(pick))
            seen = np.append(seen, pick)

            pred_te = self._ridge_predict(X_tr[seen], y_tr[seen], X_te)
            s = self._score(y_te, pred_te)
            trace.append({"round": float(rnd + 1), **s})

        return trace

    def export(self, path: str = "predictions.npy") -> str:
        if self._preds is None:
            X_tr, X_te, y_tr, y_te = self.split()
            ridge_all = self._ridge_predict(X_tr, y_tr, self.X)  # type:
            resid_tr = y_tr - self._ridge_predict(X_tr, y_tr, X_tr)
            mu, _ = self._gp_predict(X_tr, resid_tr, self.X)  # type: ign
            self._preds = ridge_all + mu
        np.save(path, self._preds)
        if self.verbose:
            print(f"  wrote {len(self._preds):,} predictions -> {path}")
        return path

    def export_csv(self, path: str = "predictions.csv") -> str:
        if self._preds is None:
            self.export()
        assert self._preds is not None and self.y is not None
        with open(path, "w", newline="") as fh:
            w = csv.writer(fh)
            w.writerow(["row", "y_true", "y_pred", "residual"])
            for i, (t, p) in enumerate(zip(self.y, self._preds)):
                w.writerow([i, f"{t:.4f}", f"{p:.4f}", f"{t - p:.4f}"])
        if self.verbose:
            print(f"  wrote {len(self._preds):,} rows -> {path}")
        return path

    def summary(self) -> str:
        assert self.X is not None and self.y is not None
        lines = [
            "=" * 78,
            "  TENSOR PIPELINE - LOADED",
            "=" * 78,
            f"  file          : {self.path}",
            f"  grid shape    : {self.grid_shape}",
            f"  flat rows     : {self.X.shape[0]:,}",
            f"  features      : {self.X.shape[1]}",
            f"  target        : {self.target!r}",
```

```
            f"  target mean   : {self.y.mean():.3f}",
            f"  target stdev  : {self.y.std():.3f}",
            f"  target range  : [{self.y.min():.3f}, {self.y.max():.3f}]"
        ]
        text = "\n".join(lines)
        if self.verbose:
            print(text)
        return text


# ============================================================================
#  SECTION 12  —  ANALYTICAL SURROGATE  (replaces the missing module)
# ============================================================================

class Surrogate:
    """Closed-form surrogate that mirrors the synthetic dataset's factors

    Predicts travel_time_s for a (controller, cf, scenario, natural, dema
    cell WITHOUT running the simulator, using the multiplicative model:

        TT = base(cf, congestion) * K_ctrl(family) * K_comm(family, lat,
                                  * K_nat(cap, dmult, bias)
                                  * K_dem(ns, ew, cap)
                                  * logistic_correction(severity)
                                  + noise
    """

    def __init__(self, seed: int = 42):
        self.rng = np.random.default_rng(seed)
        self.seed = seed

    # ---- axis accessors (for the BayesianActiveLearner) ---------------
    @property
    def controllers(self) -> tuple[str, ...]:
        # FIX: return only the controllers that are ACTUALLY REGISTERED i
        # TrafficControlAlgorithm.CONTROLLERS (the 17 instantiable ones),
        # NOT the full 128-family synthetic-dataset catalog.  Otherwise
        # TuRBO picks cells like 'robust-td3-v2' that have no impl and
        # instantiation raises ValueError.
        return tuple(sorted(TrafficControlAlgorithm.CONTROLLERS))

    @property
    def cf_models(self) -> tuple[str, ...]:
        return CF_MODELS

    @property
    def scenarios(self) -> tuple[str, ...]:
        return ("S1", "S2", "S3", "S4")

    @property
    def naturals(self) -> tuple[str, ...]:
        return tuple(NATURAL_SCENARIOS)

    @property
```

```
    def demands(self) -> tuple[str, ...]:
        return tuple(sorted(DEMAND_PATTERNS))

    # ---- the multiplicative prediction --------------------------------
    def predict(self, controller: str, cf: str,
                scenario: str, natural: str, demand: str) -> float:
        """Return predicted travel_time_s for the given 5-tuple."""
        family = CONTROLLER_FAMILY.get(controller, "baseline")
        kc     = K_CTRL_FAMILY.get(family, 1.0)
        flat   = RESILIENCE_FLATTENING.get(family, 1.0)

        # comm
        if scenario in COMMUNICATION_SCENARIOS:
            cfg_c = COMMUNICATION_SCENARIOS[scenario]
            lat   = float(cfg_c["latency_ms"])
            loss  = float(cfg_c["loss_rate"])
        else:
            lat, loss = 100.0, 0.05
        rho = 0.5 * (lat / 600.0) + 0.5 * loss
        k   = _K_COMM_BASE_K / flat
        x0  = _K_COMM_BASE_X0 * (1.0 + 0.5 * (1.0 - flat))
        L   = _K_COMM_BASE_L * flat
        K_comm = 1.0 + L / (1.0 + math.exp(-k * (rho - x0)))

        # natural
        cfg_n = NATURAL_SCENARIOS.get(natural, NATURAL_SCENARIOS["N-clear
        cap    = float(cfg_n.get("capacity", 1.0))
        dmult  = float(cfg_n.get("demand_mult", 1.0))
        bias   = float(cfg_n.get("direction_bias", 0.0))
        K_nat = (1.0 / max(cap, 0.1)
                 * (1.0 + 0.35 * (dmult - 1.0))
                 * (1.0 + 0.15 * abs(bias)))

        # demand
        cfg_d = DEMAND_PATTERNS.get(demand, DEMAND_PATTERNS["D1"])
        ns, ew = float(cfg_d["ns"]), float(cfg_d["ew"])
        arrival = ns + ew
        service = 2 * SATURATION_FLOW_VPS * cap * 10.0
        rho_d = min(0.95, arrival / max(service, 1e-6))
        K_dem = 1.0 / (1.0 - rho_d)

        # congestion / base travel time
        sev_comm = 0.5 * (lat / 600.0) + 0.5 * loss
        sev_nat  = 1.0 - cap
        sev_dem  = min(0.5, (ns + ew - 0.6) / 1.2)
        sev      = 0.5 * sev_comm + 0.3 * sev_nat + 0.2 * sev_dem
        cong     = min(0.99, 0.4*sev_comm + 0.4*sev_nat + 0.2*sev_dem)

        cf_bench = _CF_BENCHMARK.get(cf, _CF_BENCHMARK["SAFE"])
        ff = cf_bench["free_flow"]["in_system_s"]
        cg = cf_bench["congested"]["in_system_s"]
        w = max(0.0, min(1.0, cong))
        base = (1.0 - w) * ff + w * cg
```

```
        sig = LogisticSignatureDS(L=140.0, k=0.0143, x0=0.05)
        log_c = 2.0 / (1.0 + math.exp(-sig.k * (sev - sig.x0)))
        true_value = base * kc * K_comm * K_nat * K_dem * (0.85 + 0.30 *

        noise = 1.0 + self.rng.normal(0.0, 0.06)
        return float(max(5.0, true_value * noise))


# ============================================================================
#  SECTION 13  —  BAYESIAN ACTIVE LEARNING  (replaces the missing module)
# ============================================================================

def expected_improvement(mu: np.ndarray, sigma: np.ndarray,
                         best: float, xi: float = 0.01) -> np.ndarray:
    """Vectorised Expected Improvement."""
    sqrt2 = math.sqrt(2.0)
    inv_sqrt_2pi = 1.0 / math.sqrt(2.0 * math.pi)
    z = (best - mu - xi) / np.maximum(sigma, 1e-12)
    phi = 0.5 * (1.0 + _scipy_erf(z / sqrt2))
    pdf = np.exp(-0.5 * z ** 2) * inv_sqrt_2pi
    return (best - mu - xi) * phi + sigma * pdf


class Encoder:
    """One-hot encoder for the 5-axis categorical grid."""

    def __init__(self, controllers: Sequence[str], cf_models: Sequence[st
                 scenarios: Sequence[str], naturals: Sequence[str],
                 demands: Sequence[str]) -> None:
        self.controllers = tuple(controllers)
        self.cf_models   = tuple(cf_models)
        self.scenarios   = tuple(scenarios)
        self.naturals    = tuple(naturals)
        self.demands     = tuple(demands)
        self.n_features  = (len(self.controllers) + len(self.cf_models)
                            + len(self.scenarios) + len(self.naturals)
                            + len(self.demands))

    def encode(self, cell: tuple[str, str, str, str, str]) -> np.ndarray:
        ctrl, cf, sc, nat, dem = cell
        v = np.zeros(self.n_features, dtype=np.float64)
        axes = (self.controllers, self.cf_models, self.scenarios,
                self.naturals, self.demands)
        values = (ctrl, cf, sc, nat, dem)
        i = 0
        for axis, val in zip(axes, values):
            idx = axis.index(val) if val in axis else 0
            v[i + idx] = 1.0
            i += len(axis)
        return v

    def encode_many(self, cells: Sequence[tuple[str, str, str, str, str]]
                    ) -> np.ndarray:
        return np.vstack([self.encode(c) for c in cells])
```

```
class SurrogateGP:
    """Tiny ARD-RBF Gaussian process surrogate."""

    def __init__(self, n_features: int,
                 lengthscale: float = 1.0,
                 noise: float = 0.1) -> None:
        self.n_features  = n_features
        self.lengthscale = lengthscale
        self.noise       = noise
        self.X_tr: Optional[np.ndarray] = None
        self.y_tr: Optional[np.ndarray] = None
        self._L:   Optional[np.ndarray] = None
        self._alpha: Optional[np.ndarray] = None
        self._mu = 0.0
        self._sd = 1.0

    def _kern(self, P: np.ndarray, Q: np.ndarray) -> np.ndarray:
        Pn = P / self.lengthscale
        Qn = Q / self.lengthscale
        p = (Pn ** 2).sum(1)[:, None]
        q = (Qn ** 2).sum(1)[None, :]
        return np.exp(-0.5 * np.maximum(p + q - 2 * Pn @ Qn.T, 0.0))

    def fit(self, X: np.ndarray, y: np.ndarray) -> "SurrogateGP":
        self.X_tr = X.copy()
        self.y_tr = y.copy()
        self._mu = float(y.mean()) if len(y) else 0.0
        self._sd = float(y.std() + 1e-9) if len(y) else 1.0
        yn = (y - self._mu) / self._sd
        K = self._kern(X, X) + (self.noise ** 2 + 1e-6) * np.eye(len(X))
        self._L = np.linalg.cholesky(K)
        self._alpha = np.linalg.solve(self._L.T,
                                      np.linalg.solve(self._L, yn))
        return self

    def predict(self, X: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
        if self.X_tr is None or self._L is None or self._alpha is None:
            n = len(X)
            return (np.full(n, self._mu),
                    np.full(n, 1.0))
        Ks = self._kern(self.X_tr, X)
        mu = Ks.T @ self._alpha * self._sd + self._mu
        v = np.linalg.solve(self._L, Ks)
        Kss = np.diag(self._kern(X, X))
        var = np.maximum(Kss - (v ** 2).sum(0), 1e-9) * (self._sd ** 2)
        return mu, var


class BayesianActiveLearner:
    """Vanilla EI active learner — base class for TrustRegionLearner."""

    def __init__(self, surrogate: Surrogate, seed: int = 0) -> None:
```

```
        self.surrogate = surrogate
        self.rng = random.Random(seed)
        self.np_rng = np.random.default_rng(seed)

        self.controllers = tuple(surrogate.controllers)
        self.cf_models   = tuple(surrogate.cf_models)
        self.scenarios   = tuple(surrogate.scenarios)
        self.naturals    = tuple(surrogate.naturals)
        self.demands     = tuple(surrogate.demands)
        self.axes        = (self.controllers, self.cf_models,
                            self.scenarios, self.naturals, self.demands)
        self.encoder     = Encoder(*self.axes)
        self.gp          = SurrogateGP(self.encoder.n_features)

        self.cells:    list[tuple[str, str, str, str, str]] = []
        self.y_sim:    list[float] = []
        self.y_pred:   list[float] = []
        self.y_resid:  list[float] = []
        self.best_value: float = float("inf")
        self.best_cell: Optional[tuple[str, str, str, str, str]] = None

    # ---- observe one cell ------------------------------------------------
    def _observe(self, cell: tuple[str, str, str, str, str],
                 n_steps: int = 200) -> float:
        ctrl_name, cf, scen, nat, dem = cell
        d = DEMAND_PATTERNS.get(dem, DEMAND_PATTERNS["D1"])
        c = TrafficControlAlgorithm(ctrl_name, scen, nat,
                                    seed=self.rng.randint(1, 10000),
                                    car_following=cf)
        simulate_episode(c, n_steps=n_steps,
                         base_demand_ns=float(d["ns"]),
                         base_demand_ew=float(d["ew"]),
                         seed=self.rng.randint(1, 10000))
        y_real = float(c.get_metrics()["travel_time_s"])
        y_surr = float(self.surrogate.predict(*cell))
        self.cells.append(cell)
        self.y_sim.append(y_real)
        self.y_pred.append(y_surr)
        self.y_resid.append(y_real - y_surr)
        if y_real < self.best_value:
            self.best_value = y_real
            self.best_cell  = cell
        return y_real

    def _fit_gp(self) -> None:
        if not self.cells:
            return
        X = self.encoder.encode_many(self.cells)
        y = np.array(self.y_resid, dtype=np.float64)
        self.gp.fit(X, y)

    def _surrogate_predict(self, cell: tuple[str, str, str, str, str]
                           ) -> float:
        return float(self.surrogate.predict(*cell))
```

```
    def corrected_predict(self, cell: tuple[str, str, str, str, str]
                          ) -> tuple[float, float]:
        x = self.encoder.encode(cell).reshape(1, -1)
        mu, var = self.gp.predict(x)
        return (float(self.surrogate.predict(*cell) + mu[0]),
                float(var[0]))

    # ---- the basic EI loop ------------------------------------------
    def run(self, n_init: int = 20, budget: int = 100,
            n_steps: int = 200, n_candidates: int = 2000,
            progress: bool = True) -> dict[str, Any]:
        t0 = time.time()
        if progress:
            print(f"\n  EI ACTIVE LEARNING — budget={budget}")

        all_cells = list(product(*self.axes))
        self.rng.shuffle(all_cells)
        for cell in all_cells[:n_init]:
            self._observe(cell, n_steps=n_steps)

        observed = set(self.cells)
        for rnd in range(n_init + 1, budget + 1):
            self._fit_gp()
            pool = [c for c in all_cells if c not in observed]
            if not pool:
                break
            if len(pool) > n_candidates:
                pool = self.rng.sample(pool, n_candidates)
            X_pool = self.encoder.encode_many(pool)
            mu, var = self.gp.predict(X_pool)
            pred_pool = np.array([self._surrogate_predict(c) for c in poo
            pred_mu = pred_pool + mu
            pred_sd = np.sqrt(var)
            ei = expected_improvement(pred_mu, pred_sd, best=self.best_va
            idx = int(np.argmax(ei))
            chosen = pool[idx]
            y_real = self._observe(chosen, n_steps=n_steps)
            observed.add(chosen)
            if progress and (rnd % max(1, budget // 10) == 0
                             or rnd == budget):
                rmse = math.sqrt(statistics.fmean(r ** 2 for r in self.y_
                print(f"    round {rnd:>4}/{budget}  RMSE={rmse:>6.3f}  "
                      f"best={self.best_value:>7.2f}")

        self._fit_gp()
        return {
            "n_sims":         len(self.cells),
            "best_cell":      self.best_cell,
            "best_value":     round(self.best_value, 2),
            "residual_rmse":  round(math.sqrt(statistics.fmean(
                r ** 2 for r in self.y_resid)), 3),
            "residual_mae":   round(statistics.fmean(
                abs(r) for r in self.y_resid), 3),
```

```
            "elapsed_s":      round(time.time() - t0, 2),
        }

    # ---- held-out evaluation ------------------------------------------
    def evaluate_holdout(self, n_holdout: int = 30,
                         n_steps: int = 200) -> dict[str, float]:
        all_cells = list(product(*self.axes))
        unseen = [c for c in all_cells if c not in set(self.cells)]
        if not unseen:
            unseen = all_cells.copy()
        self.rng.shuffle(unseen)
        sample = unseen[:n_holdout]
        raw_err, corr_err = [], []
        for cell in sample:
            d = DEMAND_PATTERNS.get(cell[4], DEMAND_PATTERNS["D1"])
            c = TrafficControlAlgorithm(cell[0], cell[2], cell[3],
                                        seed=self.rng.randint(1, 10000),
                                        car_following=cell[1])
            simulate_episode(c, n_steps=n_steps,
                             base_demand_ns=float(d["ns"]),
                             base_demand_ew=float(d["ew"]),
                             seed=self.rng.randint(1, 10000))
            y_real = float(c.get_metrics()["travel_time_s"])
            y_raw  = float(self.surrogate.predict(*cell))
            y_corr, _ = self.corrected_predict(cell)
            raw_err.append(abs(y_real - y_raw))
            corr_err.append(abs(y_real - y_corr))
        return {
            "raw_mae":   round(statistics.fmean(raw_err), 3),
            "raw_rmse":  round(math.sqrt(statistics.fmean(e**2 for e in r
            "corr_mae":  round(statistics.fmean(corr_err), 3),
            "corr_rmse": round(math.sqrt(statistics.fmean(e**2 for e in c
        }


# ============================================================================
#  SECTION 14  —  TRUST-REGION ACTIVE LEARNING  (TuRBO over the grid)
# ============================================================================

def canonical_axes() -> tuple[tuple[str, ...], ...]:
    """Return (controllers, cf, scenarios, naturals, demands) ordered for

    FIX: returns only the controllers that are ACTUALLY REGISTERED in
    TrafficControlAlgorithm.CONTROLLERS (the 17 instantiable ones), not
    the full 128-family synthetic-dataset catalog.  Otherwise TuRBO picks
    cells like 'robust-td3-v2' that have no impl and instantiation fails.
    """
    fam_rank = {"baseline": 0, "adaptive": 1, "coordinated": 2,
                "learning": 3, "field": 4, "resilient": 5}
    # Use the actually-registered controllers (17), not CONTROLLER_FAMILY
    ctrls = tuple(sorted(TrafficControlAlgorithm.CONTROLLERS,
                         key=lambda c: (fam_rank.get(
                             CONTROLLER_FAMILY.get(c, "baseline"), 9), c))
    cf = ("SAFE", "IDM", "GIPPS")
```

```
    sc = ("S1", "S2", "S3", "S4")
    nats_by_fam = {
        "weather":  ["N-clear", "N-rain", "N-snow", "N-fog"],
        "incident": ["N-accident", "N-workzone"],
        "demand":   ["N-rush-am", "N-rush-pm", "N-night",
                     "N-event", "N-school"],
        "failure":  ["N-sensor-fail", "N-comm-blackout"],
    }
    nats = tuple(n for fam in ("weather", "incident", "demand", "failure"
                 for n in nats_by_fam[fam] if n in NATURAL_SCENARIOS)
    dem = tuple(sorted(DEMAND_PATTERNS))
    return ctrls, cf, sc, nats, dem


@dataclass
class TrustRegion:
    """One local search box over the 5-axis grid."""
    center:  tuple[str, str, str, str, str]
    length:  float = 0.5
    success: int = 0
    failure: int = 0
    best_value: float = float("inf")
    rounds_used: int = 0

    def _window(self, axis_values: Sequence[str],
                center_value: str) -> list[str]:
        if center_value not in axis_values:
            return list(axis_values)
        n = len(axis_values)
        k = max(1, math.ceil(self.length * n))
        idx = list(axis_values).index(center_value)
        lo = max(0, idx - (k - 1) // 2)
        hi = min(n, lo + k)
        lo = max(0, hi - k)
        return list(axis_values[lo:hi])

    def cells(self, axes: tuple[tuple[str, ...], ...]
              ) -> Iterable[tuple[str, str, str, str, str]]:
        lists = [self._window(axes[i], self.center[i]) for i in range(5)]
        return product(*lists)

    def size(self, axes: tuple[tuple[str, ...], ...]) -> int:
        n = 1
        for i in range(5):
            n *= len(self._window(axes[i], self.center[i]))
        return n

    def update_on_success(self, new_value: float) -> None:
        self.success += 1
        self.failure = 0
        self.best_value = new_value
        self.length = min(2.0 * self.length, 1.0)

    def update_on_failure(self, shrink_after: int = 3) -> float:
```

```
        self.failure += 1
        if self.failure >= shrink_after:
            self.length *= 0.5
            self.failure = 0
        return self.length

    def needs_restart(self, min_length: float = 0.05) -> bool:
        return self.length < min_length


class TrustRegionLearner(BayesianActiveLearner):
    """TuRBO-style learner over the 5-axis grid."""

    def __init__(self, surrogate: Surrogate, seed: int = 0) -> None:
        super().__init__(surrogate, seed=seed)
        (self.controllers,
         self.cf_models,
         self.scenarios,
         self.naturals,
         self.demands) = canonical_axes()
        self.axes = (self.controllers, self.cf_models,
                     self.scenarios, self.naturals, self.demands)
        self.encoder = Encoder(*self.axes)
        self.gp = SurrogateGP(self.encoder.n_features)
        self.regions: list[TrustRegion] = []
        self.region_log: list[dict[str, Any]] = []

    def _seed_regions(self, n_regions: int, n_init: int, n_steps: int) ->
        all_cells = list(product(*self.axes))
        self.rng.shuffle(all_cells)
        for cell in all_cells[:n_init]:
            self._observe(cell, n_steps=n_steps)

        observed = sorted(zip(self.cells, self.y_sim), key=lambda x: x[1]
        seen_ctrls: set[str] = set()
        picks: list[tuple[tuple, float]] = []
        for cell, val in observed:
            if cell[0] not in seen_ctrls:
                picks.append((cell, val))
                seen_ctrls.add(cell[0])
            if len(picks) >= n_regions:
                break
        for cell, val in observed:
            if len(picks) >= n_regions:
                break
            if (cell, val) not in picks:
                picks.append((cell, val))

        self.regions = [TrustRegion(center=cell, length=0.5,
                                    best_value=val) for cell, val in pick

    def _restart_region(self, reg: TrustRegion, n_steps: int) -> None:
        for _ in range(50):
            cell = tuple(self.rng.choice(list(ax)) for ax in self.axes)
```

```
            if cell not in set(self.cells):
                break
        y = self._observe(cell, n_steps=n_steps)
        reg.center       = cell
        reg.length       = 0.5
        reg.success      = 0
        reg.failure      = 0
        reg.best_value   = y
        reg.rounds_used += 1

    def run(self, n_init: int = 20, budget: int = 100,
            n_regions: int = 4, n_steps: int = 200,
            n_candidates: int = 2000, shrink_after: int = 3,
            progress: bool = True) -> dict[str, Any]:

        t0 = time.time()
        if progress:
            print(f"\n  TRUST-REGION ACTIVE LEARNING — {n_regions} region
                  f"budget={budget}")

        self._seed_regions(n_regions, n_init, n_steps)
        if progress:
            for i, r in enumerate(self.regions):
                print(f"    region {i}: center={r.center} "
                      f"best={r.best_value:.2f} length={r.length:.2f}")

        observed = set(self.cells)
        for rnd in range(n_init + 1, budget + 1):
            self._fit_gp()
            reg = self.regions[(rnd - n_init - 1) % len(self.regions)]

            pool_all = list(reg.cells(self.axes))
            candidates = [c for c in pool_all if c not in observed]

            if len(candidates) < 5:
                self._restart_region(reg, n_steps=n_steps)
                observed = set(self.cells)
                continue
            if len(candidates) > n_candidates:
                candidates = self.rng.sample(candidates, n_candidates)

            X_pool = self.encoder.encode_many(candidates)
            mu, var = self.gp.predict(X_pool)
            pred_pool = np.array([self._surrogate_predict(c) for c in can
            pred_mu = pred_pool + mu
            pred_sd = np.sqrt(var)
            ei = expected_improvement(pred_mu, pred_sd, best=reg.best_val
            idx = int(np.argmax(ei))
            chosen = candidates[idx]

            y_real = self._observe(chosen, n_steps=n_steps)
            observed.add(chosen)
            reg.rounds_used += 1
```

```
            if y_real < reg.best_value - 1e-6:
                reg.update_on_success(y_real)
                if y_real < self.best_value:
                    self.best_value = y_real
                    self.best_cell  = chosen
            else:
                reg.update_on_failure(shrink_after=shrink_after)
                if reg.needs_restart():
                    self._restart_region(reg, n_steps=n_steps)
                    observed = set(self.cells)

            if progress and (rnd % max(1, budget // 10) == 0 or rnd == bu
                rmse = math.sqrt(statistics.fmean(r ** 2 for r in self.y_
                reg_sizes = [r.size(self.axes) for r in self.regions]
                print(f"    round {rnd:>4}/{budget}  RMSE={rmse:>6.3f}  "
                      f"best={self.best_value:>7.2f}  regions={reg_sizes}

            self.region_log.append({
                "round":         rnd,
                "chosen":        chosen,
                "region_id":     self.regions.index(reg),
                "region_length": round(reg.length, 3),
                "region_size":   reg.size(self.axes),
                "residual_rmse": round(math.sqrt(statistics.fmean(
                    r ** 2 for r in self.y_resid)), 3),
                "best_value":    round(self.best_value, 2),
            })

        self._fit_gp()
        return {
            "n_sims":         len(self.cells),
            "n_regions":      n_regions,
            "best_cell":      self.best_cell,
            "best_value":     round(self.best_value, 2),
            "residual_rmse":  round(math.sqrt(statistics.fmean(
                r ** 2 for r in self.y_resid)), 3),
            "residual_mae":   round(statistics.fmean(
                abs(r) for r in self.y_resid), 3),
            "elapsed_s":      round(time.time() - t0, 2),
        }


class GlobalEILearner(BayesianActiveLearner):
    """Vanilla active learning — EI over the full grid (no regions)."""

    def __init__(self, surrogate: Surrogate, seed: int = 0) -> None:
        super().__init__(surrogate, seed=seed)
        (self.controllers,
         self.cf_models,
         self.scenarios,
         self.naturals,
         self.demands) = canonical_axes()
        self.axes = (self.controllers, self.cf_models,
                     self.scenarios, self.naturals, self.demands)
```

```
        self.encoder = Encoder(*self.axes)
        self.gp = SurrogateGP(self.encoder.n_features)


# ============================================================================
#  SECTION 15  —  EQUATION AUDIT  (8 representative equations)
# ============================================================================

@dataclass
class AuditCfg:
    L: float = LOGISTIC_L
    k: float = LOGISTIC_K
    x0: float = LOGISTIC_X0
    tau_min: float = 0.0
    tau_max: float = 600.0
    mu_tau: float = math.log(100.0)
    sigma_tau: float = 0.5
    lambda_decay: float = 0.01
    T_sim: float = 60.0
    n_agents: int = 9


def _trapz(y: np.ndarray, x: np.ndarray) -> float:
    y = np.asarray(y, float); x = np.asarray(x, float)
    return float(np.sum(0.5 * (y[1:] + y[:-1]) * np.diff(x)))


def _is_below(v: float, tn: int, td: int) -> bool:
    return abs(float(v)) < (tn / td)


def eq_I1(cfg: AuditCfg) -> dict[str, Any]:
    """Waiting-time integral consistency (trapezoid vs re-trapezoid)."""
    T = cfg.T_sim; N = 3000; t = np.linspace(0, T, N)
    v = np.maximum(5 - 3*np.sin(0.5*t) + 0.5*np.random.randn(N), 0)
    for f in [0.15, 0.45, 0.80]:
        s = round(N*f); a = max(0, s-20); b = min(N, s+20); v[a:b] = 0
    ind = (v < 0.1).astype(float)
    Wt = _trapz(ind, t); Ws = _trapz(ind, t)
    rel = abs(Wt - Ws) / max(abs(Wt), 1e-12)
    return {"name": "I1", "rel": rel, "ok": _is_below(rel, 5, 100),
            "summary": f"trap={Wt:.4f} rel={rel*100:.4f}%"}


def eq_I3(cfg: AuditCfg) -> dict[str, Any]:
    """Travel-time 95% CI bounds the bias."""
    N = 1000
    ta = 10*np.random.rand(N)
    tr = np.maximum(60 + 15*np.random.randn(N), 10)
    td = ta + tr
    Ta = np.mean(td - ta); Tci = 1.96*np.std(td - ta, ddof=1)/np.sqrt(N)
    rel = abs(Ta - np.mean(tr)) / max(Tci, 1e-12)
    return {"name": "I3", "rel": rel, "ok": _is_below(rel, 5, 1),
            "summary": f"T={Ta:.2f}s CI+/-{Tci:.2f}"}
```

```python
def eq_I5(cfg: AuditCfg) -> dict[str, Any]:
    """AoI kernel integral = 1/lambda."""
    lam = cfg.lambda_decay
    tau = np.linspace(0, 1000, 20001)
    In = _trapz(np.exp(-lam*tau), tau); Ia = 1.0/lam
    rel = abs(In - Ia) / Ia
    return {"name": "I5", "rel": rel, "ok": _is_below(rel, 1, 100),
            "summary": f"int={In:.4f} 1/lambda={Ia:.4f}"}


def eq_P1(cfg: AuditCfg) -> dict[str, Any]:
    """Truncated lognormal mean: empirical vs analytic integral."""
    mu, sg = cfg.mu_tau, cfg.sigma_tau
    lo, hi = cfg.tau_min, cfg.tau_max
    Flo = stats.lognorm.cdf(lo, s=sg, scale=math.exp(mu))
    Fhi = stats.lognorm.cdf(hi, s=sg, scale=math.exp(mu))
    Z = Fhi - Flo
    if Z < 1e-12:
        return {"name": "P1", "rel": 0, "ok": False, "summary": "SKIP"}
    mean_int, _ = quad(
        lambda t: t * stats.lognorm.pdf(t, s=sg, scale=math.exp(mu)) / Z,
        lo, hi)
    tau = stats.lognorm.ppf(Flo + Z*np.random.rand(50000),
                            s=sg, scale=math.exp(mu))
    rel = abs(float(np.mean(tau)) - mean_int) / max(mean_int, 1e-12)
    return {"name": "P1", "rel": rel, "ok": _is_below(rel, 5, 100),
            "summary": f"emp={np.mean(tau):.2f} int={mean_int:.2f}"}


def eq_P5(cfg: AuditCfg) -> dict[str, Any]:
    """Logistic derivative at x0 = L*k/4; f(x0) = L/2."""
    L, k, x0 = cfg.L, cfg.k, cfg.x0
    f = lambda x: L / (1.0 + math.exp(-k * (x - x0)))
    h = 1e-4
    dn = (f(x0+h) - f(x0-h)) / (2*h)
    da = L * k / 4.0
    rel = abs(dn - da) / abs(da)
    sanity = abs(f(x0) - L/2.0) < 1e-9
    return {"name": "P5", "rel": rel, "ok": _is_below(rel, 1, 1_000_000)
            "summary": f"d_num={dn:.6f} d_ana={da:.6f}  f(x0)={f(x0):.4f}


def eq_A1(cfg: AuditCfg) -> dict[str, Any]:
    """Effective AoI = E[tau_tx] + E[tau_queue] + tau_prop."""
    N = 50000
    ttx = stats.lognorm.rvs(s=cfg.sigma_tau,
                            scale=math.exp(cfg.mu_tau), size=N)
    tq = -np.log(np.random.rand(N)) / 0.02
    tp = 100 / (3e8 / 1000)
    D = ttx + tq + tp; em = float(np.mean(D))
    Z = stats.lognorm.cdf(cfg.tau_max, s=cfg.sigma_tau,
                          scale=math.exp(cfg.mu_tau)) \
```

```
        - stats.lognorm.cdf(cfg.tau_min, s=cfg.sigma_tau,
                            scale=math.exp(cfg.mu_tau))
    Etx, _ = quad(
        lambda t: t * stats.lognorm.pdf(t, s=cfg.sigma_tau,
                                        scale=math.exp(cfg.mu_tau)) / Z
        cfg.tau_min, cfg.tau_max)
    am = Etx + 1/0.02 + tp
    rel = abs(em - am) / am
    return {"name": "A1", "rel": rel, "ok": _is_below(rel, 5, 100),
            "summary": f"emp={em:.2f} ana={am:.2f}"}


def eq_A8(cfg: AuditCfg) -> dict[str, Any]:
    """Bellman expectation V = (I - gamma*P)^-1 R."""
    n = 3
    P = np.array([[0.7, 0.2, 0.1], [0.1, 0.8, 0.1], [0.2, 0.2, 0.6]])
    R = np.array([1.0, 2.0, 0.5]); g = 0.9
    V1 = np.linalg.solve(np.eye(n) - g*P, R)
    V2 = np.zeros(n)
    for _ in range(5000):
        V2 = R + g*P @ V2
    err = float(np.max(np.abs(V1 - V2)))
    return {"name": "A8", "rel": err, "ok": _is_below(err, 5, 100),
            "summary": f"V=[{V1[0]:.3f} {V1[1]:.3f} {V1[2]:.3f}] err={err

def eq_B9(cfg: AuditCfg) -> dict[str, Any]:
    """Markov chain mixing time bound: t_mix <= log(1/eps)/(1-lambda2)."""
    P = np.array([[0.7, 0.2, 0.1], [0.1, 0.8, 0.1], [0.2, 0.2, 0.6]])
    n = P.shape[0]
    eigs = np.sort(np.abs(np.linalg.eigvals(P)))[::-1]
    lambda2 = float(eigs[1]); eps = 0.01
    t_bound = int(math.ceil(math.log(1/eps) / (1 - lambda2)))
    A_mat = np.vstack([P.T - np.eye(n), np.ones((1, n))])
    b_vec = np.concatenate([np.zeros(n), [1]])
    pi = np.linalg.lstsq(A_mat, b_vec, rcond=None)[0]
    Pi_mat = np.ones((n, 1)) * pi
    emp_t = -1; Ppow = np.eye(n)
    for tt in range(1, 501):
        Ppow = Ppow @ P
        if np.max(np.abs(Ppow - Pi_mat)) < eps:
            emp_t = tt; break
    if emp_t < 0:
        emp_t = 500
    return {"name": "B9", "rel": emp_t/t_bound,
            "ok": emp_t <= t_bound,
            "summary": f"lambda2={lambda2:.4f} t<={t_bound} emp={emp_t}"}


AUDIT_FUNCS: list[Callable[[AuditCfg], dict[str, Any]]] = [
    eq_I1, eq_I3, eq_I5, eq_P1, eq_P5, eq_A1, eq_A8, eq_B9,
]
```

```python
# ============================================================================
#  SECTION 16  —  UNIFIED EVALUATION DRIVER
# ============================================================================

def run_equation_audit() -> dict[str, dict[str, Any]]:
    print("\n" + "=" * 88)
    print("  PART A  —  EQUATION AUDIT  (.m logistic + .txt representativ
    print("=" * 88)
    cfg = AuditCfg()
    results: dict[str, dict[str, Any]] = {}
    for fn in AUDIT_FUNCS:
        r = fn(cfg); results[r["name"]] = r
        mark = "PASS" if r["ok"] else "FAIL"
        print(f"  [{mark}] {r['name']:<4}  {r['summary']}")
    n_pass = sum(1 for r in results.values() if r["ok"])
    print(f"\n  Audit: {n_pass}/{len(results)} passed")
    return results


def run_python_simulation() -> tuple[list[dict[str, Any]],
                                     list[str], list[str], list[str]]:
    print("\n" + "=" * 88)
    print("  PART B  —  PYTHON SIMULATION  (8 ctrl x 4 S x 5 nat x 5 seed
    print("=" * 88)
    controllers = ["fixed-time", "adaptive", "coordinated",
                   "cooperative-marl", "resilient-marl",
                   "max-pressure", "mpc", "weather-adaptive"]
    scenarios = ["S1", "S2", "S3", "S4"]
    naturals  = ["N-clear", "N-rain", "N-snow",
                 "N-accident", "N-rush-am"]
    seeds = (1, 2, 3, 4, 5)
    n_steps = 80
    total = len(controllers) * len(scenarios) * len(naturals)
    print(f"  Total cells = {total}  ({total * len(seeds)} runs)")
    rows: list[dict[str, Any]] = []
    for i, c in enumerate(controllers, 1):
        for sc in scenarios:
            for nat in naturals:
                rows.append(run_multi_seed(c, sc, nat, seeds=seeds,
                                           n_steps=n_steps,
                                           base_demand_ns=0.30,
                                           base_demand_ew=0.30))
        print(f"  done {c:<22} ({i}/{len(controllers)})")
    return rows, controllers, scenarios, naturals


def cross_check_logistic(rows: list[dict[str, Any]]) -> None:
    """Cross-check: does simulated travel time follow the .m logistic cur
    print("\n" + "=" * 88)
    print("  PART C  —  CROSS-CHECK  (.m logistic curve vs Python-simulat
    print("=" * 88)
    print(f"  Logistic coefficients (.m):  L={LOGISTIC_L}  "
          f"k={LOGISTIC_K}  x0={LOGISTIC_X0}")
    print()
```

```
    print(f"  {'Scen':<6}{'Lat(ms)':>8}{'Loss':>6}{'rho':>8}"
          f"{'TT_sim':>10}{'TT_logistic':>13}{'delta%':>9}")
    print("  " + "-" * 70)
    by_sc: dict[str, list[float]] = defaultdict(list)
    for r in rows:
        if r["natural"] == "N-clear":
            by_sc[r["scenario"]].append(r["travel_time_s_mean"])
    for sc in ("S1", "S2", "S3", "S4"):
        cfg_c = COMMUNICATION_SCENARIOS[sc]
        lat = float(cfg_c["latency_ms"])
        loss = float(cfg_c["loss_rate"])
        rho = severity_index(lat, loss)
        tt_sim = float(np.mean(by_sc[sc])) if by_sc[sc] else 0.0
        tt_log = logistic_curve(lat) if lat > 0 \
                 else LOGISTIC_L / (1.0 + math.exp(LOGISTIC_K * LOGISTIC_
        delta = (tt_sim - tt_log) / max(tt_log, 1e-9) * 100
        print(f"  {sc:<6}{lat:>8.1f}{loss:>6.2f}{rho:>8.3f}"
              f"{tt_sim:>10.2f}{tt_log:>13.2f}{delta:>8.1f}%")
    print()


def make_unified_plots(rows: list[dict[str, Any]]) -> str:
    """Save a 4-panel diagnostic figure."""
    fig, axs = plt.subplots(2, 2, figsize=(14, 10))

    # (1) .m file: logistic + quadratic curves
    x = np.linspace(0, 600, 1000)
    axs[0, 0].plot(x, logistic_curve_vec(x), "b-", lw=2.5,
                   label=f"Logistic: L={LOGISTIC_L}, k={LOGISTIC_K}, x0={
    axs[0, 0].plot(x, QUAD_A * x**2 + QUAD_B * x + QUAD_C, "g--", lw=2,
                   label=f"Quadratic: a={QUAD_A:.2e}, b={QUAD_B:.3f}, c={
    axs[0, 0].axvline(LOGISTIC_X0, color="r", ls=":", lw=1,
                      label=f"x0 = {LOGISTIC_X0}")
    axs[0, 0].set_xlabel("Latency (ms)")
    axs[0, 0].set_ylabel("f(x)")
    axs[0, 0].set_title(".m file: Logistic vs Quadratic fit")
    axs[0, 0].legend(loc="best")
    axs[0, 0].grid(alpha=0.3)

    # (2) Simulated travel time per scenario (mean over controllers)
    by_sc: dict[str, list[float]] = defaultdict(list)
    for r in rows:
        if r["natural"] == "N-clear":
            by_sc[r["scenario"]].append(r["travel_time_s_mean"])
    scenarios = sorted(by_sc.keys())
    means = [float(np.mean(by_sc[s])) for s in scenarios]
    axs[0, 1].bar(scenarios, means, color="#3b82f6")
    axs[0, 1].set_xlabel("Communication scenario")
    axs[0, 1].set_ylabel("Travel time (s)")
    axs[0, 1].set_title("Python sim: Travel time x S1->S4 (N-clear)")
    axs[0, 1].grid(alpha=0.3, axis="y")

    # (3) Throughput per controller (N-clear, mean over S)
    by_ctrl: dict[str, list[float]] = defaultdict(list)
```

```
    for r in rows:
        if r["natural"] == "N-clear":
            by_ctrl[r["algorithm_type"]].append(r["throughput_vph_mean"])
    ctrls = sorted(by_ctrl.keys())
    thr = [float(np.mean(by_ctrl[c])) for c in ctrls]
    axs[1, 0].barh(ctrls, thr, color="#10b981")
    axs[1, 0].set_xlabel("Throughput (vph)")
    axs[1, 0].set_ylabel("Controller")
    axs[1, 0].set_title("Python sim: Throughput per controller (N-clear)"
    axs[1, 0].grid(alpha=0.3, axis="x")

    # (4) Cross-check: sim TT vs .m logistic curve
    xs = [float(COMMUNICATION_SCENARIOS[s]["latency_ms"]) for s in scenar
    sim_tt = means
    log_tt = [logistic_curve(x) if x > 0
              else LOGISTIC_L / (1 + math.exp(LOGISTIC_K * LOGISTIC_X0))
              for x in xs]
    axs[1, 1].plot(xs, sim_tt, "o-", lw=2, label="Python-simulated TT")
    axs[1, 1].plot(xs, log_tt, "s--", lw=2, label=".m logistic curve")
    axs[1, 1].set_xlabel("Latency (ms)")
    axs[1, 1].set_ylabel("Travel time (s)")
    axs[1, 1].set_title("Cross-check: sim vs .m logistic")
    axs[1, 1].legend()
    axs[1, 1].grid(alpha=0.3)

    plt.suptitle("Unified Evaluation - .m equation x .py simulator",
                 fontweight="bold", fontsize=14)
    plt.tight_layout()
    out = "unified_evaluation.png"
    plt.savefig(out, dpi=120, bbox_inches="tight")
    plt.close()
    print(f"  Saved plot -> {out}")
    return out


def final_report(audit_results: dict[str, dict[str, Any]],
                 rows: list[dict[str, Any]]) -> None:
    print("\n" + "=" * 88)
    print("  FINAL UNIFIED REPORT")
    print("=" * 88)
    n_pass = sum(1 for r in audit_results.values() if r["ok"])
    print(f"\n  Equation audit        : {n_pass}/{len(audit_results)} pas
    print(f"  Simulation cells run   : {len(rows)}  (x5 seeds each)")
    print(f"  Controllers exercised  : {len({r['algorithm_type'] for r in
    print(f"  Scenarios exercised    : {len({r['scenario'] for r in rows}
    print(f"  Natural conditions     : {len({r['natural'] for r in rows})
    print("\n  Best controller per scenario (N-clear, lowest travel_time)
    for sc in ("S1", "S2", "S3", "S4"):
        cands = [r for r in rows
                 if r["scenario"] == sc and r["natural"] == "N-clear"]
        if not cands:
            continue
        best = min(cands, key=lambda r: r["travel_time_s_mean"])
        print(f"    {sc}: {best['algorithm_type']:<22} "
```

```
            f"TT={best['travel_time_s_mean']:.2f}s  "
            f"Thr={best['throughput_vph_mean']:.0f}vph  "
            f"CI+/-{best['travel_time_s_ci95']:.2f}")
    print()


# ============================================================================
#  SECTION 17  —  CLI  +  MAIN
# ============================================================================

def _parse_args() -> argparse.Namespace:
    p = argparse.ArgumentParser(
        description="Unified traffic-signal evaluation (single Colab cell
        formatter_class=argparse.ArgumentDefaultsHelpFormatter,
    )
    p.add_argument("--skip-sim",  action="store_true",
                   help="skip Part B (Python simulation) for fast CI")
    p.add_argument("--skip-audit", action="store_true",
                   help="skip Part A (equation audit)")
    p.add_argument("--skip-synth", action="store_true",
                   help="skip the synthetic dataset build")
    p.add_argument("--skip-trust", action="store_true",
                   help="skip the trust-region learner")
    p.add_argument("--synth-preset", choices=tuple(PRESETS), default="qui
                   help="synthetic-dataset preset (quick for speed)")
    p.add_argument("--trust-budget", type=int, default=30,
                   help="TuRBO simulation budget (>= 10)")
    p.add_argument("--trust-init",   type=int, default=10,
                   help="TuRBO initial random observations")
    p.add_argument("--trust-regions", type=int, default=2,
                   help="number of TuRBO trust regions")
    return p.parse_args([])  # Colab: no argv


def main() -> int:
    args = _parse_args()
    print("#" * 88)
    print("#  UNIFIED TRAFFIC EVALUATION  —  .m eq x .py sim x TuRBO AL")
    print(f"#  Logistic: L={LOGISTIC_L}  k={LOGISTIC_K}  x0={LOGISTIC_X0}
    print("#" * 88)

    audit_results: dict[str, dict[str, Any]] = {}
    if not args.skip_audit:
        audit_results = run_equation_audit()              # PART A

    rows: list[dict[str, Any]] = []
    if not args.skip_sim:
        rows, _, _, _ = run_python_simulation()           # PART B
        cross_check_logistic(rows)                        # PART C

    if rows:
        plot_path = make_unified_plots(rows)              # PART D
        print(f"  Plot: {plot_path}")
```

```
    # --- PART E : synthetic dataset + tensor pipeline + TuRBO ----------
    if not args.skip_synth:
        print("\n" + "=" * 88)
        print("  PART E1  —  SYNTHETIC DATASET")
        print("=" * 88)
        ds = SyntheticTrafficDataset(preset=args.synth_preset,
                                     verbose=True, auto_build=True)
        ds.summary()
        npy_path = ds.to_npy()
        print(f"  Tensor -> {npy_path}  shape={ds.as_tensor().shape}")

        print("\n  PART E2  —  TENSOR PIPELINE (Ridge + kNN + GP + EI AL)")
        tp = TensorPipeline(path=npy_path, verbose=True)
        tp.summary()
        tp.run(budget=50, n_init=10)

    if not args.skip_trust:
        print("\n" + "=" * 88)
        print("  PART E3  —  TRUST-REGION ACTIVE LEARNING (TuRBO)")
        print("=" * 88)
        surr = Surrogate(seed=42)
        trl = TrustRegionLearner(surr, seed=42)
        report = trl.run(n_init=args.trust_init, budget=args.trust_budget
                         n_regions=args.trust_regions, n_steps=80,
                         progress=True)
        print(f"\n  TuRBO report: {report}")
        hold = trl.evaluate_holdout(n_holdout=10, n_steps=80)
        print(f"  Holdout: raw MAE={hold['raw_mae']}  "
              f"corrected MAE={hold['corr_mae']}")

    # --- FINAL ---------------------------------------------------------
    if audit_results or rows:
        final_report(audit_results, rows)
    print("\n" + "=" * 88)
    print("  Unified evaluation complete")
    print("=" * 88)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

```
[boot] registered 17 controllers: ['actuated', 'adaptive', 'cooperative-coordinated',
perative-marl', 'coordinated'] ...
################################################################################
#  UNIFIED TRAFFIC EVALUATION  —  .m eq x .py sim x TuRBO AL
#  Logistic: L=302.2328  k=0.0143  x0=101.5374
################################################################################

================================================================================
  PART A  —  EQUATION AUDIT  (.m logistic + .txt representative eqs)
================================================================================
  [PASS] I1    trap=2.4008 rel=0.0000%
  [PASS] I3    T=59.10s CI+/-0.92
  [PASS] I5    int=99.9955 1/lambda=100.0000
  [PASS] P1    emp=113.11 int=113.22
  [PASS] P5    d_num=1.080482 d_ana=1.080482  f(x0)=151.1164
  [PASS] A1    emp=163.19 ana=163.22
  [PASS] A8    V=[13.095 15.269 12.186] err=1.78e-14
  [PASS] B9    lambda2=0.6000 t<=12 emp=8

  Audit: 8/8 passed

================================================================================
  PART B  —  PYTHON SIMULATION  (8 ctrl x 4 S x 5 nat x 5 seeds)
================================================================================
  Total cells = 160  (800 runs)
  done fixed-time             (1/8)
  done adaptive               (2/8)
  done coordinated            (3/8)
  done cooperative-marl       (4/8)
  done resilient-marl         (5/8)
  done max-pressure           (6/8)
  done mpc                    (7/8)
  done weather-adaptive       (8/8)

================================================================================
  PART C  —  CROSS-CHECK  (.m logistic curve vs Python-simulated TT)
================================================================================
  Logistic coefficients (.m):  L=302.2328  k=0.0143  x0=101.5374

  Scen   Lat(ms)  Loss     rho    TT_sim  TT_logistic   delta%
  ------------------------------------------------------------------
  S1         0.0  0.00   0.000  58102.84        57.33101243.7%
  S2       100.0  0.05   0.108  58102.65       149.46 38776.3%
  S3       300.0  0.15   0.325  58090.59       285.52 20245.7%
  S4       600.0  0.30   0.650  58066.38       301.99 19127.9%

  Saved plot -> unified_evaluation.png
  Plot: unified_evaluation.png

================================================================================
  PART E1  —  SYNTHETIC DATASET
================================================================================
================================================================================
  SYNTHETIC DATASET  -  preset='quick'
================================================================================
```

```
  shape          : (16, 8, 4)
  total cells    : 512
  axis 0 (ctrl)  : 16
  axis 1 (scen)  : 8
  axis 2 (cond)  : 4
  seed           : 42
  car-following  : SAFE
  noise frac     : 0.06

  metric                   mean     stdev       min       max
  ------------------------------------------------------------
  travel_time_s         366.289   177.333   140.112  1458.922
  queue_veh              26.073    14.426     6.146   120.640
  waiting_time_s         43.301    24.081    11.290   200.418
  throughput_vph        394.996   134.763    96.879   717.050
  mean_aoi_ms           116.189    38.294    44.011   202.215
  delivery_ratio          0.841     0.073     0.683     1.000
  K_ctrl                  1.260     0.000     1.260     1.260
  K_comm                  1.302     0.294     1.000     1.600
  K_nat                   1.403     0.342     1.000     1.872
  K_dem                   1.066     0.006     1.057     1.072
  severity                0.158     0.075     0.005     0.305
  true_value            353.182   147.713   155.004   653.838
  wrote tensor (16, 8, 4, 12) -> synthetic_tensor.npy
  Tensor -> synthetic_tensor.npy  shape=(16, 8, 4, 12)

  PART E2  —  TENSOR PIPELINE (Ridge + kNN + GP + EI AL)
  loaded synthetic_tensor.npy  shape=(16, 8, 4, 12)  features=10  rows=512
================================================================================
  TENSOR PIPELINE - LOADED
================================================================================
  file           : synthetic_tensor.npy
  grid shape     : (16, 8, 4)
  flat rows      : 512
  features       : 10
  target         : 'travel_time_s'
  target mean    : 366.289
  target stdev   : 177.333
  target range   : [140.112, 1458.922]
================================================================================
  PIPELINE REPORT  -  target = 'travel_time_s'
================================================================================
  train / test   : 410 / 102
  feature count  : 10

  model                     MAE      RMSE        R2
  ------------------------------------------------
  mean                 118.4773  147.7644   -0.0382
  ridge                  5.8084    7.7181    0.9972
  knn5                   9.3931   14.3948    0.9901
  ridge+gp               1.9865    2.5566    0.9997

  Active-learning trace (EI loop)
   round       MAE      RMSE        R2
  -----------------------------------
```

```
   1.0   92.8536  139.3391    0.0768
   2.0   87.7665  132.1573    0.1696
   3.0   80.8658  122.2923    0.2889
   4.0   75.2150  113.7679    0.3846
   5.0   71.4235  107.9980    0.4454
   6.0   67.1505  101.8364    0.5069
   7.0   63.6033   96.2769    0.5593
   8.0   60.5316   91.5507    0.6015
   9.0   58.8897   89.0880    0.6226
  10.0   57.5209   86.9597    0.6404
  11.0   55.5092   83.9055    0.6653
  12.0   54.4384   82.3006    0.6779
  13.0   53.2073   80.4143    0.6925
  14.0   52.5644   79.4622    0.6998
  15.0   51.5089   77.7993    0.7122
  16.0   50.7609   76.6600    0.7206
  17.0   49.9011   75.3613    0.7300
  18.0   49.5144   74.7716    0.7342
  19.0   49.0878   74.1157    0.7388
  20.0   48.6182   73.2614    0.7448
  21.0   47.6190   71.6250    0.7561
  22.0   45.8779   68.8498    0.7746
  23.0   46.0474   68.7405    0.7753
  24.0   45.5627   68.0094    0.7801
  25.0   45.2092   67.4650    0.7836
  26.0   44.7107   66.6746    0.7886
  27.0   43.3886   64.7922    0.8004
  28.0   43.2759   64.4254    0.8026
  29.0   42.7340   63.5541    0.8079
  30.0   42.4790   63.1537    0.8104
  31.0   42.0737   62.4822    0.8144
  32.0   42.2032   62.6104    0.8136
  33.0   38.0181   55.8935    0.8515
  34.0   36.6873   53.8878    0.8619
  35.0   36.4235   53.3789    0.8645
  36.0   36.3318   53.2505    0.8652
  37.0   35.7269   52.2415    0.8702
  38.0   35.6144   52.0598    0.8711
  39.0   35.5572   51.9852    0.8715
  40.0   35.1983   51.4250    0.8743
  41.0   35.1331   51.2726    0.8750
  42.0   34.9964   50.9929    0.8764
  43.0   34.8497   50.7228    0.8777
  44.0   26.6023   38.1742    0.9307
  45.0   21.0948   29.8633    0.9576
  46.0   17.7376   24.9381    0.9704
  47.0   16.2855   22.8595    0.9752
  48.0   15.8837   22.2794    0.9764
  49.0   15.3878   21.5969    0.9778
  50.0   14.4735   20.3304    0.9803

================================================================================
  PART E3  —  TRUST-REGION ACTIVE LEARNING (TuRBO)
================================================================================
```

```
  TRUST-REGION ACTIVE LEARNING — 2 regions, budget=30
    region 0: center=('fixed-time', 'SAFE', 'S4', 'N-rush-pm', 'D2') best=7068.87 len
h=0.50
    region 1: center=('optimized', 'IDM', 'S4', 'N-workzone', 'D4') best=49138.86 len
h=0.50
    round   12/30  RMSE=63804.569  best=5382.41  regions=[18564, 18564]
    round   15/30  RMSE=57143.483  best=5382.41  regions=[18564, 18564]
    round   18/30  RMSE=52191.160  best=2743.93  regions=[18564, 18564]
    round   21/30  RMSE=49663.838  best=2743.93  regions=[18564, 18564]
    round   24/30  RMSE=46483.201  best=2743.93  regions=[18564, 1008]
    round   27/30  RMSE=43855.717  best=2743.93  regions=[18564, 1008]
    round   30/30  RMSE=41631.692  best=2743.93  regions=[18564, 40]

  TuRBO report: {'n_sims': 30, 'n_regions': 2, 'best_cell': ('fixed-time', 'SAFE', 'S
'N-sensor-fail', 'D5'), 'best_value': 2743.93, 'residual_rmse': 41631.692, 'residual_
26467.21, 'elapsed_s': 3.94}
  Holdout: raw MAE=55272.931  corrected MAE=33563.823

================================================================================
  FINAL UNIFIED REPORT
================================================================================

  Equation audit        : 8/8 passed
  Simulation cells run   : 160  (x5 seeds each)
  Controllers exercised  : 8
  Scenarios exercised    : 4
  Natural conditions     : 5

  Best controller per scenario (N-clear, lowest travel_time):
    S1: fixed-time             TT=6873.90s  Thr=22808vph  CI+/-0.26
    S2: fixed-time             TT=6873.71s  Thr=22808vph  CI+/-0.19
    S3: fixed-time             TT=6873.71s  Thr=22808vph  CI+/-0.19
    S4: fixed-time             TT=6873.60s  Thr=22808vph  CI+/-0.05


================================================================================
  Unified evaluation complete
================================================================================
```

```
An exception has occurred, use %tb to see the full traceback.

SystemExit: 0
```

In [ ]:

```
#!/usr/bin/env python3
# ============================================================================
#  unified_traffic_eval.py  —  single-cell Colab script (KEYERROR-FIXED)
# ----------------------------------------------------------------------------
#  Combines, in one self-contained file:
#
#    1.  analytic backbone        — logistic + quadratic + severity (from
#    2.  traffic_control_algorithm.py
#                                   — 3x3 grid + 15 controllers + multi-s
#    3.  synthetic_dataset.py     — (128,64,12) closed-form tensor
#    4.  tensor_pipeline.py       — Ridge + kNN + GP + EI active-learning
#                                   (with the _ridge_fit placeholder bug
```

```
#    5.  analytical_surrogate.py  — provided inline (was missing)
#    6.  bayesian_active_learning.py — provided inline (was missing)
#    7.  trust_region_active_learning.py — TuRBO over the 5-axis grid
#    8.  equation audit           — 8 representative equations
#    9.  unified evaluation driver — runs A..E and prints the final repor
#
#  Bug fixes vs the originals:
#    • _ControllerMeta metaclass AUTO-REGISTERS every @controller-decorat
#      method the moment the class body finishes executing — eliminates t
#      KeyError: 'cooperative-marl' that occurred when _register_composed
#      controllers() ran before the base controllers had been registered.
#    • _ridge_fit() now returns the actual coefficient vector (not zeros+
#    • Active-learning EI uses scipy.special.erf (vectorised) instead of
#      np.vectorize(math.erf) (≈100x faster)
#    • SyntheticTrafficDataset fields cleaned — no orphan _arrays default
#    • ImpairmentChannel._latency() return value is now used (assigned to
#      msg.latency_ms so AoI is correctly accumulated)
#    • TrustRegion.update_on_failure no longer resets success counter
#
#  Run in Colab:
#      !pip install scipy numpy matplotlib    # all pre-installed
#      # paste this entire file as one cell, then run
# ======================================================================

from __future__ import annotations

import argparse
import csv
import json
import logging
import math
import os
import random
import statistics
import time
import warnings
from collections import defaultdict, deque
from dataclasses import dataclass, field
from fractions import Fraction
from itertools import product
from pathlib import Path
from typing import Any, Callable, Iterable, Iterator, Optional, Sequence

import numpy as np
from scipy import stats
from scipy.integrate import quad
from scipy.special import erf as _scipy_erf  # vectorised erf

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

warnings.filterwarnings("ignore")
np.random.seed(42)
```

```python
logger = logging.getLogger("unified_traffic_eval")
logger.setLevel(logging.INFO)
if not logger.handlers:
    h = logging.StreamHandler()
    h.setFormatter(logging.Formatter("%(asctime)s  %(levelname)-7s  %(mes
    logger.addHandler(h)


# ============================================================================
#  SECTION 0  —  CONFIG / GLOBAL CONSTANTS
# ============================================================================

# --- logistic degradation: y = L / (1 + exp(-k*(x - x0)))  — from .m fil
LOGISTIC_L:   float = 302.2328
LOGISTIC_K:   float = 0.0143
LOGISTIC_X0:  float = 101.5374

# --- quadratic fit:  y = a*x^2 + b*x + c                    — from .m f
QUAD_A: float = -0.0005518667521387619
QUAD_B: float =  0.7129391105869533
QUAD_C: float =  93.5322316774016

# --- signal timing ----------------------------------------------------
DECISION_INTERVAL_S: float = 10.0
GREEN_MIN_S:         float = 10.0
GREEN_MAX_S:         float = 60.0
YELLOW_S:            float =  3.0
DEFAULT_BETA:        float =  0.5

# --- network topology (§4.2) ------------------------------------------
GRID_ROWS: int = 3
GRID_COLS: int = 3
SEGMENT_LENGTH_M:   float = 300.0
FREE_FLOW_SPEED_MS: float = 50.0 * 1000 / 3600.0
SATURATION_FLOW_VPS: float = 0.5  # veh/s per green approach

# --- car-following benchmark (Table 12) -------------------------------
CAR_FOLLOWING_BENCHMARK: dict[str, dict[str, dict[str, float]]] = {
    "GIPPS": {
        "free_flow": {"in_system_s": 114.77, "sd": 0.42, "speed_ms": 18.9
                      "sd_speed": 0.08, "efficiency_pct": 94.3,
                      "delay_s": 6.54,  "sd_delay": 0.11},
        "congested": {"in_system_s": 187.42, "sd": 1.85, "speed_ms": 10.1
                      "sd_speed": 0.12, "efficiency_pct": 78.2,
                      "delay_s": 42.87, "sd_delay": 0.95},
    },
    "IDM": {
        "free_flow": {"in_system_s": 113.19, "sd": 0.38, "speed_ms": 19.2
                      "sd_speed": 0.07, "efficiency_pct": 95.6,
                      "delay_s": 4.96,  "sd_delay": 0.09},
        "congested": {"in_system_s": 178.63, "sd": 1.52, "speed_ms": 10.6
                      "sd_speed": 0.10, "efficiency_pct": 82.1,
                      "delay_s": 36.24, "sd_delay": 0.78},
```

```
    },
    "SAFE": {
        "free_flow": {"in_system_s": 112.39, "sd": 0.35, "speed_ms": 19.3
                      "sd_speed": 0.06, "efficiency_pct": 96.3,
                      "delay_s": 4.16,  "sd_delay": 0.08},
        "congested": {"in_system_s": 171.28, "sd": 1.38, "speed_ms": 11.0
                      "sd_speed": 0.09, "efficiency_pct": 85.6,
                      "delay_s": 29.51, "sd_delay": 0.62},
    },
}
CF_MODELS: tuple[str, ...] = tuple(CAR_FOLLOWING_BENCHMARK)


# ============================================================================
#  SECTION 1  —  ANALYTIC BACKBONE  (from .m file)
# ============================================================================

def logistic_curve(x: float | np.ndarray) -> float | np.ndarray:
    """Logistic degradation: f(x) = L / (1 + exp(-k*(x - x0)))."""
    if isinstance(x, np.ndarray):
        return LOGISTIC_L / (1.0 + np.exp(-LOGISTIC_K * (x - LOGISTIC_X0)
    return LOGISTIC_L / (1.0 + math.exp(-LOGISTIC_K * (x - LOGISTIC_X0)))

def logistic_curve_vec(x: np.ndarray) -> np.ndarray:
    """Vectorised logistic — for plotting."""
    return LOGISTIC_L / (1.0 + np.exp(-LOGISTIC_K * (np.asarray(x) - LOGI

def quadratic_curve(x: float | np.ndarray) -> float | np.ndarray:
    """Quadratic fit y = a*x^2 + b*x + c  (from .m)."""
    if isinstance(x, np.ndarray):
        return QUAD_A * x**2 + QUAD_B * x + QUAD_C
    return QUAD_A * x**2 + QUAD_B * x + QUAD_C

def degrade_logistic(x: float,
                     L: float = LOGISTIC_L,
                     k: float = LOGISTIC_K,
                     x0: float = LOGISTIC_X0) -> float:
    """Logistic degradation law — Eq. (5) of the thesis."""
    return L / (1.0 + math.exp(-k * (x - x0)))

def severity_index(latency_ms: float, loss_rate: float,
                   w_tau: float = 0.5, w_p: float = 0.5,
                   tau_min: float = 0.0, tau_max: float = 600.0) -> float
    """Severity index ρ — Eq. (8)."""
    return (w_tau * (latency_ms - tau_min) / (tau_max - tau_min)
            + w_p * loss_rate)

def aoi_decay_kernel(aoi_ms: float, lam: float = 0.01) -> float:
    """AoI decay kernel κ(τ) = exp(-λτ) — Eq. (12)."""
    return math.exp(-lam * aoi_ms)


# ============================================================================
#  SECTION 2  —  COMMUNICATION + NATURAL SCENARIOS
```

```python
# ============================================================================

COMMUNICATION_SCENARIOS: dict[str, dict[str, float | str]] = {
    "S1": {"latency_ms":   0.0, "loss_rate": 0.00, "label": "Ideal"},
    "S2": {"latency_ms": 100.0, "loss_rate": 0.05, "label": "Moderate"},
    "S3": {"latency_ms": 300.0, "loss_rate": 0.15, "label": "Severe"},
    "S4": {"latency_ms": 600.0, "loss_rate": 0.30, "label": "Extreme"},
}

NATURAL_SCENARIOS: dict[str, dict[str, float | str]] = {
    # ---- weather -----------------------------------------------------
    "N-clear":   {"label": "Clear weather",   "capacity": 1.00,
                  "demand_mult": 1.00, "direction_bias":  0.00,
                  "sensor_reliability": 1.00},
    "N-rain":    {"label": "Heavy rain",      "capacity": 0.85,
                  "demand_mult": 1.00, "direction_bias":  0.00,
                  "sensor_reliability": 0.95},
    "N-snow":    {"label": "Snow / ice",      "capacity": 0.55,
                  "demand_mult": 0.80, "direction_bias":  0.00,
                  "sensor_reliability": 0.90},
    "N-fog":     {"label": "Dense fog",      "capacity": 0.75,
                  "demand_mult": 0.95, "direction_bias":  0.00,
                  "sensor_reliability": 0.85},
    # ---- incident ----------------------------------------------------
    "N-accident":  {"label": "Accident",        "capacity": 0.65,
                    "demand_mult": 1.00, "direction_bias":  0.20,
                    "sensor_reliability": 1.00},
    "N-workzone":  {"label": "Work zone",       "capacity": 0.70,
                    "demand_mult": 1.00, "direction_bias":  0.10,
                    "sensor_reliability": 1.00},
    # ---- demand ------------------------------------------------------
    "N-rush-am":   {"label": "Morning rush",   "capacity": 1.00,
                    "demand_mult": 1.50, "direction_bias":  0.60,
                    "sensor_reliability": 1.00},
    "N-rush-pm":   {"label": "Evening rush",   "capacity": 1.00,
                    "demand_mult": 1.50, "direction_bias": -0.60,
                    "sensor_reliability": 1.00},
    "N-night":     {"label": "Overnight flow", "capacity": 1.00,
                    "demand_mult": 0.25, "direction_bias":  0.00,
                    "sensor_reliability": 1.00},
    "N-event":     {"label": "Stadium egress", "capacity": 1.00,
                    "demand_mult": 1.80, "direction_bias":  0.80,
                    "sensor_reliability": 1.00},
    "N-school":    {"label": "School pick-up", "capacity": 0.90,
                    "demand_mult": 1.35, "direction_bias":  0.30,
                    "sensor_reliability": 1.00},
    # ---- failure -----------------------------------------------------
    "N-sensor-fail":   {"label": "Detector failure", "capacity": 1.00,
                        "demand_mult": 1.00, "direction_bias": 0.00,
                        "sensor_reliability": 0.40},
    "N-comm-blackout": {"label": "V2X blackout",     "capacity": 1.00,
                        "demand_mult": 1.00, "direction_bias": 0.00,
                        "sensor_reliability": 1.00,
                        "comm_override": "S3"},
```

```
}

NATURAL_GROUPS: dict[str, list[str]] = {
    "weather":  ["N-clear", "N-rain", "N-snow", "N-fog"],
    "incident": ["N-accident", "N-workzone"],
    "demand":   ["N-rush-am", "N-rush-pm", "N-night",
                 "N-event", "N-school"],
    "failure":  ["N-sensor-fail", "N-comm-blackout"],
}

DEMAND_PATTERNS: dict[str, dict[str, float | str]] = {
    "D1": {"label": "Uniform  ~300 vph",        "ns": 0.30, "ew": 0.30},
    "D2": {"label": "AM peak  ~450 vph (60 %)", "ns": 0.40, "ew": 0.18},
    "D3": {"label": "PM peak  ~450 vph (60 %)", "ns": 0.18, "ew": 0.40},
    "D4": {"label": "Oversaturated ~600 vph",   "ns": 0.60, "ew": 0.50},
    "D5": {"label": "Weekend leisure  ~260 vph", "ns": 0.22, "ew": 0.26},
    "D6": {"label": "Freight corridor ~500 vph", "ns": 0.42, "ew": 0.28},
    "D7": {"label": "School peak  ~380 vph",     "ns": 0.36, "ew": 0.16},
}


# ============================================================================
#  SECTION 3  —  MESSAGE + IMPAIRMENT CHANNEL  (Eqs. 6, 7, 14)
# ============================================================================

@dataclass
class Message:
    """V2X message exchanged between intersections."""
    sender:    str
    receiver:  str
    t_send_ms: float
    seq:       int
    phase:     int
    queue_ns:  float
    queue_ew:  float
    elapsed_s: float
    # FIX: latency_ms is now stored on the delivered message so AoI accum
    latency_ms: float = 0.0


class ImpairmentChannel:
    """Truncated log-normal latency + Gilbert-Elliott correlated loss."""

    def __init__(self, latency_mean_ms: float, loss_rate: float,
                 latency_sigma: float = 0.5,
                 p_gb: float = 0.30, p_bg: float = 0.70,
                 rng: Optional[random.Random] = None) -> None:
        self.latency_mean  = latency_mean_ms
        self.latency_sigma = latency_sigma
        self.loss_rate     = loss_rate
        self.rng           = rng or random.Random()
        self.p_gb, self.p_bg, self.p_b = p_gb, p_bg, 0.90
        pi_b = self.p_gb / (self.p_gb + self.p_bg)
        pi_g = 1.0 - pi_b
```

```
        self.p_g = max(0.0, (loss_rate - pi_b * self.p_b) / max(pi_g, 1e-
        self.last_seq: defaultdict[str, int] = defaultdict(lambda: -1)
        self.ge_state: defaultdict[str, str] = defaultdict(lambda: "G")
        self.sent_count = 0
        self.delivered_count = 0

    def _latency(self) -> float:
        """Sample a truncated log-normal latency."""
        if self.latency_mean <= 0:
            return 0.0
        mu, sigma = math.log(self.latency_mean), self.latency_sigma
        for _ in range(16):
            tau = self.rng.lognormvariate(mu, sigma)
            if 0.1 * self.latency_mean <= tau <= 5 * self.latency_mean:
                return tau
        return self.latency_mean

    def _loss(self, link: str) -> bool:
        """Gilbert-Elliott two-state loss model."""
        state = self.ge_state[link]
        if state == "G":
            p_loss = self.p_g
            if self.rng.random() < self.p_gb:
                self.ge_state[link] = "B"
        else:
            p_loss = self.p_b
            if self.rng.random() < self.p_bg:
                self.ge_state[link] = "G"
        return self.rng.random() < p_loss

    def transmit(self, msg: Message) -> Optional[Message]:
        """Return delivered message (with latency_ms set) or None if drop
        self.sent_count += 1
        if self._loss(f"{msg.sender}->{msg.receiver}"):
            return None
        tau = self._latency()                          # FIX: actually us
        if msg.seq <= self.last_seq[msg.sender]:
            return None                                # out-of-order / d
        self.last_seq[msg.sender] = msg.seq
        self.delivered_count += 1
        msg.latency_ms = tau
        return msg


# ============================================================================
#  SECTION 4  —  CAR-FOLLOWING MODEL  (Table 12)
# ============================================================================

class CarFollowingModel:
    """SAFE / GIPPS / IDM empirical model — Table 12."""

    def __init__(self, name: str = "SAFE",
                 rng: Optional[random.Random] = None) -> None:
        if name not in CAR_FOLLOWING_BENCHMARK:
```

```
            raise ValueError(f"Unknown car-following model '{name}'.")
        self.name  = name
        self.rng   = rng or random.Random()
        self.bench = CAR_FOLLOWING_BENCHMARK[name]

    def simulate_step(self, regime: str, phase: int,
                      q_ns: float, q_ew: float,
                      congestion: float) -> dict[str, float]:
        base    = self.bench[regime]
        jitter  = self.rng.gauss(0.0, base["sd"])
        total_q = max(q_ns + q_ew, 1.0)
        in_sys  = base["in_system_s"] + jitter + 0.35 * total_q * (0.5 +
        if   self.name == "GIPPS": in_sys *= 1.02
        elif self.name == "IDM":   in_sys *= 1.01
        return {
            "in_system_s":      in_sys,
            "speed_ms":         max(0.5, base["speed_ms"] - 0.02 * total_
            "delay_s":          max(0.0, base["delay_s"] + 0.15 * total_q
            "efficiency_pct":   base["efficiency_pct"],
        }


# ============================================================================
#  SECTION 5  —  NETWORK  (3x3 grid of intersections)
# ============================================================================

class Intersection:
    """One signalised four-way intersection — real queue dynamics."""

    def __init__(self, node_id: str, rng: random.Random) -> None:
        self.id, self.rng = node_id, rng
        self.q: dict[str, float] = {"N": 0.0, "S": 0.0, "E": 0.0, "W": 0.
        self.W: dict[str, float] = {"N": 0.0, "S": 0.0, "E": 0.0, "W": 0.
        self.phase, self.phase_elapsed = 0, 0.0
        self.arrivals = self.departures = self.completed = 0.0
        self.seq = 0
        self.row = self.col = 0

    def step(self, dt: float, arrive_ns: float, arrive_ew: float,
             green_active_phase: Optional[int], green_time: float) -> Non
        """Advance one timestep: enqueue arrivals, dequeue serviced vehic
        for a in ("N", "S"):
            self.q[a] += 0.5 * arrive_ns * dt
            self.arrivals += 0.5 * arrive_ns * dt
        for a in ("E", "W"):
            self.q[a] += 0.5 * arrive_ew * dt
            self.arrivals += 0.5 * arrive_ew * dt

        serviceable = (("N", "S") if green_active_phase == 0 else
                       ("E", "W") if green_active_phase == 1 else ())
        for a in serviceable:
            d = min(self.q[a], SATURATION_FLOW_VPS * green_time)
            self.q[a]         -= d
            self.departures   += d
```

```
            self.completed    += d

        for a in self.q:
            self.W[a] += self.q[a] * dt
        self.phase_elapsed += dt
        if green_active_phase is not None and green_active_phase != self.
            self.phase, self.phase_elapsed = green_active_phase, 0.0

    def total_queue(self) -> float: return sum(self.q.values())
    def queue_ns(self)    -> float: return self.q["N"] + self.q["S"]
    def queue_ew(self)    -> float: return self.q["E"] + self.q["W"]

    def snapshot(self) -> dict[str, Any]:
        return {
            "id":         self.id,
            "seq":        self.seq,
            "phase":      self.phase,
            "queue_ns":   self.queue_ns(),
            "queue_ew":   self.queue_ew(),
            "elapsed_s":  self.phase_elapsed,
        }


class TrafficNetwork:
    """3x3 grid of signalised intersections."""

    def __init__(self, rng: random.Random) -> None:
        self.rng = rng
        self.nodes: dict[str, Intersection] = {}
        for r in range(GRID_ROWS):
            for c in range(GRID_COLS):
                nid = f"J{r+1}{c+1}"
                self.nodes[nid] = Intersection(nid, rng)
                self.nodes[nid].row, self.nodes[nid].col = r, c

    def neighbour_snapshots(self, node_id: str) -> list[dict[str, Any]]:
        r, c = self.nodes[node_id].row, self.nodes[node_id].col
        out: list[dict[str, Any]] = []
        for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < GRID_ROWS and 0 <= nc < GRID_COLS:
                out.append(self.nodes[f"J{nr+1}{nc+1}"].snapshot())
        return out


# ============================================================================
#  SECTION 6  —  CONTROLLER REGISTRY  +  AUTO-REGISTERING METACLASS
# ----------------------------------------------------------------------------
#  FIX for the KeyError: 'cooperative-marl' bug.
#  The original @controller decorator only ATTACHED _controller_spec to e
#  method but never registered them into cls.CONTROLLERS (because the cla
#  itself didn't exist yet when the decorator ran inside the class body).
#  The _ControllerMeta metaclass below scans the class namespace at class
#  creation time (i.e. immediately after the class body finishes) and
```

```python
#  registers every method carrying _controller_spec into cls.CONTROLLERS.
#  By the time _register_composed_controllers() runs in Section 8, all 15
#  base controllers are already registered, so compose() finds them.
# ============================================================================

@dataclass(frozen=True)
class ControllerSpec:
    """One registered controller expression."""
    name:     str
    category: str
    inputs:   tuple[str, ...]
    fn:       Callable
    doc:      str = ""


def controller(name: str, category: str, inputs: tuple[str, ...],
               doc: str = "") -> Callable:
    """Decorator — tag a method as a named controller expression.
    The actual registration is done by the _ControllerMeta metaclass
    at class-creation time (see below)."""
    def _decorator(fn: Callable) -> Callable:
        fn._controller_spec = ControllerSpec(name, category, inputs, fn,
        return fn
    return _decorator


class _ControllerMeta(type):
    """Metaclass that auto-registers every @controller-decorated method
    into cls.CONTROLLERS at class-creation time.

    This eliminates the KeyError that occurred when _register_composed_
    controllers() ran before the base controllers had been registered.
    """
    def __new__(mcs, name, bases, namespace, **kwargs):
        cls = super().__new__(mcs, name, bases, namespace, **kwargs)
        # Walk the namespace for any function carrying _controller_spec.
        for attr_name, attr_val in namespace.items():
            if callable(attr_val) and hasattr(attr_val, "_controller_spec
                spec = attr_val._controller_spec
                # Initialise CONTROLLERS on this class (own copy) if it
                # inherited an empty one or has none of its own.
                if "CONTROLLERS" not in cls.__dict__:
                    cls.CONTROLLERS = dict(getattr(cls, "CONTROLLERS", {}
                cls.CONTROLLERS[spec.name] = spec
        return cls


# ============================================================================
#  SECTION 7  —  MAIN CLASS  (TrafficControlAlgorithm)
# ----------------------------------------------------------------------------
#  NOTE: uses metaclass=_ControllerMeta so every @controller-decorated
#  method below is auto-registered the moment the class body finishes.
# ============================================================================
```

```
class TrafficControlAlgorithm(metaclass=_ControllerMeta):
    """Unified traffic-signal controller with expression-selection dispat

    CONTROLLERS: dict[str, ControllerSpec] = {}
    CATEGORIES: tuple[str, ...] = (
        "classical", "coordinated", "learning", "mixed", "field",
    )

    # ------------------------------------------------------------------ init
    def __init__(self,
                 algorithm_type: str = "adaptive",
                 scenario: str = "S1",
                 natural: str = "N-clear",
                 seed: int = 0,
                 car_following: str = "SAFE",
                 beta: float = DEFAULT_BETA,
                 lambda_decay_per_ms: float = 0.01,
                 stale_threshold_ms: float = 500.0,
                 p_drop: float = 0.10,
                 training: bool = False) -> None:
        if algorithm_type not in self.CONTROLLERS:
            raise ValueError(
                f"Unknown algorithm '{algorithm_type}'. "
                f"Available: {sorted(self.CONTROLLERS)}")
        if scenario not in COMMUNICATION_SCENARIOS:
            raise ValueError(f"Unknown scenario '{scenario}'.")
        if natural not in NATURAL_SCENARIOS:
            raise ValueError(f"Unknown natural scenario '{natural}'.")

        override = NATURAL_SCENARIOS[natural].get("comm_override")
        if override:
            scenario = override  # type: ignore[assignment]

        self.algorithm_type = algorithm_type
        self.scenario       = scenario
        self.natural        = natural
        self.comm_cfg       = COMMUNICATION_SCENARIOS[scenario]
        self.nat_cfg        = NATURAL_SCENARIOS[natural]
        self.rng            = random.Random(seed)
        self.car_model      = CarFollowingModel(car_following, self.rng)

        self.beta         = beta
        self.lambda_decay = lambda_decay_per_ms
        self.stale_thresh = stale_threshold_ms
        self.p_drop       = p_drop
        self.training     = training

        self.channel = ImpairmentChannel(
            self.comm_cfg["latency_ms"],  # type: ignore[arg-type]
            self.comm_cfg["loss_rate"],   # type: ignore[arg-type]
            rng=self.rng,
        )
        self.aoi_ms:      defaultdict[str, float] = defaultdict(float)
        self.fwd_history: defaultdict[str, deque] = defaultdict(lambda: d
```

```
        self.optimization_events = 0
        self.phase_decisions: list[dict[str, Any]] = []
        self.seq             = 0
        self.current_time_ms = 0.0
        self.cycle_length_s  = 60.0

        self.m: dict[str, float | int] = {
            "travel_time_sum_s":  0.0,
            "queue_sum":          0.0,
            "waiting_sum_s":      0.0,
            "vehicles_completed": 0.0,
            "samples":            0,
        }

    # ------------------------------------------------------- registry A
    @classmethod
    def register(cls, spec: ControllerSpec) -> None:
        cls.CONTROLLERS[spec.name] = spec

    @classmethod
    def list_controllers(cls, category: Optional[str] = None) -> list[str
        if category is None:
            return sorted(cls.CONTROLLERS)
        return sorted(n for n, s in cls.CONTROLLERS.items()
                      if s.category == category)

    @classmethod
    def get_expression(cls, name: str) -> ControllerSpec:
        return cls.CONTROLLERS[name]

    @classmethod
    def compose(cls, name_a: str, name_b: str, weight_a: float = 0.5,
                new_name: Optional[str] = None) -> str:
        """Register a NEW controller that blends two existing expressions
        spec_a = cls.CONTROLLERS[name_a]
        spec_b = cls.CONTROLLERS[name_b]
        inputs = tuple(sorted(set(spec_a.inputs) | set(spec_b.inputs)))
        w = max(0.0, min(1.0, weight_a))
        cname = new_name or f"{name_a}|{name_b}:{w:.2f}"

        def blended(self, **kw: Any) -> tuple[int, float]:
            phase_a, dur_a = spec_a.fn(self, **{k: kw[k] for k in spec_a.
            phase_b, dur_b = spec_b.fn(self, **{k: kw[k] for k in spec_b.
            phase = phase_a if w >= 0.5 else phase_b
            return phase, w * dur_a + (1 - w) * dur_b

        cls.register(ControllerSpec(
            cname, "composed", inputs, blended,
            f"{name_a} ({w:.0%}) + {name_b} ({1-w:.0%})",
        ))
        return cname

    # --------------------------------- communication-augmented observat
```

```
    def build_observation(self, local_state: dict[str, Any],
                          neighbours: Optional[list[dict[str, Any]]]
                          ) -> dict[str, Any]:
        """Communication-augmented observation — Eq. (2)."""
        self.seq += 1
        received: list[tuple[Message, float]] = []
        for nb in (neighbours or []):
            msg = Message(
                sender=nb.get("id", "nb"),
                receiver="self",
                t_send_ms=self.current_time_ms,
                seq=nb.get("seq", 0),
                phase=nb.get("phase", 0),
                queue_ns=nb.get("queue_ns", 0.0),
                queue_ew=nb.get("queue_ew", 0.0),
                elapsed_s=nb.get("elapsed_s", 0.0),
            )
            delivered = self.channel.transmit(msg)
            if delivered is None:
                self.aoi_ms[msg.sender] += DECISION_INTERVAL_S * 1000.0
                continue
            aoi = self.aoi_ms[msg.sender] + delivered.latency_ms
            self.aoi_ms[msg.sender] = aoi
            received.append((delivered, aoi))
            self.fwd_history[delivered.sender].append(
                (self.current_time_ms, delivered.queue_ns, delivered.queu
        return {"local": local_state, "received": received}

    # ======================================================================
    #  CONTROLLER EXPRESSIONS  (registered via @controller)
    #  The _ControllerMeta metaclass auto-registers each into CONTROLLERS
    #  the moment the class body finishes — no manual scan needed.
    # ======================================================================

    # ------- classical ------------------------------------------------
    @controller("fixed-time", "classical", ("current_time",))
    def fixed_time_control(self, current_time: float) -> tuple[int, float
        return int((current_time % 60.0) / 30.0), 30.0

    @controller("actuated", "classical",
                ("veh_ns", "veh_ew", "gap_ns", "gap_ew"))
    def actuated_control(self, veh_ns: float, veh_ew: float,
                         gap_ns: float, gap_ew: float) -> tuple[int, floa
        if veh_ns >= veh_ew:
            phase, demand, gap = 0, veh_ns, gap_ns
        else:
            phase, demand, gap = 1, veh_ew, gap_ew
        total = max(veh_ns + veh_ew, 1.0)
        green = max(GREEN_MIN_S, min(GREEN_MAX_S,
                                     GREEN_MAX_S * demand / total))
        if gap >= 3.0:
            green = GREEN_MIN_S
        self.optimization_events += 1
        return phase, green
```

```
    @controller("adaptive", "classical",
                ("veh_ns", "veh_ew", "congestion_eff"))
    def adaptive_control(self, veh_ns: float, veh_ew: float,
                         congestion_eff: float) -> tuple[int, float]:
        ns_r = veh_ns / max(veh_ns + veh_ew, 1.0)
        if congestion_eff > 0.7:
            ns_t, ew_t = (35.0, 20.0) if ns_r > 0.6 else (20.0, 35.0)
        else:
            ns_t = ew_t = 30.0
        phase = 0 if ns_r > 0.5 else 1
        self.optimization_events += 1
        return phase, (ns_t if phase == 0 else ew_t)

    @controller("optimized", "classical",
                ("veh_ns", "veh_ew", "waiting_ns", "waiting_ew",
                 "congestion_eff"))
    def optimized_control(self, veh_ns: float, veh_ew: float,
                          waiting_ns: float, waiting_ew: float,
                          congestion_eff: float) -> tuple[int, float]:
        u_ns = 0.4 * veh_ns + 0.6 * waiting_ns
        u_ew = 0.4 * veh_ew + 0.6 * waiting_ew
        ns_p = u_ns / max(u_ns + u_ew, 1e-6)
        cycle = 60.0 * (0.8 if congestion_eff > 0.8
                        else 1.2 if congestion_eff < 0.3 else 1.0)
        green = max(GREEN_MIN_S, cycle - YELLOW_S)
        phase = 0 if ns_p > 0.5 else 1
        self.optimization_events += 1
        return phase, green * (ns_p if phase == 0 else 1 - ns_p)

    # ------- coordinated ------------------------------------------------
    @controller("coordinated", "coordinated",
                ("veh_ns", "veh_ew", "congestion_eff", "row", "col"))
    def coordinated_control(self, veh_ns: float, veh_ew: float,
                            congestion_eff: float,
                            row: int, col: int) -> tuple[int, float]:
        Y     = min(0.85, 0.35 + 0.5 * congestion_eff)
        L     = 2.0 * YELLOW_S
        cycle = max(40.0, min(120.0, (1.5 * L + 5.0) / max(1.0 - Y, 0.05)
        self.cycle_length_s = cycle
        _ = ((row + col) * (SEGMENT_LENGTH_M / FREE_FLOW_SPEED_MS)) % cyc
        ns_r  = veh_ns / max(veh_ns + veh_ew, 1.0)
        green = max(GREEN_MIN_S, cycle - YELLOW_S)
        phase = 0 if ns_r > 0.5 else 1
        self.optimization_events += 1
        return phase, green * (ns_r if phase == 0 else 1 - ns_r)

    # ------- learning ---------------------------------------------------
    @controller("independent-dqn", "learning",
                ("veh_ns", "veh_ew", "waiting_ns", "waiting_ew",
                 "congestion_eff"))
    def independent_dqn_control(self, veh_ns: float, veh_ew: float,
                                waiting_ns: float, waiting_ew: float,
                                congestion_eff: float) -> tuple[int, floa
```

```
        phase = (0 if (veh_ns + 0.5 * waiting_ns)
                 >= (veh_ew + 0.5 * waiting_ew) else 1)
        self.optimization_events += 1
        return phase, 30.0 + 10.0 * congestion_eff

    @controller("cooperative-marl", "learning",
                ("veh_ns", "veh_ew", "congestion_eff", "obs"))
    def cooperative_marl_control(self, veh_ns: float, veh_ew: float,
                                 congestion_eff: float,
                                 obs: dict[str, Any]) -> tuple[int, floa
        agg_ns, agg_ew = veh_ns, veh_ew
        for msg, _ in obs["received"]:
            agg_ns += msg.queue_ns
            agg_ew += msg.queue_ew
        phase = 0 if agg_ns >= agg_ew else 1
        self.optimization_events += 1
        return phase, 30.0 + 15.0 * congestion_eff

    @controller("resilient-marl", "learning",
                ("veh_ns", "veh_ew", "congestion_eff", "obs"))
    def resilient_marl_control(self, veh_ns: float, veh_ew: float,
                               congestion_eff: float,
                               obs: dict[str, Any]) -> tuple[int, float]
        """Resilient MARL with M1-M5 active."""
        agg_ns, agg_ew = veh_ns, veh_ew

        # M2 + M5 — AoI-weighted aggregation with training-time dropout
        for msg, aoi in obs["received"]:
            if self.training and self.rng.random() < self.p_drop:
                continue
            w = aoi_decay_kernel(aoi, self.lambda_decay)
            agg_ns += w * msg.queue_ns
            agg_ew += w * msg.queue_ew

        # M3 — predictive compensation for stale neighbours
        for nb_id, hist in self.fwd_history.items():
            if not hist or self.aoi_ms[nb_id] <= self.stale_thresh:
                continue
            if len(hist) >= 2:
                t0, ns0, ew0 = hist[-2]
                t1, ns1, ew1 = hist[-1]
                dt = max(t1 - t0, 1.0)
                pred_ns = ns1 + (ns1 - ns0) / dt * self.aoi_ms[nb_id]
                pred_ew = ew1 + (ew1 - ew0) / dt * self.aoi_ms[nb_id]
            else:
                pred_ns, pred_ew = hist[-1][1], hist[-1][2]
            w = aoi_decay_kernel(self.aoi_ms[nb_id], self.lambda_decay)
            agg_ns += w * pred_ns
            agg_ew += w * pred_ew

        phase = 0 if agg_ns >= agg_ew else 1
        self.optimization_events += 1
        return phase, 30.0 + 12.0 * congestion_eff
```

```
    # ------- field-inspired ------------------------------------------
    @controller("max-pressure", "field",
                ("veh_ns", "veh_ew", "current_phase"))
    def max_pressure_control(self, veh_ns: float, veh_ew: float,
                             current_phase: int) -> tuple[int, float]:
        diff   = (veh_ns - 0.6 * veh_ew) - (veh_ew - 0.6 * veh_ns)
        switch = (diff < -3.0) if current_phase == 0 else (diff > 3.0)
        new_phase = (1 - current_phase) if switch else current_phase
        dur = 20.0 + 0.6 * abs(diff) + 0.4 * (veh_ns + veh_ew)
        self.optimization_events += 1
        return new_phase, max(GREEN_MIN_S, min(GREEN_MAX_S, dur))

    @controller("scats", "field",
                ("veh_ns", "veh_ew", "congestion_eff", "row", "col"))
    def scats_control(self, veh_ns: float, veh_ew: float,
                      congestion_eff: float,
                      row: int, col: int) -> tuple[int, float]:
        deg_sat = min(1.0, (veh_ns + veh_ew) / 40.0 * (0.6 + congestion_e
        cycle   = max(60.0, min(120.0, 60.0 * (1.0 + 0.4 * deg_sat)))
        self.cycle_length_s = cycle
        ns_r  = veh_ns / max(veh_ns + veh_ew, 1.0)
        green = max(GREEN_MIN_S, cycle - 2 * YELLOW_S)
        phase = 0 if ns_r > 0.5 else 1
        self.optimization_events += 1
        return phase, green * (ns_r if phase == 0 else 1 - ns_r)

    @controller("scoot", "field", ("veh_ns", "veh_ew", "congestion_eff"))
    def scoot_control(self, veh_ns: float, veh_ew: float,
                      congestion_eff: float) -> tuple[int, float]:
        cfp   = (veh_ns + veh_ew) * (0.5 + congestion_eff)
        cycle = (max(50.0, self.cycle_length_s - 4.0) if cfp > 60.0
                 else min(120.0, self.cycle_length_s + 4.0) if cfp < 25.0
                 else self.cycle_length_s)
        self.cycle_length_s = cycle
        ns_r  = veh_ns / max(veh_ns + veh_ew, 1.0)
        green = max(GREEN_MIN_S, cycle - 2 * YELLOW_S)
        phase = 0 if ns_r > 0.5 else 1
        self.optimization_events += 1
        return phase, green * (ns_r if phase == 0 else 1 - ns_r)

    @controller("multiband", "field",
                ("veh_ns", "veh_ew", "congestion_eff"))
    def multiband_control(self, veh_ns: float, veh_ew: float,
                          congestion_eff: float) -> tuple[int, float]:
        bias  = self.nat_cfg["direction_bias"]
        cycle = 90.0 if abs(bias) > 0.4 else 70.0
        self.cycle_length_s = cycle
        w_dom = 0.60 + 0.15 * abs(bias)
        if bias >= 0:
            phase, dur = 0, (cycle - 2 * YELLOW_S) * w_dom
        else:
            phase, dur = 1, (cycle - 2 * YELLOW_S) * w_dom
        self.optimization_events += 1
        return phase, dur
```

```
    @controller("fuzzy-logic", "field",
                ("veh_ns", "veh_ew", "congestion_eff"))
    def fuzzy_logic_control(self, veh_ns: float, veh_ew: float,
                            congestion_eff: float) -> tuple[int, float]:
        def tri(x: float, lo: float, mid: float, hi: float) -> float:
            if x <= lo or x >= hi:
                return 0.0
            return (x - lo) / (mid - lo) if x <= mid else (hi - x) / (hi
        nsL, ewL = tri(veh_ns, 5, 30, 60), tri(veh_ew, 5, 30, 60)
        high_c   = max(0.0, (congestion_eff - 0.5) / 0.5)
        r1, r2, r3 = min(nsL, 1 - ewL), min(ewL, 1 - nsL), min(nsL, ewL)
        r4 = min(1 - nsL, 1 - ewL) * (1 - high_c)
        ns_score = r1 + 0.5 * r3 + 0.3 * r4
        ew_score = r2 + 0.5 * r3 + 0.3 * r4
        ns_p = ns_score / max(ns_score + ew_score, 1e-6)
        cycle = 60.0 * (1.0 - 0.3 * high_c)
        green = max(GREEN_MIN_S, cycle - 2 * YELLOW_S)
        phase = 0 if ns_p >= 0.5 else 1
        self.optimization_events += 1
        return phase, green * (ns_p if phase == 0 else 1 - ns_p)

    @controller("mpc", "field", ("veh_ns", "veh_ew", "congestion_eff"))
    def mpc_control(self, veh_ns: float, veh_ew: float,
                    congestion_eff: float) -> tuple[int, float]:
        H, best_phase, best_cost = 4, 0, float("inf")
        arrival = (veh_ns + veh_ew) / max(DECISION_INTERVAL_S, 1.0) * 0.3
        for cand in (0, 1):
            qn, qe, cost = veh_ns, veh_ew, 0.0
            for _ in range(H):
                if cand == 0:
                    qn = max(0.0, qn - 3.0 + 0.5 * arrival)
                    qe = max(0.0, qe + 0.5 * arrival)
                else:
                    qn = max(0.0, qn + 0.5 * arrival)
                    qe = max(0.0, qe - 3.0 + 0.5 * arrival)
                cost += qn + qe + 0.05 * (qn * qn + qe * qe) / 20.0
            if cost < best_cost:
                best_cost, best_phase = cost, cand
        dur = 25.0 + 15.0 * congestion_eff + 0.2 * (veh_ns + veh_ew)
        self.optimization_events += 1
        return best_phase, min(GREEN_MAX_S, max(GREEN_MIN_S, dur))

    @controller("weather-adaptive", "field",
                ("veh_ns", "veh_ew", "congestion_eff"))
    def weather_adaptive_control(self, veh_ns: float, veh_ew: float,
                                 congestion_eff: float) -> tuple[int, fl
        cap   = self.nat_cfg["capacity"]
        qn    = veh_ns / max(cap, 0.1)
        qe    = veh_ew / max(cap, 0.1)
        ns_r  = qn / max(qn + qe, 1.0)
        cycle = 60.0 + 30.0 * (1.0 - cap)
        green = max(GREEN_MIN_S, cycle - 2 * YELLOW_S - 2.0 * (1.0 - cap)
        phase = 0 if ns_r > 0.5 else 1
```

```
        self.optimization_events += 1
        return phase, green * (ns_r if phase == 0 else 1 - ns_r)

    # ================================================================
    #  DISPATCHER  –  single registry lookup
    # ================================================================
    def control_signal(self,
                       intersection_id: str,
                       vehicle_count_ns: float,
                       vehicle_count_ew: float,
                       waiting_ns: float,
                       waiting_ew: float,
                       current_time: float,
                       current_congestion: float,
                       gap_ns: float = 0.0,
                       gap_ew: float = 0.0,
                       neighbours: Optional[list[dict[str, Any]]] = None,
                       row: int = 0, col: int = 0,
                       current_phase: int = 0) -> tuple[int, float]:
        self.current_time_ms = current_time * 1000.0

        # ---- natural-condition preprocessing -------------------------
        cfg       = self.nat_cfg
        nat_total = (vehicle_count_ns + vehicle_count_ew) * cfg["demand_m
        bias      = cfg["direction_bias"]
        veh_ns    = nat_total * (0.5 + 0.5 * bias)
        veh_ew    = nat_total - veh_ns
        congestion_eff = min(0.99,
                             current_congestion / max(cfg["capacity"], 0.

        if cfg["sensor_reliability"] < 1.0:
            if self.rng.random() > cfg["sensor_reliability"]:
                veh_ns, veh_ew, waiting_ns, waiting_ew = 0.0, 0.0, 0.0, 0

        obs = self.build_observation(
            {"phase": current_phase, "elapsed_s": current_time % 30.0,
             "queue_ns": veh_ns, "queue_ew": veh_ew},
            neighbours)

        # ---- expression selection ------------------------------------
        spec = self.CONTROLLERS[self.algorithm_type]
        context: dict[str, Any] = {
            "intersection_id":     intersection_id,
            "veh_ns":              veh_ns,
            "veh_ew":              veh_ew,
            "waiting_ns":          waiting_ns,
            "waiting_ew":          waiting_ew,
            "current_time":        current_time,
            "current_congestion":  current_congestion,
            "congestion_eff":      congestion_eff,
            "gap_ns":              gap_ns,
            "gap_ew":              gap_ew,
            "obs":                 obs,
            "row":                 row,
```

```
            "col":                  col,
            "current_phase":        current_phase,
        }
        kwargs = {k: context[k] for k in spec.inputs}
        phase, duration = spec.fn(self, **kwargs)
        duration = max(GREEN_MIN_S, min(GREEN_MAX_S, duration))

        self.phase_decisions.append({
            "intersection": intersection_id, "phase": phase,
            "duration": duration, "algorithm": self.algorithm_type,
            "scenario": self.scenario, "natural": self.natural,
            "time": current_time})

        # ---- metric accumulation (Eqs. 15-18) -------------------------
        self.m["samples"]            += 1
        self.m["waiting_sum_s"]      += waiting_ns + waiting_ew
        self.m["queue_sum"]          += veh_ns + veh_ew
        cleared = min(2.5, 0.025 * (veh_ns + veh_ew) * cfg["capacity"])
        self.m["vehicles_completed"] += cleared

        cf = self.car_model.simulate_step(
            "free_flow" if congestion_eff < 0.5 else "congested",
            phase, veh_ns, veh_ew, congestion_eff)
        self.m["travel_time_sum_s"] += (
            0.6 * cf["in_system_s"]
            + 0.4 * (0.5 * (waiting_ns + waiting_ew) + 100.0))
        return phase, duration

    # ========================================================================
    #  COOPERATIVE REWARD — Eq. (10)
    # ========================================================================
    def cooperative_reward(self, local_queues: Sequence[float],
                           global_queues: Sequence[float]) -> float:
        r_l = -sum(local_queues)
        r_g = -(sum(global_queues) / max(len(global_queues), 1))
        return self.beta * r_l + (1.0 - self.beta) * r_g

    # ========================================================================
    #  METRICS — Eqs. (15)-(20)
    # ========================================================================
    def get_metrics(self) -> dict[str, Any]:
        s     = max(self.m["samples"], 1)
        sim_s = s * DECISION_INTERVAL_S
        sent  = max(self.channel.sent_count, 1)
        mean_aoi = (sum(self.aoi_ms.values()) / len(self.aoi_ms)
                    if self.aoi_ms else 0.0)
        return {
            "algorithm_type":      self.algorithm_type,
            "scenario":            self.scenario,
            "natural":             self.natural,
            "car_following":       self.car_model.name,
            "travel_time_s":       round(self.m["travel_time_sum_s"] / s,
            "queue_veh":           round(self.m["queue_sum"] / s, 2),
            "waiting_time_s":      round(self.m["waiting_sum_s"] / s, 2),
```

```
            "throughput_vph":      round(self.m["vehicles_completed"]
                                         / max(sim_s, 1.0) * 3600.0, 1),
            "mean_aoi_ms":         round(mean_aoi, 1),
            "delivery_ratio":      round(self.channel.delivered_count / s
            "msg_overhead":        round(self.channel.sent_count / s, 2),
            "severity_index":      round(severity_index(
                self.comm_cfg["latency_ms"],  # type: ignore[arg-type]
                self.comm_cfg["loss_rate"]),   # type: ignore[arg-type]
                3),
            "optimization_events": self.optimization_events,
        }


# ============================================================================
#  SECTION 8  —  POST-CLASS REGISTRATION  (mixed controllers via composit
# ----------------------------------------------------------------------------
#  NOTE: by the time this runs, the _ControllerMeta metaclass has already
#  auto-registered all 15 base controllers (fixed-time, actuated, adaptiv
#  optimized, coordinated, independent-dqn, cooperative-marl, resilient-m
#  max-pressure, scats, scoot, multiband, fuzzy-logic, mpc, weather-adapt
#  into TrafficControlAlgorithm.CONTROLLERS, so compose() finds both keys
# ============================================================================

def _register_composed_controllers() -> None:
    """Register the two pre-baked composed controllers."""
    TrafficControlAlgorithm.compose("cooperative-marl", "coordinated", 0.
                                    new_name="cooperative-coordinated")
    TrafficControlAlgorithm.compose("resilient-marl",   "coordinated", 0.
                                    new_name="resilient-coordinated")

_register_composed_controllers()

print(f"[boot] registered {len(TrafficControlAlgorithm.CONTROLLERS)} cont
      f"{sorted(TrafficControlAlgorithm.CONTROLLERS)[:5]} ...")


# ============================================================================
#  SECTION 9  —  SIMULATION DRIVER  +  MULTI-SEED EVALUATION
# ============================================================================

def simulate_episode(controller: TrafficControlAlgorithm,
                     n_steps: int = 200,
                     base_demand_ns: float = 0.15,
                     base_demand_ew: float = 0.10,
                     congestion_start: float = 0.40,
                     congestion_end:   float = 0.90,
                     seed: Optional[int] = None) -> TrafficControlAlgorit
    """Run the 3x3 grid for n_steps decision intervals."""
    rng = random.Random(seed) if seed is not None else controller.rng
    net = TrafficNetwork(rng)
    dt  = DECISION_INTERVAL_S
    net_wait_sum = net_queue_samples = 0.0
    net_samples  = 0
```

```
    for t in range(n_steps):
        c = (congestion_start
             + (congestion_end - congestion_start) * t / max(n_steps - 1,
        cur_time = t * dt
        for nid, node in net.nodes.items():
            phase, dur = controller.control_signal(
                intersection_id    = nid,
                vehicle_count_ns   = node.queue_ns(),
                vehicle_count_ew   = node.queue_ew(),
                waiting_ns         = node.W["N"] + node.W["S"],
                waiting_ew         = node.W["E"] + node.W["W"],
                current_time       = cur_time,
                current_congestion = c,
                gap_ns             = 1.0 if node.queue_ns() > 0 else 5.0,
                gap_ew             = 1.0 if node.queue_ew() > 0 else 5.0,
                neighbours         = net.neighbour_snapshots(nid),
                row                = node.row,
                col                = node.col,
                current_phase      = node.phase)
            node.step(dt,
                      arrive_ns         = base_demand_ns * (0.8 + 0.6 * c
                      arrive_ew         = base_demand_ew * (0.8 + 0.6 * c
                      green_active_phase= phase,
                      green_time        = min(dur, dt))
        for node in net.nodes.values():
            net_wait_sum      += sum(node.W.values())
            net_queue_samples += node.total_queue()
        net_samples += len(net.nodes)

    controller.m["waiting_sum_s"]      = net_wait_sum / max(net_samples,
    controller.m["queue_sum"]           = net_queue_samples / max(net_sam
    controller.m["vehicles_completed"]  = sum(n.completed for n in net.no
    controller.m["samples"]             = n_steps
    return controller


def run_multi_seed(algorithm_type: str,
                   scenario: str = "S1",
                   natural: str = "N-clear",
                   seeds: Iterable[int] = (1, 2, 3, 4, 5),
                   n_steps: int = 200,
                   base_demand_ns: float = 0.15,
                   base_demand_ew: float = 0.10,
                   car_following: str = "SAFE") -> dict[str, Any]:
    """Run one cell across n seeds; return mean / stdev / 95% CI per metr
    keys = ("travel_time_s", "queue_veh", "waiting_time_s",
            "throughput_vph", "mean_aoi_ms", "delivery_ratio")
    per_seed: dict[str, list[float]] = defaultdict(list)
    seeds = tuple(seeds)

    for s in seeds:
        c = TrafficControlAlgorithm(algorithm_type, scenario, natural,
                                    seed=s, car_following=car_following)
        simulate_episode(c, n_steps=n_steps,
```

```
                          base_demand_ns=base_demand_ns,
                          base_demand_ew=base_demand_ew, seed=s)
        m = c.get_metrics()
        for k in keys:
            per_seed[k].append(float(m[k]))

    out: dict[str, Any] = {"algorithm_type": algorithm_type,
                           "scenario": scenario,
                           "natural":  natural,
                           "n_seeds":  len(seeds)}
    for k, xs in per_seed.items():
        mean = statistics.fmean(xs) if xs else 0.0
        if len(xs) > 1:
            sd   = statistics.stdev(xs)
            ci95 = 1.96 * sd / math.sqrt(len(xs))
        else:
            sd = ci95 = 0.0
        out[f"{k}_mean"]   = round(mean, 3)
        out[f"{k}_sd"]     = round(sd,   3)
        out[f"{k}_ci95"]   = round(ci95, 3)
        out[f"{k}_values"] = [round(x, 3) for x in xs]
    return out


def export_csv(rows: list[dict[str, Any]],
               path: str = "traffic_benchmark.csv") -> None:
    """Write a list of dicts (from run_multi_seed) to CSV."""
    if not rows:
        return
    fieldnames = sorted({k for r in rows for k in r})
    with open(path, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=fieldnames, extrasaction="ignore
        w.writeheader()
        for r in rows:
            w.writerow(r)


# ============================================================================
#  SECTION 10  —  SYNTHETIC DATASET  (closed-form tensor generator)
# ============================================================================

PRESETS: dict[str, dict[str, int | None]] = {
    "quick":    {"n_ctrl": 16,  "n_scen": 8,  "n_cond": 4,  "n_steps": 20
    "standard": {"n_ctrl": 64,  "n_scen": 32, "n_cond": 8,  "n_steps": 50
    "full":     {"n_ctrl": 128, "n_scen": 64, "n_cond": 12, "n_steps": No
}

_CF_BENCHMARK: dict[str, dict[str, dict[str, float]]] = {
    "SAFE":  {"free_flow": {"in_system_s": 100.0},
              "congested": {"in_system_s": 230.0}},
    "GIPPS": {"free_flow": {"in_system_s": 105.0},
              "congested": {"in_system_s": 250.0}},
    "IDM":   {"free_flow": {"in_system_s": 102.0},
              "congested": {"in_system_s": 240.0}},
```

```
}

K_CTRL_FAMILY: dict[str, float] = {
    "baseline": 1.26, "adaptive": 1.06, "coordinated": 1.00,
    "field": 0.98, "learning": 1.02, "resilient": 0.99,
}

RESILIENCE_FLATTENING: dict[str, float] = {
    "baseline": 1.00, "adaptive": 0.95, "coordinated": 0.90,
    "field": 0.85, "learning": 0.75, "resilient": 0.35,
}

_K_COMM_BASE_K  = 1.43 * 100.0
_K_COMM_BASE_X0 = 101.54 / 600.0
_K_COMM_BASE_L  = 0.60

_NUMERIC_COLS: tuple[str, ...] = (
    "travel_time_s", "queue_veh", "waiting_time_s", "throughput_vph",
    "mean_aoi_ms", "delivery_ratio",
    "K_ctrl", "K_comm", "K_nat", "K_dem",
    "severity", "true_value",
)


@dataclass
class DatasetConfig:
    """All tunable parameters for the synthetic dataset, validated at con
    seed:          int   = 42
    car_following: str   = "SAFE"
    sample_cf:     bool  = False
    noise_frac:    float = 0.06
    outlier_frac:  float = 0.02
    outlier_mult:  float = 2.2
    preset:        str   = "full"
    csv_path:      str   = "synthetic_dataset.csv"
    npy_path:      str   = "synthetic_tensor.npy"
    progress:      bool  = True
    auto_build:    bool  = False
    verbose:       bool  = True

    def __post_init__(self) -> None:
        if self.preset not in PRESETS:
            raise ValueError(f"Unknown preset {self.preset!r}. "
                             f"Choose from {tuple(PRESETS)}.")
        if self.car_following not in _CF_BENCHMARK:
            raise ValueError(f"Unknown car_following {self.car_following!
        if not 0.0 <= self.noise_frac < 1.0:
            raise ValueError(f"noise_frac must be in [0, 1); got {self.no
        if not 0.0 <= self.outlier_frac <= 1.0:
            raise ValueError(f"outlier_frac must be in [0, 1]; got {self.
        if self.outlier_mult <= 0.0:
            raise ValueError(f"outlier_mult must be > 0; got {self.outlie
```

```python
# --- 128 controllers, built family-by-family ---------------------------

_FAMILY_SPEC: dict[str, dict[str, Any]] = {
    "baseline":    {"count": 16, "bases": ["fixed-time", "actuated",
                                           "semi-actuated", "pretimed",
                                           "gap-out", "cycle-72", "phase-s
                                           "dual-ring"]},
    "adaptive":    {"count": 16, "bases": ["adaptive", "adapt-p", "adapt-
                                           "adapt-pid", "adapt-q", "adapt-
                                           "adapt-lin", "adapt-nonlin"]},
    "coordinated": {"count": 16, "bases": ["coord", "coord-green", "offse
                                           "platoon", "green-wave", "green
                                           "flow-sync", "bidir-wave"]},
    "field":       {"count": 32, "bases": ["max-pressure", "scats", "scoo
                                           "multiband", "mpc", "fuzzy", "w
                                           "bike", "transit", "emergency",
                                           "ped-adapt", "rideshare", "max-
                                           "split-opt", "cycle-opt", "offs
    "learning":    {"count": 24, "bases": ["idqn", "coop-marl", "ppo-marl
                                           "sac-marl", "td3-marl", "a2c-ma
                                           "ddpg-marl", "rainbow", "iqn",
                                           "iql", "bc-marl"]},
    "resilient":   {"count": 24, "bases": ["res-marl", "res-dqn", "res-pp
                                           "byz-robust", "fed-marl", "sec-
                                           "robust-td3", "res-sac", "res-t
                                           "res-a2c", "res-ddpg", "res-iqn
}
assert sum(s["count"] for s in _FAMILY_SPEC.values()) == 128


def _build_controllers() -> dict[str, str]:
    """Return an ordered mapping {controller_name: family}."""
    out: dict[str, str] = {}
    for family, spec in _FAMILY_SPEC.items():
        bases = list(spec["bases"])
        while len(bases) < spec["count"]:
            i = len(bases) - len(spec["bases"])
            bases.append(
                f"{spec['bases'][i % len(spec['bases'])]}"
                f"-v{2 + (i // len(spec['bases']))}"
            )
        for name in bases[: spec["count"]]:
            out[name] = family
    return out


CONTROLLER_FAMILY: dict[str, str] = _build_controllers()
CONTROLLERS_TUPLE: tuple[str, ...] = tuple(CONTROLLER_FAMILY)
assert len(CONTROLLERS_TUPLE) == 128


# --- 64 scenarios (8 latencies x 8 loss rates) -------------------------

_LATENCY_GRID = np.linspace(10.0, 1000.0, 8)
```

```python
_LOSS_GRID    = np.linspace(0.005, 0.650, 8)

SCENARIOS_DS: dict[str, dict[str, float]] = {}
SCENARIO_KEYS_DS: list[str] = []
for _i, _lat in enumerate(_LATENCY_GRID):
    for _j, _loss in enumerate(_LOSS_GRID):
        _key = f"S{_i * 8 + _j + 1:02d}"
        SCENARIOS_DS[_key] = {"latency_ms": float(_lat),
                              "loss_rate":  float(_loss)}
        SCENARIO_KEYS_DS.append(_key)
SCENARIO_KEYS_DS_TUPLE: tuple[str, ...] = tuple(SCENARIO_KEYS_DS)
assert len(SCENARIO_KEYS_DS_TUPLE) == 64


# --- 12 conditions ----------------------------------------------------

CONDITIONS: dict[str, dict[str, float]] = {
    "C01-clear":        {"capacity": 1.00, "demand_mult": 1.00, "directio
    "C02-rain":         {"capacity": 0.85, "demand_mult": 0.95, "directio
    "C03-snow":         {"capacity": 0.60, "demand_mult": 0.80, "directio
    "C04-hailstorm":    {"capacity": 0.45, "demand_mult": 0.55, "directio
    "C05-flooding":     {"capacity": 0.35, "demand_mult": 0.50, "directio
    "C06-accident":     {"capacity": 0.55, "demand_mult": 0.90, "directio
    "C07-major-crash":  {"capacity": 0.40, "demand_mult": 0.85, "directio
    "C08-rush-am":      {"capacity": 0.95, "demand_mult": 1.35, "directio
    "C09-rush-pm":      {"capacity": 0.95, "demand_mult": 1.35, "directio
    "C10-oversat":      {"capacity": 1.00, "demand_mult": 1.60, "directio
    "C11-blackout":     {"capacity": 1.00, "demand_mult": 0.85, "directio
    "C12-power-outage": {"capacity": 0.90, "demand_mult": 0.75, "directio
}
CONDITION_KEYS_TUPLE: tuple[str, ...] = tuple(CONDITIONS)
assert len(CONDITION_KEYS_TUPLE) == 12


@dataclass(frozen=True, slots=True)
class LogisticSignatureDS:
    L:  float
    k:  float
    x0: float
    def __call__(self, x: float) -> float:
        return self.L / (1.0 + math.exp(-self.k * (x - self.x0)))


SCENARIO_SIGNATURES: dict[str, LogisticSignatureDS] = {
    f"S{i:02d}": LogisticSignatureDS(
        L  = 140.0 + 2.4 * (i - 1),
        k  = 0.0143 + 0.00030 * (i - 1),
        x0 = 0.05 + 0.0050 * (i - 1),
    )
    for i in range(1, 65)
}


@dataclass
```

```python
class SyntheticTrafficDataset:
    """Closed-form (128,64,12) synthetic dataset."""

    seed:          int   = 42
    car_following: str   = "SAFE"
    sample_cf:     bool  = False
    noise_frac:    float = 0.06
    outlier_frac:  float = 0.02
    outlier_mult:  float = 2.2
    csv_path:      str   = "synthetic_dataset.csv"
    npy_path:      str   = "synthetic_tensor.npy"
    preset:        str   = "full"
    progress:      bool  = True
    auto_build:    bool  = False
    verbose:       bool  = True

    _cfg:       DatasetConfig = field(init=False, repr=False, compare=Fal
    _ctrls:     tuple[str, ...] = field(init=False, repr=False, compare=F
    _scens:     tuple[str, ...] = field(init=False, repr=False, compare=F
    _conds:     tuple[str, ...] = field(init=False, repr=False, compare=F
    _shape:     tuple[int, int, int] = field(init=False, repr=False, comp
    _arrays:    Optional[dict[str, np.ndarray]] = field(init=False, repr=
    _M:         Optional[np.ndarray] = field(init=False, repr=False, comp

    NUMERIC_COLS: tuple[str, ...] = _NUMERIC_COLS

    def __post_init__(self) -> None:
        self._cfg = DatasetConfig(
            seed=self.seed, car_following=self.car_following,
            sample_cf=self.sample_cf, noise_frac=self.noise_frac,
            outlier_frac=self.outlier_frac, outlier_mult=self.outlier_mul
            preset=self.preset, csv_path=self.csv_path,
            npy_path=self.npy_path, progress=self.progress,
            auto_build=self.auto_build, verbose=self.verbose,
        )
        p = PRESETS[self._cfg.preset]
        self._ctrls = CONTROLLERS_TUPLE[: p["n_ctrl"]]
        self._scens = SCENARIO_KEYS_DS_TUPLE[: p["n_scen"]]
        self._conds = CONDITION_KEYS_TUPLE[: p["n_cond"]]
        self._shape = (len(self._ctrls), len(self._scens), len(self._cond
        if self._cfg.auto_build:
            self.build()

    @classmethod
    def default(cls, **overrides: Any) -> "SyntheticTrafficDataset":
        cfg = DatasetConfig(**overrides) if overrides else DatasetConfig(
        return cls(**cfg.__dict__)

    @property
    def shape(self) -> tuple[int, int, int]:
        return self._shape

    @property
    def size(self) -> int:
```

```
        return self._shape[0] * self._shape[1] * self._shape[2]

    def __len__(self) -> int: return self.size

    def __repr__(self) -> str:
        state = "built" if self._M is not None else "empty"
        return (f"SyntheticTrafficDataset(preset={self.preset!r}, "
                f"shape={self._shape}, seed={self.seed}, {state})")

    def __getitem__(self, key: tuple[int, int, int]) -> dict[str, Any]:
        i, j, k = key
        return self._ensure_records()[i, j, k]

    def __iter__(self) -> Iterator[dict[str, Any]]:
        M = self._ensure_records()
        for i, j, k in product(range(self._shape[0]),
                               range(self._shape[1]),
                               range(self._shape[2])):
            yield M[i, j, k]

    # ----- vectorised factors ----------------------------------------
    @staticmethod
    def _K_comm_vector(family_flat: np.ndarray, latency: np.ndarray,
                       loss: np.ndarray) -> np.ndarray:
        rho = 0.5 * (latency / 600.0) + 0.5 * loss
        k   = _K_COMM_BASE_K / family_flat
        x0  = _K_COMM_BASE_X0 * (1.0 + 0.5 * (1.0 - family_flat))
        L   = _K_COMM_BASE_L * family_flat
        return 1.0 + L / (1.0 + np.exp(-k * (rho - x0)))

    @staticmethod
    def _K_nat_vector(capacity: np.ndarray, demand_mult: np.ndarray,
                      bias: np.ndarray) -> np.ndarray:
        return ((1.0 / np.maximum(capacity, 0.1))
                * (1.0 + 0.35 * (demand_mult - 1.0))
                * (1.0 + 0.15 * np.abs(bias)))

    @staticmethod
    def _K_dem_vector(ns: np.ndarray, ew: np.ndarray,
                      capacity: np.ndarray) -> np.ndarray:
        arrival = ns + ew
        service = 2 * SATURATION_FLOW_VPS * capacity * 10.0
        rho = np.minimum(0.95, arrival / np.maximum(service, 1e-6))
        return 1.0 / (1.0 - rho)

    def _compute_arrays(self, force: bool = False) -> dict[str, np.ndarra
        if self._arrays is not None and not force:
            return self._arrays

        n1, n2, n3 = self._shape
        rng = np.random.default_rng(self._cfg.seed)

        ctrl_family = np.array([CONTROLLER_FAMILY[c] for c in self._ctrls
        kc   = np.array([K_CTRL_FAMILY[f] for f in ctrl_family])
```

```
        flat = np.array([RESILIENCE_FLATTENING[f] for f in ctrl_family])

        latency = np.array([SCENARIOS_DS[s]["latency_ms"] for s in self._
        loss    = np.array([SCENARIOS_DS[s]["loss_rate"]  for s in self._

        sig_L  = np.array([SCENARIO_SIGNATURES[s].L  for s in self._scens
        sig_k  = np.array([SCENARIO_SIGNATURES[s].k  for s in self._scens
        sig_x0 = np.array([SCENARIO_SIGNATURES[s].x0 for s in self._scens

        cap   = np.array([CONDITIONS[c]["capacity"]       for c in self._
        dmult = np.array([CONDITIONS[c]["demand_mult"]    for c in self._
        bias  = np.array([CONDITIONS[c]["direction_bias"] for c in self._
        ns    = np.array([CONDITIONS[c]["ns"]             for c in self._
        ew    = np.array([CONDITIONS[c]["ew"]             for c in self._

        kc_b    = kc[:, None, None]
        flat_b  = flat[:, None, None]
        lat_b   = latency[None, :, None]
        loss_b  = loss[None, :, None]
        sigL_b  = sig_L[None, :, None]
        sigk_b  = sig_k[None, :, None]
        sigx0_b = sig_x0[None, :, None]
        cap_b   = cap[None, None, :]
        dmult_b = dmult[None, None, :]
        bias_b  = bias[None, None, :]
        ns_b    = ns[None, None, :]
        ew_b    = ew[None, None, :]

        sev_comm   = 0.5 * (lat_b / 600.0) + 0.5 * loss_b
        sev_nat    = 1.0 - cap_b
        sev_dem    = np.minimum(0.5, (ns_b + ew_b - 0.6) / 1.2)
        sev        = 0.5 * sev_comm + 0.3 * sev_nat + 0.2 * sev_dem
        congestion = np.minimum(0.99,
                                0.4 * sev_comm + 0.4 * sev_nat + 0.2 * se

        ff_lut = np.array([_CF_BENCHMARK[c]["free_flow"]["in_system_s"]
                           for c in CF_MODELS])
        cg_lut = np.array([_CF_BENCHMARK[c]["congested"]["in_system_s"]
                           for c in CF_MODELS])
        if self._cfg.sample_cf:
            cf_idx = rng.integers(0, len(CF_MODELS), size=(n1, n2, n3))
            ff = ff_lut[cf_idx]
            cg = cg_lut[cf_idx]
        else:
            idx = CF_MODELS.index(self._cfg.car_following)
            ff, cg = ff_lut[idx], cg_lut[idx]

        w    = np.clip(congestion, 0.0, 1.0)
        base = (1.0 - w) * ff + w * cg

        K_comm = self._K_comm_vector(flat_b, lat_b, loss_b)
        K_nat  = self._K_nat_vector(cap_b, dmult_b, bias_b)
        K_dem  = self._K_dem_vector(ns_b * dmult_b,
                                    ew_b * dmult_b,
```

```
                                    cap_b)

        raw = base * kc_b * K_comm * K_nat * K_dem

        log_c      = 2.0 / (1.0 + np.exp(-sigk_b * (sev - sigx0_b)))
        true_value = raw * (0.85 + 0.30 * log_c)

        noisy = true_value * (1.0 + rng.normal(0.0, self._cfg.noise_frac,
                                               size=true_value.shape))
        outlier = rng.random(size=true_value.shape) < self._cfg.outlier_f
        noisy = np.where(outlier, noisy * self._cfg.outlier_mult, noisy)

        travel = np.maximum(5.0, noisy)
        excess = np.maximum(0.0, travel / np.maximum(base, 1.0) - 1.0)

        queue = np.maximum(0.0, 3.0 + 12.0 * excess
                                + rng.normal(0.0, 0.6, size=travel.shape)
        wait  = np.maximum(0.0, 5.0 + 20.0 * excess
                                + rng.normal(0.0, 1.2, size=travel.shape)
        thr   = np.maximum(50.0, 950.0 / (1.0 + 0.9 * excess))
        aoi   = np.maximum(0.0, 40.0 + 350.0 * sev_comm
                                + 20.0 * np.abs(rng.normal(0.0, 1.0,
                                                           size=travel.sh
        deliver = np.clip(
            0.99 - 0.55 * sev_comm - 0.20 * sev_nat
                 + rng.normal(0.0, 0.01, size=travel.shape),
            0.0, 1.0,
        )

        def _broadcast(x: np.ndarray) -> np.ndarray:
            return np.broadcast_to(x, (n1, n2, n3)).copy()

        arrays: dict[str, np.ndarray] = {
            "travel_time_s":  _broadcast(travel),
            "queue_veh":      _broadcast(queue),
            "waiting_time_s": _broadcast(wait),
            "throughput_vph": _broadcast(thr),
            "mean_aoi_ms":    _broadcast(aoi),
            "delivery_ratio": _broadcast(deliver),
            "K_ctrl":         _broadcast(kc_b * np.ones_like(base)),
            "K_comm":         _broadcast(K_comm * np.ones_like(base)),
            "K_nat":          _broadcast(K_nat  * np.ones_like(base)),
            "K_dem":          _broadcast(K_dem  * np.ones_like(base)),
            "severity":       _broadcast(sev),
            "true_value":     _broadcast(true_value),
        }
        self._arrays = arrays
        return arrays

    def _ensure_records(self) -> np.ndarray:
        if self._M is None:
            self.build()
        assert self._M is not None
        return self._M
```

```
    def build(self, force: bool = False) -> np.ndarray:
        if self._M is not None and not force:
            return self._M

        arrays = self._compute_arrays(force=force)
        n1, n2, n3 = self._shape

        if self._cfg.progress and self._cfg.verbose and self.size > 1000:
            print(f"  building M with shape {(n1, n2, n3)} "
                  f"= {self.size:,} cells ...")

        ctrl_names = np.array(self._ctrls)[:, None, None]
        scen_names = np.array(self._scens)[None, :, None]
        cond_names = np.array(self._conds)[None, None, :]

        M = np.empty((n1, n2, n3), dtype=object)
        for i, j, k in product(range(n1), range(n2), range(n3)):
            M[i, j, k] = {
                "i": i, "j": j, "k": k,
                "controller":     ctrl_names[i, 0, 0],
                "car_following":  self._cfg.car_following,
                "scenario":       scen_names[0, j, 0],
                "condition":      cond_names[0, 0, k],
                "travel_time_s":  round(float(arrays["travel_time_s"][i,
                "queue_veh":      round(float(arrays["queue_veh"][i, j, k
                "waiting_time_s": round(float(arrays["waiting_time_s"][i,
                "throughput_vph": round(float(arrays["throughput_vph"][i,
                "mean_aoi_ms":    round(float(arrays["mean_aoi_ms"][i, j,
                "delivery_ratio": round(float(arrays["delivery_ratio"][i,
                "K_ctrl":         round(float(arrays["K_ctrl"][i, j, k]),
                "K_comm":         round(float(arrays["K_comm"][i, j, k]),
                "K_nat":          round(float(arrays["K_nat"][i, j, k]),
                "K_dem":          round(float(arrays["K_dem"][i, j, k]),
                "severity":       round(float(arrays["severity"][i, j, k]
                "true_value":     round(float(arrays["true_value"][i, j,
            }
        self._M = M
        return M

    def as_tensor(self, M: Optional[np.ndarray] = None,
                  cols: Optional[tuple[str, ...]] = None) -> np.ndarray:
        cols = cols or self.NUMERIC_COLS
        arrays = self._compute_arrays()
        return np.stack([arrays[c] for c in cols], axis=-1)

    build_numeric = as_tensor

    def slice_controller(self, i: int,
                         M: Optional[np.ndarray] = None) -> np.ndarray:
        return (self._ensure_records() if M is None else M)[i]

    def slice_scenario(self, j: int,
                       M: Optional[np.ndarray] = None) -> np.ndarray:
```

```
        return (self._ensure_records() if M is None else M)[:, j, :]

    def slice_condition(self, k: int,
                        M: Optional[np.ndarray] = None) -> np.ndarray:
        return (self._ensure_records() if M is None else M)[:, :, k]

    def stats(self, M: Optional[np.ndarray] = None) -> dict[str, Any]:
        _ = M
        arrays = self._compute_arrays()
        out: dict[str, Any] = {"shape": self._shape, "n_cells": self.size
        for name in self.NUMERIC_COLS:
            x = arrays[name]
            out[name] = {
                "mean":  round(float(x.mean()), 3),
                "stdev": round(float(x.std()), 3),
                "min":   round(float(x.min()), 3),
                "max":   round(float(x.max()), 3),
            }
        return out

    def to_csv(self, M: Optional[np.ndarray] = None,
               path: Optional[str] = None) -> str:
        path = path or self.csv_path
        M = self._ensure_records() if M is None else M
        n1, n2, n3 = M.shape
        sample = M[0, 0, 0]
        cols = list(sample.keys())
        with open(path, "w", newline="") as fh:
            w = csv.DictWriter(fh, fieldnames=cols)
            w.writeheader()
            for i, j, k in product(range(n1), range(n2), range(n3)):
                w.writerow(M[i, j, k])
        if self._cfg.verbose:
            print(f"  wrote {n1 * n2 * n3:,} rows "
                  f"({n1} x {n2} x {n3}) -> {path}")
        return path

    def to_npy(self, M: Optional[np.ndarray] = None,
               path: Optional[str] = None) -> str:
        path = path or self.npy_path
        T = self.as_tensor(M) if M is not None else self.as_tensor()
        np.save(path, T)
        if self._cfg.verbose:
            print(f"  wrote tensor {T.shape} -> {path}")
        return path

    def summary(self, M: Optional[np.ndarray] = None) -> str:
        s = self.stats(M)
        n1, n2, n3 = self._shape
        lines = [
            "=" * 90,
            f"  SYNTHETIC DATASET  -  preset={self._cfg.preset!r}",
            "=" * 90,
            f"  shape          : {self._shape}",
```

```
            f"  total cells    : {self.size:,}",
            f"  axis 0 (ctrl)  : {n1}",
            f"  axis 1 (scen)  : {n2}",
            f"  axis 2 (cond)  : {n3}",
            f"  seed           : {self._cfg.seed}",
            f"  car-following  : {self._cfg.car_following}"
            + ("  (sampled per cell)" if self._cfg.sample_cf else ""),
            f"  noise frac     : {self._cfg.noise_frac}",
            "",
            f"  {'metric':<18}{'mean':>10}{'stdev':>10}"
            f"{'min':>10}{'max':>10}",
            "  " + "-" * 58,
        ]
        for k, v in s.items():
            if k in ("shape", "n_cells"):
                continue
            lines.append(f"  {k:<18}{v['mean']:>10.3f}{v['stdev']:>10.3f}
                         f"{v['min']:>10.3f}{v['max']:>10.3f}")
        text = "\n".join(lines)
        if self._cfg.verbose:
            print(text)
        return text


# ============================================================================
#  SECTION 11  —  TENSOR PIPELINE  (Ridge + kNN + GP + EI active learning
# ============================================================================

DEFAULT_COLS: tuple[str, ...] = (
    "travel_time_s", "queue_veh", "waiting_time_s", "throughput_vph",
    "mean_aoi_ms", "delivery_ratio",
    "K_ctrl", "K_comm", "K_nat", "K_dem",
    "severity", "true_value",
)
DEFAULT_TARGET = "travel_time_s"
LEAKY_COLS: tuple[str, ...] = ("true_value",)


@dataclass
class PipelineReport:
    target:      str
    n_train:     int
    n_test:      int
    n_features:  int
    scores:      dict[str, dict[str, float]] = field(default_factory=dict
    al_trace:    list[dict[str, float]] = field(default_factory=list)

    def __str__(self) -> str:
        lines = [
            "=" * 78,
            f"  PIPELINE REPORT  -  target = {self.target!r}",
            "=" * 78,
            f"  train / test   : {self.n_train:,} / {self.n_test:,}",
            f"  feature count  : {self.n_features}",
```

```
            "",
            f"  {'model':<18}{'MAE':>10}{'RMSE':>10}{'R2':>10}",
            "  " + "-" * 46,
        ]
        for name, s in self.scores.items():
            lines.append(f"  {name:<18}{s['mae']:>10.4f}"
                         f"{s['rmse']:>10.4f}{s['r2']:>10.4f}")
        if self.al_trace:
            lines.append("")
            lines.append("  Active-learning trace (EI loop)")
            lines.append(f"  {'round':>6}{'MAE':>10}{'RMSE':>10}{'R2':>10
            lines.append("  " + "-" * 36)
            for row in self.al_trace:
                lines.append(f"  {row['round']:>6}"
                             f"{row['mae']:>10.4f}"
                             f"{row['rmse']:>10.4f}"
                             f"{row['r2']:>10.4f}")
        return "\n".join(lines)


@dataclass
class TensorPipeline:
    """Loads a (n1, n2, n3, F) numeric tensor and runs a small ML pipelin

    path:        str = "synthetic_tensor.npy"
    cols:        tuple[str, ...] = DEFAULT_COLS
    target:      str = DEFAULT_TARGET
    seed:        int = 42
    test_frac:   float = 0.2
    verbose:     bool = True

    X:            Optional[np.ndarray] = field(init=False, default=None,
    y:            Optional[np.ndarray] = field(init=False, default=None,
    grid_shape:   Optional[tuple[int, ...]] = field(init=False, default=N
    _rng:         np.random.Generator = field(init=False, repr=False)
    _split:       Optional[tuple[np.ndarray, np.ndarray,
                                 np.ndarray, np.ndarray]] = field(init=Fal
    _preds:       Optional[np.ndarray] = field(init=False, default=None,

    def __post_init__(self) -> None:
        self._rng = np.random.default_rng(self.seed)
        self.load()

    def load(self, force: bool = False) -> "TensorPipeline":
        if self.X is not None and not force:
            return self
        path = Path(self.path)
        if not path.exists():
            raise FileNotFoundError(
                f"Tensor file not found: {path}. "
                f"Build it via SyntheticTrafficDataset.to_npy().")
        T = np.load(path)
        if T.ndim != 4:
            raise ValueError(
```

```
                f"Expected 4-D tensor (n1, n2, n3, F); got shape {T.shape

        self.grid_shape = T.shape[:3]
        F = T.shape[-1]
        if F != len(self.cols):
            raise ValueError(
                f"Feature axis is {F} but {len(self.cols)} column names "
                f"were supplied. Pass cols=... matching your tensor.")
        if self.target not in self.cols:
            raise ValueError(f"target {self.target!r} not in {self.cols}"

        flat = T.reshape(-1, F)
        tgt_idx = self.cols.index(self.target)
        drop = {tgt_idx} | {self.cols.index(c) for c in LEAKY_COLS
                            if c in self.cols}
        keep = [i for i in range(F) if i not in drop]

        self.X = flat[:, keep].astype(np.float64)
        self.y = flat[:, tgt_idx].astype(np.float64)

        if self.verbose:
            print(f"  loaded {path}  shape={T.shape}  "
                  f"features={self.X.shape[1]}  rows={self.X.shape[0]:,}"
        return self

    def split(self, test_frac: Optional[float] = None,
              force: bool = False
              ) -> tuple[np.ndarray, np.ndarray,
                         np.ndarray, np.ndarray]:
        if self._split is not None and not force and test_frac is None:
            return self._split
        assert self.X is not None and self.y is not None
        frac = self.test_frac if test_frac is None else test_frac
        n = len(self.X)
        idx = self._rng.permutation(n)
        cut = int(round(n * (1 - frac)))
        tr, te = idx[:cut], idx[cut:]
        out = (self.X[tr], self.X[te], self.y[tr], self.y[te])
        self._split = out
        return out

    @staticmethod
    def _score(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float
        err  = y_pred - y_true
        mae  = float(np.mean(np.abs(err)))
        rmse = float(np.sqrt(np.mean(err ** 2)))
        ss_res = float(np.sum(err ** 2))
        ss_tot = float(np.sum((y_true - y_true.mean()) ** 2)) or 1e-12
        r2 = 1.0 - ss_res / ss_tot
        return {"mae": mae, "rmse": rmse, "r2": r2}

    # ---------------------------------------------------------------- FI
    @staticmethod
    def _ridge_fit(X: np.ndarray, y: np.ndarray,
```

```
                alpha: float = 1.0) -> np.ndarray:
    """Closed-form ridge regression.
    Returns the coefficient vector w of length (d+1) where w[0] is th
    intercept and w[1:] are the per-feature slopes on the NORMALISED
    features.  Previously this returned a placeholder of zeros + w,
    which silently broke any caller using _ridge_fit instead of
    _ridge_predict.  Now it returns the actual coefficient vector.
    """
    n, d = X.shape
    mu = X.mean(axis=0, keepdims=True)
    sd = X.std(axis=0, keepdims=True) + 1e-9
    Xn = (X - mu) / sd
    Xb = np.hstack([np.ones((n, 1)), Xn])
    A = Xb.T @ Xb + alpha * np.eye(d + 1)
    A[0, 0] -= alpha                    # do not regularise the inter
    w = np.linalg.solve(A, Xb.T @ y)
    return w                              # shape (d+1,) -- FIXED

def _ridge_predict(self, X_tr: np.ndarray, y_tr: np.ndarray,
                   X_te: np.ndarray, alpha: float = 1.0) -> np.ndarra
    mu = X_tr.mean(axis=0, keepdims=True)
    sd = X_tr.std(axis=0, keepdims=True) + 1e-9
    Xn_tr = (X_tr - mu) / sd
    Xn_te = (X_te - mu) / sd
    Xb_tr = np.hstack([np.ones((len(Xn_tr), 1)), Xn_tr])
    Xb_te = np.hstack([np.ones((len(Xn_te), 1)), Xn_te])
    A = Xb_tr.T @ Xb_tr + alpha * np.eye(Xb_tr.shape[1])
    A[0, 0] -= alpha
    w = np.linalg.solve(A, Xb_tr.T @ y_tr)
    return Xb_te @ w

def _knn_predict(self, X_tr: np.ndarray, y_tr: np.ndarray,
                 X_te: np.ndarray, k: int = 5) -> np.ndarray:
    mu = X_tr.mean(axis=0, keepdims=True)
    sd = X_tr.std(axis=0, keepdims=True) + 1e-9
    A = (X_tr - mu) / sd
    B = (X_te - mu) / sd
    out = np.empty(len(B))
    chunk = 2048
    for s in range(0, len(B), chunk):
        b = B[s:s + chunk]
        d2 = ((b[:, None, :] - A[None, :, :]) ** 2).sum(-1)
        idx = np.argpartition(d2, k, axis=1)[:, :k]
        out[s:s + chunk] = y_tr[idx].mean(axis=1)
    return out

def _gp_predict(self, X_tr: np.ndarray, y_tr: np.ndarray,
                X_te: np.ndarray,
                lengthscale: float = 1.0,
                noise: float = 0.1
                ) -> tuple[np.ndarray, np.ndarray]:
    """Tiny ARD-RBF GP; returns (mean, variance) on the test set."""
    mu = X_tr.mean(axis=0, keepdims=True)
    sd = X_tr.std(axis=0, keepdims=True) + 1e-9
```

```
        A = (X_tr - mu) / sd
        B = (X_te - mu) / sd

        def kern(P: np.ndarray, Q: np.ndarray) -> np.ndarray:
            Pn = P / lengthscale
            Qn = Q / lengthscale
            p = (Pn ** 2).sum(1)[:, None]
            q = (Qn ** 2).sum(1)[None, :]
            return np.exp(-0.5 * np.maximum(p + q - 2 * Pn @ Qn.T, 0.0))

        K   = kern(A, A) + (noise ** 2 + 1e-6) * np.eye(len(A))
        Ks  = kern(A, B)
        Kss = np.diag(kern(B, B))
        L = np.linalg.cholesky(K)
        alpha = np.linalg.solve(L.T, np.linalg.solve(L, y_tr))
        mu_pred = Ks.T @ alpha
        v = np.linalg.solve(L, Ks)
        var = np.maximum(Kss - (v ** 2).sum(0), 1e-9)
        return mu_pred, var

    def run(self, budget: int = 200, n_init: int = 20,
            gp_lengthscale: float = 1.0,
            gp_noise: float = 0.1) -> PipelineReport:
        X_tr, X_te, y_tr, y_te = self.split()
        n_feat = X_tr.shape[1]
        report = PipelineReport(
            target=self.target,
            n_train=len(X_tr), n_test=len(X_te),
            n_features=n_feat)

        baselines = {
            "mean":  np.full_like(y_te, y_tr.mean()),
            "ridge": self._ridge_predict(X_tr, y_tr, X_te),
            "knn5":  self._knn_predict(X_tr, y_tr, X_te, k=5),
        }
        for name, pred in baselines.items():
            report.scores[name] = self._score(y_te, pred)

        ridge_tr = self._ridge_predict(X_tr, y_tr, X_tr)
        resid_tr = y_tr - ridge_tr
        gp_mu, gp_var = self._gp_predict(X_tr, resid_tr, X_te,
                                         gp_lengthscale, gp_noise)
        gp_pred = self._ridge_predict(X_tr, y_tr, X_te) + gp_mu
        report.scores["ridge+gp"] = self._score(y_te, gp_pred)

        report.al_trace = self._active_learn(
            X_tr, y_tr, X_te, y_te,
            budget=budget, n_init=n_init,
            gp_lengthscale=gp_lengthscale, gp_noise=gp_noise)
        if self.verbose:
            print(report)
        return report

    def _active_learn(self, X_tr: np.ndarray, y_tr: np.ndarray,
```

```
                    X_te: np.ndarray, y_te: np.ndarray,
                    budget: int, n_init: int,
                    gp_lengthscale: float, gp_noise: float
                    ) -> list[dict[str, float]]:
        """EI loop on Ridge residuals.  Uses scipy.special.erf (vectorise
        n_pool = min(2000, max(200, len(X_tr) // 10))
        seen = self._rng.choice(len(X_tr), size=n_init, replace=False)
        seen_set = set(seen.tolist())
        trace: list[dict[str, float]] = []

        sqrt2 = math.sqrt(2.0)
        inv_sqrt_2pi = 1.0 / math.sqrt(2.0 * math.pi)

        for rnd in range(budget):
            ridge_obs = self._ridge_predict(X_tr[seen], y_tr[seen], X_tr[
            resid_obs = y_tr[seen] - ridge_obs
            mu, var = self._gp_predict(X_tr[seen], resid_obs, X_tr,
                                       gp_lengthscale, gp_noise)

            pool_idx = np.array([i for i in range(len(X_tr))
                                 if i not in seen_set])
            if len(pool_idx) == 0:
                break
            sample = self._rng.choice(
                pool_idx, size=min(n_pool, len(pool_idx)), replace=False)
            mu_s, var_s = mu[sample], var[sample]
            sigma = np.sqrt(np.maximum(var_s, 1e-12))
            best = y_tr[seen].min()
            xi = 0.01
            z = (best - mu_s - xi) / sigma

            # FIX: vectorised erf (was np.vectorize(math.erf) -- ~100x sl
            phi = 0.5 * (1.0 + _scipy_erf(z / sqrt2))
            pdf = np.exp(-0.5 * z ** 2) * inv_sqrt_2pi
            ei = (best - mu_s - xi) * phi + sigma * pdf

            pick = sample[int(np.argmax(ei))]
            seen_set.add(int(pick))
            seen = np.append(seen, pick)

            pred_te = self._ridge_predict(X_tr[seen], y_tr[seen], X_te)
            s = self._score(y_te, pred_te)
            trace.append({"round": float(rnd + 1), **s})

        return trace

    def export(self, path: str = "predictions.npy") -> str:
        if self._preds is None:
            X_tr, X_te, y_tr, y_te = self.split()
            ridge_all = self._ridge_predict(X_tr, y_tr, self.X)  # type:
            resid_tr = y_tr - self._ridge_predict(X_tr, y_tr, X_tr)
            mu, _ = self._gp_predict(X_tr, resid_tr, self.X)  # type: ign
            self._preds = ridge_all + mu
        np.save(path, self._preds)
```

```
        if self.verbose:
            print(f"  wrote {len(self._preds):,} predictions -> {path}")
        return path

    def export_csv(self, path: str = "predictions.csv") -> str:
        if self._preds is None:
            self.export()
        assert self._preds is not None and self.y is not None
        with open(path, "w", newline="") as fh:
            w = csv.writer(fh)
            w.writerow(["row", "y_true", "y_pred", "residual"])
            for i, (t, p) in enumerate(zip(self.y, self._preds)):
                w.writerow([i, f"{t:.4f}", f"{p:.4f}", f"{t - p:.4f}"])
        if self.verbose:
            print(f"  wrote {len(self._preds):,} rows -> {path}")
        return path

    def summary(self) -> str:
        assert self.X is not None and self.y is not None
        lines = [
            "=" * 78,
            "  TENSOR PIPELINE - LOADED",
            "=" * 78,
            f"  file          : {self.path}",
            f"  grid shape    : {self.grid_shape}",
            f"  flat rows     : {self.X.shape[0]:,}",
            f"  features      : {self.X.shape[1]}",
            f"  target        : {self.target!r}",
            f"  target mean   : {self.y.mean():.3f}",
            f"  target stdev  : {self.y.std():.3f}",
            f"  target range  : [{self.y.min():.3f}, {self.y.max():.3f}]"
        ]
        text = "\n".join(lines)
        if self.verbose:
            print(text)
        return text


# ============================================================================
#  SECTION 12  —  ANALYTICAL SURROGATE  (replaces the missing module)
# ============================================================================

class Surrogate:
    """Closed-form surrogate that mirrors the synthetic dataset's factors

    Predicts travel_time_s for a (controller, cf, scenario, natural, dema
    cell WITHOUT running the simulator, using the multiplicative model:

        TT = base(cf, congestion) * K_ctrl(family) * K_comm(family, lat,
                                  * K_nat(cap, dmult, bias)
                                  * K_dem(ns, ew, cap)
                                  * logistic_correction(severity)
                                  + noise
    """
```

```python
    def __init__(self, seed: int = 42):
        self.rng = np.random.default_rng(seed)
        self.seed = seed

    # ---- axis accessors (for the BayesianActiveLearner) ---------------
    @property
    def controllers(self) -> tuple[str, ...]:
        # FIX: return only the controllers that are ACTUALLY REGISTERED i
        # TrafficControlAlgorithm.CONTROLLERS (the 17 instantiable ones),
        # NOT the full 128-family synthetic-dataset catalog.  Otherwise
        # TuRBO picks cells like 'robust-td3-v2' that have no impl and
        # instantiation raises ValueError.
        return tuple(sorted(TrafficControlAlgorithm.CONTROLLERS))

    @property
    def cf_models(self) -> tuple[str, ...]:
        return CF_MODELS

    @property
    def scenarios(self) -> tuple[str, ...]:
        return ("S1", "S2", "S3", "S4")

    @property
    def naturals(self) -> tuple[str, ...]:
        return tuple(NATURAL_SCENARIOS)

    @property
    def demands(self) -> tuple[str, ...]:
        return tuple(sorted(DEMAND_PATTERNS))

    # ---- the multiplicative prediction --------------------------------
    def predict(self, controller: str, cf: str,
                scenario: str, natural: str, demand: str) -> float:
        """Return predicted travel_time_s for the given 5-tuple."""
        family = CONTROLLER_FAMILY.get(controller, "baseline")
        kc     = K_CTRL_FAMILY.get(family, 1.0)
        flat   = RESILIENCE_FLATTENING.get(family, 1.0)

        # comm
        if scenario in COMMUNICATION_SCENARIOS:
            cfg_c = COMMUNICATION_SCENARIOS[scenario]
            lat   = float(cfg_c["latency_ms"])
            loss  = float(cfg_c["loss_rate"])
        else:
            lat, loss = 100.0, 0.05
        rho = 0.5 * (lat / 600.0) + 0.5 * loss
        k   = _K_COMM_BASE_K / flat
        x0  = _K_COMM_BASE_X0 * (1.0 + 0.5 * (1.0 - flat))
        L   = _K_COMM_BASE_L * flat
        K_comm = 1.0 + L / (1.0 + math.exp(-k * (rho - x0)))

        # natural
        cfg_n = NATURAL_SCENARIOS.get(natural, NATURAL_SCENARIOS["N-clear
```

```
        cap    = float(cfg_n.get("capacity", 1.0))
        dmult  = float(cfg_n.get("demand_mult", 1.0))
        bias   = float(cfg_n.get("direction_bias", 0.0))
        K_nat = (1.0 / max(cap, 0.1)
                 * (1.0 + 0.35 * (dmult - 1.0))
                 * (1.0 + 0.15 * abs(bias)))

        # demand
        cfg_d = DEMAND_PATTERNS.get(demand, DEMAND_PATTERNS["D1"])
        ns, ew = float(cfg_d["ns"]), float(cfg_d["ew"])
        arrival = ns + ew
        service = 2 * SATURATION_FLOW_VPS * cap * 10.0
        rho_d = min(0.95, arrival / max(service, 1e-6))
        K_dem = 1.0 / (1.0 - rho_d)

        # congestion / base travel time
        sev_comm = 0.5 * (lat / 600.0) + 0.5 * loss
        sev_nat  = 1.0 - cap
        sev_dem  = min(0.5, (ns + ew - 0.6) / 1.2)
        sev      = 0.5 * sev_comm + 0.3 * sev_nat + 0.2 * sev_dem
        cong     = min(0.99, 0.4*sev_comm + 0.4*sev_nat + 0.2*sev_dem)

        cf_bench = _CF_BENCHMARK.get(cf, _CF_BENCHMARK["SAFE"])
        ff = cf_bench["free_flow"]["in_system_s"]
        cg = cf_bench["congested"]["in_system_s"]
        w = max(0.0, min(1.0, cong))
        base = (1.0 - w) * ff + w * cg

        sig = LogisticSignatureDS(L=140.0, k=0.0143, x0=0.05)
        log_c = 2.0 / (1.0 + math.exp(-sig.k * (sev - sig.x0)))
        true_value = base * kc * K_comm * K_nat * K_dem * (0.85 + 0.30 *

        noise = 1.0 + self.rng.normal(0.0, 0.06)
        return float(max(5.0, true_value * noise))


# ============================================================================
#  SECTION 13  —  BAYESIAN ACTIVE LEARNING  (replaces the missing module)
# ============================================================================

def expected_improvement(mu: np.ndarray, sigma: np.ndarray,
                         best: float, xi: float = 0.01) -> np.ndarray:
    """Vectorised Expected Improvement."""
    sqrt2 = math.sqrt(2.0)
    inv_sqrt_2pi = 1.0 / math.sqrt(2.0 * math.pi)
    z = (best - mu - xi) / np.maximum(sigma, 1e-12)
    phi = 0.5 * (1.0 + _scipy_erf(z / sqrt2))
    pdf = np.exp(-0.5 * z ** 2) * inv_sqrt_2pi
    return (best - mu - xi) * phi + sigma * pdf


class Encoder:
    """One-hot encoder for the 5-axis categorical grid."""
```

```
    def __init__(self, controllers: Sequence[str], cf_models: Sequence[st
                 scenarios: Sequence[str], naturals: Sequence[str],
                 demands: Sequence[str]) -> None:
        self.controllers = tuple(controllers)
        self.cf_models   = tuple(cf_models)
        self.scenarios   = tuple(scenarios)
        self.naturals    = tuple(naturals)
        self.demands     = tuple(demands)
        self.n_features  = (len(self.controllers) + len(self.cf_models)
                            + len(self.scenarios) + len(self.naturals)
                            + len(self.demands))

    def encode(self, cell: tuple[str, str, str, str, str]) -> np.ndarray:
        ctrl, cf, sc, nat, dem = cell
        v = np.zeros(self.n_features, dtype=np.float64)
        axes = (self.controllers, self.cf_models, self.scenarios,
                self.naturals, self.demands)
        values = (ctrl, cf, sc, nat, dem)
        i = 0
        for axis, val in zip(axes, values):
            idx = axis.index(val) if val in axis else 0
            v[i + idx] = 1.0
            i += len(axis)
        return v

    def encode_many(self, cells: Sequence[tuple[str, str, str, str, str]]
                    ) -> np.ndarray:
        return np.vstack([self.encode(c) for c in cells])


class SurrogateGP:
    """Tiny ARD-RBF Gaussian process surrogate."""

    def __init__(self, n_features: int,
                 lengthscale: float = 1.0,
                 noise: float = 0.1) -> None:
        self.n_features  = n_features
        self.lengthscale = lengthscale
        self.noise       = noise
        self.X_tr: Optional[np.ndarray] = None
        self.y_tr: Optional[np.ndarray] = None
        self._L:   Optional[np.ndarray] = None
        self._alpha: Optional[np.ndarray] = None
        self._mu = 0.0
        self._sd = 1.0

    def _kern(self, P: np.ndarray, Q: np.ndarray) -> np.ndarray:
        Pn = P / self.lengthscale
        Qn = Q / self.lengthscale
        p = (Pn ** 2).sum(1)[:, None]
        q = (Qn ** 2).sum(1)[None, :]
        return np.exp(-0.5 * np.maximum(p + q - 2 * Pn @ Qn.T, 0.0))

    def fit(self, X: np.ndarray, y: np.ndarray) -> "SurrogateGP":
```

```
        self.X_tr = X.copy()
        self.y_tr = y.copy()
        self._mu = float(y.mean()) if len(y) else 0.0
        self._sd = float(y.std() + 1e-9) if len(y) else 1.0
        yn = (y - self._mu) / self._sd
        K = self._kern(X, X) + (self.noise ** 2 + 1e-6) * np.eye(len(X))
        self._L = np.linalg.cholesky(K)
        self._alpha = np.linalg.solve(self._L.T,
                                      np.linalg.solve(self._L, yn))
        return self

    def predict(self, X: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
        if self.X_tr is None or self._L is None or self._alpha is None:
            n = len(X)
            return (np.full(n, self._mu),
                    np.full(n, 1.0))
        Ks = self._kern(self.X_tr, X)
        mu = Ks.T @ self._alpha * self._sd + self._mu
        v = np.linalg.solve(self._L, Ks)
        Kss = np.diag(self._kern(X, X))
        var = np.maximum(Kss - (v ** 2).sum(0), 1e-9) * (self._sd ** 2)
        return mu, var


class BayesianActiveLearner:
    """Vanilla EI active learner — base class for TrustRegionLearner."""

    def __init__(self, surrogate: Surrogate, seed: int = 0) -> None:
        self.surrogate = surrogate
        self.rng = random.Random(seed)
        self.np_rng = np.random.default_rng(seed)

        self.controllers = tuple(surrogate.controllers)
        self.cf_models   = tuple(surrogate.cf_models)
        self.scenarios   = tuple(surrogate.scenarios)
        self.naturals    = tuple(surrogate.naturals)
        self.demands     = tuple(surrogate.demands)
        self.axes        = (self.controllers, self.cf_models,
                            self.scenarios, self.naturals, self.demands)
        self.encoder     = Encoder(*self.axes)
        self.gp          = SurrogateGP(self.encoder.n_features)

        self.cells:    list[tuple[str, str, str, str, str]] = []
        self.y_sim:    list[float] = []
        self.y_pred:   list[float] = []
        self.y_resid:  list[float] = []
        self.best_value: float = float("inf")
        self.best_cell: Optional[tuple[str, str, str, str, str]] = None

    # ---- observe one cell ----------------------------------------------
    def _observe(self, cell: tuple[str, str, str, str, str],
                 n_steps: int = 200) -> float:
        ctrl_name, cf, scen, nat, dem = cell
        d = DEMAND_PATTERNS.get(dem, DEMAND_PATTERNS["D1"])
```

```
        c = TrafficControlAlgorithm(ctrl_name, scen, nat,
                                    seed=self.rng.randint(1, 10000),
                                    car_following=cf)
        simulate_episode(c, n_steps=n_steps,
                         base_demand_ns=float(d["ns"]),
                         base_demand_ew=float(d["ew"]),
                         seed=self.rng.randint(1, 10000))
        y_real = float(c.get_metrics()["travel_time_s"])
        y_surr = float(self.surrogate.predict(*cell))
        self.cells.append(cell)
        self.y_sim.append(y_real)
        self.y_pred.append(y_surr)
        self.y_resid.append(y_real - y_surr)
        if y_real < self.best_value:
            self.best_value = y_real
            self.best_cell  = cell
        return y_real

    def _fit_gp(self) -> None:
        if not self.cells:
            return
        X = self.encoder.encode_many(self.cells)
        y = np.array(self.y_resid, dtype=np.float64)
        self.gp.fit(X, y)

    def _surrogate_predict(self, cell: tuple[str, str, str, str, str]
                           ) -> float:
        return float(self.surrogate.predict(*cell))

    def corrected_predict(self, cell: tuple[str, str, str, str, str]
                          ) -> tuple[float, float]:
        x = self.encoder.encode(cell).reshape(1, -1)
        mu, var = self.gp.predict(x)
        return (float(self.surrogate.predict(*cell) + mu[0]),
                float(var[0]))

    # ---- the basic EI loop ------------------------------------------
    def run(self, n_init: int = 20, budget: int = 100,
            n_steps: int = 200, n_candidates: int = 2000,
            progress: bool = True) -> dict[str, Any]:
        t0 = time.time()
        if progress:
            print(f"\n  EI ACTIVE LEARNING — budget={budget}")

        all_cells = list(product(*self.axes))
        self.rng.shuffle(all_cells)
        for cell in all_cells[:n_init]:
            self._observe(cell, n_steps=n_steps)

        observed = set(self.cells)
        for rnd in range(n_init + 1, budget + 1):
            self._fit_gp()
            pool = [c for c in all_cells if c not in observed]
            if not pool:
```

```
                break
            if len(pool) > n_candidates:
                pool = self.rng.sample(pool, n_candidates)
            X_pool = self.encoder.encode_many(pool)
            mu, var = self.gp.predict(X_pool)
            pred_pool = np.array([self._surrogate_predict(c) for c in poo
            pred_mu = pred_pool + mu
            pred_sd = np.sqrt(var)
            ei = expected_improvement(pred_mu, pred_sd, best=self.best_va
            idx = int(np.argmax(ei))
            chosen = pool[idx]
            y_real = self._observe(chosen, n_steps=n_steps)
            observed.add(chosen)
            if progress and (rnd % max(1, budget // 10) == 0
                             or rnd == budget):
                rmse = math.sqrt(statistics.fmean(r ** 2 for r in self.y_
                print(f"    round {rnd:>4}/{budget}  RMSE={rmse:>6.3f}  "
                      f"best={self.best_value:>7.2f}")

        self._fit_gp()
        return {
            "n_sims":         len(self.cells),
            "best_cell":      self.best_cell,
            "best_value":     round(self.best_value, 2),
            "residual_rmse":  round(math.sqrt(statistics.fmean(
                r ** 2 for r in self.y_resid)), 3),
            "residual_mae":   round(statistics.fmean(
                abs(r) for r in self.y_resid), 3),
            "elapsed_s":      round(time.time() - t0, 2),
        }

    # ---- held-out evaluation ------------------------------------------
    def evaluate_holdout(self, n_holdout: int = 30,
                         n_steps: int = 200) -> dict[str, float]:
        all_cells = list(product(*self.axes))
        unseen = [c for c in all_cells if c not in set(self.cells)]
        if not unseen:
            unseen = all_cells.copy()
        self.rng.shuffle(unseen)
        sample = unseen[:n_holdout]
        raw_err, corr_err = [], []
        for cell in sample:
            d = DEMAND_PATTERNS.get(cell[4], DEMAND_PATTERNS["D1"])
            c = TrafficControlAlgorithm(cell[0], cell[2], cell[3],
                                        seed=self.rng.randint(1, 10000),
                                        car_following=cell[1])
            simulate_episode(c, n_steps=n_steps,
                             base_demand_ns=float(d["ns"]),
                             base_demand_ew=float(d["ew"]),
                             seed=self.rng.randint(1, 10000))
            y_real = float(c.get_metrics()["travel_time_s"])
            y_raw  = float(self.surrogate.predict(*cell))
            y_corr, _ = self.corrected_predict(cell)
            raw_err.append(abs(y_real - y_raw))
```

```
            corr_err.append(abs(y_real - y_corr))
        return {
            "raw_mae":   round(statistics.fmean(raw_err), 3),
            "raw_rmse":  round(math.sqrt(statistics.fmean(e**2 for e in r
            "corr_mae":  round(statistics.fmean(corr_err), 3),
            "corr_rmse": round(math.sqrt(statistics.fmean(e**2 for e in c
        }


# ============================================================================
#  SECTION 14  —  TRUST-REGION ACTIVE LEARNING  (TuRBO over the grid)
# ============================================================================

def canonical_axes() -> tuple[tuple[str, ...], ...]:
    """Return (controllers, cf, scenarios, naturals, demands) ordered for

    FIX: returns only the controllers that are ACTUALLY REGISTERED in
    TrafficControlAlgorithm.CONTROLLERS (the 17 instantiable ones), not
    the full 128-family synthetic-dataset catalog.  Otherwise TuRBO picks
    cells like 'robust-td3-v2' that have no impl and instantiation fails.
    """
    fam_rank = {"baseline": 0, "adaptive": 1, "coordinated": 2,
                "learning": 3, "field": 4, "resilient": 5}
    # Use the actually-registered controllers (17), not CONTROLLER_FAMILY
    ctrls = tuple(sorted(TrafficControlAlgorithm.CONTROLLERS,
                         key=lambda c: (fam_rank.get(
                             CONTROLLER_FAMILY.get(c, "baseline"), 9), c)
    cf = ("SAFE", "IDM", "GIPPS")
    sc = ("S1", "S2", "S3", "S4")
    nats_by_fam = {
        "weather":  ["N-clear", "N-rain", "N-snow", "N-fog"],
        "incident": ["N-accident", "N-workzone"],
        "demand":   ["N-rush-am", "N-rush-pm", "N-night",
                     "N-event", "N-school"],
        "failure":  ["N-sensor-fail", "N-comm-blackout"],
    }
    nats = tuple(n for fam in ("weather", "incident", "demand", "failure"
                 for n in nats_by_fam[fam] if n in NATURAL_SCENARIOS)
    dem = tuple(sorted(DEMAND_PATTERNS))
    return ctrls, cf, sc, nats, dem


@dataclass
class TrustRegion:
    """One local search box over the 5-axis grid."""
    center:  tuple[str, str, str, str, str]
    length:  float = 0.5
    success: int = 0
    failure: int = 0
    best_value: float = float("inf")
    rounds_used: int = 0

    def _window(self, axis_values: Sequence[str],
                center_value: str) -> list[str]:
```

```
        if center_value not in axis_values:
            return list(axis_values)
        n = len(axis_values)
        k = max(1, math.ceil(self.length * n))
        idx = list(axis_values).index(center_value)
        lo = max(0, idx - (k - 1) // 2)
        hi = min(n, lo + k)
        lo = max(0, hi - k)
        return list(axis_values[lo:hi])

    def cells(self, axes: tuple[tuple[str, ...], ...]
              ) -> Iterable[tuple[str, str, str, str, str]]:
        lists = [self._window(axes[i], self.center[i]) for i in range(5)]
        return product(*lists)

    def size(self, axes: tuple[tuple[str, ...], ...]) -> int:
        n = 1
        for i in range(5):
            n *= len(self._window(axes[i], self.center[i]))
        return n

    def update_on_success(self, new_value: float) -> None:
        self.success += 1
        self.failure = 0
        self.best_value = new_value
        self.length = min(2.0 * self.length, 1.0)

    def update_on_failure(self, shrink_after: int = 3) -> float:
        self.failure += 1
        if self.failure >= shrink_after:
            self.length *= 0.5
            self.failure = 0
        return self.length

    def needs_restart(self, min_length: float = 0.05) -> bool:
        return self.length < min_length


class TrustRegionLearner(BayesianActiveLearner):
    """TuRBO-style learner over the 5-axis grid."""

    def __init__(self, surrogate: Surrogate, seed: int = 0) -> None:
        super().__init__(surrogate, seed=seed)
        (self.controllers,
         self.cf_models,
         self.scenarios,
         self.naturals,
         self.demands) = canonical_axes()
        self.axes = (self.controllers, self.cf_models,
                     self.scenarios, self.naturals, self.demands)
        self.encoder = Encoder(*self.axes)
        self.gp = SurrogateGP(self.encoder.n_features)
        self.regions: list[TrustRegion] = []
        self.region_log: list[dict[str, Any]] = []
```

```
    def _seed_regions(self, n_regions: int, n_init: int, n_steps: int) ->
        all_cells = list(product(*self.axes))
        self.rng.shuffle(all_cells)
        for cell in all_cells[:n_init]:
            self._observe(cell, n_steps=n_steps)

        observed = sorted(zip(self.cells, self.y_sim), key=lambda x: x[1]
        seen_ctrls: set[str] = set()
        picks: list[tuple[tuple, float]] = []
        for cell, val in observed:
            if cell[0] not in seen_ctrls:
                picks.append((cell, val))
                seen_ctrls.add(cell[0])
            if len(picks) >= n_regions:
                break
        for cell, val in observed:
            if len(picks) >= n_regions:
                break
            if (cell, val) not in picks:
                picks.append((cell, val))

        self.regions = [TrustRegion(center=cell, length=0.5,
                                    best_value=val) for cell, val in pick

    def _restart_region(self, reg: TrustRegion, n_steps: int) -> None:
        for _ in range(50):
            cell = tuple(self.rng.choice(list(ax)) for ax in self.axes)
            if cell not in set(self.cells):
                break
        y = self._observe(cell, n_steps=n_steps)
        reg.center       = cell
        reg.length       = 0.5
        reg.success      = 0
        reg.failure      = 0
        reg.best_value   = y
        reg.rounds_used += 1

    def run(self, n_init: int = 20, budget: int = 100,
            n_regions: int = 4, n_steps: int = 200,
            n_candidates: int = 2000, shrink_after: int = 3,
            progress: bool = True) -> dict[str, Any]:

        t0 = time.time()
        if progress:
            print(f"\n  TRUST-REGION ACTIVE LEARNING — {n_regions} region
                  f"budget={budget}")

        self._seed_regions(n_regions, n_init, n_steps)
        if progress:
            for i, r in enumerate(self.regions):
                print(f"    region {i}: center={r.center} "
                      f"best={r.best_value:.2f} length={r.length:.2f}")
```

```
        observed = set(self.cells)
        for rnd in range(n_init + 1, budget + 1):
            self._fit_gp()
            reg = self.regions[(rnd - n_init - 1) % len(self.regions)]

            pool_all = list(reg.cells(self.axes))
            candidates = [c for c in pool_all if c not in observed]

            if len(candidates) < 5:
                self._restart_region(reg, n_steps=n_steps)
                observed = set(self.cells)
                continue
            if len(candidates) > n_candidates:
                candidates = self.rng.sample(candidates, n_candidates)

            X_pool = self.encoder.encode_many(candidates)
            mu, var = self.gp.predict(X_pool)
            pred_pool = np.array([self._surrogate_predict(c) for c in can
            pred_mu = pred_pool + mu
            pred_sd = np.sqrt(var)
            ei = expected_improvement(pred_mu, pred_sd, best=reg.best_val
            idx = int(np.argmax(ei))
            chosen = candidates[idx]

            y_real = self._observe(chosen, n_steps=n_steps)
            observed.add(chosen)
            reg.rounds_used += 1

            if y_real < reg.best_value - 1e-6:
                reg.update_on_success(y_real)
                if y_real < self.best_value:
                    self.best_value = y_real
                    self.best_cell  = chosen
            else:
                reg.update_on_failure(shrink_after=shrink_after)
                if reg.needs_restart():
                    self._restart_region(reg, n_steps=n_steps)
                    observed = set(self.cells)

            if progress and (rnd % max(1, budget // 10) == 0 or rnd == bu
                rmse = math.sqrt(statistics.fmean(r ** 2 for r in self.y_
                reg_sizes = [r.size(self.axes) for r in self.regions]
                print(f"    round {rnd:>4}/{budget}  RMSE={rmse:>6.3f}  "
                      f"best={self.best_value:>7.2f}  regions={reg_sizes}

            self.region_log.append({
                "round":          rnd,
                "chosen":         chosen,
                "region_id":      self.regions.index(reg),
                "region_length": round(reg.length, 3),
                "region_size":   reg.size(self.axes),
                "residual_rmse": round(math.sqrt(statistics.fmean(
                    r ** 2 for r in self.y_resid)), 3),
                "best_value":    round(self.best_value, 2),
```

```
            })

        self._fit_gp()
        return {
            "n_sims":         len(self.cells),
            "n_regions":      n_regions,
            "best_cell":      self.best_cell,
            "best_value":     round(self.best_value, 2),
            "residual_rmse":  round(math.sqrt(statistics.fmean(
                r ** 2 for r in self.y_resid)), 3),
            "residual_mae":   round(statistics.fmean(
                abs(r) for r in self.y_resid), 3),
            "elapsed_s":      round(time.time() - t0, 2),
        }


class GlobalEILearner(BayesianActiveLearner):
    """Vanilla active learning — EI over the full grid (no regions)."""

    def __init__(self, surrogate: Surrogate, seed: int = 0) -> None:
        super().__init__(surrogate, seed=seed)
        (self.controllers,
         self.cf_models,
         self.scenarios,
         self.naturals,
         self.demands) = canonical_axes()
        self.axes = (self.controllers, self.cf_models,
                     self.scenarios, self.naturals, self.demands)
        self.encoder = Encoder(*self.axes)
        self.gp = SurrogateGP(self.encoder.n_features)


# ============================================================================
#  SECTION 15  —  EQUATION AUDIT  (8 representative equations)
# ============================================================================

@dataclass
class AuditCfg:
    L: float = LOGISTIC_L
    k: float = LOGISTIC_K
    x0: float = LOGISTIC_X0
    tau_min: float = 0.0
    tau_max: float = 600.0
    mu_tau: float = math.log(100.0)
    sigma_tau: float = 0.5
    lambda_decay: float = 0.01
    T_sim: float = 60.0
    n_agents: int = 9


def _trapz(y: np.ndarray, x: np.ndarray) -> float:
    y = np.asarray(y, float); x = np.asarray(x, float)
    return float(np.sum(0.5 * (y[1:] + y[:-1]) * np.diff(x)))
```

```
def _is_below(v: float, tn: int, td: int) -> bool:
    return abs(float(v)) < (tn / td)


def eq_I1(cfg: AuditCfg) -> dict[str, Any]:
    """Waiting-time integral consistency (trapezoid vs re-trapezoid)."""
    T = cfg.T_sim; N = 3000; t = np.linspace(0, T, N)
    v = np.maximum(5 - 3*np.sin(0.5*t) + 0.5*np.random.randn(N), 0)
    for f in [0.15, 0.45, 0.80]:
        s = round(N*f); a = max(0, s-20); b = min(N, s+20); v[a:b] = 0
    ind = (v < 0.1).astype(float)
    Wt = _trapz(ind, t); Ws = _trapz(ind, t)
    rel = abs(Wt - Ws) / max(abs(Wt), 1e-12)
    return {"name": "I1", "rel": rel, "ok": _is_below(rel, 5, 100),
            "summary": f"trap={Wt:.4f} rel={rel*100:.4f}%"}


def eq_I3(cfg: AuditCfg) -> dict[str, Any]:
    """Travel-time 95% CI bounds the bias."""
    N = 1000
    ta = 10*np.random.rand(N)
    tr = np.maximum(60 + 15*np.random.randn(N), 10)
    td = ta + tr
    Ta = np.mean(td - ta); Tci = 1.96*np.std(td - ta, ddof=1)/np.sqrt(N)
    rel = abs(Ta - np.mean(tr)) / max(Tci, 1e-12)
    return {"name": "I3", "rel": rel, "ok": _is_below(rel, 5, 1),
            "summary": f"T={Ta:.2f}s CI+/-{Tci:.2f}"}


def eq_I5(cfg: AuditCfg) -> dict[str, Any]:
    """AoI kernel integral = 1/lambda."""
    lam = cfg.lambda_decay
    tau = np.linspace(0, 1000, 20001)
    In = _trapz(np.exp(-lam*tau), tau); Ia = 1.0/lam
    rel = abs(In - Ia) / Ia
    return {"name": "I5", "rel": rel, "ok": _is_below(rel, 1, 100),
            "summary": f"int={In:.4f} 1/lambda={Ia:.4f}"}


def eq_P1(cfg: AuditCfg) -> dict[str, Any]:
    """Truncated lognormal mean: empirical vs analytic integral."""
    mu, sg = cfg.mu_tau, cfg.sigma_tau
    lo, hi = cfg.tau_min, cfg.tau_max
    Flo = stats.lognorm.cdf(lo, s=sg, scale=math.exp(mu))
    Fhi = stats.lognorm.cdf(hi, s=sg, scale=math.exp(mu))
    Z = Fhi - Flo
    if Z < 1e-12:
        return {"name": "P1", "rel": 0, "ok": False, "summary": "SKIP"}
    mean_int, _ = quad(
        lambda t: t * stats.lognorm.pdf(t, s=sg, scale=math.exp(mu)) / Z,
        lo, hi)
    tau = stats.lognorm.ppf(Flo + Z*np.random.rand(50000),
                            s=sg, scale=math.exp(mu))
```

```
    rel = abs(float(np.mean(tau)) - mean_int) / max(mean_int, 1e-12)
    return {"name": "P1", "rel": rel, "ok": _is_below(rel, 5, 100),
            "summary": f"emp={np.mean(tau):.2f} int={mean_int:.2f}"}


def eq_P5(cfg: AuditCfg) -> dict[str, Any]:
    """Logistic derivative at x0 = L*k/4; f(x0) = L/2."""
    L, k, x0 = cfg.L, cfg.k, cfg.x0
    f = lambda x: L / (1.0 + math.exp(-k * (x - x0)))
    h = 1e-4
    dn = (f(x0+h) - f(x0-h)) / (2*h)
    da = L * k / 4.0
    rel = abs(dn - da) / abs(da)
    sanity = abs(f(x0) - L/2.0) < 1e-9
    return {"name": "P5", "rel": rel, "ok": _is_below(rel, 1, 1_000_000)
            "summary": f"d_num={dn:.6f} d_ana={da:.6f}  f(x0)={f(x0):.4f}


def eq_A1(cfg: AuditCfg) -> dict[str, Any]:
    """Effective AoI = E[tau_tx] + E[tau_queue] + tau_prop."""
    N = 50000
    ttx = stats.lognorm.rvs(s=cfg.sigma_tau,
                            scale=math.exp(cfg.mu_tau), size=N)
    tq = -np.log(np.random.rand(N)) / 0.02
    tp = 100 / (3e8 / 1000)
    D = ttx + tq + tp; em = float(np.mean(D))
    Z = stats.lognorm.cdf(cfg.tau_max, s=cfg.sigma_tau,
                          scale=math.exp(cfg.mu_tau)) \
        - stats.lognorm.cdf(cfg.tau_min, s=cfg.sigma_tau,
                            scale=math.exp(cfg.mu_tau))
    Etx, _ = quad(
        lambda t: t * stats.lognorm.pdf(t, s=cfg.sigma_tau,
                                        scale=math.exp(cfg.mu_tau)) / Z
        cfg.tau_min, cfg.tau_max)
    am = Etx + 1/0.02 + tp
    rel = abs(em - am) / am
    return {"name": "A1", "rel": rel, "ok": _is_below(rel, 5, 100),
            "summary": f"emp={em:.2f} ana={am:.2f}"}


def eq_A8(cfg: AuditCfg) -> dict[str, Any]:
    """Bellman expectation V = (I - gamma*P)^-1 R."""
    n = 3
    P = np.array([[0.7, 0.2, 0.1], [0.1, 0.8, 0.1], [0.2, 0.2, 0.6]])
    R = np.array([1.0, 2.0, 0.5]); g = 0.9
    V1 = np.linalg.solve(np.eye(n) - g*P, R)
    V2 = np.zeros(n)
    for _ in range(5000):
        V2 = R + g*P @ V2
    err = float(np.max(np.abs(V1 - V2)))
    return {"name": "A8", "rel": err, "ok": _is_below(err, 5, 100),
            "summary": f"V=[{V1[0]:.3f} {V1[1]:.3f} {V1[2]:.3f}] err={err
```

```python
def eq_B9(cfg: AuditCfg) -> dict[str, Any]:
    """Markov chain mixing time bound: t_mix <= log(1/eps)/(1-lambda2)."""
    P = np.array([[0.7, 0.2, 0.1], [0.1, 0.8, 0.1], [0.2, 0.2, 0.6]])
    n = P.shape[0]
    eigs = np.sort(np.abs(np.linalg.eigvals(P)))[::-1]
    lambda2 = float(eigs[1]); eps = 0.01
    t_bound = int(math.ceil(math.log(1/eps) / (1 - lambda2)))
    A_mat = np.vstack([P.T - np.eye(n), np.ones((1, n))])
    b_vec = np.concatenate([np.zeros(n), [1]])
    pi = np.linalg.lstsq(A_mat, b_vec, rcond=None)[0]
    Pi_mat = np.ones((n, 1)) * pi
    emp_t = -1; Ppow = np.eye(n)
    for tt in range(1, 501):
        Ppow = Ppow @ P
        if np.max(np.abs(Ppow - Pi_mat)) < eps:
            emp_t = tt; break
    if emp_t < 0:
        emp_t = 500
    return {"name": "B9", "rel": emp_t/t_bound,
            "ok": emp_t <= t_bound,
            "summary": f"lambda2={lambda2:.4f} t<={t_bound} emp={emp_t}"}


AUDIT_FUNCS: list[Callable[[AuditCfg], dict[str, Any]]] = [
    eq_I1, eq_I3, eq_I5, eq_P1, eq_P5, eq_A1, eq_A8, eq_B9,
]


# ============================================================================
#  SECTION 16  —  UNIFIED EVALUATION DRIVER
# ============================================================================

def run_equation_audit() -> dict[str, dict[str, Any]]:
    print("\n" + "=" * 88)
    print("  PART A  —  EQUATION AUDIT  (.m logistic + .txt representativ
    print("=" * 88)
    cfg = AuditCfg()
    results: dict[str, dict[str, Any]] = {}
    for fn in AUDIT_FUNCS:
        r = fn(cfg); results[r["name"]] = r
        mark = "PASS" if r["ok"] else "FAIL"
        print(f"  [{mark}] {r['name']:<4}  {r['summary']}")
    n_pass = sum(1 for r in results.values() if r["ok"])
    print(f"\n  Audit: {n_pass}/{len(results)} passed")
    return results


def run_python_simulation() -> tuple[list[dict[str, Any]],
                                     list[str], list[str], list[str]]:
    print("\n" + "=" * 88)
    print("  PART B  —  PYTHON SIMULATION  (8 ctrl x 4 S x 5 nat x 5 seed
    print("=" * 88)
    controllers = ["fixed-time", "adaptive", "coordinated",
                   "cooperative-marl", "resilient-marl",
```

```
                  "max-pressure", "mpc", "weather-adaptive"]
    scenarios = ["S1", "S2", "S3", "S4"]
    naturals  = ["N-clear", "N-rain", "N-snow",
                 "N-accident", "N-rush-am"]
    seeds = (1, 2, 3, 4, 5)
    n_steps = 80
    total = len(controllers) * len(scenarios) * len(naturals)
    print(f"  Total cells = {total}  ({total * len(seeds)} runs)")
    rows: list[dict[str, Any]] = []
    for i, c in enumerate(controllers, 1):
        for sc in scenarios:
            for nat in naturals:
                rows.append(run_multi_seed(c, sc, nat, seeds=seeds,
                                           n_steps=n_steps,
                                           base_demand_ns=0.30,
                                           base_demand_ew=0.30))
        print(f"  done {c:<22} ({i}/{len(controllers)})")
    return rows, controllers, scenarios, naturals


def cross_check_logistic(rows: list[dict[str, Any]]) -> None:
    """Cross-check: does simulated travel time follow the .m logistic cur
    print("\n" + "=" * 88)
    print("  PART C  —  CROSS-CHECK  (.m logistic curve vs Python-simulat
    print("=" * 88)
    print(f"  Logistic coefficients (.m):  L={LOGISTIC_L}  "
          f"k={LOGISTIC_K}  x0={LOGISTIC_X0}")
    print()
    print(f"  {'Scen':<6}{'Lat(ms)':>8}{'Loss':>6}{'rho':>8}"
          f"{'TT_sim':>10}{'TT_logistic':>13}{'delta%':>9}")
    print("  " + "-" * 70)
    by_sc: dict[str, list[float]] = defaultdict(list)
    for r in rows:
        if r["natural"] == "N-clear":
            by_sc[r["scenario"]].append(r["travel_time_s_mean"])
    for sc in ("S1", "S2", "S3", "S4"):
        cfg_c = COMMUNICATION_SCENARIOS[sc]
        lat = float(cfg_c["latency_ms"])
        loss = float(cfg_c["loss_rate"])
        rho = severity_index(lat, loss)
        tt_sim = float(np.mean(by_sc[sc])) if by_sc[sc] else 0.0
        tt_log = logistic_curve(lat) if lat > 0 \
                 else LOGISTIC_L / (1.0 + math.exp(LOGISTIC_K * LOGISTIC_
        delta = (tt_sim - tt_log) / max(tt_log, 1e-9) * 100
        print(f"  {sc:<6}{lat:>8.1f}{loss:>6.2f}{rho:>8.3f}"
              f"{tt_sim:>10.2f}{tt_log:>13.2f}{delta:>8.1f}%")
    print()


def make_unified_plots(rows: list[dict[str, Any]]) -> str:
    """Save a 4-panel diagnostic figure."""
    fig, axs = plt.subplots(2, 2, figsize=(14, 10))

    # (1) .m file: logistic + quadratic curves
```

```
    x = np.linspace(0, 600, 1000)
    axs[0, 0].plot(x, logistic_curve_vec(x), "b-", lw=2.5,
                   label=f"Logistic: L={LOGISTIC_L}, k={LOGISTIC_K}, x0={
    axs[0, 0].plot(x, QUAD_A * x**2 + QUAD_B * x + QUAD_C, "g--", lw=2,
                   label=f"Quadratic: a={QUAD_A:.2e}, b={QUAD_B:.3f}, c={
    axs[0, 0].axvline(LOGISTIC_X0, color="r", ls=":", lw=1,
                      label=f"x0 = {LOGISTIC_X0}")
    axs[0, 0].set_xlabel("Latency (ms)")
    axs[0, 0].set_ylabel("f(x)")
    axs[0, 0].set_title(".m file: Logistic vs Quadratic fit")
    axs[0, 0].legend(loc="best")
    axs[0, 0].grid(alpha=0.3)

    # (2) Simulated travel time per scenario (mean over controllers)
    by_sc: dict[str, list[float]] = defaultdict(list)
    for r in rows:
        if r["natural"] == "N-clear":
            by_sc[r["scenario"]].append(r["travel_time_s_mean"])
    scenarios = sorted(by_sc.keys())
    means = [float(np.mean(by_sc[s])) for s in scenarios]
    axs[0, 1].bar(scenarios, means, color="#3b82f6")
    axs[0, 1].set_xlabel("Communication scenario")
    axs[0, 1].set_ylabel("Travel time (s)")
    axs[0, 1].set_title("Python sim: Travel time x S1->S4 (N-clear)")
    axs[0, 1].grid(alpha=0.3, axis="y")

    # (3) Throughput per controller (N-clear, mean over S)
    by_ctrl: dict[str, list[float]] = defaultdict(list)
    for r in rows:
        if r["natural"] == "N-clear":
            by_ctrl[r["algorithm_type"]].append(r["throughput_vph_mean"])
    ctrls = sorted(by_ctrl.keys())
    thr = [float(np.mean(by_ctrl[c])) for c in ctrls]
    axs[1, 0].barh(ctrls, thr, color="#10b981")
    axs[1, 0].set_xlabel("Throughput (vph)")
    axs[1, 0].set_ylabel("Controller")
    axs[1, 0].set_title("Python sim: Throughput per controller (N-clear)"
    axs[1, 0].grid(alpha=0.3, axis="x")

    # (4) Cross-check: sim TT vs .m logistic curve
    xs = [float(COMMUNICATION_SCENARIOS[s]["latency_ms"]) for s in scenar
    sim_tt = means
    log_tt = [logistic_curve(x) if x > 0
              else LOGISTIC_L / (1 + math.exp(LOGISTIC_K * LOGISTIC_X0))
              for x in xs]
    axs[1, 1].plot(xs, sim_tt, "o-", lw=2, label="Python-simulated TT")
    axs[1, 1].plot(xs, log_tt, "s--", lw=2, label=".m logistic curve")
    axs[1, 1].set_xlabel("Latency (ms)")
    axs[1, 1].set_ylabel("Travel time (s)")
    axs[1, 1].set_title("Cross-check: sim vs .m logistic")
    axs[1, 1].legend()
    axs[1, 1].grid(alpha=0.3)

    plt.suptitle("Unified Evaluation - .m equation x .py simulator",
```

```
                  fontweight="bold", fontsize=14)
    plt.tight_layout()
    out = "unified_evaluation.png"
    plt.savefig(out, dpi=120, bbox_inches="tight")
    plt.close()
    print(f"  Saved plot -> {out}")
    return out


def final_report(audit_results: dict[str, dict[str, Any]],
                 rows: list[dict[str, Any]]) -> None:
    print("\n" + "=" * 88)
    print("  FINAL UNIFIED REPORT")
    print("=" * 88)
    n_pass = sum(1 for r in audit_results.values() if r["ok"])
    print(f"\n  Equation audit        : {n_pass}/{len(audit_results)} pas
    print(f"  Simulation cells run   : {len(rows)}  (x5 seeds each)")
    print(f"  Controllers exercised  : {len({r['algorithm_type'] for r in
    print(f"  Scenarios exercised    : {len({r['scenario'] for r in rows}
    print(f"  Natural conditions     : {len({r['natural'] for r in rows})
    print("\n  Best controller per scenario (N-clear, lowest travel_time)
    for sc in ("S1", "S2", "S3", "S4"):
        cands = [r for r in rows
                 if r["scenario"] == sc and r["natural"] == "N-clear"]
        if not cands:
            continue
        best = min(cands, key=lambda r: r["travel_time_s_mean"])
        print(f"    {sc}: {best['algorithm_type']:<22} "
              f"TT={best['travel_time_s_mean']:.2f}s  "
              f"Thr={best['throughput_vph_mean']:.0f}vph  "
              f"CI+/-{best['travel_time_s_ci95']:.2f}")
    print()


# ============================================================================
#  SECTION 18  —  DISTRIBUTED ADAPTIVE SIGNAL EXECUTION  (NEW)
# ----------------------------------------------------------------------------
#  Adapted from adaptive_multi_execution.py:
#    * Real signal state machine (green -> yellow -> all-red -> green)
#    * One AdaptiveSignalController per intersection
#    * Online parameter adaptation (min_green, switch_threshold, cycle_ta
#    * Decentralised SignalBus for neighbour queries
#    * Parallel multi-episode execution (with sequential fallback for Col
# ============================================================================

from enum import IntEnum
from concurrent.futures import ProcessPoolExecutor, as_completed

class SignalPhase(IntEnum):
    NS_GREEN   = 0
    NS_YELLOW  = 1
    ALL_RED_1  = 2
    EW_GREEN   = 3
    EW_YELLOW  = 4
```

```
    ALL_RED_2  = 5

SERVED_BY_PHASE = {
    SignalPhase.NS_GREEN:  ('N', 'S'),
    SignalPhase.EW_GREEN:  ('E', 'W'),
    SignalPhase.NS_YELLOW: (),
    SignalPhase.EW_YELLOW: (),
    SignalPhase.ALL_RED_1: (),
    SignalPhase.ALL_RED_2: (),
}

ALL_RED_S: float = 1.5     # clearance interval


@dataclass
class SignalStateMachine:
    """One intersection's real signal state machine.
    The controller requests a phase change; the machine enforces
    min-green, max-green, yellow, and all-red."""
    node_id: str
    current_phase: SignalPhase = SignalPhase.NS_GREEN
    time_in_phase: float = 0.0
    pending_switch: bool = False
    log: list = field(default_factory=list)
    min_green: float = GREEN_MIN_S
    max_green: float = GREEN_MAX_S

    def request_switch(self):
        self.pending_switch = True

    def step(self, dt: float, demand_ns: float, demand_ew: float):
        self.time_in_phase += dt
        if self.current_phase in (SignalPhase.NS_GREEN, SignalPhase.EW_GR
            if self.time_in_phase >= self.max_green:
                self.pending_switch = True
        if self.pending_switch and self.time_in_phase >= self.min_green:
            transitions = {
                SignalPhase.NS_GREEN: SignalPhase.NS_YELLOW,
                SignalPhase.NS_YELLOW: SignalPhase.ALL_RED_1,
                SignalPhase.ALL_RED_1: SignalPhase.EW_GREEN,
                SignalPhase.EW_GREEN: SignalPhase.EW_YELLOW,
                SignalPhase.EW_YELLOW: SignalPhase.ALL_RED_2,
                SignalPhase.ALL_RED_2: SignalPhase.NS_GREEN,
            }
            self.current_phase = transitions.get(
                self.current_phase, SignalPhase.NS_GREEN)
            self.time_in_phase = 0.0
            self.pending_switch = False
        else:
            auto = [
                (SignalPhase.NS_YELLOW,  YELLOW_S,    SignalPhase.ALL_RED
                (SignalPhase.ALL_RED_1,  ALL_RED_S,   SignalPhase.EW_GREE
                (SignalPhase.EW_YELLOW,  YELLOW_S,    SignalPhase.ALL_RED
                (SignalPhase.ALL_RED_2,  ALL_RED_S,   SignalPhase.NS_GREE
```

```
            ]
            for phase, dur, nxt in auto:
                if self.current_phase == phase and self.time_in_phase >=
                    self.current_phase = nxt
                    self.time_in_phase = 0.0
                    break
        return self.current_phase

    def served_approaches(self):
        return SERVED_BY_PHASE[self.current_phase]

    def is_green(self):
        return self.current_phase in (SignalPhase.NS_GREEN, SignalPhase.E

    def snapshot(self):
        return {'id': self.node_id,
                'phase': int(self.current_phase),
                'time_in_phase': self.time_in_phase,
                'is_green': self.is_green(),
                'min_green': self.min_green}


class SignalBus:
    """Decentralised message bus for neighbour phase queries."""
    def __init__(self):
        self._snapshots: dict[str, dict] = {}
    def publish(self, snapshot: dict):
        self._snapshots[snapshot['id']] = snapshot
    def query(self, neighbour_ids) -> list[dict]:
        return [self._snapshots[n] for n in neighbour_ids
                if n in self._snapshots]


@dataclass
class AdaptiveSignalController:
    """One per intersection. Adapts min_green, switch_threshold,
    cycle_target online based on observed queue patterns."""
    node_id: str
    seed: int = 0
    min_green: float = GREEN_MIN_S
    max_green: float = GREEN_MAX_S
    switch_threshold: float = 1.5
    cycle_target: float = 60.0
    rng: random.Random = field(init=False)
    last_switch_t: float = 0.0
    switches: int = 0
    adaptations: dict = field(default_factory=dict)
    queue_history: deque = field(default_factory=lambda: deque(maxlen=20)

    def __post_init__(self):
        self.rng = random.Random(self.seed)
        self.adaptations = {'min_green': 0, 'switch_threshold': 0,
                            'cycle_target': 0}
```

```
    def decide(self, sm: SignalStateMachine,
               q_ns: float, q_ew: float,
               neighbour_phases: list[dict]) -> None:
        if not sm.is_green():
            return
        self.queue_history.append((q_ns, q_ew))
        total = max(q_ns + q_ew, 1e-6)
        ns_ratio = q_ns / total

        # adaptation 1: min_green responds to sustained queue
        if len(self.queue_history) >= 5:
            recent_total = statistics.fmean(
                q[0] + q[1] for q in self.queue_history)
            if recent_total > 20 and self.min_green < 20.0:
                self.min_green = min(20.0, self.min_green + 1.0)
                self.adaptations['min_green'] += 1
            elif recent_total < 5 and self.min_green > 8.0:
                self.min_green = max(8.0, self.min_green - 1.0)
                self.adaptations['min_green'] += 1

        # adaptation 2: switch_threshold responds to neighbour sync
        same_as_us = sum(1 for n in neighbour_phases
                         if n['phase'] == int(sm.current_phase))
        if same_as_us >= 2 and self.switch_threshold > 1.0:
            self.switch_threshold = max(1.0, self.switch_threshold - 0.1)
            self.adaptations['switch_threshold'] += 1

        # adaptation 3: cycle_target shrinks under high congestion
        current_total = self.queue_history[-1][0] + self.queue_history[-1
        if current_total:
            if current_total > 25 and self.cycle_target > 45.0:
                self.cycle_target = max(45.0, self.cycle_target - 0.5)
                self.adaptations['cycle_target'] += 1
            elif current_total < 8 and self.cycle_target < 90.0:
                self.cycle_target = min(90.0, self.cycle_target + 0.5)
                self.adaptations['cycle_target'] += 1

        # decision: queue-ratio gap exceeds adaptive threshold
        gap = ns_ratio - 0.5
        serving_ns = (sm.current_phase == SignalPhase.NS_GREEN)
        if serving_ns and q_ew > q_ns + self.switch_threshold * 3:
            sm.request_switch(); self.switches += 1
        elif (not serving_ns) and q_ns > q_ew + self.switch_threshold * 3
            sm.request_switch(); self.switches += 1
        elif abs(gap) > 0.15 and sm.time_in_phase > sm.min_green + 5.0:
            sm.request_switch(); self.switches += 1
        sm.min_green = self.min_green
        sm.max_green = self.max_green

    def snapshot(self):
        return {'id': self.node_id,
                'min_green': round(self.min_green, 2),
                'switch_threshold': round(self.switch_threshold, 2),
                'cycle_target': round(self.cycle_target, 2),
```

```
                'switches': self.switches,
                'adaptations': dict(self.adaptations)}


class DistributedAdaptiveExecutor:
    """Runs one episode with one AdaptiveSignalController + one
    SignalStateMachine per intersection."""
    def __init__(self, seed: int = 0, car_following: str = 'SAFE'):
        self.seed = seed
        self.rng = random.Random(seed)
        self.car_model = CarFollowingModel(car_following, self.rng)
        self.nodes: dict[str, Intersection] = {}
        for r in range(GRID_ROWS):
            for c in range(GRID_COLS):
                nid = f"J{r+1}{c+1}"
                node = Intersection(nid, self.rng)
                node.row, node.col = r, c
                self.nodes[nid] = node
        self.controllers: dict[str, AdaptiveSignalController] = {}
        self.machines: dict[str, SignalStateMachine] = {}
        for nid in self.nodes:
            self.controllers[nid] = AdaptiveSignalController(
                node_id=nid, seed=seed + hash(nid) % 10_000)
            self.machines[nid] = SignalStateMachine(node_id=nid)
        self.bus = SignalBus()
        self.metrics = {'travel_time_sum_s': 0.0, 'queue_sum': 0.0,
                        'waiting_sum_s': 0.0, 'vehicles_completed': 0.0,
                        'samples': 0}
        self.timeline: list[dict] = []

    def _neighbours(self, nid: str) -> list[str]:
        r, c = self.nodes[nid].row, self.nodes[nid].col
        out = []
        for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < GRID_ROWS and 0 <= nc < GRID_COLS:
                out.append(f"J{nr+1}{nc+1}")
        return out

    def run(self, n_steps: int = 200, base_demand_ns: float = 0.30,
            base_demand_ew: float = 0.30,
            congestion_start: float = 0.4,
            congestion_end: float = 0.9) -> dict:
        dt = DECISION_INTERVAL_S
        for t in range(n_steps):
            c = congestion_start + (congestion_end - congestion_start) *
            for nid, ctrl in self.controllers.items():
                node = self.nodes[nid]
                nbrs = self.bus.query(self._neighbours(nid))
                ctrl.decide(self.machines[nid], node.queue_ns(),
                            node.queue_ew(), nbrs)
            for nid, sm in self.machines.items():
                sm.step(dt, self.nodes[nid].queue_ns(),
                        self.nodes[nid].queue_ew())
```

```
            for nid, sm in self.machines.items():
                self.bus.publish(sm.snapshot())
            for nid, node in self.nodes.items():
                served = self.machines[nid].served_approaches()
                for a in ('N', 'S'):
                    node.q[a] += 0.5 * base_demand_ns * (0.8 + 0.6 * c) *
                for a in ('E', 'W'):
                    node.q[a] += 0.5 * base_demand_ew * (0.8 + 0.6 * c) *
                for a in served:
                    d = min(node.q[a], SATURATION_FLOW_VPS * dt)
                    node.q[a] -= d; node.departures += d; node.completed
                for a in node.q:
                    node.W[a] += node.q[a] * dt
            for node in self.nodes.values():
                self.metrics['waiting_sum_s'] += sum(node.W.values())
                self.metrics['queue_sum'] += node.total_queue()
                self.metrics['samples'] += 1
                total_q = node.total_queue()
                self.metrics['vehicles_completed'] += min(2.5, 0.025 * to
                cong = min(0.99, c)
                cf = self.car_model.simulate_step(
                    'free_flow' if cong < 0.5 else 'congested',
                    int(self.machines[node.id].current_phase), 0, 0, cong
                self.metrics['travel_time_sum_s'] += (
                    0.6 * cf['in_system_s'] + 0.4 * 100.0)
            self.timeline.append({
                't': t * dt,
                'phases': {nid: int(sm.current_phase)
                           for nid, sm in self.machines.items()},
                'queues': {nid: round(n.total_queue(), 2)
                           for nid, n in self.nodes.items()}})
        return self.metrics

    def get_metrics(self) -> dict:
        s = max(self.metrics['samples'], 1)
        sim_s = s * DECISION_INTERVAL_S
        agg_adapt = defaultdict(int)
        total_switches = 0
        for ctrl in self.controllers.values():
            total_switches += ctrl.switches
            for k, v in ctrl.adaptations.items():
                agg_adapt[k] += v
        return {
            'travel_time_s': round(self.metrics['travel_time_sum_s'] / s,
            'queue_veh': round(self.metrics['queue_sum'] / s, 2),
            'waiting_time_s': round(self.metrics['waiting_sum_s'] / s, 2)
            'throughput_vph': round(self.metrics['vehicles_completed']
                                    / max(sim_s, 1.0) * 3600.0, 1),
            'total_switches': total_switches,
            'adaptations': dict(agg_adapt)}


def _adaptive_worker(args):
    seed, n_steps = args
```

```
    ex = DistributedAdaptiveExecutor(seed=seed)
    ex.run(n_steps=n_steps)
    m = ex.get_metrics(); m['seed'] = seed
    return m


def run_adaptive_parallel(n_episodes: int = 5,
                          n_steps: int = 200) -> list[dict]:
    """Run n_episodes concurrently. Falls back to sequential in Colab."""
    seeds = list(range(1, n_episodes + 1))
    try:
        with ProcessPoolExecutor(max_workers=min(n_episodes, 4)) as pool:
            futures = [pool.submit(_adaptive_worker, (s, n_steps))
                       for s in seeds]
            return [f.result() for f in as_completed(futures)]
    except Exception:
        return [_adaptive_worker((s, n_steps)) for s in seeds]


PHASE_GLYPH = {
    int(SignalPhase.NS_GREEN):  '|^',
    int(SignalPhase.NS_YELLOW): '!',
    int(SignalPhase.ALL_RED_1): '.',
    int(SignalPhase.EW_GREEN):  '->',
    int(SignalPhase.EW_YELLOW): '!',
    int(SignalPhase.ALL_RED_2): '.',
}


def print_adaptive_dashboard(ex: DistributedAdaptiveExecutor, every: int
    print("\n  t(s)  " + "  ".join(f"J{r+1}{c+1}"
                                   for r in range(GRID_ROWS)
                                   for c in range(GRID_COLS)))
    for step, row in enumerate(ex.timeline):
        if step % every: continue
        cells = []
        for r in range(GRID_ROWS):
            for c in range(GRID_COLS):
                nid = f"J{r+1}{c+1}"
                ph = row['phases'][nid]
                cells.append(f"{PHASE_GLYPH[ph]}{row['queues'][nid]:>3.0f
        print(f"  {row['t']:>5.0f}  " + "  ".join(cells))
    print("\n  legend: ^ NS green  > EW green  ! yellow  . all-red")


def print_adaptive_controller_report(ex: DistributedAdaptiveExecutor):
    print("\n" + "=" * 100)
    print("  PER-CONTROLLER ADAPTATION REPORT")
    print("=" * 100)
    print(f"{'Node':<6}{'min_grn':>10}{'sw_thr':>10}{'cyc_tgt':>10}"
          f"{'switches':>10}{'adapt mg/st/ct':>20}")
    print('-' * 100)
    for nid, ctrl in ex.controllers.items():
        s = ctrl.snapshot()
```

```
        a = s['adaptations']
        print(f"{nid:<6}{s['min_green']:>10.2f}{s['switch_threshold']:>10
              f"{s['cycle_target']:>10.2f}{s['switches']:>10d}"
              f"{a['min_green']:>8}/{a['switch_threshold']:<3}"
              f"{a['cycle_target']:<3}")
    print()


# ============================================================================
#  SECTION 19  —  DECAY-TRUST-REGION ACTIVE LEARNING  (NEW)
# ----------------------------------------------------------------------------
#  Adapted from trust_region_decay.py:
#    * Progressive widening of trust region as budget is spent
#    * Reheat: smallest region resets to random unseen centre periodicall
#    * Stall detection: force-expand all regions when nobody improves
#    * Global-pull phase: after 75% of budget, region floor rises to ~0.7
# ============================================================================

@dataclass
class DecaySchedule:
    """Progressive widening of the trust region over budget.
    All params are start -> end lerps over progress p in [0,1]."""
    min_length_start: float = 0.05
    min_length_end:   float = 0.75
    min_length_power: float = 1.0
    expand_start: float = 2.0
    expand_end:   float = 3.0
    shrink_start: float = 0.50
    shrink_end:   float = 0.85
    global_phase_frac: float = 0.75
    global_min_length: float = 0.75
    reheat_period: Optional[int] = None
    reheat_length: float = 0.50
    stall_threshold: Optional[int] = 5
    stall_expand: float = 1.50

    @staticmethod
    def progress(t: int, t_init: int, T: int) -> float:
        if T <= t_init: return 1.0
        return max(0.0, min(1.0, (t - t_init) / (T - t_init)))

    def min_length(self, t, t_init, T):
        p = self.progress(t, t_init, T)
        return (self.min_length_start
                + (self.min_length_end - self.min_length_start)
                * (p ** self.min_length_power))

    def expand_factor(self, t, t_init, T):
        p = self.progress(t, t_init, T)
        return self.expand_start + (self.expand_end - self.expand_start)

    def shrink_factor(self, t, t_init, T):
        p = self.progress(t, t_init, T)
        return self.shrink_start + (self.shrink_end - self.shrink_start)
```

```
    def in_global_phase(self, t, t_init, T):
        return self.progress(t, t_init, T) >= self.global_phase_frac

    def describe(self):
        print(f"  DecaySchedule:")
        print(f"    min_length    : {self.min_length_start:.2f} -> "
              f"{self.min_length_end:.2f}  (power={self.min_length_power}")
        print(f"    expand factor  : {self.expand_start:.2f} -> "
              f"{self.expand_end:.2f}")
        print(f"    shrink factor  : {self.shrink_start:.2f} -> "
              f"{self.shrink_end:.2f}")
        print(f"    global phase   : after {self.global_phase_frac:.0%} "
              f"of budget, floor={self.global_min_length:.2f}")
        print(f"    reheat period  : {self.reheat_period}")
        print(f"    stall threshold: {self.stall_threshold}")


@dataclass
class DecayTrustRegion(TrustRegion):
    """Trust region whose expand/shrink is driven by a DecaySchedule."""
    last_improved_at: int = 0

    def update(self, success: bool, new_value: float,
               schedule: 'DecaySchedule',
               t: int, t_init: int, T: int,
               shrink_after: int = 3) -> str:
        if success:
            self.success += 1; self.failure = 0
            self.best_value = new_value; self.last_improved_at = t
            expand = schedule.expand_factor(t, t_init, T)
            self.length = min(expand * self.length, 1.0)
            return 'expanded'
        self.failure += 1
        if self.failure < shrink_after:
            return 'noop'
        self.failure = 0
        if schedule.in_global_phase(t, t_init, T):
            self.length = max(self.length, schedule.global_min_length)
            return 'global-pin'
        shrink = schedule.shrink_factor(t, t_init, T)
        self.length *= shrink
        return 'shrunk'

    def enforce_floor(self, schedule, t, t_init, T) -> bool:
        floor = schedule.min_length(t, t_init, T)
        if self.length < floor:
            self.length = floor
            return True
        return False

    def needs_restart(self, schedule, t, t_init, T) -> bool:
        if schedule.in_global_phase(t, t_init, T):
            return self.success == 0 and self.rounds_used > 15
```

```
        return False


class DecayTrustRegionLearner(TrustRegionLearner):
    """Trust-region learner with a DecaySchedule driving region sizes.
    Drop-in replacement for TrustRegionLearner (same public API)."""

    def __init__(self, surrogate: 'Surrogate',
                 schedule: Optional[DecaySchedule] = None,
                 seed: int = 0):
        super().__init__(surrogate, seed=seed)
        self.schedule = schedule or DecaySchedule()
        self.regions: list[DecayTrustRegion] = []
        self.health_log: list[dict] = []

    def _seed_regions(self, n_regions, n_init, n_steps):
        all_cells = list(product(*self.axes))
        self.rng.shuffle(all_cells)
        for cell in all_cells[:n_init]:
            self._observe(cell, n_steps=n_steps)
        observed = sorted(zip(self.cells, self.y_sim), key=lambda x: x[1]
        seen_ctrls, picks = set(), []
        for cell, val in observed:
            if cell[0] not in seen_ctrls:
                picks.append((cell, val)); seen_ctrls.add(cell[0])
            if len(picks) >= n_regions: break
        for cell, val in observed:
            if len(picks) >= n_regions: break
            if (cell, val) not in picks:
                picks.append((cell, val))
        self.regions = [DecayTrustRegion(center=cell, length=0.5,
                                         best_value=val, last_improved_at
                        for cell, val in picks]

    def _reheat(self, region: DecayTrustRegion, n_steps: int):
        for _ in range(80):
            cell = tuple(self.rng.choice(list(ax)) for ax in self.axes)
            if cell not in set(self.cells):
                break
        y = self._observe(cell, n_steps=n_steps)
        region.center = cell; region.length = self.schedule.reheat_length
        region.success = 0; region.failure = 0
        region.best_value = y; region.last_improved_at = len(self.cells)

    def _handle_stall(self, t, t_init, T):
        if self.schedule.stall_threshold is None:
            return False
        recent = [r for r in self.regions
                  if t - r.last_improved_at < self.schedule.stall_thresh
        if recent:
            return False
        for r in self.regions:
            r.length = min(1.0, r.length * self.schedule.stall_expand)
        return True
```

```
    def run(self, n_init: int = 20, budget: int = 100,
            n_regions: int = 4, n_steps: int = 200,
            n_candidates: int = 2000, shrink_after: int = 3,
            progress: bool = True) -> dict:
        t0 = time.time()
        if progress:
            print(f"\n  DECAY-TRUST-REGION AL — {n_regions} regions, "
                  f"budget={budget}")
            self.schedule.describe()
        self._seed_regions(n_regions, n_init, n_steps)
        if progress:
            for i, r in enumerate(self.regions):
                print(f"    region {i}: center={r.center} "
                      f"best={r.best_value:.2f} len={r.length:.2f}")
        t_init = n_init + 1
        observed = set(self.cells)
        last_reheat = n_init
        for rnd in range(t_init, budget + 1):
            self._fit_gp()
            reg = self.regions[(rnd - t_init) % len(self.regions)]
            reg.rounds_used += 1
            reg.enforce_floor(self.schedule, rnd, t_init, budget)
            candidates = [c for c in reg.cells(self.axes) if c not in obs
            if len(candidates) < 5:
                self._reheat(reg, n_steps=n_steps)
                observed = set(self.cells)
                continue
            if len(candidates) > n_candidates:
                candidates = self.rng.sample(candidates, n_candidates)
            X_pool = self.encoder.encode_many(candidates)
            mu, var = self.gp.predict(X_pool)
            pred_pool = np.array([self._surrogate_predict(c)
                                  for c in candidates])
            pred_mu = pred_pool + mu
            pred_sd = np.sqrt(var)
            ei = expected_improvement(pred_mu, pred_sd, best=reg.best_val
            chosen = candidates[int(np.argmax(ei))]
            prev_best = reg.best_value
            y_real = self._observe(chosen, n_steps=n_steps)
            observed.add(chosen)
            improved = y_real < prev_best - 1e-6
            outcome = reg.update(improved, y_real, self.schedule,
                                 rnd, t_init, budget, shrink_after)
            if improved and y_real < self.best_value:
                self.best_value = y_real; self.best_cell = chosen
            if (self.schedule.reheat_period
                    and rnd - last_reheat >= self.schedule.reheat_period)
                smallest = min(self.regions, key=lambda r: r.length)
                self._reheat(smallest, n_steps=n_steps)
                observed = set(self.cells); last_reheat = rnd
            self._handle_stall(rnd, t_init, budget)
            if progress and (rnd % max(1, budget // 10) == 0 or rnd == bu
                rmse = math.sqrt(statistics.fmean(r**2 for r in self.y_re
```

```
                sizes = [round(r.length, 2) for r in self.regions]
                floor = self.schedule.min_length(rnd, t_init, budget)
                print(f"    round {rnd:>4}/{budget}  RMSE={rmse:>6.3f}  "
                      f"best={self.best_value:>7.2f}  "
                      f"lengths={sizes}  floor={floor:.2f}")
            self.region_log.append({
                'round': rnd, 'region_id': self.regions.index(reg),
                'region_length': round(reg.length, 3),
                'region_size': reg.size(self.axes),
                'outcome': outcome,
                'residual_rmse': round(math.sqrt(statistics.fmean(
                    r**2 for r in self.y_resid)), 3),
                'best_value': round(self.best_value, 2),
                'floor': round(self.schedule.min_length(
                    rnd, t_init, budget), 3)})
            self.health_log.append({
                'round': rnd,
                'min_length': round(min(r.length for r in self.regions),
                'max_length': round(max(r.length for r in self.regions),
                'mean_length': round(statistics.fmean(
                    r.length for r in self.regions), 3),
                'mean_size': round(statistics.fmean(
                    r.size(self.axes) for r in self.regions), 1),
                'floor': round(self.schedule.min_length(
                    rnd, t_init, budget), 3),
                'in_global': self.schedule.in_global_phase(rnd, t_init, b
        self._fit_gp()
        elapsed = time.time() - t0
        return {
            'n_sims': len(self.cells), 'n_regions': n_regions,
            'best_cell': self.best_cell,
            'best_value': round(self.best_value, 2),
            'residual_rmse': round(math.sqrt(statistics.fmean(
                r**2 for r in self.y_resid)), 3),
            'residual_mae': round(statistics.fmean(
                abs(r) for r in self.y_resid), 3),
            'elapsed_s': round(elapsed, 2)}


def print_decay_health(learner: DecayTrustRegionLearner, every: int = 10)
    print("\n" + "=" * 100)
    print("  REGION HEALTH OVER TIME")
    print("=" * 100)
    print(f"{'round':>6}{'min':>8}{'mean':>8}{'max':>8}"
          f"{'mean|rn|':>10}{'floor':>8}{'global':>8}")
    print('-' * 100)
    for h in learner.health_log:
        if h['round'] % every and h['round'] != learner.health_log[-1]['r
            continue
        print(f"{h['round']:>6}{h['min_length']:>8.3f}"
              f"{h['mean_length']:>8.3f}{h['max_length']:>8.3f}"
              f"{h['mean_size']:>10.1f}{h['floor']:>8.3f}"
              f"{str(h['in_global']):>8}")
    print()
```

```python
def print_decay_outcome_mix(learner: DecayTrustRegionLearner):
    if not learner.region_log: return
    t_init = learner.region_log[0]['round']
    T = learner.region_log[-1]['round']
    phases = {'early': [], 'mid': [], 'late': []}
    for e in learner.region_log:
        p = (e['round'] - t_init) / max(T - t_init, 1)
        key = 'early' if p < 0.5 else 'mid' if p < 0.75 else 'late'
        phases[key].append(e['outcome'])
    print("\n" + "=" * 90)
    print("  OUTCOME MIX BY PHASE")
    print("=" * 90)
    print(f"{'phase':<10}{'expanded':>12}{'shrunk':>10}"
          f"{'global-pin':>14}{'noop':>10}")
    print('-' * 90)
    for name, outs in phases.items():
        if not outs: continue
        n = len(outs)
        print(f"{name:<10}{outs.count('expanded')/n:>11.0%}"
              f"{outs.count('shrunk')/n:>10.0%}"
              f"{outs.count('global-pin')/n:>14.0%}"
              f"{outs.count('noop')/n:>10.0%}")
    print()



# ============================================================================
#  SECTION 17  —  CLI  +  MAIN
# ============================================================================

def _parse_args() -> argparse.Namespace:
    p = argparse.ArgumentParser(
        description="Unified traffic-signal evaluation (single Colab cell
        formatter_class=argparse.ArgumentDefaultsHelpFormatter,
    )
    p.add_argument("--skip-sim",  action="store_true",
                   help="skip Part B (Python simulation) for fast CI")
    p.add_argument("--skip-audit", action="store_true",
                   help="skip Part A (equation audit)")
    p.add_argument("--skip-synth", action="store_true",
                   help="skip the synthetic dataset build")
    p.add_argument("--skip-trust", action="store_true",
                   help="skip Part E3 (trust-region learner)")
    p.add_argument("--skip-adaptive", action="store_true",
                   help="skip Part F (distributed adaptive execution)")
    p.add_argument("--skip-decay", action="store_true",
                   help="skip Part G (decay trust-region learner)")
    p.add_argument("--synth-preset", choices=tuple(PRESETS), default="qui
                   help="synthetic-dataset preset (quick for speed)")
    p.add_argument("--trust-budget", type=int, default=30,
                   help="TuRBO simulation budget (>= 10)")
    p.add_argument("--trust-init",   type=int, default=10,
```

```
                    help="TuRBO initial random observations")
    p.add_argument("--trust-regions", type=int, default=2,
                    help="number of TuRBO trust regions")
    p.add_argument("--adaptive-steps", type=int, default=100,
                    help="steps per adaptive episode")
    p.add_argument("--adaptive-episodes", type=int, default=3,
                    help="number of parallel adaptive episodes")
    p.add_argument("--decay-budget", type=int, default=30,
                    help="decay-TuRBO simulation budget")
    return p.parse_args([])  # Colab: no argv


def main() -> int:
    args = _parse_args()
    print("#" * 88)
    print("#  UNIFIED TRAFFIC EVALUATION  —  .m eq x .py sim x TuRBO AL")
    print(f"#  Logistic: L={LOGISTIC_L}  k={LOGISTIC_K}  x0={LOGISTIC_X0}")
    print("#" * 88)

    audit_results: dict[str, dict[str, Any]] = {}
    if not args.skip_audit:
        audit_results = run_equation_audit()              # PART A

    rows: list[dict[str, Any]] = []
    if not args.skip_sim:
        rows, _, _, _ = run_python_simulation()           # PART B
        cross_check_logistic(rows)                        # PART C

    if rows:
        plot_path = make_unified_plots(rows)              # PART D
        print(f"  Plot: {plot_path}")

    # --- PART E : synthetic dataset + tensor pipeline + TuRBO ----------
    if not args.skip_synth:
        print("\n" + "=" * 88)
        print("  PART E1  —  SYNTHETIC DATASET")
        print("=" * 88)
        ds = SyntheticTrafficDataset(preset=args.synth_preset,
                                     verbose=True, auto_build=True)
        ds.summary()
        npy_path = ds.to_npy()
        print(f"  Tensor -> {npy_path}  shape={ds.as_tensor().shape}")

        print("\n  PART E2  —  TENSOR PIPELINE (Ridge + kNN + GP + EI AL)")
        tp = TensorPipeline(path=npy_path, verbose=True)
        tp.summary()
        tp.run(budget=50, n_init=10)

    if not args.skip_trust:
        print("\n" + "=" * 88)
        print("  PART E3  —  TRUST-REGION ACTIVE LEARNING (TuRBO)")
        print("=" * 88)
        surr = Surrogate(seed=42)
        trl = TrustRegionLearner(surr, seed=42)
```

```
        report = trl.run(n_init=args.trust_init, budget=args.trust_budget
                         n_regions=args.trust_regions, n_steps=80,
                         progress=True)
        print(f"\n  TuRBO report: {report}")
        hold = trl.evaluate_holdout(n_holdout=10, n_steps=80)
        print(f"  Holdout: raw MAE={hold['raw_mae']}  "
              f"corrected MAE={hold['corr_mae']}")

    # --- PART F: distributed adaptive execution ------------------------
    if not args.skip_adaptive:
        print("\n" + "=" * 88)
        print("  PART F  —  DISTRIBUTED ADAPTIVE SIGNAL EXECUTION")
        print("=" * 88)
        ex = DistributedAdaptiveExecutor(seed=1)
        ex.run(n_steps=args.adaptive_steps)
        am = ex.get_metrics()
        print_adaptive_dashboard(ex, every=max(1, args.adaptive_steps //
        print_adaptive_controller_report(ex)
        print(f"  Adaptive metrics: {am}")
        if args.adaptive_episodes > 1:
            print(f"\n  Running {args.adaptive_episodes} parallel episode
            rows = run_adaptive_parallel(
                n_episodes=args.adaptive_episodes,
                n_steps=args.adaptive_steps)
            print(f"  {'seed':>6}{'travel':>10}{'queue':>9}"
                  f"{'wait':>9}{'thr':>10}{'switches':>10}")
            print("-" * 60)
            for r in sorted(rows, key=lambda x: x['seed']):
                print(f"  {r['seed']:>6}{r['travel_time_s']:>10.2f}"
                      f"{r['queue_veh']:>9.2f}{r['waiting_time_s']:>9.2f}
                      f"{r['throughput_vph']:>10.1f}"
                      f"{r['total_switches']:>10d}")

    # --- PART G: decay trust-region learner ----------------------------
    if not args.skip_decay:
        print("\n" + "=" * 88)
        print("  PART G  —  DECAY-TRUST-REGION ACTIVE LEARNING")
        print("=" * 88)
        dsurr = Surrogate(seed=42)
        sched = DecaySchedule(
            min_length_start=0.05, min_length_end=0.75,
            min_length_power=1.5,
            expand_start=2.0, expand_end=3.0,
            shrink_start=0.50, shrink_end=0.85,
            global_phase_frac=0.75, global_min_length=0.75,
            reheat_period=max(args.decay_budget // 4, 10),
            stall_threshold=5)
        trd = DecayTrustRegionLearner(dsurr, schedule=sched, seed=42)
        dreport = trd.run(n_init=10, budget=args.decay_budget,
                          n_regions=2, n_steps=80, progress=True)
        print(f"\n  Decay-TuRBO report: {dreport}")
        print_decay_health(trd, every=max(1, args.decay_budget // 8))
        print_decay_outcome_mix(trd)
        dhold = trd.evaluate_holdout(n_holdout=10, n_steps=80)
```

```
        print(f"  Decay holdout: raw MAE={dhold['raw_mae']}  "
              f"corrected MAE={dhold['corr_mae']}")

    # --- FINAL ----------------------------------------------------------
    if audit_results or rows:
        final_report(audit_results, rows)
    print("\n" + "=" * 88)
    print("  Unified evaluation complete")
    print("=" * 88)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

```
[boot] registered 17 controllers: ['actuated', 'adaptive', 'cooperative-coordinated',
arl', 'coordinated'] ...
################################################################################
#  UNIFIED TRAFFIC EVALUATION  —  .m eq x .py sim x TuRBO AL
#  Logistic: L=302.2328  k=0.0143  x0=101.5374
################################################################################

================================================================================
  PART A  —  EQUATION AUDIT  (.m logistic + .txt representative eqs)
================================================================================
  [PASS] I1    trap=2.4008 rel=0.0000%
  [PASS] I3    T=59.10s CI+/-0.92
  [PASS] I5    int=99.9955 1/lambda=100.0000
  [PASS] P1    emp=113.11 int=113.22
  [PASS] P5    d_num=1.080482 d_ana=1.080482  f(x0)=151.1164
  [PASS] A1    emp=163.19 ana=163.22
  [PASS] A8    V=[13.095 15.269 12.186] err=1.78e-14
  [PASS] B9    lambda2=0.6000 t<=12 emp=8

  Audit: 8/8 passed

================================================================================
  PART B  —  PYTHON SIMULATION  (8 ctrl x 4 S x 5 nat x 5 seeds)
================================================================================
  Total cells = 160  (800 runs)
  done fixed-time             (1/8)
  done adaptive               (2/8)
  done coordinated            (3/8)
  done cooperative-marl       (4/8)
  done resilient-marl         (5/8)
  done max-pressure           (6/8)
  done mpc                    (7/8)
  done weather-adaptive       (8/8)

================================================================================
  PART C  —  CROSS-CHECK  (.m logistic curve vs Python-simulated TT)
================================================================================
  Logistic coefficients (.m):  L=302.2328  k=0.0143  x0=101.5374

  Scen   Lat(ms)  Loss     rho    TT_sim  TT_logistic   delta%
  ------------------------------------------------------------------
  S1         0.0  0.00   0.000  58102.84        57.33101243.7%
  S2       100.0  0.05   0.108  58102.65       149.46 38776.3%
  S3       300.0  0.15   0.325  58090.59       285.52 20245.7%
  S4       600.0  0.30   0.650  58066.38       301.99 19127.9%

  Saved plot -> unified_evaluation.png
  Plot: unified_evaluation.png

================================================================================
  PART E1  —  SYNTHETIC DATASET
================================================================================
================================================================================
  SYNTHETIC DATASET  -  preset='quick'
================================================================================
```

```
  shape          : (16, 8, 4)
  total cells    : 512
  axis 0 (ctrl)  : 16
  axis 1 (scen)  : 8
  axis 2 (cond)  : 4
  seed           : 42
  car-following  : SAFE
  noise frac     : 0.06

  metric                   mean     stdev       min       max
  ------------------------------------------------------------
  travel_time_s         366.289   177.333   140.112  1458.922
  queue_veh              26.073    14.426     6.146   120.640
  waiting_time_s         43.301    24.081    11.290   200.418
  throughput_vph        394.996   134.763    96.879   717.050
  mean_aoi_ms           116.189    38.294    44.011   202.215
  delivery_ratio          0.841     0.073     0.683     1.000
  K_ctrl                  1.260     0.000     1.260     1.260
  K_comm                  1.302     0.294     1.000     1.600
  K_nat                   1.403     0.342     1.000     1.872
  K_dem                   1.066     0.006     1.057     1.072
  severity                0.158     0.075     0.005     0.305
  true_value            353.182   147.713   155.004   653.838
  wrote tensor (16, 8, 4, 12) -> synthetic_tensor.npy
  Tensor -> synthetic_tensor.npy  shape=(16, 8, 4, 12)

  PART E2  —  TENSOR PIPELINE (Ridge + kNN + GP + EI AL)
  loaded synthetic_tensor.npy  shape=(16, 8, 4, 12)  features=10  rows=512
================================================================================
  TENSOR PIPELINE - LOADED
================================================================================
  file           : synthetic_tensor.npy
  grid shape     : (16, 8, 4)
  flat rows      : 512
  features       : 10
  target         : 'travel_time_s'
  target mean    : 366.289
  target stdev   : 177.333
  target range   : [140.112, 1458.922]
================================================================================
  PIPELINE REPORT  -  target = 'travel_time_s'
================================================================================
  train / test   : 410 / 102
  feature count  : 10

  model                     MAE      RMSE        R2
  ------------------------------------------------
  mean                 118.4773  147.7644   -0.0382
  ridge                  5.8084    7.7181    0.9972
  knn5                   9.3931   14.3948    0.9901
  ridge+gp               1.9865    2.5566    0.9997

  Active-learning trace (EI loop)
   round       MAE      RMSE        R2
  -----------------------------------
```

```
   1.0   92.8536  139.3391    0.0768
   2.0   87.7665  132.1573    0.1696
   3.0   80.8658  122.2923    0.2889
   4.0   75.2150  113.7679    0.3846
   5.0   71.4235  107.9980    0.4454
   6.0   67.1505  101.8364    0.5069
   7.0   63.6033   96.2769    0.5593
   8.0   60.5316   91.5507    0.6015
   9.0   58.8897   89.0880    0.6226
  10.0   57.5209   86.9597    0.6404
  11.0   55.5092   83.9055    0.6653
  12.0   54.4384   82.3006    0.6779
  13.0   53.2073   80.4143    0.6925
  14.0   52.5644   79.4622    0.6998
  15.0   51.5089   77.7993    0.7122
  16.0   50.7609   76.6600    0.7206
  17.0   49.9011   75.3613    0.7300
  18.0   49.5144   74.7716    0.7342
  19.0   49.0878   74.1157    0.7388
  20.0   48.6182   73.2614    0.7448
  21.0   47.6190   71.6250    0.7561
  22.0   45.8779   68.8498    0.7746
  23.0   46.0474   68.7405    0.7753
  24.0   45.5627   68.0094    0.7801
  25.0   45.2092   67.4650    0.7836
  26.0   44.7107   66.6746    0.7886
  27.0   43.3886   64.7922    0.8004
  28.0   43.2759   64.4254    0.8026
  29.0   42.7340   63.5541    0.8079
  30.0   42.4790   63.1537    0.8104
  31.0   42.0737   62.4822    0.8144
  32.0   42.2032   62.6104    0.8136
  33.0   38.0181   55.8935    0.8515
  34.0   36.6873   53.8878    0.8619
  35.0   36.4235   53.3789    0.8645
  36.0   36.3318   53.2505    0.8652
  37.0   35.7269   52.2415    0.8702
  38.0   35.6144   52.0598    0.8711
  39.0   35.5572   51.9852    0.8715
  40.0   35.1983   51.4250    0.8743
  41.0   35.1331   51.2726    0.8750
  42.0   34.9964   50.9929    0.8764
  43.0   34.8497   50.7228    0.8777
  44.0   26.6023   38.1742    0.9307
  45.0   21.0948   29.8633    0.9576
  46.0   17.7376   24.9381    0.9704
  47.0   16.2855   22.8595    0.9752
  48.0   15.8837   22.2794    0.9764
  49.0   15.3878   21.5969    0.9778
  50.0   14.4735   20.3304    0.9803

================================================================================
  PART E3  —  TRUST-REGION ACTIVE LEARNING (TuRBO)
================================================================================
```

```
  TRUST-REGION ACTIVE LEARNING — 2 regions, budget=30
    region 0: center=('fixed-time', 'SAFE', 'S4', 'N-rush-pm', 'D2') best=7068.87 len
    region 1: center=('optimized', 'IDM', 'S4', 'N-workzone', 'D4') best=49138.86 len
    round   12/30  RMSE=63804.569  best=5382.41  regions=[18564, 18564]
    round   15/30  RMSE=57143.483  best=5382.41  regions=[18564, 18564]
    round   18/30  RMSE=52191.160  best=2743.93  regions=[18564, 18564]
    round   21/30  RMSE=49663.838  best=2743.93  regions=[18564, 18564]
    round   24/30  RMSE=46483.201  best=2743.93  regions=[18564, 1008]
    round   27/30  RMSE=43855.717  best=2743.93  regions=[18564, 1008]
    round   30/30  RMSE=41631.692  best=2743.93  regions=[18564, 40]

  TuRBO report: {'n_sims': 30, 'n_regions': 2, 'best_cell': ('fixed-time', 'SAFE', 'S
ail', 'D5'), 'best_value': 2743.93, 'residual_rmse': 41631.692, 'residual_mae': 2646
d_s': 2.75}
  Holdout: raw MAE=55272.931  corrected MAE=33563.823

================================================================================
  PART F  —  DISTRIBUTED ADAPTIVE SIGNAL EXECUTION
================================================================================

  t(s)  J11  J12  J13  J21  J22  J23  J31  J32  J33
     0  |^  3  |^  3  |^  3  |^  3  |^  3  |^  3  |^  3  |^  3  |^  3
   100  -> 33  -> 33  -> 33  -> 33  -> 33  -> 33  -> 33  -> 33  -> 33
   200  |^ 59  |^ 59  |^ 59  |^ 59  |^ 59  |^ 59  |^ 59  |^ 59  |^ 59
   300  ! 76  ! 76  ! 76  ! 76  ! 76  ! 76  ! 76  ! 76  ! 76
   400  -> 94  -> 94  -> 94  -> 94  -> 94  -> 94  -> 94  -> 94  -> 94
   500  !115  !115  !115  !115  !115  !115  !115  !115  !115
   600  |^138  |^138  |^138  |^138  |^138  |^138  |^138  |^138  |^138
   700  !162  !162  !162  !162  !162  !162  !162  !162  !162
   800  ->188  ->188  ->188  ->188  ->188  ->188  ->188  ->188  ->188
   900  !216  !216  !216  !216  !216  !216  !216  !216  !216

  legend: ^ NS green  > EW green  ! yellow  . all-red

================================================================================
  PER-CONTROLLER ADAPTATION REPORT
================================================================================
Node     min_grn    sw_thr   cyc_tgt  switches      adapt mg/st/ct
--------------------------------------------------------------------------------
J11        20.00      1.00     45.00        37      10/5  34
J12        20.00      1.00     45.00        37      10/5  34
J13        20.00      1.00     45.00        37      10/5  34
J21        20.00      1.00     45.00        37      10/5  34
J22        20.00      1.00     45.00        37      10/5  34
J23        20.00      1.00     45.00        37      10/5  34
J31        20.00      1.00     45.00        37      10/5  34
J32        20.00      1.00     45.00        37      10/5  34
J33        20.00      1.00     45.00        37      10/5  34

  Adaptive metrics: {'travel_time_s': 135.93, 'queue_veh': 119.9, 'waiting_time_s': 4
hput_vph': 735.1, 'total_switches': 333, 'adaptations': {'min_green': 90, 'switch_th
ycle_target': 306}}

  Running 3 parallel episodes ...
    seed    travel    queue     wait       thr  switches
```

```
------------------------------------------------------------
       1    135.93   119.90 41816.80     735.1       333
       2    135.93   119.90 41816.80     735.1       333
       3    135.92   119.90 41816.80     735.1       333

================================================================================
  PART G  —  DECAY-TRUST-REGION ACTIVE LEARNING
================================================================================

  DECAY-TRUST-REGION AL — 2 regions, budget=30
  DecaySchedule:
    min_length    : 0.05 -> 0.75  (power=1.5)
    expand factor  : 2.00 -> 3.00
    shrink factor  : 0.50 -> 0.85
    global phase   : after 75% of budget, floor=0.75
    reheat period  : 10
    stall threshold: 5
    region 0: center=('fixed-time', 'SAFE', 'S4', 'N-rush-pm', 'D2') best=7068.87 len
    region 1: center=('optimized', 'IDM', 'S4', 'N-workzone', 'D4') best=49138.86 len
    round   12/30  RMSE=63804.569  best=5382.41  lengths=[1.0, 1.0]  floor=0.06
    round   15/30  RMSE=57143.483  best=5382.41  lengths=[1.0, 1.0]  floor=0.12
    round   18/30  RMSE=52191.160  best=2743.93  lengths=[1.0, 1.0]  floor=0.21
    round   21/30  RMSE=51325.729  best=2743.93  lengths=[1.0, 1.0]  floor=0.32
    round   24/30  RMSE=48178.643  best=2743.93  lengths=[1.0, 0.74]  floor=0.45
    round   27/30  RMSE=45559.358  best=2743.93  lengths=[1.0, 1.0]  floor=0.59
    round   30/30  RMSE=45326.773  best=2743.93  lengths=[0.5, 1.0]  floor=0.75

  Decay-TuRBO report: {'n_sims': 32, 'n_regions': 2, 'best_cell': ('fixed-time', 'SAF
nsor-fail', 'D5'), 'best_value': 2743.93, 'residual_rmse': 45326.773, 'residual_mae'
apsed_s': 3.45}

================================================================================
  REGION HEALTH OVER TIME
================================================================================
 round     min    mean     max  mean|rn|   floor  global
--------------------------------------------------------------------------------
    12   1.000   1.000   1.000   18564.0   0.058   False
    15   1.000   1.000   1.000   18564.0   0.118   False
    18   1.000   1.000   1.000   18564.0   0.207   False
    21   1.000   1.000   1.000   18564.0   0.317   False
    24   0.739   0.870   1.000   12792.0   0.446   False
    27   1.000   1.000   1.000   18564.0   0.591    True
    30   0.500   0.750   1.000    9786.0   0.750    True


================================================================================
  OUTCOME MIX BY PHASE
================================================================================
phase         expanded    shrunk    global-pin      noop
--------------------------------------------------------------------------------
early              60%        0%            0%       40%
mid                20%       20%            0%       60%
late               20%        0%           40%       40%

  Decay holdout: raw MAE=59011.298  corrected MAE=29360.73
```

```
================================================================================
  FINAL UNIFIED REPORT
================================================================================

  Equation audit        : 8/8 passed
  Simulation cells run   : 3  (x5 seeds each)
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_1681/1520123291.py in <cell line: 0>()
   3696
   3697 if __name__ == "__main__":
-> 3698     raise SystemExit(main())
   3699

/tmp/ipykernel_1681/1520123291.py in main()
   3688     # --- FINAL
---------------------------------------------------------------
   3689     if audit_results or rows:
-> 3690         final_report(audit_results, rows)
   3691     print("\n" + "=" * 88)
   3692     print("  Unified evaluation complete")

/tmp/ipykernel_1681/1520123291.py in final_report(audit_results, rows)
   2890     print(f"\n  Equation audit        : {n_pass}/{len(audit_result
s)} passed")
   2891     print(f"  Simulation cells run   : {len(rows)}  (x5 seeds eac
h)")
-> 2892     print(f"  Controllers exercised  : {len({r['algorithm_type'] fo
r r in rows})}")
   2893     print(f"  Scenarios exercised    : {len({r['scenario'] for r in
rows})}")
   2894     print(f"  Natural conditions     : {len({r['natural'] for r in
rows})}")

KeyError: 'algorithm_type'
```

In [ ]:

```
# ================================================================================
#  train_synthetic.py
#  ------------------------------------------------------------------------------
#  End-to-end training pipeline for synthetic_tensor.npy
#
#  Tensor shape : (16, 8, 4, 12)   -> 16 samples, 8 channels, 4 x 12 spat
#  Dtype        : float32 (cast from float64)
#
#  This script is intentionally verbose (1251 lines) so every component i
#  explicit, readable, and easy to modify. It works out-of-the-box in Col
#  with `!python train_synthetic.py` or by pasting into a cell.
#
#  Key design choices
#  ------------------
#  * The tensor has NO labels. Rather than random targets (which triviall
#    overfit), we build targets from structure inside the tensor itself:
#        - Regression  : 4 summary statistics per sample (mean/std/max/l2
```

```
#        - Classification : 4-way binning of the tensor mean (quartiles)
#        - Autoencoding : reconstruct the tensor itself
#    All three tasks are trained and compared.
#
#  * 5-fold cross-validation on the 16 samples (yes, tiny — this is a smo
#    test / sanity harness; swap `X` for a bigger array and it scales).
#
#  * Multiple architectures are provided so you can ablate quickly.
#
#  * AMP + grad clipping + cosine LR + early stopping are all on by defau
#    and disable gracefully on CPU-only Colab.
# ============================================================================

from __future__ import annotations

import argparse
import json
import math
import os
import random
import sys
import time
import warnings
from dataclasses import dataclass, field, asdict
from pathlib import Path
from typing import Any, Callable, Dict, List, Optional, Tuple

import numpy as np

# ---------------------------------------------------------------------------
# Optional heavy imports (matplotlib, sklearn) — degrade gracefully
# ---------------------------------------------------------------------------
try:
    import matplotlib
    matplotlib.use("Agg")            # headless-safe in Colab
    import matplotlib.pyplot as plt
    HAS_MPL = True
except Exception:                    # pragma: no cover
    HAS_MPL = False

try:
    from sklearn.decomposition import PCA
    from sklearn.metrics import (confusion_matrix, mean_absolute_error,
                                 mean_squared_error, r2_score,
                                 accuracy_score, f1_score)
    from sklearn.model_selection import StratifiedKFold
    HAS_SKLEARN = True
except Exception:                    # pragma: no cover
    HAS_SKLEARN = False

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
```

```python
from torch.utils.data import (DataLoader, Dataset, Subset,
                              TensorDataset, random_split)

warnings.filterwarnings("ignore", category=UserWarning)

# ============================================================================
# SECTION 0 — Global configuration
# ============================================================================

@dataclass
class Config:
    """All knobs in one place. Override via CLI or by editing defaults."""

    # ---- I/O ----
    data_path: str = "synthetic_tensor.npy"
    out_dir: str = "runs/exp_default"
    seed: int = 42

    # ---- Task selection ----
    # one of: "regression", "classification", "autoencoder"
    task: str = "regression"

    # ---- Architecture ----
    # one of: "cnn", "resnet", "mlp", "transformer"
    arch: str = "cnn"
    hidden_dim: int = 64
    dropout: float = 0.1

    # ---- Optimization ----
    epochs: int = 200
    batch_size: int = 4
    lr: float = 1e-3
    weight_decay: float = 1e-4
    grad_clip: float = 1.0
    warmup_epochs: int = 5
    early_stop_patience: int = 40
    early_stop_min_delta: float = 1e-4

    # ---- Cross-validation ----
    n_folds: int = 5
    train_frac: float = 0.75          # used when n_folds <= 1

    # ---- Reproducibility / numerics ----
    amp: bool = True                  # mixed precision on GPU
    deterministic: bool = True
    num_workers: int = 0              # 0 is safest on Colab

    # ---- Logging ----
    log_every: int = 10
    save_plots: bool = True
    verbose: bool = True

    # ---- Derived / filled in at runtime ----
    device: str = field(default="", init=False)
```

```python
# ============================================================================
# SECTION 1 — Utility helpers
# ============================================================================

def set_seed(seed: int, deterministic: bool = True) -> None:
    """Best-effort full reproducibility across Python / NumPy / Torch."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    if deterministic:
        os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False
        try:
            torch.use_deterministic_algorithms(True, warn_only=True)
        except Exception:
            pass


def pick_device() -> str:
    """Prefer CUDA, fall back to MPS, finally CPU."""
    if torch.cuda.is_available():
        return "cuda"
    if getattr(torch.backends, "mps", None) and torch.backends.mps.is_ava
        return "mps"
    return "cpu"


def ensure_dir(path: str | Path) -> Path:
    p = Path(path)
    p.mkdir(parents=True, exist_ok=True)
    return p


def humanize_seconds(s: float) -> str:
    if s < 60:
        return f"{s:.1f}s"
    m, s = divmod(s, 60)
    if m < 60:
        return f"{int(m)}m{int(s):02d}s"
    h, m = divmod(m, 60)
    return f"{int(h)}h{int(m):02d}m"


def count_params(model: nn.Module) -> int:
    return sum(p.numel() for p in model.parameters() if p.requires_grad)


def gpu_mem_str() -> str:
    if not torch.cuda.is_available():
        return "cpu"
```

```
    alloc = torch.cuda.memory_allocated() / 1e6
    reserv = torch.cuda.memory_reserved() / 1e6
    return f"{alloc:.1f}/{reserv:.1f} MB"


def cosine_lr_lambda(warmup_epochs: int, total_epochs: int) -> Callable[[
    """Linear warmup -> cosine decay to 0."""
    def fn(epoch: int) -> float:
        if epoch < warmup_epochs:
            return (epoch + 1) / max(1, warmup_epochs)
        progress = (epoch - warmup_epochs) / max(1, total_epochs - warmup
        return 0.5 * (1.0 + math.cos(math.pi * progress))
    return fn


# ============================================================================
# SECTION 2 — Data loading and target construction
# ============================================================================

def load_tensor(path: str | Path) -> np.ndarray:
    """Load the .npy file regardless of extension. Returns float32 array.
    path = Path(path)
    if not path.exists():
        raise FileNotFoundError(f"Data file not found: {path}")

    # np.load accepts file objects, so the odd .txt.txt extension is fine
    with open(path, "rb") as f:
        arr = np.load(f, allow_pickle=False)

    if arr.dtype != np.float32:
        arr = arr.astype(np.float32)

    if not np.isfinite(arr).all():
        raise ValueError("Tensor contains NaN or Inf values")

    return arr


def standardize(x: np.ndarray) -> Tuple[np.ndarray, float, float]:
    """Zero-mean, unit-std over ALL elements. Returns (x_norm, mean, std)
    mu = float(x.mean())
    sd = float(x.std())
    if sd < 1e-12:
        sd = 1.0
    return (x - mu) / sd, mu, sd


# ----------------------------------------------------------------------------
# Target builders (the tensor has no labels)
# ----------------------------------------------------------------------------

def make_regression_targets(x: np.ndarray) -> np.ndarray:
    """
    Per-sample 4-D regression target built from real structure:
```

```
        [ mean, std, max, l2_norm ]
    Normalized to zero mean / unit std across samples.
    """
    n = x.shape[0]
    flat = x.reshape(n, -1)
    feats = np.stack([
        flat.mean(axis=1),
        flat.std(axis=1),
        flat.max(axis=1),
        np.linalg.norm(flat, axis=1),
    ], axis=1).astype(np.float32)

    # standardize so MSE loss is well-scaled
    feats = (feats - feats.mean(0, keepdims=True)) / (feats.std(0, keepdi
    return feats


def make_classification_targets(x: np.ndarray, n_classes: int = 4) -> np.
    """
    Bin samples by their global mean into `n_classes` quantile buckets.
    With N=16 and 4 classes this yields 4 samples per class — balanced.
    """
    n = x.shape[0]
    scores = x.reshape(n, -1).mean(axis=1)

    if n_classes > n:
        raise ValueError("n_classes cannot exceed number of samples")

    # rank-based binning avoids duplicate-edge issues from qcut
    order = np.argsort(scores, kind="stable")
    labels = np.empty(n, dtype=np.int64)
    for rank, idx in enumerate(order):
        labels[idx] = min(n_classes - 1, (rank * n_classes) // n)
    return labels


# ============================================================================
# SECTION 3 — Datasets
# ============================================================================

class ArrayDataset(Dataset):
    """
    Generic dataset wrapping (X, y) numpy arrays.
      X : (N, C, H, W) float32
      y : (N,) or (N, K) float32/int64 depending on task
    """

    def __init__(self, X: np.ndarray, y: np.ndarray):
        assert len(X) == len(y), "X and y must have equal length"
        self.X = torch.from_numpy(np.ascontiguousarray(X))
        self.y = torch.from_numpy(np.ascontiguousarray(y))

    def __len__(self) -> int:
        return len(self.X)
```

```python
    def __getitem__(self, i: int) -> Tuple[torch.Tensor, torch.Tensor]:
        return self.X[i], self.y[i]


# ============================================================================
# SECTION 4 — Model zoo
# ============================================================================

# ---------------------------- 4.1  Simple CNN ---------------------------
class SimpleCNN(nn.Module):
    """
    Baseline CNN for 8 x 4 x 12 inputs (C x H x W).

    For regression  -> outputs (B, K) where K = n_out
    For classif.    -> outputs (B, n_classes) logits
    """

    def __init__(self, in_ch: int = 8, n_out: int = 4, hidden: int = 64,
                 dropout: float = 0.1):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(in_ch, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),                        # -> 32 x 2 x 6

            nn.Conv2d(32, hidden, kernel_size=3, padding=1),
            nn.BatchNorm2d(hidden),
            nn.ReLU(inplace=True),
            nn.AdaptiveAvgPool2d((1, 1)),           # -> hidden x 1 x 1
        )
        self.head = nn.Sequential(
            nn.Flatten(),
            nn.Linear(hidden, hidden),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(hidden, n_out),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.head(self.features(x))


# ---------------------------- 4.2  ResNet-ish ---------------------------
class ResidualBlock2D(nn.Module):
    """A two-conv residual block with BN + skip connection."""

    def __init__(self, in_ch: int, out_ch: int, stride: int = 1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride=stride, padding=1
        self.bn1 = nn.BatchNorm2d(out_ch)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_ch)
```

```
        if stride != 1 or in_ch != out_ch:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_ch, out_ch, 1, stride=stride, bias=False),
                nn.BatchNorm2d(out_ch),
            )
        else:
            self.shortcut = nn.Identity()

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        out = F.relu(self.bn1(self.conv1(x)), inplace=True)
        out = self.bn2(self.conv2(out))
        out = out + self.shortcut(x)
        return F.relu(out, inplace=True)


class SmallResNet(nn.Module):
    """
    Tiny ResNet designed for 8 x 4 x 12 inputs — the input is small so
    we avoid aggressive downsampling to keep enough spatial resolution.
    """

    def __init__(self, in_ch: int = 8, n_out: int = 4, hidden: int = 32,
                 dropout: float = 0.1):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(in_ch, hidden, 3, padding=1, bias=False),
            nn.BatchNorm2d(hidden),
            nn.ReLU(inplace=True),
        )
        self.layer1 = ResidualBlock2D(hidden, hidden)
        self.layer2 = ResidualBlock2D(hidden, hidden * 2, stride=2)  # ->
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.head = nn.Sequential(
            nn.Flatten(),
            nn.Dropout(dropout),
            nn.Linear(hidden * 2, n_out),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.stem(x)
        x = self.layer1(x)
        x = self.layer2(x)
        x = self.pool(x)
        return self.head(x)


# ---------------------------- 4.3  MLP ---------------------------------
class MLP(nn.Module):
    """Fully-connected baseline — useful as an ablation (no spatial bias)

    def __init__(self, in_ch: int = 8, h: int = 4, w: int = 12,
                 n_out: int = 4, hidden: int = 128, dropout: float = 0.1)
        super().__init__()
```

```
        in_dim = in_ch * h * w
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(in_dim, hidden),
            nn.BatchNorm1d(hidden),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(hidden, hidden),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(hidden, n_out),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)


# ---------------------------- 4.4  Transformer ------------------------
class TinyViT(nn.Module):
    """
    Vision-Transformer-lite for very small images.

    We flatten (H, W) into tokens, project each pixel-vector to `hidden`,
    add learned positional embeddings, run one Transformer encoder layer,
    then mean-pool.
    """

    def __init__(self, in_ch: int = 8, h: int = 4, w: int = 12,
                 n_out: int = 4, hidden: int = 64, n_heads: int = 4,
                 n_layers: int = 2, dropout: float = 0.1):
        super().__init__()
        assert hidden % n_heads == 0, "hidden must be divisible by n_head
        self.n_tokens = h * w
        self.patch = nn.Linear(in_ch, hidden)
        self.pos = nn.Parameter(torch.zeros(1, self.n_tokens, hidden))
        nn.init.trunc_normal_(self.pos, std=0.02)

        enc_layer = nn.TransformerEncoderLayer(
            d_model=hidden, nhead=n_heads, dim_feedforward=hidden * 2,
            dropout=dropout, batch_first=True, activation="gelu",
            norm_first=True,
        )
        self.encoder = nn.TransformerEncoder(enc_layer, num_layers=n_laye
        self.norm = nn.LayerNorm(hidden)
        self.head = nn.Linear(hidden, n_out)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (B, C, H, W) -> (B, H*W, C)
        B, C, H, W = x.shape
        x = x.permute(0, 2, 3, 1).reshape(B, H * W, C)
        x = self.patch(x) + self.pos
        x = self.encoder(x)
        x = self.norm(x.mean(dim=1))
        return self.head(x)
```

```python
# ---------------------------- 4.5  Autoencoder -------------------------
class ConvAutoencoder(nn.Module):
    """
    Reconstructs the full tensor. Encoder compresses to a latent vector,
    decoder upsamples back to (C, H, W).
    """

    def __init__(self, in_ch: int = 8, h: int = 4, w: int = 12,
                 latent: int = 32):
        super().__init__()
        self.h, self.w = h, w
        self.encoder = nn.Sequential(
            nn.Conv2d(in_ch, 16, 3, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(inplace=True),
            nn.AdaptiveAvgPool2d((1, 1)),
            nn.Flatten(),
            nn.Linear(32, latent),
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent, 32 * h * w),
            nn.ReLU(inplace=True),
            nn.Unflatten(1, (32, h, w)),
            nn.Conv2d(32, 16, 3, padding=1), nn.ReLU(inplace=True),
            nn.Conv2d(16, in_ch, 3, padding=1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        z = self.encoder(x)
        return self.decoder(z)


# ---------------------------- 4.6  Factory -----------------------------
def build_model(cfg: Config, in_ch: int, h: int, w: int,
                n_out: int) -> nn.Module:
    arch = cfg.arch.lower()
    if cfg.task == "autoencoder":
        return ConvAutoencoder(in_ch=in_ch, h=h, w=w, latent=cfg.hidden_d

    if arch == "cnn":
        return SimpleCNN(in_ch, n_out, cfg.hidden_dim, cfg.dropout)
    if arch == "resnet":
        return SmallResNet(in_ch, n_out, max(16, cfg.hidden_dim // 2), cf
    if arch == "mlp":
        return MLP(in_ch, h, w, n_out, cfg.hidden_dim, cfg.dropout)
    if arch == "transformer":
        return TinyViT(in_ch, h, w, n_out, cfg.hidden_dim, 4, 2, cfg.drop
    raise ValueError(f"Unknown arch: {cfg.arch}")


# ============================================================================
# SECTION 5 — Metrics
# ============================================================================
```

```python
def regression_metrics(pred: np.ndarray, targ: np.ndarray) -> Dict[str, f
    mse = float(mean_squared_error(targ, pred)) if HAS_SKLEARN else float
    mae = float(mean_absolute_error(targ, pred)) if HAS_SKLEARN else floa
    r2  = float(r2_score(targ, pred)) if HAS_SKLEARN else float("nan")
    return {"mse": mse, "mae": mae, "r2": r2}


def classification_metrics(pred: np.ndarray, targ: np.ndarray) -> Dict[st
    if HAS_SKLEARN:
        acc = float(accuracy_score(targ, pred))
        f1  = float(f1_score(targ, pred, average="macro", zero_division=0
    else:
        acc = float((pred == targ).mean())
        f1 = float("nan")
    return {"acc": acc, "f1_macro": f1}


# ============================================================================
# SECTION 6 — Training engine (one fold)
# ============================================================================

def _maybe_amp(device: str, use_amp: bool):
    """Return (enabled, autocast_factory, grad_scaler)."""
    enabled = use_amp and device.startswith("cuda")
    autocast = (lambda: torch.cuda.amp.autocast(enabled=True)) if enabled
    scaler = torch.cuda.amp.GradScaler(enabled=enabled)
    return enabled, autocast, scaler


class _nullcontext:
    def __enter__(self): return self
    def __exit__(self, *a): return False


def train_one_fold(
    cfg: Config,
    fold: int,
    train_idx: np.ndarray,
    val_idx: np.ndarray,
    X: np.ndarray,
    y: np.ndarray,
    out_dir: Path,
) -> Dict[str, Any]:
    """
    Train a single fold. Returns a dict with best metrics + history.
    Saves:
        out_dir/fold{fold}/best.pt
        out_dir/fold{fold}/history.json
    """
    device = cfg.device
    in_ch, h, w = X.shape[1:]
    n_out = 1 if cfg.task == "autoencoder" else (
        y.shape[1] if y.ndim > 1 else int(y.max()) + 1
```

```
    )

    # ---- Dataset ----
    X_tr, X_va = X[train_idx], X[val_idx]
    if cfg.task == "autoencoder":
        y_tr, y_va = X_tr, X_va           # target == input
    else:
        y_tr, y_va = y[train_idx], y[val_idx]

    train_ds = ArrayDataset(X_tr, y_tr)
    val_ds   = ArrayDataset(X_va, y_va)

    train_loader = DataLoader(train_ds, batch_size=cfg.batch_size,
                              shuffle=True, num_workers=cfg.num_workers,
                              drop_last=False)
    val_loader   = DataLoader(val_ds, batch_size=cfg.batch_size,
                              shuffle=False, num_workers=cfg.num_workers)

    # ---- Model / loss / optim ----
    model = build_model(cfg, in_ch, h, w, n_out).to(device)
    if cfg.task == "classification":
        criterion: nn.Module = nn.CrossEntropyLoss()
    else:
        criterion = nn.MSELoss()

    optimizer = optim.AdamW(model.parameters(), lr=cfg.lr,
                            weight_decay=cfg.weight_decay)
    scheduler = optim.lr_scheduler.LambdaLR(
        optimizer, cosine_lr_lambda(cfg.warmup_epochs, cfg.epochs)
    )

    amp_on, autocast, scaler = _maybe_amp(device, cfg.amp)

    # ---- Bookkeeping ----
    best_val = float("inf")
    best_epoch = -1
    patience = 0
    history: Dict[str, List[float]] = {"train_loss": [], "val_loss": []}
    fold_dir = ensure_dir(out_dir / f"fold{fold}")
    ckpt_path = fold_dir / "best.pt"

    t0 = time.time()

    for epoch in range(cfg.epochs):
        # ---------- train ----------
        model.train()
        running = 0.0
        for xb, yb in train_loader:
            xb = xb.to(device, non_blocking=True)
            yb = yb.to(device, non_blocking=True)

            optimizer.zero_grad(set_to_none=True)
            with autocast():
                out = model(xb)
```

```
                if cfg.task == "classification":
                    loss = criterion(out, yb.long())
                else:
                    loss = criterion(out, yb)

            if amp_on:
                scaler.scale(loss).backward()
                if cfg.grad_clip > 0:
                    scaler.unscale_(optimizer)
                    nn.utils.clip_grad_norm_(model.parameters(), cfg.grad
                scaler.step(optimizer)
                scaler.update()
            else:
                loss.backward()
                if cfg.grad_clip > 0:
                    nn.utils.clip_grad_norm_(model.parameters(), cfg.grad
                optimizer.step()

            running += loss.item() * xb.size(0)

        scheduler.step()
        train_loss = running / len(train_loader.dataset)

        # ---------- validate ----------
        model.eval()
        v_running = 0.0
        with torch.no_grad():
            for xb, yb in val_loader:
                xb = xb.to(device, non_blocking=True)
                yb = yb.to(device, non_blocking=True)
                with autocast():
                    out = model(xb)
                    if cfg.task == "classification":
                        vloss = criterion(out, yb.long())
                    else:
                        vloss = criterion(out, yb)
                v_running += vloss.item() * xb.size(0)
        val_loss = v_running / max(1, len(val_loader.dataset))

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)

        # ---------- early stopping ----------
        improved = val_loss < best_val - cfg.early_stop_min_delta
        if improved:
            best_val = val_loss
            best_epoch = epoch
            patience = 0
            torch.save({
                "model": model.state_dict(),
                "cfg":   asdict(cfg),
                "epoch": epoch,
                "val_loss": val_loss,
            }, ckpt_path)
```

```
        else:
            patience += 1

        if cfg.verbose and (epoch % cfg.log_every == 0 or epoch == 0):
            lr_now = optimizer.param_groups[0]["lr"]
            print(f"  [fold {fold}] ep {epoch:3d} | "
                  f"train {train_loss:.4f} | val {val_loss:.4f} | "
                  f"lr {lr_now:.2e} | pat {patience:2d} | {gpu_mem_str()}")

        if patience >= cfg.early_stop_patience:
            if cfg.verbose:
                print(f"  [fold {fold}] early stop at epoch {epoch} "
                      f"(best {best_val:.4f} @ ep {best_epoch})")
            break

    elapsed = time.time() - t0

    # ---- Restore best & evaluate on val ----
    if ckpt_path.exists():
        state = torch.load(ckpt_path, map_location=device)
        model.load_state_dict(state["model"])

    model.eval()
    preds, targets = [], []
    with torch.no_grad():
        for xb, yb in val_loader:
            xb = xb.to(device)
            out = model(xb).float().cpu()
            preds.append(out)
            targets.append(yb)

    preds   = torch.cat(preds).numpy()
    targets = torch.cat(targets).numpy()

    if cfg.task == "classification":
        metrics = classification_metrics(preds.argmax(1), targets.astype(
    elif cfg.task == "regression":
        metrics = regression_metrics(preds, targets)
    else:  # autoencoder
        metrics = regression_metrics(preds.reshape(len(preds), -1),
                                     targets.reshape(len(targets), -1))

    metrics.update({
        "best_val_loss": float(best_val),
        "best_epoch": best_epoch,
        "elapsed_sec": elapsed,
        "n_params": count_params(model),
        "fold": fold,
    })

    with open(fold_dir / "history.json", "w") as f:
        json.dump({"history": history, "metrics": metrics}, f, indent=2)

    print(f"  [fold {fold}] done in {humanize_seconds(elapsed)} | "
```

```
          f"best val {best_val:.4f} @ ep {best_epoch} | "
          f"metrics { {k: round(v, 4) if isinstance(v, float) else v for

    return {"metrics": metrics, "history": history, "model": model}


# ============================================================================
# SECTION 7 — Cross-validation orchestration
# ============================================================================

def make_folds(cfg: Config, y: np.ndarray, n: int) -> List[Tuple[np.ndarr
    """Return list of (train_idx, val_idx) tuples."""
    rng = np.random.default_rng(cfg.seed)

    if cfg.n_folds and cfg.n_folds > 1:
        if cfg.task == "classification" and HAS_SKLEARN:
            skf = StratifiedKFold(n_splits=cfg.n_folds, shuffle=True,
                                  random_state=cfg.seed)
            return [(tr, va) for tr, va in skf.split(np.zeros(n), y)]
        # fallback: shuffled KFold
        idx = rng.permutation(n)
        folds = np.array_split(idx, cfg.n_folds)
        out = []
        for k in range(cfg.n_folds):
            va = folds[k]
            tr = np.concatenate([folds[j] for j in range(cfg.n_folds) if
            out.append((tr, va))
        return out

    # single split
    idx = rng.permutation(n)
    cut = int(cfg.train_frac * n)
    return [(idx[:cut], idx[cut:])]


# ============================================================================
# SECTION 8 — Plotting
# ============================================================================

def plot_loss_curves(fold_histories: List[Dict[str, List[float]]],
                     out_path: Path) -> None:
    if not HAS_MPL:
        return
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    for i, hist in enumerate(fold_histories):
        axes[0].plot(hist["train_loss"], label=f"fold {i}")
        axes[1].plot(hist["val_loss"],   label=f"fold {i}")
    axes[0].set_title("Train loss"); axes[0].set_xlabel("epoch"); axes[0]
    axes[1].set_title("Val loss");   axes[1].set_xlabel("epoch"); axes[1]
    fig.tight_layout()
    fig.savefig(out_path, dpi=120)
    plt.close(fig)
```

```python
def plot_pred_vs_target(pred: np.ndarray, targ: np.ndarray, out_path: Pat
    if not HAS_MPL:
        return
    # show first output dimension only (or a scatter for 1-D)
    p = pred.reshape(len(pred), -1)[:, 0]
    t = targ.reshape(len(targ), -1)[:, 0]
    fig, ax = plt.subplots(figsize=(5, 5))
    ax.scatter(t, p, alpha=0.8)
    lo, hi = min(t.min(), p.min()), max(t.max(), p.max())
    ax.plot([lo, hi], [lo, hi], "r--", lw=1)
    ax.set_xlabel("target"); ax.set_ylabel("prediction")
    ax.set_title("Prediction vs Target")
    fig.tight_layout(); fig.savefig(out_path, dpi=120); plt.close(fig)


def plot_pca(X: np.ndarray, y: Optional[np.ndarray], out_path: Path) -> N
    if not (HAS_MPL and HAS_SKLEARN):
        return
    flat = X.reshape(len(X), -1)
    if flat.shape[1] < 2:
        return
    z = PCA(n_components=2, random_state=0).fit_transform(flat)
    fig, ax = plt.subplots(figsize=(5, 4))
    if y is not None:
        sc = ax.scatter(z[:, 0], z[:, 1], c=y, cmap="viridis", s=60)
        fig.colorbar(sc, ax=ax, label="label")
    else:
        ax.scatter(z[:, 0], z[:, 1], s=60)
    ax.set_title("PCA of samples"); ax.set_xlabel("PC1"); ax.set_ylabel("
    fig.tight_layout(); fig.savefig(out_path, dpi=120); plt.close(fig)


# ============================================================================
# SECTION 9 — Inference helper
# ============================================================================

@torch.no_grad()
def predict(cfg: Config, ckpt_path: str | Path, X_new: np.ndarray) -> np.
    """Load a saved checkpoint and run inference on X_new (N, C, H, W)."""
    device = cfg.device or pick_device()
    state = torch.load(ckpt_path, map_location=device)
    in_ch, h, w = X_new.shape[1:]
    n_out = state["model"]["head.4.weight"].shape[0] if False else None

    # Rebuild model using cfg in checkpoint
    saved_cfg = Config(**{**asdict(cfg), **state.get("cfg", {})})
    n_out = 1 if saved_cfg.task == "autoencoder" else None
    if n_out is None:
        # infer from a dummy pass
        tmp = build_model(saved_cfg, in_ch, h, w, n_out=1).to(device)
        n_out = tmp(torch.zeros(1, in_ch, h, w, device=device)).shape[-1]

    model = build_model(saved_cfg, in_ch, h, w, n_out=n_out).to(device)
    model.load_state_dict(state["model"])
```

```
    model.eval()

    X_t = torch.from_numpy(X_new.astype(np.float32)).to(device)
    out = model(X_t).float().cpu().numpy()
    if saved_cfg.task == "classification":
        return out.argmax(1)
    return out


# ============================================================================
# SECTION 10 — Main pipeline
# ============================================================================

def run(cfg: Config) -> Dict[str, Any]:
    set_seed(cfg.seed, cfg.deterministic)
    cfg.device = pick_device()
    out_dir = ensure_dir(cfg.out_dir)

    print("=" * 78)
    print(f"Config: {json.dumps(asdict(cfg), indent=2, default=str)}")
    print("=" * 78)

    # ---- Load & normalize ----
    print(f"[data] loading {cfg.data_path}")
    X_raw = load_tensor(cfg.data_path)
    print(f"[data] raw shape {X_raw.shape}  dtype {X_raw.dtype}  "
          f"min {X_raw.min():.3f}  max {X_raw.max():.3f}")

    X_norm, mu, sd = standardize(X_raw)
    print(f"[data] standardized (mu={mu:.3f}, sd={sd:.3f})")

    # ---- Build targets based on task ----
    if cfg.task == "regression":
        y = make_regression_targets(X_norm)
        print(f"[task] regression: target shape {y.shape} (mean/std/max/l
    elif cfg.task == "classification":
        y = make_classification_targets(X_norm, n_classes=4)
        print(f"[task] classification: {len(set(y.tolist()))} classes, "
              f"counts {np.bincount(y).tolist()}")
    elif cfg.task == "autoencoder":
        y = X_norm  # unused, but keeps signature uniform
        print("[task] autoencoder: reconstruct input")
    else:
        raise ValueError(f"Unknown task: {cfg.task}")

    # ---- Quick sanity visualization ----
    if cfg.save_plots:
        plot_pca(X_norm, y if cfg.task == "classification" else None,
                 out_dir / "pca.png")

    # ---- Cross-validation ----
    folds = make_folds(cfg, y if cfg.task != "autoencoder" else
                       np.zeros(len(X_norm), dtype=int), len(X_norm))
    print(f"[cv] {len(folds)} folds, sizes "
```

```
          f"{[(len(tr), len(va)) for tr, va in folds]}")

    all_metrics: List[Dict[str, Any]] = []
    fold_histories: List[Dict[str, List[float]]] = []
    t_start = time.time()

    for fold_idx, (tr, va) in enumerate(folds):
        result = train_one_fold(cfg, fold_idx, tr, va, X_norm, y, out_dir
        all_metrics.append(result["metrics"])
        fold_histories.append(result["history"])

    total = time.time() - t_start

    # ---- Aggregate ----
    print("\n" + "=" * 78)
    print(f"Training finished in {humanize_seconds(total)}")
    print("=" * 78)

    keys = [k for k in all_metrics[0] if isinstance(all_metrics[0][k], (i
    summary: Dict[str, Dict[str, float]] = {}
    for k in keys:
        vals = [m[k] for m in all_metrics]
        summary[k] = {"mean": float(np.mean(vals)), "std": float(np.std(v

    print(f"\n{'metric':<16}{'mean':>10}{'std':>10}")
    print("-" * 36)
    for k, v in summary.items():
        print(f"{k:<16}{v['mean']:>10.4f}{v['std']:>10.4f}")

    # ---- Plots ----
    if cfg.save_plots:
        plot_loss_curves(fold_histories, out_dir / "loss_curves.png")
        print(f"\n[plot] loss curves saved to {out_dir/'loss_curves.png'}

    # ---- Save summary ----
    with open(out_dir / "summary.json", "w") as f:
        json.dump({
            "config": asdict(cfg),
            "data": {"shape": list(X_raw.shape),
                     "mu": mu, "sd": sd},
            "folds": all_metrics,
            "summary": summary,
            "elapsed_sec": total,
        }, f, indent=2)
    print(f"[save] summary -> {out_dir/'summary.json'}")

    return {"summary": summary, "folds": all_metrics,
            "out_dir": str(out_dir), "cfg": cfg}


# ============================================================================
# SECTION 11 — CLI
# ============================================================================
```

```python
def parse_args() -> Config:
    parser = argparse.ArgumentParser(description="Train on synthetic_tens
    parser.add_argument("--data", type=str, default="synthetic_tensor.npy
    parser.add_argument("--out",  type=str, default="runs/exp_default")
    parser.add_argument("--task", type=str, default="regression",
                        choices=["regression", "classification", "autoenc
    parser.add_argument("--arch", type=str, default="cnn",
                        choices=["cnn", "resnet", "mlp", "transformer"])
    parser.add_argument("--epochs", type=int, default=200)
    parser.add_argument("--batch-size", type=int, default=4)
    parser.add_argument("--lr", type=float, default=1e-3)
    parser.add_argument("--hidden", type=int, default=64)
    parser.add_argument("--dropout", type=float, default=0.1)
    parser.add_argument("--weight-decay", type=float, default=1e-4)
    parser.add_argument("--folds", type=int, default=5)
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--no-amp", action="store_true")
    parser.add_argument("--log-every", type=int, default=10)
    parser.add_argument("--quiet", action="store_true")

    args = parser.parse_args()

    return Config(
        data_path=args.data,
        out_dir=args.out,
        task=args.task,
        arch=args.arch,
        epochs=args.epochs,
        batch_size=args.batch_size,
        lr=args.lr,
        hidden_dim=args.hidden,
        dropout=args.dropout,
        weight_decay=args.weight_decay,
        n_folds=args.folds,
        seed=args.seed,
        amp=not args.no_amp,
        log_every=args.log_every,
        verbose=not args.quiet,
    )


def main() -> None:
    if "ipykernel" in sys.modules:
        # In Colab, argparse would choke on the notebook's own argv.
        cfg = Config()
    else:
        cfg = parse_args()

    results = run(cfg)
    print("\nDone.  Artifacts in:", results["out_dir"])


if __name__ == "__main__":
    main()
```

```
================================================================================
Config: {
  "data_path": "synthetic_tensor.npy",
  "out_dir": "runs/exp_default",
  "seed": 42,
  "task": "regression",
  "arch": "cnn",
  "hidden_dim": 64,
  "dropout": 0.1,
  "epochs": 200,
  "batch_size": 4,
  "lr": 0.001,
  "weight_decay": 0.0001,
  "grad_clip": 1.0,
  "warmup_epochs": 5,
  "early_stop_patience": 40,
  "early_stop_min_delta": 0.0001,
  "n_folds": 5,
  "train_frac": 0.75,
  "amp": true,
  "deterministic": true,
  "num_workers": 0,
  "log_every": 10,
  "save_plots": true,
  "verbose": true,
  "device": "cpu"
}
================================================================================
[data] loading synthetic_tensor.npy
[data] raw shape (16, 8, 4, 12)  dtype float32  min 0.005  max 1458.922
[data] standardized (mu=108.838, sd=173.811)
[task] regression: target shape (16, 4) (mean/std/max/l2)
[cv] 5 folds, sizes [(12, 4), (13, 3), (13, 3), (13, 3), (13, 3)]
  [fold 0] ep   0 | train 0.9561 | val 1.2119 | lr 4.00e-04 | pat  0 | cpu
  [fold 0] ep  10 | train 0.6973 | val 1.2694 | lr 9.98e-04 | pat 10 | cpu
  [fold 0] ep  20 | train 0.3806 | val 1.4366 | lr 9.83e-04 | pat 20 | cpu
  [fold 0] ep  30 | train 0.1174 | val 1.1302 | lr 9.57e-04 | pat  2 | cpu
  [fold 0] ep  40 | train 0.0793 | val 1.4562 | lr 9.18e-04 | pat 12 | cpu
  [fold 0] ep  50 | train 0.0529 | val 1.0039 | lr 8.69e-04 | pat 22 | cpu
  [fold 0] ep  60 | train 0.0503 | val 1.0213 | lr 8.10e-04 | pat 32 | cpu
  [fold 0] early stop at epoch 68 (best 0.9845 @ ep 28)
  [fold 0] done in 1.2s | best val 0.9845 @ ep 28 | metrics {'mse': 0.9845, 'm
ae': 0.6807, 'r2': 0.1018, 'best_val_loss': 0.9845, 'best_epoch': 28, 'elapse
d_sec': 1.1967, 'n_params': 25444, 'fold': 0}
  [fold 1] ep   0 | train 0.9549 | val 1.2354 | lr 4.00e-04 | pat  0 | cpu
  [fold 1] ep  10 | train 0.6931 | val 1.0643 | lr 9.98e-04 | pat  0 | cpu
  [fold 1] ep  20 | train 0.3211 | val 0.5301 | lr 9.83e-04 | pat  0 | cpu
  [fold 1] ep  30 | train 0.1118 | val 0.5985 | lr 9.57e-04 | pat  8 | cpu
  [fold 1] ep  40 | train 0.1016 | val 0.8584 | lr 9.18e-04 | pat 18 | cpu
  [fold 1] ep  50 | train 0.1175 | val 0.7400 | lr 8.69e-04 | pat 28 | cpu
  [fold 1] ep  60 | train 0.0750 | val 0.6299 | lr 8.10e-04 | pat 38 | cpu
  [fold 1] early stop at epoch 62 (best 0.4449 @ ep 22)
  [fold 1] done in 1.0s | best val 0.4449 @ ep 22 | metrics {'mse': 0.4449, 'm
ae': 0.5767, 'r2': 0.6539, 'best_val_loss': 0.4449, 'best_epoch': 22, 'elapse
d_sec': 1.0095, 'n_params': 25444, 'fold': 1}
```

```
  [fold 2] ep   0 | train 1.1470 | val 0.4759 | lr 4.00e-04 | pat  0 | cpu
  [fold 2] ep  10 | train 0.9439 | val 0.4943 | lr 9.98e-04 | pat 10 | cpu
  [fold 2] ep  20 | train 0.2155 | val 0.3197 | lr 9.83e-04 | pat  0 | cpu
  [fold 2] ep  30 | train 0.1110 | val 0.3622 | lr 9.57e-04 | pat  9 | cpu
  [fold 2] ep  40 | train 0.4442 | val 0.0472 | lr 9.18e-04 | pat  2 | cpu
  [fold 2] ep  50 | train 0.1243 | val 0.0490 | lr 8.69e-04 | pat 12 | cpu
  [fold 2] ep  60 | train 0.0999 | val 0.0501 | lr 8.10e-04 | pat 22 | cpu
  [fold 2] ep  70 | train 0.1054 | val 0.0449 | lr 7.43e-04 | pat 32 | cpu
  [fold 2] early stop at epoch 78 (best 0.0214 @ ep 38)
  [fold 2] done in 1.0s | best val 0.0214 @ ep 38 | metrics {'mse': 0.0214, 'm
ae': 0.1123, 'r2': 0.3282, 'best_val_loss': 0.0214, 'best_epoch': 38, 'elapse
d_sec': 1.013, 'n_params': 25444, 'fold': 2}
  [fold 3] ep   0 | train 1.0916 | val 0.5904 | lr 4.00e-04 | pat  0 | cpu
  [fold 3] ep  10 | train 0.8207 | val 0.5904 | lr 9.98e-04 | pat 10 | cpu
  [fold 3] ep  20 | train 0.2613 | val 0.4091 | lr 9.83e-04 | pat  1 | cpu
  [fold 3] ep  30 | train 0.2712 | val 0.2381 | lr 9.57e-04 | pat  2 | cpu
  [fold 3] ep  40 | train 0.1002 | val 0.2738 | lr 9.18e-04 | pat 12 | cpu
  [fold 3] ep  50 | train 0.2581 | val 0.0957 | lr 8.69e-04 | pat  0 | cpu
  [fold 3] ep  60 | train 0.1076 | val 0.2109 | lr 8.10e-04 | pat 10 | cpu
  [fold 3] ep  70 | train 0.1001 | val 0.1432 | lr 7.43e-04 | pat 20 | cpu
  [fold 3] ep  80 | train 0.0650 | val 0.2215 | lr 6.70e-04 | pat  8 | cpu
  [fold 3] ep  90 | train 0.0763 | val 0.2466 | lr 5.92e-04 | pat 18 | cpu
  [fold 3] ep 100 | train 0.1048 | val 0.1651 | lr 5.12e-04 | pat 28 | cpu
  [fold 3] ep 110 | train 0.0581 | val 0.1958 | lr 4.32e-04 | pat 38 | cpu
  [fold 3] early stop at epoch 112 (best 0.0955 @ ep 72)
  [fold 3] done in 1.6s | best val 0.0955 @ ep 72 | metrics {'mse': 0.0955, 'm
ae': 0.2582, 'r2': 0.3504, 'best_val_loss': 0.0955, 'best_epoch': 72, 'elapse
d_sec': 1.5633, 'n_params': 25444, 'fold': 3}
  [fold 4] ep   0 | train 0.8882 | val 1.4976 | lr 4.00e-04 | pat  0 | cpu
  [fold 4] ep  10 | train 0.8183 | val 1.7081 | lr 9.98e-04 | pat 10 | cpu
  [fold 4] ep  20 | train 0.5601 | val 1.6764 | lr 9.83e-04 | pat 20 | cpu
  [fold 4] ep  30 | train 0.1988 | val 1.7440 | lr 9.57e-04 | pat  5 | cpu
  [fold 4] ep  40 | train 0.0831 | val 1.8999 | lr 9.18e-04 | pat  3 | cpu
  [fold 4] ep  50 | train 0.3287 | val 1.6115 | lr 8.69e-04 | pat 13 | cpu
  [fold 4] ep  60 | train 0.0808 | val 1.7478 | lr 8.10e-04 | pat 23 | cpu
  [fold 4] ep  70 | train 0.1769 | val 1.9539 | lr 7.43e-04 | pat 33 | cpu
  [fold 4] early stop at epoch 77 (best 0.8932 @ ep 37)
  [fold 4] done in 1.3s | best val 0.8932 @ ep 37 | metrics {'mse': 0.8932, 'm
ae': 0.726, 'r2': 0.3633, 'best_val_loss': 0.8932, 'best_epoch': 37, 'elapse
d_sec': 1.2861, 'n_params': 25444, 'fold': 4}

================================================================================
Training finished in 6.1s
================================================================================

metric                mean       std
------------------------------------
mse                 0.4879    0.3961
mae                 0.4708    0.2425
r2                  0.3595    0.1756
best_val_loss       0.4879    0.3961
best_epoch         39.4000   17.3390
elapsed_sec         1.2137    0.2048
n_params        25444.0000    0.0000
fold                2.0000    1.4142
```

```
[plot] loss curves saved to runs/exp_default/loss_curves.png
[save] summary -> runs/exp_default/summary.json

Done.  Artifacts in: runs/exp_default
```

In [ ]:

```python
# ============================================================
# Stage 1: Harden the regression setup
# ============================================================
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader, random_split
import json

# ---- Reproducibility ----
SEED = 42
np.random.seed(SEED); torch.manual_seed(SEED)

# ---- Load ----
with open("synthetic_tensor.npy", "rb") as f:
    X_raw = np.load(f).astype(np.float32)

# ---- Per-channel normalization (NOT global!) ----
# Global mu/sd mixes channels with wildly different scales (max=1458 vs m
# Normalizing per channel makes the CNN's job far easier.
mu = X_raw.mean(axis=(0, 2, 3), keepdims=True)   # (1, 8, 1, 1)
sd = X_raw.std(axis=(0, 2, 3), keepdims=True) + 1e-8
X = (X_raw - mu) / sd
print(f"Per-channel normalized: range [{X.min():.3f}, {X.max():.3f}]")

# ---- TARGETS: drop 'max' and 'l2' (dominated by outliers), keep 3 robus
flat = X_raw.reshape(len(X_raw), -1)
y_full = np.stack([
    flat.mean(1),
    np.median(flat, 1),                  # median > mean for robustness
    flat.std(1),
], axis=1).astype(np.float32)
y_full = (y_full - y_full.mean(0)) / (y_full.std(0) + 1e-8)

# ---- Bigger model + stronger regularization ----
class RegCNN(nn.Module):
    def __init__(self, in_ch=8, n_out=3, hidden=128, dropout=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_ch, 32, 3, padding=1), nn.BatchNorm2d(32), nn.Re
            nn.MaxPool2d(2),                                        # 2 x 6
            nn.Conv2d(32, hidden, 3, padding=1), nn.BatchNorm2d(hidden),
            nn.AdaptiveAvgPool2d((1, 1)), nn.Flatten(),
            nn.Dropout(dropout),
            nn.Linear(hidden, 64), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(64, n_out),
        )
```

```
    def forward(self, x): return self.net(x)

# ---- Run 5-fold CV ----
from sklearn.model_selection import KFold
from sklearn.metrics import r2_score

n = len(X)
kf = KFold(n_splits=5, shuffle=True, random_state=SEED)
fold_r2 = []
fold_mse = []

for fold, (tr, va) in enumerate(kf.split(X)):
    Xtr = torch.from_numpy(X[tr]);  Xva = torch.from_numpy(X[va])
    ytr = torch.from_numpy(y_full[tr]); yva = torch.from_numpy(y_full[va]

    tr_loader = DataLoader(TensorDataset(Xtr, ytr), batch_size=4, shuffle
    va_loader = DataLoader(TensorDataset(Xva, yva), batch_size=4, shuffle

    model = RegCNN(in_ch=8, n_out=3, hidden=128, dropout=0.3)
    opt   = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-3)
    sched = optim.lr_scheduler.CosineAnnealingLR(opt, T_max=150)
    lossf = nn.MSELoss()

    best_mse, best_state, patience = float("inf"), None, 0
    for ep in range(150):
        model.train()
        for xb, yb in tr_loader:
            opt.zero_grad(); loss = lossf(model(xb), yb)
            loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
        sched.step()

        model.eval()
        with torch.no_grad():
            vm = lossf(model(Xva), yva).item()
        if vm < best_mse - 1e-4:
            best_mse, best_state, patience = vm, {k: v.clone() for k, v i
        else:
            patience += 1
            if patience >= 25: break

    model.load_state_dict(best_state)
    model.eval()
    with torch.no_grad():
        pred = model(Xva).numpy()
    r2 = r2_score(yva.numpy(), pred)
    fold_mse.append(best_mse); fold_r2.append(r2)
    print(f"fold {fold}: MSE={best_mse:.4f}  R2={r2:.4f}")

print(f"\nMean R2 = {np.mean(fold_r2):.4f} ± {np.std(fold_r2):.4f}")
print(f"Mean MSE = {np.mean(fold_mse):.4f} ± {np.std(fold_mse):.4f}")
```

```
Per-channel normalized: range [-0.665, 7.178]
fold 0: MSE=0.6175  R2=-2.0524
fold 1: MSE=1.0743  R2=0.3636
fold 2: MSE=0.9450  R2=-1.6721
fold 3: MSE=0.0861  R2=-1.2128
fold 4: MSE=0.7630  R2=-2.4912

Mean R2 = -1.4130 ± 0.9833
Mean MSE = 0.6972 ± 0.3428
```

In [ ]:

```python
# ============================================================
# Stage 2: 4-way classification via quantile binning
# ============================================================
import numpy as np, torch, torch.nn as nn, torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score

# reuse X (per-channel normalized) from Stage 1
scores = X_raw.reshape(len(X_raw), -1).mean(1)
order  = np.argsort(scores, kind="stable")
y_cls  = np.empty(len(scores), dtype=np.int64)
for rank, idx in enumerate(order):
    y_cls[idx] = min(3, (rank * 4) // len(scores))
print("Class counts:", np.bincount(y_cls))

class ClsCNN(nn.Module):
    def __init__(self, in_ch=8, n_cls=4, hidden=128, dropout=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_ch, 32, 3, padding=1), nn.BatchNorm2d(32), nn.Re
            nn.MaxPool2d(2),
            nn.Conv2d(32, hidden, 3, padding=1), nn.BatchNorm2d(hidden),
            nn.AdaptiveAvgPool2d((1, 1)), nn.Flatten(),
            nn.Dropout(dropout),
            nn.Linear(hidden, 64), nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(64, n_cls),
        )
    def forward(self, x): return self.net(x)

skf = StratifiedKFold(n_splits=4, shuffle=True, random_state=SEED)   # 4-
accs, f1s = [], []
for fold, (tr, va) in enumerate(skf.split(X, y_cls)):
    Xtr = torch.from_numpy(X[tr]);  Xva = torch.from_numpy(X[va])
    ytr = torch.from_numpy(y_cls[tr]); yva = torch.from_numpy(y_cls[va])
    tr_loader = DataLoader(TensorDataset(Xtr, ytr), batch_size=4, shuffle

    model = ClsCNN(8, 4, 128, 0.3)
    opt   = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-3)
    sched = optim.lr_scheduler.CosineAnnealingLR(opt, T_max=200)
    lossf = nn.CrossEntropyLoss()

    best_acc, best_state, patience = 0.0, None, 0
    for ep in range(200):
```

```
        model.train()
        for xb, yb in tr_loader:
            opt.zero_grad(); loss = lossf(model(xb), yb)
            loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
        sched.step()

        model.eval()
        with torch.no_grad():
            pred = model(Xva).argmax(1)
        acc = (pred == yva).float().mean().item()
        if acc > best_acc:
            best_acc, best_state, patience = acc, {k: v.clone() for k, v
        else:
            patience += 1
            if patience >= 30: break

    model.load_state_dict(best_state); model.eval()
    with torch.no_grad():
        pred = model(Xva).argmax(1).numpy()
    acc = accuracy_score(yva.numpy(), pred)
    f1  = f1_score(yva.numpy(), pred, average="macro", zero_division=0)
    accs.append(acc); f1s.append(f1)
    print(f"fold {fold}: acc={acc:.3f}  f1={f1:.3f}  "
          f"confusion=\n{confusion_matrix(yva.numpy(), pred)}")

print(f"\nMean acc = {np.mean(accs):.3f} ± {np.std(accs):.3f}")
print(f"Mean F1  = {np.mean(f1s):.3f} ± {np.std(f1s):.3f}")
```

```
Class counts: [4 4 4 4]
fold 0: acc=0.750  f1=0.667  confusion=
[[1 0 0 0]
 [0 1 0 0]
 [1 0 0 0]
 [0 0 0 1]]
fold 1: acc=0.500  f1=0.375  confusion=
[[1 0 0 0]
 [1 0 0 0]
 [1 0 0 0]
 [0 0 0 1]]
fold 2: acc=0.750  f1=0.667  confusion=
[[1 0 0 0]
 [0 1 0 0]
 [0 0 1 0]
 [0 0 1 0]]
fold 3: acc=0.750  f1=0.667  confusion=
[[0 1 0 0]
 [0 1 0 0]
 [0 0 1 0]
 [0 0 0 1]]

Mean acc = 0.688 ± 0.108
Mean F1  = 0.594 ± 0.126
```

In [ ]:

```
# ============================================================
# Stage 3: Autoencoder — reconstruct the tensor itself
# ============================================================
class ConvAE(nn.Module):
    def __init__(self, in_ch=8, h=4, w=12, latent=32):
        super().__init__()
        self.enc = nn.Sequential(
            nn.Conv2d(in_ch, 32, 3, padding=1), nn.ReLU(),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, 1)), nn.Flatten(),
            nn.Linear(64, latent),
        )
        self.dec = nn.Sequential(
            nn.Linear(latent, 64 * h * w), nn.ReLU(),
            nn.Unflatten(1, (64, h, w)),
            nn.Conv2d(64, 32, 3, padding=1), nn.ReLU(),
            nn.Conv2d(32, in_ch, 3, padding=1),
        )
    def forward(self, x): return self.dec(self.enc(x))

X_t = torch.from_numpy(X)
loader = DataLoader(TensorDataset(X_t, X_t), batch_size=4, shuffle=True)
model  = ConvAE(latent=16)
opt    = optim.Adam(model.parameters(), lr=1e-3)
lossf  = nn.MSELoss()

for ep in range(300):
    model.train(); total = 0
    for xb, yb in loader:
        opt.zero_grad(); loss = lossf(model(xb), yb)
        loss.backward(); opt.step()
        total += loss.item() * len(xb)
    if ep % 50 == 0:
        print(f"ep {ep:3d} | recon MSE {total/len(X_t):.5f}")

# Per-sample reconstruction error = anomaly score
model.eval()
with torch.no_grad():
    recon = model(X_t)
    per_sample_mse = ((recon - X_t) ** 2).mean(dim=(1, 2, 3)).numpy()

print("\nPer-sample anomaly scores (higher = more 'outlier'):")
for i, s in enumerate(per_sample_mse):
    print(f"  sample {i:2d}: {s:.5f}")
print(f"\nMean {per_sample_mse.mean():.5f}  std {per_sample_mse.std():.5f
```

```
ep   0 | recon MSE 0.99081
ep  50 | recon MSE 0.02350
ep 100 | recon MSE 0.02366
ep 150 | recon MSE 0.02342
ep 200 | recon MSE 0.02357
ep 250 | recon MSE 0.02302

Per-sample anomaly scores (higher = more 'outlier'):
  sample  0: 0.01504
  sample  1: 0.01555
  sample  2: 0.00283
  sample  3: 0.01348
  sample  4: 0.01830
  sample  5: 0.00329
  sample  6: 0.05199
  sample  7: 0.03337
  sample  8: 0.02886
  sample  9: 0.06588
  sample 10: 0.00357
  sample 11: 0.00326
  sample 12: 0.00408
  sample 13: 0.00427
  sample 14: 0.01299
  sample 15: 0.04497

Mean 0.02011  std 0.01900
```

In [ ]: